

# Physlib Monthly Report

1–31 March 2026

**Contributors:** Alex Meiburg, Bolton Bailey, Daniel Morrison, devtest-boop, Gabe, Gregory Loges, ijsvogelberg, JayYarlott, Jeremias Rieser, Joseph Tooby-Smith, Leonardo A. Lessa, Leonardo Lessa, Pietro Monticone, Rodolfo R. Soldati, Ruizsolveall, Tanner Duve, Zhi Kai Pong

**Reviewers:** Alex Meiburg, Daniel Morrison, Joseph Tooby-Smith, Zhi Kai Pong

Generated 14 August 2026 from commit [f2fd447](#).

## Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–31 March 2026 on branch `master` of [leanprover-community/physlib](#), between commits `6286c6b` and `f2fd447`. Over that period 17 contributors landed 546 commits, changing 40,754 lines across 500 files and adding 2,247 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

## Contents

|          |                                                                                                   |          |
|----------|---------------------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                                                               | <b>2</b> |
| 1.1      | Contributors . . . . .                                                                            | 2        |
| 1.2      | Reviewers . . . . .                                                                               | 2        |
| 1.3      | Modified subfolders . . . . .                                                                     | 2        |
| 1.4      | New declarations by kind . . . . .                                                                | 3        |
| <b>2</b> | <b>Details by file</b>                                                                            | <b>4</b> |
| 2.1      | <code>PhysLean.ClassicalMechanics.DampedHarmonicOscillator.Basic</code> . . . . .                 | 4        |
| 2.2      | <code>PhysLean.Electromagnetism.ThreeDimension.Basic</code> . . . . .                             | 4        |
| 2.3      | <code>PhysLean.Mathematics.KroneckerDelta</code> . . . . .                                        | 5        |
| 2.4      | <code>PhysLean.QuantumMechanics.DDimensions.Operators.Commutation</code> . . . . .                | 6        |
| 2.5      | <code>PhysLean.QuantumMechanics.DDimensions.Operators.Momentum</code> . . . . .                   | 6        |
| 2.6      | <code>PhysLean.QuantumMechanics.DDimensions.Operators.Position</code> . . . . .                   | 7        |
| 2.7      | <code>PhysLean.QuantumMechanics.DDimensions.Operators.Unbounded</code> . . . . .                  | 8        |
| 2.8      | <code>PhysLean.QuantumMechanics.DDimensions.SpaceDHilbertSpace.SchwartzSubmodule</code> . . . . . | 12       |
| 2.9      | <code>PhysLean.Relativity.Tensors.Elab</code> . . . . .                                           | 13       |
| 2.10     | <code>PhysLean.SpaceAndTime.Space.Basic</code> . . . . .                                          | 14       |
| 2.11     | <code>PhysLean.SpaceAndTime.Space.Derivatives.Basic</code> . . . . .                              | 15       |
| 2.12     | <code>PhysLean.SpaceAndTime.Space.Derivatives.Curl</code> . . . . .                               | 16       |
| 2.13     | <code>PhysLean.SpaceAndTime.Space.Derivatives.Grad</code> . . . . .                               | 16       |
| 2.14     | <code>PhysLean.SpaceAndTime.Space.Integrals.Basic</code> . . . . .                                | 17       |
| 2.15     | <code>PhysLean.SpaceAndTime.Space.Integrals.NormPow</code> . . . . .                              | 18       |
| 2.16     | <code>PhysLean.SpaceAndTime.Space.Integrals.RadialAngularMeasure</code> . . . . .                 | 19       |

|      |                                                                                              |     |
|------|----------------------------------------------------------------------------------------------|-----|
| 2.17 | <a href="#">PhysLean.SpaceAndTime.Space.Module</a>                                           | 19  |
| 2.18 | <a href="#">PhysLean.SpaceAndTime.Time.Derivatives</a>                                       | 21  |
| 2.19 | <a href="#">QuantumInfo.ClassicalInfo.Capacity</a>                                           | 21  |
| 2.20 | <a href="#">QuantumInfo.ClassicalInfo.Channel</a>                                            | 22  |
| 2.21 | <a href="#">QuantumInfo.ClassicalInfo.Distribution</a>                                       | 23  |
| 2.22 | <a href="#">QuantumInfo.ClassicalInfo.Entropy</a>                                            | 28  |
| 2.23 | <a href="#">QuantumInfo.ClassicalInfo.ForMathlib.Analysis.SpecialFunctions.Log.NegMulLog</a> | 29  |
| 2.24 | <a href="#">QuantumInfo.ClassicalInfo.Prob</a>                                               | 30  |
| 2.25 | <a href="#">QuantumInfo.Finite.AxiomatizedEntropy.Defs</a>                                   | 38  |
| 2.26 | <a href="#">QuantumInfo.Finite.AxiomatizedEntropy.Renyi</a>                                  | 40  |
| 2.27 | <a href="#">QuantumInfo.Finite.Braket</a>                                                    | 41  |
| 2.28 | <a href="#">QuantumInfo.Finite.Capacity</a>                                                  | 46  |
| 2.29 | <a href="#">QuantumInfo.Finite.Channel.DegradableOrder</a>                                   | 48  |
| 2.30 | <a href="#">QuantumInfo.Finite.CPTPMap.Bundled</a>                                           | 48  |
| 2.31 | <a href="#">QuantumInfo.Finite.CPTPMap.CPTP</a>                                              | 55  |
| 2.32 | <a href="#">QuantumInfo.Finite.CPTPMap.Dual</a>                                              | 64  |
| 2.33 | <a href="#">QuantumInfo.Finite.CPTPMap.MatrixMap</a>                                         | 68  |
| 2.34 | <a href="#">QuantumInfo.Finite.CPTPMap.Unbundled</a>                                         | 72  |
| 2.35 | <a href="#">QuantumInfo.Finite.Distance.Fidelity</a>                                         | 80  |
| 2.36 | <a href="#">QuantumInfo.Finite.Distance.TraceDistance</a>                                    | 81  |
| 2.37 | <a href="#">QuantumInfo.Finite.Ensemble</a>                                                  | 81  |
| 2.38 | <a href="#">QuantumInfo.Finite.Entanglement</a>                                              | 86  |
| 2.39 | <a href="#">QuantumInfo.Finite.Entropy.DPI</a>                                               | 89  |
| 2.40 | <a href="#">QuantumInfo.Finite.Entropy.Relative</a>                                          | 93  |
| 2.41 | <a href="#">QuantumInfo.Finite.Entropy.SSA</a>                                               | 108 |
| 2.42 | <a href="#">QuantumInfo.Finite.Entropy.VonNeumann</a>                                        | 119 |
| 2.43 | <a href="#">QuantumInfo.Finite.MState</a>                                                    | 123 |
| 2.44 | <a href="#">QuantumInfo.Finite.Pinching</a>                                                  | 138 |
| 2.45 | <a href="#">QuantumInfo.Finite.POVm</a>                                                      | 141 |
| 2.46 | <a href="#">QuantumInfo.Finite.Qubit.Basic</a>                                               | 142 |
| 2.47 | <a href="#">QuantumInfo.Finite.ResourceTheory.FreeState</a>                                  | 146 |
| 2.48 | <a href="#">QuantumInfo.Finite.ResourceTheory.HypothesisTesting</a>                          | 152 |
| 2.49 | <a href="#">QuantumInfo.Finite.ResourceTheory.ResourceTheory</a>                             | 154 |
| 2.50 | <a href="#">QuantumInfo.Finite.ResourceTheory.SteinsLemma</a>                                | 155 |
| 2.51 | <a href="#">QuantumInfo.Finite.Unitary</a>                                                   | 162 |
| 2.52 | <a href="#">QuantumInfo.ForMathlib.ContinuousLinearMap</a>                                   | 163 |
| 2.53 | <a href="#">QuantumInfo.ForMathlib.ContinuousSup</a>                                         | 164 |
| 2.54 | <a href="#">QuantumInfo.ForMathlib.Filter</a>                                                | 167 |
| 2.55 | <a href="#">QuantumInfo.ForMathlib.HermitianMat.Basic</a>                                    | 167 |
| 2.56 | <a href="#">QuantumInfo.ForMathlib.HermitianMat.CFC</a>                                      | 180 |
| 2.57 | <a href="#">QuantumInfo.ForMathlib.HermitianMat.Inner</a>                                    | 191 |
| 2.58 | <a href="#">QuantumInfo.ForMathlib.HermitianMat.Jordan</a>                                   | 196 |
| 2.59 | <a href="#">QuantumInfo.ForMathlib.HermitianMat.LogExp</a>                                   | 198 |
| 2.60 | <a href="#">QuantumInfo.ForMathlib.HermitianMat.NonSingular</a>                              | 203 |
| 2.61 | <a href="#">QuantumInfo.ForMathlib.HermitianMat.Order</a>                                    | 207 |
| 2.62 | <a href="#">QuantumInfo.ForMathlib.HermitianMat.Proj</a>                                     | 213 |
| 2.63 | <a href="#">QuantumInfo.ForMathlib.HermitianMat.Reindex</a>                                  | 219 |
| 2.64 | <a href="#">QuantumInfo.ForMathlib.HermitianMat.Rpow</a>                                     | 220 |
| 2.65 | <a href="#">QuantumInfo.ForMathlib.HermitianMat.Schatten</a>                                 | 226 |
| 2.66 | <a href="#">QuantumInfo.ForMathlib.HermitianMat.Sqrt</a>                                     | 228 |
| 2.67 | <a href="#">QuantumInfo.ForMathlib.HermitianMat.Trace</a>                                    | 229 |

|      |                                                       |     |
|------|-------------------------------------------------------|-----|
| 2.68 | QuantumInfo.ForMathlib.HermitianMat.Unitary . . . . . | 233 |
| 2.69 | QuantumInfo.ForMathlib.IsMaximalSelfAdjoint . . . . . | 234 |
| 2.70 | QuantumInfo.ForMathlib.Isometry . . . . .             | 234 |
| 2.71 | QuantumInfo.ForMathlib.Lieb . . . . .                 | 240 |
| 2.72 | QuantumInfo.ForMathlib.LimSupInf . . . . .            | 240 |
| 2.73 | QuantumInfo.ForMathlib.LinearEquiv . . . . .          | 242 |
| 2.74 | QuantumInfo.ForMathlib.Majorization . . . . .         | 244 |
| 2.75 | QuantumInfo.ForMathlib.Matrix . . . . .               | 250 |
| 2.76 | QuantumInfo.ForMathlib.MatrixNorm.TraceNorm . . . . . | 260 |
| 2.77 | QuantumInfo.ForMathlib.Minimax . . . . .              | 262 |
| 2.78 | QuantumInfo.ForMathlib.Misc . . . . .                 | 264 |
| 2.79 | QuantumInfo.ForMathlib.SionMinimax . . . . .          | 265 |
| 2.80 | QuantumInfo.ForMathlib.Superadditive . . . . .        | 270 |
| 2.81 | QuantumInfo.ForMathlib.ULift . . . . .                | 270 |
| 2.82 | QuantumInfo.ForMathlib.Unitary . . . . .              | 271 |
| 2.83 | QuantumInfo.Regularized . . . . .                     | 272 |
| 2.84 | QuantumInfo.StatMech.Hamiltonian . . . . .            | 274 |
| 2.85 | QuantumInfo.StatMech.IdealGas . . . . .               | 275 |
| 2.86 | QuantumInfo.StatMech.ThermoQuantities . . . . .       | 275 |

# 1 Introduction

During Mar 2026, 17 contributors submitted 546 commits that changed 40,754 lines ([+37,415](#) / [-3,339](#)) across 500 files spanning 33 top-level areas of the repository. Of those, [89](#) files were newly added and [2](#) were removed outright. This report catalogues 2,247 newly added Lean declarations: 1,297 theorems, 417 lemmas, 318 defs, 167 instances, 20 structures, 15 abbrevs, 11 classes, 1 opaque, and 1 axiom. We're delighted to recognize the following first-time contributors this month: [Alex Meiburg](#), Bolton Bailey, devtest-boop, Gabe, ijsvogelberg, [JayYarlott](#), [Jeremias Rieser](#), Leonardo A. Lessa, Leonardo Lessa, Rodolfo R. Soldati, Ruizsolveall, and Tanner Duve.

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

## 1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

| Contributor                        | Lines Changed | Commits |
|------------------------------------|---------------|---------|
| <a href="#">Alex Meiburg</a>       | 91,184        | 372     |
| Bolton Bailey                      | 157           | 3       |
| <a href="#">Daniel Morrison</a>    | 82            | 1       |
| devtest-boop                       | 82            | 5       |
| Gabe                               | 578           | 2       |
| Gregory Loges                      | 2,593         | 11      |
| ijsvogelberg                       | 4             | 1       |
| <a href="#">JayYarlott</a>         | 48            | 1       |
| <a href="#">Jeremias Rieser</a>    | 192           | 6       |
| <a href="#">Joseph Tooby-Smith</a> | 3,355         | 29      |
| Leonardo A. Lessa                  | 2,143         | 31      |
| Leonardo Lessa                     | 1,561         | 43      |
| <a href="#">Pietro Monticone</a>   | 84            | 3       |
| Rodolfo R. Soldati                 | 746           | 19      |
| Ruizsolveall                       | 80            | 1       |
| Tanner Duve                        | 17            | 2       |
| <a href="#">Zhi Kai Pong</a>       | 4,835         | 16      |

## 1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

| Reviewer                           | PRs Reviewed |
|------------------------------------|--------------|
| <a href="#">Alex Meiburg</a>       | 1            |
| <a href="#">Daniel Morrison</a>    | 11           |
| <a href="#">Joseph Tooby-Smith</a> | 17           |
| <a href="#">Zhi Kai Pong</a>       | 6            |

## 1.3 Modified subfolders

| Subfolder | Files | New | Deleted | Additions | Deletions |
|-----------|-------|-----|---------|-----------|-----------|
|-----------|-------|-----|---------|-----------|-----------|

|                               |    |     |    |         |        |
|-------------------------------|----|-----|----|---------|--------|
| QuantumInfo/Finite            | 31 | +31 | —  | +14,555 | -0     |
| QuantumInfo/ForMathlib        | 34 | +34 | —  | +12,440 | -0     |
| PhysLean/SpaceAndTime         | 32 | +4  | -1 | +1,933  | -1,047 |
| PhysLean/QuantumMechanics     | 29 | —   | —  | +1,397  | -779   |
| QuantumInfo                   | 4  | +4  | —  | +2,015  | -0     |
| QuantumInfo/ClassicalInfo     | 6  | +6  | —  | +1,119  | -0     |
| PhysLean.lean                 | 1  | —   | —  | +410    | -404   |
| PhysLean/Relativity           | 81 | —   | —  | +520    | -154   |
| PhysLean/QFT                  | 72 | —   | —  | +446    | -153   |
| PhysLean/Mathematics          | 33 | —   | —  | +409    | -116   |
| PhysLean/Particles            | 64 | —   | —  | +372    | -113   |
| QuantumInfo/StatMech          | 3  | +3  | —  | +442    | -0     |
| PhysLean/ClassicalMechanics   | 18 | —   | —  | +275    | -146   |
| PhysLean/Electromagnetism     | 23 | +1  | —  | +206    | -62    |
| PhysLean/Meta                 | 15 | —   | —  | +128    | -61    |
| README.md                     | 1  | —   | —  | +105    | -67    |
| lakefile.toml                 | 1  | —   | -1 | +0      | -157   |
| lakefile.lean                 | 1  | +1  | —  | +117    | -0     |
| PhysLean/StringTheory         | 13 | —   | —  | +75     | -25    |
| PhysLean/Units                | 14 | —   | —  | +77     | -21    |
| docs                          | 2  | +2  | —  | +86     | -0     |
| .github/workflows             | 1  | +1  | —  | +82     | -0     |
| QuantumInfo/InfiniteDim       | 1  | +1  | —  | +74     | -0     |
| PhysLean/StatisticalMechanics | 5  | —   | —  | +33     | -13    |
| QuantumInfo.lean              | 1  | +1  | —  | +42     | -0     |
| PhysLean/Thermodynamics       | 4  | —   | —  | +23     | -7     |
| PhysLean/CondensedMatter      | 2  | —   | —  | +11     | -5     |
| PhysLean/Cosmology            | 2  | —   | —  | +9      | -3     |
| PhysLean/Optics               | 2  | —   | —  | +8      | -2     |
| .codespellignore              | 1  | —   | —  | +3      | -0     |
| .gitignore                    | 1  | —   | —  | +1      | -2     |
| lake-manifest.json            | 1  | —   | —  | +1      | -1     |
| scripts                       | 1  | —   | —  | +1      | -1     |

#### 1.4 New declarations by kind

| Kind      | Count |
|-----------|-------|
| theorem   | 1,297 |
| lemma     | 417   |
| def       | 318   |
| instance  | 167   |
| structure | 20    |
| abbrev    | 15    |
| class     | 11    |
| opaque    | 1     |
| axiom     | 1     |

## 2 Details by file

### 2.1 PhysLean.ClassicalMechanics.DampedHarmonicOscillator.Basic

[PhysLean/ClassicalMechanics/DampedHarmonicOscillator/Basic.lean](#) on GitHub.

#### lemma kineticEnergy\_eq

[\[source\]](#)

*Equalities for energy eqns*

```
lemma kineticEnergy_eq (x: Time → ℝ) :
  kineticEnergy S x = fun t => (1 / 2 : ℝ) * S.m * (Time.deriv x t)^2
```

#### lemma potentialEnergy\_eq

[\[source\]](#)

```
lemma potentialEnergy_eq (x: Time → ℝ) :
  potentialEnergy S x = fun t => (1 / 2 : ℝ) * S.k * (x t)^2
```

#### lemma energy\_dissipation\_rate

[\[source\]](#)

*Derives the energy dissipation rate from the equation of motion*

```
lemma energy_dissipation_rate (x: Time → ℝ) (h1 : S.EquationOfMotion x)
  (hx : ContDiff ℝ ∞ x) :
  Time.deriv (energy S x) t = - S.γ * (Time.deriv x t)^2
```

#### lemma energy\_not\_conserved

[\[source\]](#)

```
lemma energy_not_conserved (x: Time → ℝ) (h1 : S.EquationOfMotion x)
  (hx : ContDiff ℝ ∞ x)
  (hdx : Time.deriv x t ≠ 0)
  (hy : 0 < S.γ) :
  Time.deriv (energy S x) t < 0
```

### 2.2 PhysLean.Electromagnetism.ThreeDimension.Basic

[PhysLean/Electromagnetism/ThreeDimension/Basic.lean](#) on GitHub.

#### theorem electricField\_eq\_3D

[\[source\]](#)

*The electric field written in terms of the scalar and vector potentials as  $-\nabla \phi - \partial_t A$ .*

```
theorem electricField_eq_3D :
  E = fun t x => - ∇ (φ t) x - ∂_t (fun t => A t x) t
```

#### theorem magneticField\_eq\_3D

[\[source\]](#)

*The magnetic field written as the curl of the vector potential as  $\nabla \times A$ .*

```
theorem magneticField_eq_3D :
  B = fun t x => (∇ × (A t)) x
```

## 2.3 PhysLean.Mathematics.KroneckerDelta

[PhysLean/Mathematics/KroneckerDelta.lean](#) on GitHub.

**lemma eq\_one\_of\_same** [\[source\]](#)

```
@[simp]
lemma eq_one_of_same (i :  $\alpha$ ) :  $\delta[i,i] = 1$ 
```

**lemma eq\_zero\_of\_ne** [\[source\]](#)

```
lemma eq_zero_of_ne {i j :  $\alpha$ } (h : i  $\neq$  j) :  $\delta[i,j] = 0$ 
```

**lemma eq\_of\_coe** [\[source\]](#)

```
@[simp]
lemma eq_of_coe {p :  $\alpha \rightarrow \text{Prop}$ } (i j : Subtype p) :  $\delta[(i : \alpha),j] = \delta[i,j]$ 
```

**lemma eq\_zero\_of\_not** [\[source\]](#)

```
lemma eq_zero_of_not {p :  $\alpha \rightarrow \text{Prop}$ } {i j :  $\alpha$ } (hi :  $\neg p\ i$ ) (hj : p\ j) :  $\delta[i,j] = 0$ 
```

**lemma smul\_of\_eq\_zero** [\[source\]](#)

```
lemma smul_of_eq_zero [AddMonoid M] (i j :  $\alpha$ ) {f :  $\alpha \rightarrow \alpha \rightarrow M$ } (hf : f\ i\ i = 0) :
   $\delta[i,j] \cdot f\ i\ j = 0$ 
```

**lemma smul\_eq\_zero\_iff** [\[source\]](#)

```
lemma smul_eq_zero_iff [AddMonoid M] (i j :  $\alpha$ ) (f :  $\alpha \rightarrow \alpha \rightarrow M$ ) :
   $\delta[i,j] \cdot f\ i\ j = 0 \leftrightarrow i \neq j \vee f\ i\ i = 0$ 
```

**lemma smul\_eq\_zero\_iff'** [\[source\]](#)

```
lemma smul_eq_zero_iff' [AddMonoid M] (i :  $\alpha$ ) (f :  $\alpha \rightarrow \alpha \rightarrow M$ ) :
   $(\forall j : \alpha, \delta[i,j] \cdot f\ i\ j = 0) \leftrightarrow f\ i\ i = 0$ 
```

**lemma smul\_eq\_zero\_iff''** [\[source\]](#)

```
lemma smul_eq_zero_iff'' [AddMonoid M] (f :  $\alpha \rightarrow \alpha \rightarrow M$ ) :
   $(\forall i\ j : \alpha, \delta[i,j] \cdot f\ i\ j = 0) \leftrightarrow \forall i : \alpha, f\ i\ i = 0$ 
```

**lemma symm** [\[source\]](#)

```
lemma symm (i j :  $\alpha$ ) :  $\delta[i,j] = \delta[j,i]$ 
```

**lemma smul\_symm** [\[source\]](#)

```
lemma smul_symm [AddMonoid M] (i j :  $\alpha$ ) (f :  $\alpha \rightarrow \alpha \rightarrow M$ ) :  $\delta[i,j] \cdot f\ j\ i = \delta[i,j] \cdot f\ i\ j$ 
```

**lemma symmetrize**[\[source\]](#)

```
lemma symmetrize [AddMonoid M] (i j : α) (f : α → α → M) :
  δ[i,j] • (f i j + f j i) = (2 * δ[i,j]) • f i j
```

**lemma symmetrize'**[\[source\]](#)

```
lemma symmetrize' [AddCommMonoid M] {K : Type*} [Semifield K] [CharZero K] [Module K M]
  (i j : α) (f : α → α → M) : δ[i,j] • (2 : K)-1 • (f i j + f j i) = δ[i,j] • f i j
```

**lemma smul\_sub\_eq\_zero**[\[source\]](#)

```
@[simp]
lemma smul_sub_eq_zero [AddGroup M] (i j : α) (f : α → α → M) : δ[i,j] • (f i j - f j i) = 0
```

**lemma sum\_mul**[\[source\]](#)

```
@[simp]
lemma sum_mul [Fintype α] (i j : α) : ∑ k : α, δ[i,k] * δ[k,j] = δ[i,j]
```

**lemma sum\_smul**[\[source\]](#)

```
@[simp]
lemma sum_smul [Fintype α] (i : α) (f : α → M) : ∑ j : α, δ[i,j] • f j = f i
```

**lemma sum\_sum\_smul\_eq\_zero**[\[source\]](#)

```
lemma sum_sum_smul_eq_zero [Fintype α] {f : α → α → M} (hf : ∀ i : α, f i i = 0) :
  ∑ i : α, ∑ j : α, δ[i,j] • f i j = 0
```

**lemma finset\_sum\_smul**[\[source\]](#)

```
lemma finset_sum_smul (s : Finset α) (i : α) (f : α → M) :
  ∑ j ∈ s, δ[i,j] • f j = if i ∈ s then f i else 0
```

**lemma finset\_sum\_sum\_smul\_eq\_zero**[\[source\]](#)

```
lemma finset_sum_sum_smul_eq_zero {s s' : Finset α} {f : α → α → M}
  (hf : ∀ i ∈ s ∩ s', f i i = 0) : ∑ i ∈ s, ∑ j ∈ s', δ[i,j] • f i j = 0
```

**2.4 PhysLean.QuantumMechanics.DDimensions.Operators.Commutation**

[PhysLean/QuantumMechanics/DDimensions/Operators/Commutation.lean](#) on GitHub.

**lemma angularMomentum\_comp\_antisymm\_sum**[\[source\]](#)

```
private lemma angularMomentum_comp_antisymm_sum {d : ℕ} (a b : Fin d) :
  ∑ x : Fin d, L[x,a] ∘L L[b,x] = ∑ x : Fin d, L[a,x] ∘L L[x,b]
```

**2.5 PhysLean.QuantumMechanics.DDimensions.Operators.Momentum**

[PhysLean/QuantumMechanics/DDimensions/Operators/Momentum.lean](#) on GitHub.



**def momentumOperatorSchwartz**[\[source\]](#)

*The momentum operators defined on the Schwartz submodule.*

```
def momentumOperatorSchwartz : schwartzSubmodule d →l[C] schwartzSubmodule d
```

**lemma momentumOperatorSchwartz\_isSymmetric**[\[source\]](#)

```
lemma momentumOperatorSchwartz_isSymmetric :  
  (momentumOperatorSchwartz i).IsSymmetric
```

**def momentumUnboundedOperator**[\[source\]](#)

*The symmetric momentum unbounded operators with domain the Schwartz submodule of the Hilbert space.*

```
def momentumUnboundedOperator :  
  UnboundedOperator (SpaceDHilbertSpace d) (SpaceDHilbertSpace d)
```

**2.6 PhysLean.QuantumMechanics.DDimensions.Operators.Position**

[PhysLean/QuantumMechanics/DDimensions/Operators/Position.lean](#) on GitHub.

**lemma norm\_sq\_add\_unit\_sq\_pos**[\[source\]](#)

```
lemma norm_sq_add_unit_sq_pos {d : ℕ} (ε : ℝ+) (x : Space d) : 0 < ||x|| ^ 2 + ε ^ 2
```

**lemma normRegularizedPow\_pos**[\[source\]](#)

```
lemma normRegularizedPow_pos (d : ℕ) (ε : ℝ+) (s : ℝ) (x : Space d) :  
  0 < normRegularizedPow d ε s x
```

**def positionOperatorSchwartz**[\[source\]](#)

*The position operators defined on the Schwartz submodule.*

```
def positionOperatorSchwartz : schwartzSubmodule d →l[C] schwartzSubmodule d
```

**lemma positionOperatorSchwartz\_isSymmetric**[\[source\]](#)

```
lemma positionOperatorSchwartz_isSymmetric : (positionOperatorSchwartz i).IsSymmetric
```

**def positionUnboundedOperator**[\[source\]](#)

*The symmetric position unbounded operators with domain the Schwartz submodule of the Hilbert space.*

```
def positionUnboundedOperator : UnboundedOperator (SpaceDHilbertSpace d) (SpaceDHilbertSpace d)
```

**def radiusRegPowOperatorSchwartz**[\[source\]](#)

*The (regularized) radius operators defined on the Schwartz submodule.*

```
def radiusRegPowOperatorSchwartz {d : ℕ} (ε : ℝ+) (s : ℝ) :
  schwartzSubmodule d →ℓ[ℂ] schwartzSubmodule d
```

**lemma radiusRegPowOperatorSchwartz\_isSymmetric**[\[source\]](#)

```
lemma radiusRegPowOperatorSchwartz_isSymmetric {d : ℕ} (ε : ℝ+) (s : ℝ) :
  (radiusRegPowOperatorSchwartz (d := d) ε s).IsSymmetric
```

**def radiusRegPowUnboundedOperator**[\[source\]](#)

*The symmetric (regularized) radius unbounded operators with domain the Schwartz submodule of the Hilbert space.*

```
def radiusRegPowUnboundedOperator {d : ℕ} (ε : ℝ+) (s : ℝ) :
  UnboundedOperator (SpaceDHilbertSpace d) (SpaceDHilbertSpace d)
```

**2.7 PhysLean.QuantumMechanics.DDimensions.Operators.Unbounded**

[PhysLean/QuantumMechanics/DDimensions/Operators/Unbounded.lean](#) on GitHub.

**lemma ext**[\[source\]](#)

```
@[ext]
lemma ext {U1 U2 : UnboundedOperator H H'} (h : U1.toLinearPMap = U2.toLinearPMap) :
  U1 = U2
```

**instance CoeFun (UnboundedOperator H H') (fun U ↦ U.domain → H')**[\[source\]](#)

```
instance : CoeFun (UnboundedOperator H H') (fun U ↦ U.domain → H')
```

**lemma inner\_map\_polarization**[\[source\]](#)

```
lemma inner_map_polarization {T : UnboundedOperator H H} (x y : T.domain) :
  ⟨⟨T x, ↑y⟩⟩ℂ = (⟨⟨T (x + y), ↑(x + y)⟩⟩ℂ - ⟨⟨T (x - y), ↑(x - y)⟩⟩ℂ
  - I * ⟨⟨T (x + I • y), ↑(x + I • y)⟩⟩ℂ + I * ⟨⟨T (x - I • y), ↑(x - I • y)⟩⟩ℂ) / 4
```

**lemma inner\_map\_polarization'**[\[source\]](#)

```
lemma inner_map_polarization' {T : UnboundedOperator H H} (x y : T.domain) :
  ⟨⟨↑x, T y⟩⟩ℂ = (⟨⟨↑(x + y), T (x + y)⟩⟩ℂ - ⟨⟨↑(x - y), T (x - y)⟩⟩ℂ
  - I * ⟨⟨↑(x + I • y), T (x + I • y)⟩⟩ℂ + I * ⟨⟨↑(x - I • y), T (x - I • y)⟩⟩ℂ) / 4
```

**instance PartialOrder (UnboundedOperator H H')**[\[source\]](#)

```
instance : PartialOrder (UnboundedOperator H H') where
```

**lemma isClosable\_of\_zero**[\[source\]](#)

```
lemma isClosable_of_zero {E F : Type*} [NormedAddCommGroup E] [InnerProductSpace ℂ E]
  [NormedAddCommGroup F] [InnerProductSpace ℂ F] {f : LinearPMap ℂ E F} (hf : f.toFun' = 0) :
  f.IsClosable
```

**instance Zero (UnboundedOperator H H')**[\[source\]](#)

```
instance : Zero (UnboundedOperator H H')
```

**lemma zero\_toLinearPMap**[\[source\]](#)

```
@[simp]
lemma zero_toLinearPMap : (0 : UnboundedOperator H H').toLinearPMap = 0
```

**instance Inhabited (UnboundedOperator H H')**[\[source\]](#)

```
instance : Inhabited (UnboundedOperator H H')
```

**instance Add (UnboundedOperator H H')**[\[source\]](#)

```
noncomputable instance : Add (UnboundedOperator H H') where
```

**lemma add\_domain\_of\_dense**[\[source\]](#)

*The domain of  $U_1 + U_2$  is  $D(U_1) \cap D(U_2)$  when it is dense.*

```
lemma add_domain_of_dense {U1 U2 : UnboundedOperator H H'}
  (hD : Dense (U1.domain ∩ U2.domain : Set H)) : (U1 + U2).domain = U1.domain ∩ U2.domain
```

**lemma add\_domain\_of\_not\_dense**[\[source\]](#)

*The junk value for  $U_1 + U_2$  has domain all of  $H$  when  $D(U_1) \cap D(U_2)$  is not dense.*

```
lemma add_domain_of_not_dense {U1 U2 : UnboundedOperator H H'}
  (hD : ¬Dense (U1.domain ∩ U2.domain : Set H)) : (U1 + U2).domain = ⊤
```

**lemma mem\_domain\_of\_dense**[\[source\]](#)

```
lemma mem_domain_of_dense {U1 U2 : UnboundedOperator H H'}
  (hD : Dense (U1.domain ∩ U2.domain : Set H)) (ψ : (U1 + U2).domain) :
  ↑ψ ∈ U1.domain ∧ ↑ψ ∈ U2.domain
```

**lemma add\_toLinearPMap\_of\_dense\_closable**[\[source\]](#)

*$U_1 + U_2$  is the unbounded operator corresponding to  $U_1.toLinearPMap + U_2.toLinearPMap$ , provided it is both densely defined and closable.*

```
lemma add_toLinearPMap_of_dense_closable {U1 U2 : UnboundedOperator H H'}
  (hD : Dense (U1.domain ∩ U2.domain : Set H))
  (hC : (U1.toLinearPMap + U2.toLinearPMap).IsClosable) :
  (U1 + U2).toLinearPMap = U1.toLinearPMap + U2.toLinearPMap
```

**lemma add\_toLinearPMap\_of\_dense\_not\_closable**[\[source\]](#)

*The junk value for  $U_1 + U_2$  when  $D(U_1) \cap D(U_2)$  is dense and  $U_1 + U_2$  is not closable.*

```
lemma add_toLinearPMap_of_dense_not_closable {U1 U2 : UnboundedOperator H H'}
  (hD : Dense (U1.domain  $\sqcap$  U2.domain : Set H))
  (hC :  $\neg$ (U1.toLinearPMap + U2.toLinearPMap).IsClosable) :
  (U1 + U2).toLinearPMap = (U1.domain  $\sqcap$  U2.domain, 0)
```

**lemma add\_toLinearPMap\_of\_not\_dense**[\[source\]](#)

*The junk value for  $U_1 + U_2$  when  $D(U_1) \cap D(U_2)$  is not dense.*

```
lemma add_toLinearPMap_of_not_dense {U1 U2 : UnboundedOperator H H'}
  (hD :  $\neg$ Dense (U1.domain  $\sqcap$  U2.domain : Set H)) : (U1 + U2).toLinearPMap = 0
```

**lemma add\_apply\_of\_dense\_closable**[\[source\]](#)

*$(U_1 + U_2)\psi = U_1\psi + U_2\psi$  provided  $U_1 + U_2$  is not given by a junk value.*

```
lemma add_apply_of_dense_closable {U1 U2 : UnboundedOperator H H'}
  (hD : Dense (U1.domain  $\sqcap$  U2.domain : Set H))
  (hC : (U1.toLinearPMap + U2.toLinearPMap).IsClosable) ( $\psi$  : (U1 + U2).domain) :
  (U1 + U2)  $\psi$  = U1 ( $\psi$ , (mem_domain_of_dense hD  $\psi$ ).1) + U2 ( $\psi$ , (mem_domain_of_dense hD  $\psi$ ).2)
```

**instance AddZeroClass (UnboundedOperator H H')**[\[source\]](#)

```
noncomputable instance : AddZeroClass (UnboundedOperator H H') where
```

**lemma add\_assoc**[\[source\]](#)

*Addition of unbounded operators is associative when neither of the intermediate additions,  $U_1 + U_2$  and  $U_2 + U_3$ , is given by a junk value.*

```
lemma add_assoc {U1 U2 U3 : UnboundedOperator H H'}
  (hD12 : Dense (U1.domain  $\sqcap$  U2.domain : Set H)) (hC12 : (U1.1 + U2.1).IsClosable)
  (hD23 : Dense (U2.domain  $\sqcap$  U3.domain : Set H)) (hC23 : (U2.1 + U3.1).IsClosable) :
  U1 + U2 + U3 = U1 + (U2 + U3)
```

**lemma smul\_mem\_graph\_of\_mem\_smul\_graph**[\[source\]](#)

```
lemma smul_mem_graph_of_mem_smul_graph {E F : Type*} [NormedAddCommGroup E] [InnerProductSpace  $\mathbb{C}$  E]
  [NormedAddCommGroup F] [InnerProductSpace  $\mathbb{C}$  F] {f : LinearPMap  $\mathbb{C}$  E F} {c :  $\mathbb{C}$ } (hc : c  $\neq$  0)
  {x : E  $\times$  F} (h : x  $\in$  (c • f).graph.topologicalClosure) :
  (x.fst, c-1 • x.snd)  $\in$  f.graph.topologicalClosure
```

**lemma smul\_isClosable\_of\_isClosable**[\[source\]](#)

```
lemma smul_isClosable_of_isClosable {E F : Type*} [NormedAddCommGroup E] [InnerProductSpace  $\mathbb{C}$  E]
  [NormedAddCommGroup F] [InnerProductSpace  $\mathbb{C}$  F] {f : LinearPMap  $\mathbb{C}$  E F} (hf : f.IsClosable)
  (c :  $\mathbb{C}$ ) : (c • f).IsClosable
```

**instance SMul  $\mathbb{C}$  (UnboundedOperator H H')** [\[source\]](#)

**noncomputable instance** : SMul  $\mathbb{C}$  (UnboundedOperator H H') **where**

**lemma smul\_domain** [\[source\]](#)

@[simp]  
**lemma** smul\_domain (c :  $\mathbb{C}$ ) : (c • U).domain = U.domain

**lemma smul\_toLinearPMap** [\[source\]](#)

@[simp]  
**lemma** smul\_toLinearPMap (c :  $\mathbb{C}$ ) : (c • U).toLinearPMap = c • U.toLinearPMap

**lemma zero\_smul\_le\_zero** [\[source\]](#)

**lemma** zero\_smul\_le\_zero : ( $0 : \mathbb{C}$ ) • U  $\leq$  0

**instance DistribSMul  $\mathbb{C}$  (UnboundedOperator H H')** [\[source\]](#)

**noncomputable instance** : DistribSMul  $\mathbb{C}$  (UnboundedOperator H H') **where**

**lemma le\_closure** [\[source\]](#)

**lemma** le\_closure : U  $\leq$  U.closure

**lemma isClosed\_def** [\[source\]](#)

**lemma** isClosed\_def : IsClosed U  $\leftrightarrow$  U.closure = U

**lemma adjoint\_dense\_of\_isClosable** [\[source\]](#)

*The adjoint of a densely defined, closable LinearPMap is densely defined.*

**lemma** adjoint\_dense\_of\_isClosable {f : LinearPMap  $\mathbb{C}$  H H'} (h\_dense : Dense (f.domain : Set H))  
(h\_closable : f.IsClosable) : Dense (f†.domain : Set H')

**lemma adjoint\_closure\_eq\_adjoint** [\[source\]](#)

**lemma** adjoint\_closure\_eq\_adjoint : U†.closure = U†

**lemma le\_adjoint\_adjoint** [\[source\]](#)

**lemma** le\_adjoint\_adjoint : U  $\leq$  U††

**lemma isClosed\_iff** [\[source\]](#)

**lemma** isClosed\_iff : IsClosed U  $\leftrightarrow$  U†† = U

**lemma adjoint\_ge\_adjoint\_of\_le**[\[source\]](#)

```
lemma adjoint_ge_adjoint_of_le {U1 U2 : UnboundedOperator H H'} (h : U1 ≤ U2) : U2† ≤ U1†
```

**lemma closure\_mono**[\[source\]](#)

```
lemma closure_mono {U1 U2 : UnboundedOperator H H'} (h : U1 ≤ U2) : U1.closure ≤ U2.closure
```

**def IsSymmetric**[\[source\]](#)

*T is symmetric if  $\langle\langle T x, y \rangle\rangle = \langle\langle x, T y \rangle\rangle$  for all  $x, y \in T.\text{domain}$ .*

```
def IsSymmetric : Prop
```

**lemma isSymmetric\_iff\_inner\_map\_self\_real**[\[source\]](#)

```
lemma isSymmetric_iff_inner_map_self_real :
  IsSymmetric T ↔ ∀ x : T.domain, (starRingEnd ℂ) ⟨⟨T x, x⟩⟩_ℂ = ⟨⟨T x, x⟩⟩_ℂ
```

**lemma isSymmetric\_iff\_le\_adjoint**[\[source\]](#)

```
lemma isSymmetric_iff_le_adjoint : IsSymmetric T ↔ T ≤ T†
```

**lemma isClosed\_of\_isSelfAdjoint**[\[source\]](#)

```
lemma isClosed_of_isSelfAdjoint {T : UnboundedOperator H H} (hT : IsSelfAdjoint T) : IsClosed T
```

**lemma isSymmetric\_of\_isSelfAdjoint**[\[source\]](#)

```
lemma isSymmetric_of_isSelfAdjoint {T : UnboundedOperator H H} (hT : IsSelfAdjoint T) :
  IsSymmetric T
```

**def IsEssentiallySelfAdjoint**[\[source\]](#)

*T is essentially self-adjoint if its closure is self-adjoint.*

```
def IsEssentiallySelfAdjoint : Prop
```

**lemma isEssentiallySelfAdjoint\_def**[\[source\]](#)

```
lemma isEssentiallySelfAdjoint_def : IsEssentiallySelfAdjoint T ↔ T† = T.closure
```

**lemma isSelfAdjoint\_isEssentiallySelfAdjoint**[\[source\]](#)

```
lemma isSelfAdjoint_isEssentiallySelfAdjoint {T : UnboundedOperator H H} (hT : IsSelfAdjoint T) :
  :
  IsEssentiallySelfAdjoint T
```

**2.8 PhysLean.QuantumMechanics.DDimensions.SpaceDHilbertSpace.SchwartzSubmodule**

[PhysLean/QuantumMechanics/DDimensions/SpaceDHilbertSpace/SchwartzSubmodule.lean](#) on GitHub.

**instance CoeFun (schwartzSubmodule d) fun \_ ↦ Space d →  $\mathbb{C}$**  [\[source\]](#)

**instance** : CoeFun (schwartzSubmodule d) **fun** \_ ↦ Space d →  $\mathbb{C}$

**lemma val\_eq\_coe** [\[source\]](#)

@[simp]

**lemma** val\_eq\_coe ( $\psi$  : schwartzSubmodule d) ( $x$  : Space d) :  $\psi.\text{val } x = \psi x$

**def schwartzEquiv** [\[source\]](#)

*The linear equivalence between the Schwartz functions  $\mathcal{S}(\text{Space } d, \mathbb{C})$  and the Schwartz submodule of  $\text{SpaceDHilbertSpace } d$ .*

**def** schwartzEquiv :  $\mathcal{S}(\text{Space } d, \mathbb{C}) \approx_l[\mathbb{C}]$  schwartzSubmodule d

**lemma schwartzEquiv\_coe\_ae** [\[source\]](#)

**lemma** schwartzEquiv\_coe\_ae : (schwartzEquiv f) =<sup>[volume]</sup> f

**lemma schwartzEquiv\_inner** [\[source\]](#)

**lemma** schwartzEquiv\_inner :

$\langle\langle \text{schwartzEquiv } f, \text{schwartzEquiv } g \rangle\rangle_{\mathbb{C}} = \int x : \text{Space } d, \text{starRingEnd } \mathbb{C} (f x) * g x$

**lemma schwartzEquiv\_ae\_eq** [\[source\]](#)

**lemma** schwartzEquiv\_ae\_eq ( $h$  : schwartzEquiv f =<sup>[volume]</sup> schwartzEquiv g) : f = g

## 2.9 PhysLean.Relativity.Tensors.Elab

[PhysLean/Relativity/Tensors/Elab.lean](#) on GitHub.

**def indexExprIsBracketEval** [\[source\]](#)

*Bool which is true if an index is evaluated bracket  $[\mu]$ .*

**def** indexExprIsBracketEval(stx : Syntax) : Bool

**def getEvalBracketPos** [\[source\]](#)

*For list of indexExpr e.g.  $[\alpha, 3, \beta, 2, [\gamma]]$ , getEvalPos returns a list of pairs  $\mathbb{N} \times \text{Term}$  related to indices which are evaluated e.g.  $[\mu]$ . The second element of each pair is the value corresponding to that index. The first element is the position of that number in the list of indices when all other numbered indices before it are removed. Thus for the example given getEvalBracketPos outputs  $[(4, \gamma)]$ .*

**def** getEvalBracketPos (ind : List (TSyntax `indexExpr)) : TermElabM (List ( $\mathbb{N} \times \text{Term}$ ))

**def evalTermBracketMap**[\[source\]](#)

Given a list  $l$  of pairs  $\mathbb{N} \times \text{Term}$  and a term  $T$  corresponding to a tensor tree, for each  $(a, b)$  in  $l$ , `evalSyntax` applies `TensorTree.eval a b` to  $T$  recursively. Here  $a$  is the position of the index to be evaluated and  $b$  is the value it is evaluated to from the  $[\mu]$  syntax.

For example, if  $l$  is  $[(1, \mu), (1, \nu)]$  and  $T$  is a tensor tree then `evalSyntax l T` is `TensorTree.eval 1 \nu (TensorTree.eval 1 \mu T)`.

The list  $l$  is expected to be the output of `getEvalBracketPos`.

```
def evalTermBracketMap (l : List (ℕ × Term)) (T : Term) : Term
```

**2.10 PhysLean.SpaceAndTime.Space.Basic**

[PhysLean/SpaceAndTime/Space/Basic.lean](#) on GitHub.

**instance Nonempty (Space d)**[\[source\]](#)

```
instance {d} : Nonempty (Space d)
```

**instance VSub (EuclideanSpace ℝ (Fin d)) (Space d)**[\[source\]](#)

```
noncomputable instance {d} : VSub (EuclideanSpace ℝ (Fin d)) (Space d) where
```

**lemma vsub\_apply**[\[source\]](#)

```
@[simp]
lemma vsub_apply {d} (s1 s2 : Space d) (i : Fin d) :
  (s1 -v s2) i = s1 i - s2 i
```

**instance AddTorsor (EuclideanSpace ℝ (Fin d)) (Space d)**[\[source\]](#)

```
noncomputable instance {d} : AddTorsor (EuclideanSpace ℝ (Fin d)) (Space d) where
```

**instance Dist (Space d)**[\[source\]](#)

```
noncomputable instance {d} : Dist (Space d) where
```

**instance PseudoMetricSpace (Space d)**[\[source\]](#)

```
noncomputable instance {d} : PseudoMetricSpace (Space d) where
```

**instance NormedAddTorsor (EuclideanSpace ℝ (Fin d)) (Space d)**[\[source\]](#)

```
noncomputable instance {d} : NormedAddTorsor (EuclideanSpace ℝ (Fin d)) (Space d) where
```

**instance MetricSpace (Space d)**[\[source\]](#)

```
noncomputable instance {d} : MetricSpace (Space d) where
```



**instance Nontrivial (Space d.succ)**

[\[source\]](#)

```
instance {d : ℕ} : Nontrivial (Space d.succ) where
```

**def manifoldStructure**

[\[source\]](#)

*The manifold structure on Space d.*

```
noncomputable def manifoldStructure (d : ℕ) :  
  ModelWithCorners ℝ (EuclideanSpace ℝ (Fin d)) (Space d) where
```

**lemma manifoldStructure\_comp\_manifoldStructure\_symm**

[\[source\]](#)

```
@[simp]  
lemma manifoldStructure_comp_manifoldStructure_symm {d : ℕ} :  
  (↑(manifoldStructure d) ∘ ↑(manifoldStructure d).symm) = id
```

**lemma manifoldStructure\_comp\_manifoldStructure\_symm\_apply**

[\[source\]](#)

```
lemma manifoldStructure_comp_manifoldStructure_symm_apply {d : ℕ}  
  (x : EuclideanSpace ℝ (Fin d)) :  
  (manifoldStructure d) ((manifoldStructure d).symm x) = x
```

**lemma range\_manifoldStructure**

[\[source\]](#)

```
@[simp]  
lemma range_manifoldStructure {d : ℕ} :  
  (Set.range ↑(manifoldStructure d)) = Set.univ
```

**lemma contMDiff\_vaddConst**

[\[source\]](#)

```
lemma contMDiff_vaddConst (d : ℕ) : ContMDiff  
  (manifoldStructure d) (ℐ(ℝ, EuclideanSpace ℝ (Fin d))) ↑ (manifoldStructure d).toFun
```

## 2.11 PhysLean.SpaceAndTime.Space.Derivatives.Basic

[PhysLean/SpaceAndTime/Space/Derivatives/Basic.lean](#) on GitHub.

**lemma deriv\_eq\_fderiv\_fun**

[\[source\]](#)

```
lemma deriv_eq_fderiv_fun [AddCommGroup M] [Module ℝ M] [TopologicalSpace M]  
  (μ : Fin d) (f : Space d → M) :  
  deriv μ f = fun x => fderiv ℝ (fun x => f x) x (basis μ)
```

**lemma deriv\_eq\_mfderiv**

[\[source\]](#)

*The spatial-derivative in terms of the derivative of functions between manifolds.*

```
lemma deriv_eq_mfderiv {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]  
  (μ : Fin d) (f : Space d → M) (x : Space d) :  
  deriv μ f x = mfderiv ℐ(ℝ, Space d) ℐ(ℝ, M) f x (basis μ)
```

**lemma mdifferentiable\_manifoldStructure\_iff\_differentiable**[\[source\]](#)

```

lemma mdifferentiable_manifoldStructure_iff_differentiable {M d} [NormedAddCommGroup M]
  [NormedSpace ℝ M] {f : Space d → M} {x : Space d} :
  MDifferentiableAt (Space.manifoldStructure d)  $\mathcal{I}(\mathbb{R}, M)$  f x ↔ DifferentiableAt ℝ f x

```

**lemma deriv\_eq\_mfderiv\_manifoldStructure**[\[source\]](#)

*The spatial-derivative in terms of the derivative of functions between manifolds with the manifold structure `Space.manifoldStructure d`.*

```

lemma deriv_eq_mfderiv_manifoldStructure {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
  (μ : Fin d) (f : Space d → M) (x : Space d) :
  deriv μ f x = mfderiv (Space.manifoldStructure d)  $\mathcal{I}(\mathbb{R}, M)$  f x (EuclideanSpace.single μ 1)

```

**lemma deriv\_contDiff**[\[source\]](#)

```

lemma deriv_contDiff {d} {f : Space d → ℝ} (hf : ContDiff ℝ (n + 1) f) :
  ContDiff ℝ n fun x i => deriv i f x

```

**2.12 PhysLean.SpaceAndTime.Space.Derivatives.Curl**

[PhysLean/SpaceAndTime/Space/Derivatives/Curl.lean](#) on GitHub.

**lemma curl\_neg**[\[source\]](#)

```

lemma curl_neg (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : Differentiable ℝ f) :
   $\nabla \times (-f) = -\nabla \times f$ 

```

**lemma curl\_sub**[\[source\]](#)

```

lemma curl_sub (f1 f2 : Space → EuclideanSpace ℝ (Fin 3))
  (hf1 : Differentiable ℝ f1) (hf2 : Differentiable ℝ f2) :
   $\nabla \times (f1 - f2) = \nabla \times f1 - \nabla \times f2$ 

```

**lemma curl\_of\_grad\_eq\_zero**[\[source\]](#)

```

lemma curl_of_grad_eq_zero (f : Space → ℝ) (hf : ContDiff ℝ 2 f) :
   $\nabla \times (\nabla f) = 0$ 

```

**2.13 PhysLean.SpaceAndTime.Space.Derivatives.Grad**

[PhysLean/SpaceAndTime/Space/Derivatives/Grad.lean](#) on GitHub.

**lemma contDiff\_grad**[\[source\]](#)

```

@[fun_prop]
lemma contDiff_grad {n} {f : Space → ℝ} (hf : ContDiff ℝ (n + 1) f) :
  ContDiff ℝ n (fun x =>  $\nabla f$  x)

```

**def gradSchwartz**[\[source\]](#)*The gradient on Schwartz maps.*

```
noncomputable def gradSchwartz {d} :  $\mathcal{S}(\text{Space } d, \mathbb{R}) \rightarrow L[\mathbb{R}] \mathcal{S}(\text{Space } d, \text{EuclideanSpace } \mathbb{R} \text{ (Fin } d))$ 
```

**lemma gradSchwartz\_apply\_eq\_grad**[\[source\]](#)

```
lemma gradSchwartz_apply_eq_grad {d} (η :  $\mathcal{S}(\text{Space } d, \mathbb{R})$ ) (x : Space d) :  
  gradSchwartz η x = grad η x
```

## 2.14 PhysLean.SpaceAndTime.Space.Integrals.Basic

[PhysLean/SpaceAndTime/Space/Integrals/Basic.lean](#) on GitHub.**lemma volume\_metricBall\_three\_real**[\[source\]](#)

```
@[simp]  
lemma volume_metricBall_three_real :  
  (volume.real (Metric.ball (0 : Space 3) 1)) = 4 / 3 * Real.pi
```

**lemma integral\_one\_dim\_eq\_integral\_real**[\[source\]](#)

```
lemma integral_one_dim_eq_integral_real {f : Space 1 → ℝ} :  
  ∫ x, f x ∂volume = ∫ x, f (oneEquiv.symm x) ∂volume
```

**lemma integral\_volume\_eq\_spherical**[\[source\]](#)

```
lemma integral_volume_eq_spherical (d : ℕ) (f : Space d.succ → F)  
  [NormedAddCommGroup F] [NormedSpace ℝ F] :  
  ∫ x, f x ∂volume = ∫ x, f (x.2.1 • x.1.1) ∂(volume (α := Space d.succ).toSphere.prod  
    (Measure.volumeIoiPow (Module.finrank ℝ (Space d.succ) - 1)))
```

**instance SFinite (@Measure.comap ↑(Set.Ioi 0) ℝ Subtype.instMeasurableSpace Real...** [\[source\]](#)

```
instance : SFinite (@Measure.comap ↑(Set.Ioi 0) ℝ Subtype.instMeasurableSpace  
  Real.measureSpace.toMeasurableSpace Subtype.val volume)
```

**lemma lintegral\_volume\_eq\_spherical**[\[source\]](#)

```
lemma lintegral_volume_eq_spherical (d : ℕ) (f : Space d.succ → ENNReal) (hf : Measurable f) :  
  ∫- x, f x ∂volume = ∫- x, f (x.2.1 • x.1.1) ∂(volume (α := Space d.succ).toSphere.prod  
    (Measure.volumeIoiPow (Module.finrank ℝ (Space d.succ) - 1)))
```

**lemma lintegral\_volume\_eq\_spherical\_mul**[\[source\]](#)

```
lemma lintegral_volume_eq_spherical_mul (d : ℕ) (f : Space d.succ → ENNReal) (hf : Measurable f)  
  ) :  
  ∫- x, f x ∂volume = ∫- x, f (x.2.1 • x.1.1) * .ofReal (x.2.1 ^ d)  
    ∂(volume (α := Space d.succ).toSphere.prod (Measure.volumeIoiPow 0))
```

## 2.15 PhysLean.SpaceAndTime.Space.Integrals.NormPow

[PhysLean/SpaceAndTime/Space/Integrals/NormPow.lean](#) on GitHub.

**lemma npow\_indicator\_rpow\_eq** [\[source\]](#)

```
private lemma npow_indicator_rpow_eq {n : ℕ} {s : Set ℝ} (hs : 0 ∉ s) (p : ℝ) :
  (fun r ↦ r ^ n • s.indicator (fun r ↦ r ^ p) r) = s.indicator (fun r ↦ r ^ (n + p))
```

**lemma integrable0n\_indicator\_iff** [\[source\]](#)

```
private lemma _root_.MeasureTheory.integrable0n_indicator_iff {s : Set ℝ} (hs : MeasurableSet s)
  (t : Set ℝ) (f : ℝ → ℝ) :
  Integrable0n (s.indicator f) t volume ↔ Integrable0n f (s ∩ t) volume
```

**lemma integrable0n\_norm\_rpow\_ball\_iff** [\[source\]](#)

*The function  $x \mapsto \|x\|^p$  is integrable on  $\{x : \text{Space } d \mid 0 \leq \|x\| < b\}$  iff  $0 < d + p$ .*

```
lemma integrable0n_norm_rpow_ball_iff {d : ℕ} (hd : 0 < d) {b : ℝ} (hb : 0 < b) (p : ℝ) :
  Integrable0n (fun x : Space d ↦ \|x\| ^ p) (Metric.ball 0 b) ↔ 0 < d + p
```

**lemma integrable0n\_norm\_rpow\_ball\_compl\_iff** [\[source\]](#)

*The function  $x \mapsto \|x\|^p$  is integrable on  $\{x : \text{Space } d \mid 0 < a \leq \|x\|\}$  iff  $d + p < 0$ .*

```
lemma integrable0n_norm_rpow_ball_compl_iff {d : ℕ} (hd : 0 < d) {a : ℝ} (ha : 0 < a) (p : ℝ) :
  Integrable0n (fun x : Space d ↦ \|x\| ^ p) (Metric.ball 0 a)c ↔ d + p < 0
```

**lemma integrable0n\_norm\_rpow\_shell** [\[source\]](#)

*The function  $x \mapsto \|x\|^p$  is integrable on the shell  $\{x : \text{Space } d \mid 0 < a \leq \|x\| \wedge \|x\| < b\}$ .*

```
lemma integrable0n_norm_rpow_shell {d : ℕ} {a : ℝ} (ha : 0 < a) (b p : ℝ) :
  Integrable0n (fun x : Space d ↦ \|x\| ^ p) ((Metric.ball 0 b) ∩ (Metric.ball 0 a)c)
```

**lemma integrable0n\_norm\_rpow\_iff\_of\_isBounded\_nhds** [\[source\]](#)

*The function  $x \mapsto \|x\|^p$  is integrable on a bounded neighborhood of the origin iff  $0 < d + p$ .*

```
lemma integrable0n_norm_rpow_iff_of_isBounded_nhds {d : ℕ} (hd : 0 < d) {s : Set (Space d)}
  (hs : Bornology.IsBounded s) (hs' : s ∈ nhds 0) (p : ℝ) :
  Integrable0n (fun x : Space d ↦ \|x\| ^ p) s ↔ 0 < d + p
```

**lemma integrable0n\_norm\_rpow\_of\_isBounded\_compl\_nhds** [\[source\]](#)

*The function  $x \mapsto \|x\|^p$  is integrable on a bounded subset with the origin in its exterior.*

```
lemma integrable0n_norm_rpow_of_isBounded_compl_nhds {d : ℕ} {s : Set (Space d)}
  (hs : Bornology.IsBounded s) (hs' : sc ∈ nhds 0) (p : ℝ) :
  Integrable0n (fun x : Space d ↦ \|x\| ^ p) s
```

**lemma integrable0n\_norm\_rpow\_of\_compl\_nhds**[\[source\]](#)

*The function  $x \mapsto \|x\|^p$  is integrable on a subset with the origin in its exterior provided the decay at infinity is fast enough.*

```
lemma integrable0n_norm_rpow_of_compl_nhds {d : ℕ} (hd : 0 < d) {s : Set (Space d)}
  (hs : sc ∈ nhds 0) {p : ℝ} (h : d + p < 0) : Integrable0n (fun x : Space d ↦ ‖x‖ ^ p) s
```

**2.16 PhysLean.SpaceAndTime.Space.Integrals.RadialAngularMeasure**

[PhysLean/SpaceAndTime/Space/Integrals/RadialAngularMeasure.lean](#) on GitHub.

**instance SFinite (radialAngularMeasure (d**[\[source\]](#)

```
instance (d : ℕ) : SFinite (radialAngularMeasure (d := d))
```

**lemma lintegral\_radialMeasure**[\[source\]](#)

```
lemma lintegral_radialMeasure {d : ℕ} (f : Space d → ENNReal) (hf : Measurable f) :
  ∫- x, f x ∂radialAngularMeasure = ∫- x, ENNReal.ofReal (1 / ‖x‖ ^ (d - 1)) * f x
```

**lemma lintegral\_radialMeasure\_eq\_spherical\_mul**[\[source\]](#)

```
lemma lintegral_radialMeasure_eq_spherical_mul (d : ℕ) (f : Space d.succ → ENNReal)
  (hf : Measurable f) :
  ∫- x, f x ∂radialAngularMeasure = ∫- x, f (x.2.1 • x.1.1)
    ∂(volume (α := Space d.succ).toSphere.prod (Measure.volumeIoiPow 0))
```

**lemma radialAngularMeasure\_closedBall**[\[source\]](#)

```
@[simp]
lemma radialAngularMeasure_closedBall (r : ℝ) :
  radialAngularMeasure (Metric.closedBall (0 : Space 3) r) = ENNReal.ofReal (4 * π * r)
```

**lemma radialAngularMeasure\_real\_closedBall**[\[source\]](#)

```
lemma radialAngularMeasure_real_closedBall (r : ℝ) (hr : 0 < r) :
  radialAngularMeasure.real (Metric.closedBall (0 : Space 3) r) = 4 * π * r
```

**lemma integrable\_radialAngularMeasure\_of\_spherical**[\[source\]](#)

```
lemma integrable_radialAngularMeasure_of_spherical {d : ℕ} (f : Space d.succ → F)
  (hae : StronglyMeasurable f)
  (hf : Integrable (fun x => f (x.2.1 • x.1.1))
    (volume (α := Space d.succ).toSphere.prod (Measure.volumeIoiPow 0))) :
  Integrable f radialAngularMeasure
```

**2.17 PhysLean.SpaceAndTime.Space.Module**

[PhysLean/SpaceAndTime/Space/Module.lean](#) on GitHub.

**lemma dist\_eq\_norm**[\[source\]](#)

```
@[simp]
lemma dist_eq_norm {d} (p q : Space d) :
  dist p q = ||p - q||
```

**lemma contDiff0n\_vadd**[\[source\]](#)

```
@[fun_prop]
lemma contDiff0n_vadd (s : Space d) :
  ContDiff0n ℝ ω (fun (v : EuclideanSpace ℝ (Fin d)) => v +v s) Set.univ
```

**lemma vadd\_differentiable**[\[source\]](#)

```
@[fun_prop]
lemma vadd_differentiable {d} (s : Space d) :
  Differentiable ℝ (fun (v : EuclideanSpace ℝ (Fin d)) => v +v s)
```

**lemma contDiff0n\_vsub**[\[source\]](#)

```
@[fun_prop]
lemma contDiff0n_vsub (s1 : Space d) :
  ContDiff0n ℝ ω (fun (s : Space d) => s -v s1) Set.univ
```

**lemma vsub\_differentiable**[\[source\]](#)

```
@[fun_prop]
lemma vsub_differentiable {d} (s1 : Space d) :
  Differentiable ℝ (fun (s : Space d) => s -v s1)
```

**lemma fderiv\_space\_components**[\[source\]](#)

```
lemma fderiv_space_components {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
  (μ : Fin d) (f : M → Space d) (hf : Differentiable ℝ f) (m dm : M):
  fderiv ℝ f m dm μ = fderiv ℝ (fun m' => f m' μ) m dm
```

**def modelDiffeo**[\[source\]](#)

*A diffeomorphism between the two different manifold structures on Space d, that equivalent to manifoldStructure d and that equivalent to  $\mathcal{I}(\mathbb{R}, \text{Space } d)$*

```
noncomputable def modelDiffeo {d} :
  Diffeomorph (manifoldStructure d)  $\mathcal{I}(\mathbb{R}, \text{Space } d)$  (Space d) (Space d) τ where
```

**lemma modelDiffeo\_apply**[\[source\]](#)

```
@[simp]
lemma modelDiffeo_apply (p : Space d) :
  modelDiffeo p = p
```

**lemma basis\_eq\_mfderiv\_modelDiffeo\_single**[\[source\]](#)

*The derivative of `modelDiffeo` provides an equivalence between `Space d` and `EuclideanSpace ℝ (Fin d)`. This equivalence takes the basis of `EuclideanSpace ℝ (Fin d)` to the basis of `Space d`, and vice versa.*

```
lemma basis_eq_mfderiv_modelDiffeo_single (d : ℕ) (μ : Fin d) (x : Space d) :
  basis μ = mfderiv (manifoldStructure d)  $\mathcal{I}(\mathbb{R}, \text{Space } d)$  (modelDiffeo (d := d)) x
  (EuclideanSpace.single μ 1)
```

**lemma norm\_vadd\_le\_add**[\[source\]](#)

```
lemma norm_vadd_le_add {d} (v : EuclideanSpace ℝ (Fin d)) (s : Space d) :
  ||v +v s|| ≤ ||v|| + ||s||
```

**lemma differentiable\_vadd**[\[source\]](#)

```
@[fun_prop]
lemma differentiable_vadd {d} (v : EuclideanSpace ℝ (Fin d)) :
  Differentiable ℝ (fun (s : Space d) => v +v s)
```

**lemma fderiv\_vadd**[\[source\]](#)

```
@[simp]
lemma fderiv_vadd {d} (v : EuclideanSpace ℝ (Fin d)) :
  fderiv ℝ (fun s => v +v s) = fun _ : Space d => ContinuousLinearMap.id ℝ _
```

**lemma vadd\_hasTemperateGrowth**[\[source\]](#)

```
@[fun_prop]
lemma vadd_hasTemperateGrowth {d} (v : EuclideanSpace ℝ (Fin d)) :
  Function.HasTemperateGrowth (fun s : Space d => v +v s)
```

**2.18 PhysLean.SpaceAndTime.Time.Derivatives**

[PhysLean/SpaceAndTime/Time/Derivatives.lean](#) on GitHub.

**lemma deriv\_contDiff\_of\_space**[\[source\]](#)

```
@[fun_prop]
lemma deriv_contDiff_of_space {n} {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M]
  (f : Time → Space d → M) (hf : ContDiff ℝ (n + 1) f) :
  ContDiff ℝ n fun (x : Space d) => (∂t fun t => f t x) t
```

**2.19 QuantumInfo.ClassicalInfo.Capacity**

[QuantumInfo/ClassicalInfo/Capacity.lean](#) on GitHub.

**structure Code**[\[source\]](#)

*Here we define a `*Code*` by an encoder and a decoder. The encoder is a function that takes strings (Lists) of any length over an alphabet  $A$ , and returns strings over  $I$ ; the decoder takes  $O$  and gives back strings over  $A$ . The idea is that a channel would*

map from strings of  $I$  to  $O$ . Important special cases are where  $I=O$  (the channel doesn't change the symbol set), and `FixedLengthCode` where the output lengths only depend on input lengths.

```
structure Code where
  encoder : List A → List I
  decoder : List O → List A
```

### structure FixedLengthCode

[\[source\]](#)

A `FixedLengthCode` is a `Code` whose output lengths don't depend on the input content, only the input length.

```
structure FixedLengthCode extends Code A I O where
  enc_length : ℕ → ℕ
  enc_maps_length : ∀ as, (encoder as).length = enc_length (as.length)
  dec_length : ℕ → ℕ
  dec_maps_length : ∀ is, (decoder is).length = dec_length (is.length)
```

### structure BlockCode

[\[source\]](#)

A `BlockCode` is a `FixedLengthCode` that (1) maps symbols in discrete blocks of fixed length, (2) the encoded alphabet  $I$  has a canonical injection into  $O$ , and (3) has encoder and decoders that are inverses of each other. This is well-suited to describing noise and erasure channels. If the channel merely "corrupts" the data then this  $O = I$ ; the erasure channel might for instance take  $O = I \oplus \text{Unit}$  or  $\text{Option } I$ .

We define the behavior of a block code to "fail" if the input is not a multiple of the block size, by having it return an empty list.

```
structure BlockCode (io : I → O) extends FixedLengthCode A I O where
  block_in : ℕ
  block_out : ℕ
  block_enc : (Fin block_in → A) → (Fin block_out → I)
  block_dec : (Fin block_out → O) → (Fin block_in → A)
  block_enc_dec_inv : ∀ as, block_dec (io ∘ (block_enc as)) = as
  enc_length na := if na % block_in != 0 then 0 else (na / block_in) * block_out
  dec_length no := if no % block_out != 0 then 0 else (no / block_out) * block_in
  encoder as := if as.length % block_in != 0 then [] else
    sorry
  decoder os := if os.length % block_out != 0 then [] else
    sorry
  enc_maps_length := sorry
  dec_maps_length := sorry
```

## 2.20 QuantumInfo.ClassicalInfo.Channel

[QuantumInfo/ClassicalInfo/Channel.lean](#) on GitHub.

### structure DMChannel

[\[source\]](#)

Discrete Memoryless Channel. Each input symbol  $I$  has a corresponding output distribution `Distribution O`, and this process is applied independently on each symbol in the list.



```
structure DMChannel where
  symb_dist : I → Distribution 0
```

**def on\_fin**

[source]

*Apply a discrete memoryless channel to an  $n$ -character string.*

```
def on_fin (C : DMChannel I 0) {n : ℕ} (is : Fin n → I) : ProbDistribution (Fin n → 0)
```

**def on\_list**

[source]

*Apply a discrete memoryless channel to a list.*

```
def on_list (C : DMChannel I 0) (is : List I) : ProbDistribution (Fin (is.length) → 0)
```

## 2.21 QuantumInfo.ClassicalInfo.Distribution

[QuantumInfo/ClassicalInfo/Distribution.lean](#) on GitHub.

**def ProbDistribution**

[source]

*We define our own (discrete) probability distribution notion here, instead of using PMF from Mathlib, because that uses ENNReals everywhere to maintain compatibility with MeasureTheory.Measure.*

*The probabilities internal to a Distribution are NNReals. This lets us more easily write the statement that they sum to 1, since NNReals can be added. (Probabilities, on their own, cannot.) But the FunLike instance gives Prob out, which carry the information that they are all in the range  $[0,1]$ .*

```
def ProbDistribution (α : Type u) [Fintype α] : Type u
```

**def mk'**

[source]

*Make a distribution, proving only that the values are nonnegative and that the sum is 1. The fact that the values are at most 1 is derived as a consequence.*

```
def mk' (f : α → ℝ) (h₁ : ∀ i, 0 ≤ f i) (hN : ∑ i, f i = 1) : ProbDistribution α
```

**instance instFunLikeProb**

[source]

```
instance instFunLikeProb : FunLike (ProbDistribution α) α Prob where
```

**theorem normalized**

[source]

@[simp]

```
theorem normalized (d : ProbDistribution α) : Finset.sum Finset.univ (fun i ↦ (d i : ℝ)) = 1
```

**abbrev prob**

[source]

```
abbrev prob (d : ProbDistribution α)
```

**theorem fun\_eq\_val**[\[source\]](#)

```
@[simp]
theorem fun_eq_val (d : ProbDistribution  $\alpha$ ) : d.val = d
```

**theorem funlike\_apply**[\[source\]](#)

```
@[simp]
theorem funlike_apply (d :  $\alpha \rightarrow \text{Prob}$ ) (h : _) (x :  $\alpha$ ) :
  DFunLike.coe (self := instFunLikeProb) ⟨d, h⟩ x = d x
```

**theorem ext**[\[source\]](#)

```
@[ext]
theorem ext {p q : ProbDistribution  $\alpha$ } (h :  $\forall x, p x = q x$ ) : p = q
```

**theorem nonempty**[\[source\]](#)

*A distribution provides a witness that d is nonempty.*

```
theorem nonempty (d : ProbDistribution  $\alpha$ ) : Nonempty  $\alpha$ 
```

**def constant**[\[source\]](#)

*Make an constant distribution: supported on a single element. This is also called, variously, a "One-point distribution", a "Degenerate distribution", a "Deterministic distribution", a "Delta function", or a "Point mass distribution".*

```
def constant (x :  $\alpha$ ) : ProbDistribution  $\alpha$ 
```

**theorem constant\_def**[\[source\]](#)

```
theorem constant_def (x :  $\alpha$ ) : (constant x :  $\alpha \rightarrow \text{Prob}$ ) = fun y ↦ if x = y then 1 else 0
```

**theorem constant\_eq**[\[source\]](#)

```
@[simp]
theorem constant_eq (x :  $\alpha$ ) : constant x y = if x = y then 1 else 0
```

**theorem constant\_def'**[\[source\]](#)

```
@[simp]
theorem constant_def' (x y :  $\alpha$ ) : (constant x :  $\alpha \rightarrow \text{Prob}$ ) y = if x = y then 1 else 0
```

**theorem constant\_of\_exists\_one**[\[source\]](#)

*If a distribution has an element with probability 1, the distribution has a constant.*

```
theorem constant_of_exists_one {D : ProbDistribution  $\alpha$ } {x :  $\alpha$ } (h : D x = 1) : D =
  ProbDistribution.constant x
```

**def uniform**[\[source\]](#)*Make an uniform distribution.*

```
def uniform [n : Nonempty α] : ProbDistribution α
```

**theorem uniform\_def**[\[source\]](#)`@[simp]`

```
theorem uniform_def [Nonempty α] (y : α) : ((uniform y) : ℝ) = 1 / (Finset.univ.card (α := α))
```

**def prod**[\[source\]](#)*Make a distribution on a product of two Fintypes.*

```
def prod (d1 : ProbDistribution α) (d2 : ProbDistribution β) : ProbDistribution (Prod α β)
```

**theorem prod\_def**[\[source\]](#)`@[simp]`

```
theorem prod_def (x : α) (y : β) : prod d1 d2 ⟨x, y⟩ = (d1 x) * (d2 y)
```

**def extend\_right**[\[source\]](#)*Given a distribution on  $\alpha$ , extend it to a distribution on  $\text{Sum } \alpha \beta$  by giving it no support on  $\beta$ .*

```
def extend_right (d : ProbDistribution α) : ProbDistribution (α ⊕ β)
```

**def extend\_left**[\[source\]](#)*Given a distribution on  $\alpha$ , extend it to a distribution on  $\text{Sum } \beta \alpha$  by giving it no support on  $\beta$ .*

```
def extend_left (d : ProbDistribution α) : ProbDistribution (β ⊕ α)
```

**instance instMixable**[\[source\]](#)*Make a convex mixture of two distributions on the same set.*

```
instance instMixable : Mixable (α → ℝ) (ProbDistribution α)
```

**def relabel**[\[source\]](#)*Given a distribution on type  $\alpha$  and an equivalence to type  $\beta$ , get the corresponding distribution on type  $\beta$ .*

```
def relabel (d : ProbDistribution α) (σ : β ≈ α) : ProbDistribution β
```

**def congr**[\[source\]](#)*ProbDistribution on  $\alpha$  and  $\beta$  are equivalent for equivalent types  $\alpha \simeq \beta$ .*

```
def congr (σ : α ≈ β) : ProbDistribution α ≈ ProbDistribution β
```

**theorem congr\_apply**[\[source\]](#)

```
@[simp]
theorem congr_apply (σ : α ≈ β) (d : ProbDistribution α) (j : β): (congr σ d) j = d (σ.symm j)
```

**theorem congr\_symm\_apply**[\[source\]](#)

*The inverse and congruence operations for distributions commute*

```
@[simp]
theorem congr_symm_apply (σ : α ≈ β) : (ProbDistribution.congr σ).symm = ProbDistribution.congr
σ.symm
```

**def coin**[\[source\]](#)

*The distribution on Fin 2 corresponding to a coin with probability p. Chance p of 1, 1-p of 0.*

```
def coin (p : Prob) : ProbDistribution (Fin 2)
```

**theorem coin\_val\_zero**[\[source\]](#)

```
@[simp]
theorem coin_val_zero (p : Prob) : coin p 0 = p
```

**theorem coin\_val\_one**[\[source\]](#)

```
@[simp]
theorem coin_val_one (p : Prob) : coin p 1 = 1 - p
```

**theorem fin\_two\_eq\_coin**[\[source\]](#)

*Every distribution on two variable is some coin.*

```
theorem fin_two_eq_coin (d : ProbDistribution (Fin 2)) : d = coin (d 0)
```

**theorem coin\_eq\_iff**[\[source\]](#)

```
theorem coin_eq_iff (p : Prob) (f : ProbDistribution (Fin 2)) :
  ProbDistribution.coin p = f ↔ p = f 0
```

**structure RandVar**[\[source\]](#)

*A T-valued random variable over α is a map var : α → T along with a probability distribution distr : Distribution α.*

```
structure RandVar (α : Type*) [Fintype α] (T : Type*) where
  var : α → T
  distr : ProbDistribution α
```

**instance instFunctor**[\[source\]](#)

```
instance instFunctor : Functor (RandVar α) where map f e
```

**instance instLawfulFunctor**[\[source\]](#)

```
instance instLawfulFunctor : LawfulFunctor (RandVar  $\alpha$ ) where
```

**def expect\_val**[\[source\]](#)

*Distribution.exp\_val is the expectation value of a random variable  $X$ . Under the hood, it is the "convex combination over a finite family" on the type  $T$ , afforded by the Mixable instance, with the probability distribution of  $X$  as weights.*

```
def expect_val (X : RandVar  $\alpha$  T) : T
```

**theorem expect\_val\_eq\_mixable\_mix**[\[source\]](#)

*The expectation value of a random variable over  $\alpha = \text{Fin } 2$  is the same as Mixable.mix with probability weight  $X.\text{distr } 0$*

```
theorem expect_val_eq_mixable_mix (d : ProbDistribution (Fin 2)) (x1 x2 : T) : expect_val (![x1, x2], d) = Mixable.mix (d 0) x1 x2
```

**theorem expect\_val\_constant**[\[source\]](#)

*The expectation value of a random variable with constant probability distribution constant  $x$  is its value at  $x$*

```
theorem expect_val_constant (x :  $\alpha$ ) (f :  $\alpha \rightarrow T$ ) : expect_val {f, (constant x)} = f x
```

**theorem zero\_le\_expect\_val**[\[source\]](#)

*The expectation value of a nonnegative real random variable is also nonnegative*

```
theorem zero_le_expect_val (d : ProbDistribution  $\alpha$ ) (f :  $\alpha \rightarrow \mathbb{R}$ ) (hpos :  $0 \leq f$ ) :  $0 \leq \text{expect\_val } \{f, d\}$ 
```

**def congrRandVar**[\[source\]](#)

*$T$ -valued random variables on  $\alpha$  and  $\beta$  are equivalent if  $\alpha \approx \beta$*

```
def congrRandVar ( $\sigma$  :  $\alpha \approx \beta$ ) : RandVar  $\alpha$  T  $\approx$  RandVar  $\beta$  T
```

**def map\_congr\_eq\_congr\_map**[\[source\]](#)

*Given a  $T$ -valued random variable  $X$  over  $\alpha$ , mapping over  $T$  commutes with the equivalence over  $\alpha$*

```
def map_congr_eq_congr_map {S : Type _} [Mixable U S] (f : T  $\rightarrow$  S) ( $\sigma$  :  $\alpha \approx \beta$ ) (X : RandVar  $\alpha$  T) :  
f <$> congrRandVar  $\sigma$  X = congrRandVar  $\sigma$  (f <$> X)
```

**theorem expect\_val\_congr\_eq\_expect\_val**[\[source\]](#)

*The expectation value is invariant under equivalence of random variables*

```
@[simp]
theorem expect_val_congr_eq_expect_val (σ : α ≈ β) (X : RandVar α T) : expect_val (congrRandVar
  σ X) = expect_val X
```

## 2.22 QuantumInfo.ClassicalInfo.Entropy

[QuantumInfo/ClassicalInfo/Entropy.lean](#) on GitHub.

**def H<sub>1</sub>** [source]

*The one-event entropy function,  $H_1(p) = -p \ln(p)$ . Uses nits.*

```
def H1 : Prob → ℝ
```

**def H<sub>1</sub>\_zero\_eq\_zero** [source]

*$H_1$  of 0 is zero.*

```
@[simp]
def H1_zero_eq_zero : H1 0 = 0
```

**def H<sub>1</sub>\_one\_eq\_zero** [source]

*$H_1$  of 1 is zero.*

```
@[simp]
def H1_one_eq_zero : H1 1 = 0
```

**theorem H<sub>1</sub>\_nonneg** [source]

*Entropy is nonnegative.*

```
theorem H1_nonneg (p : Prob) : 0 ≤ H1 p
```

**theorem H<sub>1</sub>\_le\_1** [source]

*Entropy is less than 1.*

```
theorem H1_le_1 (p : Prob) : H1 p < 1
```

**theorem H<sub>1</sub>\_le\_exp\_m1** [source]

*Entropy is at most  $1/e$ .*

```
theorem H1_le_exp_m1 (p : Prob) : H1 p ≤ Real.exp (-1)
```

**theorem H<sub>1</sub>\_concave** [source]

```
theorem H1_concave : ∀ (x y : Prob), ∀ (p : Prob), p[H1 x ↔ H1 y] ≤ H1 (p[x ↔ y])
```

**def**  $H_s$  [\[source\]](#)

*The Shannon entropy of a discrete distribution,  $H(X) = \sum H_1(p_x)$ .*

```
def  $H_s$  (d : ProbDistribution  $\alpha$ ) :  $\mathbb{R}$ 
```

**theorem**  $H_s\_nonneg$  [\[source\]](#)

*Shannon entropy of a distribution is nonnegative.*

```
theorem  $H_s\_nonneg$  (d : ProbDistribution  $\alpha$ ) :  $0 \leq H_s$  d
```

**theorem**  $H_s\_le\_log\_d$  [\[source\]](#)

*Shannon entropy of a distribution is at most  $\ln d$ .*

```
theorem  $H_s\_le\_log\_d$  (d : ProbDistribution  $\alpha$ ) :  $H_s$  d  $\leq \text{Real.log } (\text{Fintype.card } \alpha)$ 
```

**theorem**  $H_s\_constant\_eq\_zero$  [\[source\]](#)

*The shannon entropy of a constant variable is zero.*

```
@[simp]
```

```
theorem  $H_s\_constant\_eq\_zero$  {i :  $\alpha$ } :  $H_s$  (ProbDistribution.constant i) = 0
```

**theorem**  $H_s\_uniform$  [\[source\]](#)

*Shannon entropy of a uniform distribution is  $\ln d$ .*

```
theorem  $H_s\_uniform$  [Nonempty  $\alpha$ ] :  
   $H_s$  (ProbDistribution.uniform ( $\alpha := \alpha$ )) =  $\text{Real.log } (\text{Finset.univ.card } (\alpha := \alpha))$ 
```

**theorem**  $H_s\_coin$  [\[source\]](#)

*Shannon entropy of two-event distribution.*

```
theorem  $H_s\_coin$  (p : Prob) :  $H_s$  (ProbDistribution.coin p) =  $\text{Real.binEntropy } p$ 
```

**lemma**  $H_s\_eq\_of\_multiset\_map\_eq$  [\[source\]](#)

```
lemma  $H_s\_eq\_of\_multiset\_map\_eq$  (d1 : ProbDistribution  $\alpha$ ) (d2 : ProbDistribution  $\beta$ )  
  (h : Multiset.map d1.prob Finset.univ.val = Multiset.map d2.prob Finset.univ.val) :  
   $H_s$  d1 =  $H_s$  d2
```

## 2.23 QuantumInfo.ClassicalInfo.ForMathlib.Analysis.SpecialFunctions.Log.NegMulLog

QuantumInfo/ClassicalInfo/ForMathlib/Analysis/SpecialFunctions/Log/NegMulLog.lean on GitHub.

**theorem**  $\text{negMulLog\_strictMonoOn}$  [\[source\]](#)

```
theorem  $\text{negMulLog\_strictMonoOn}$  : StrictMonoOn  $\text{Real.negMulLog } (\text{Set.Icc } 0 \text{ (exp } (-1)))$ 
```

**theorem negMulLog\_strictAnti0n**[\[source\]](#)

```
theorem negMulLog_strictAnti0n : StrictAnti0n Real.negMulLog (Set.Ici (exp (-1)))
```

**theorem negMulLog\_le\_rexp\_neg\_one**[\[source\]](#)

```
theorem negMulLog_le_rexp_neg_one {x : ℝ} (hx : 0 ≤ x) : negMulLog x ≤ exp (-1)
```

**2.24 QuantumInfo.ClassicalInfo.Prob**

[QuantumInfo/ClassicalInfo/Prob.lean](#) on GitHub.

**def Prob**[\[source\]](#)

*Prob is a real number in the interval  $[0,1]$ . Similar to `NNReal` in many definitions, but this allows other nice things more 'safely' such as convex combination.*

```
@[reducible]
```

```
def Prob
```

**instance Coe Prob ℝ**[\[source\]](#)

```
instance : Coe Prob ℝ
```

**instance canLift**[\[source\]](#)

```
instance canLift : CanLift ℝ Prob Subtype.val fun r => 0 ≤ r ∧ r ≤ 1
```

**instance instZero**[\[source\]](#)

```
instance instZero : Zero Prob
```

**instance instOne**[\[source\]](#)

```
instance instOne : One Prob
```

**instance instMul**[\[source\]](#)

```
instance instMul : Mul Prob
```

**theorem coe\_zero**[\[source\]](#)

```
@[simp, norm_cast]
```

```
theorem coe_zero : (0 : Prob).val = 0
```

**theorem coe\_one**[\[source\]](#)

```
@[simp, norm_cast]
```

```
theorem coe_one : (1 : Prob).val = 1
```



**theorem coe\_mul**[\[source\]](#)

```
@[simp, norm_cast]
theorem coe_mul (x y : Prob) : (x * y).val = x.val * y.val
```

**theorem coe\_inf**[\[source\]](#)

```
@[simp, norm_cast]
theorem coe_inf (x y : Prob) : (x  $\sqcap$  y).val = x.val  $\sqcap$  y.val
```

**theorem coe\_sup**[\[source\]](#)

```
@[simp, norm_cast]
theorem coe_sup (x y : Prob) : (x  $\sqcup$  y).val = x.val  $\sqcup$  y.val
```

**instance instCommMonoidWithZero**[\[source\]](#)

```
instance instCommMonoidWithZero : CommMonoidWithZero Prob where
```

**instance instDenselyOrdered**[\[source\]](#)

```
instance instDenselyOrdered : DenselyOrdered Prob
```

**instance instCompleteLinearOrder**[\[source\]](#)

```
instance instCompleteLinearOrder : CompleteLinearOrder Prob
```

**instance instInhabited**[\[source\]](#)

```
instance instInhabited : Inhabited Prob where
```

**instance LinearOrderedCommMonoidWithZero Prob**[\[source\]](#)

```
instance : LinearOrderedCommMonoidWithZero Prob where
```

**theorem zero\_le\_coe**[\[source\]](#)

```
@[simp]
theorem zero_le_coe {p : Prob} : 0  $\leq$  (p :  $\mathbb{R}$ )
```

**theorem coe\_le\_one**[\[source\]](#)

```
@[simp]
theorem coe_le_one {p : Prob} : (p :  $\mathbb{R}$ )  $\leq$  1
```

**theorem zero\_le**[\[source\]](#)

```
@[simp]
theorem zero_le {p : Prob} : 0  $\leq$  p
```

**theorem le\_one**[\[source\]](#)

```
@[simp]
theorem le_one {p : Prob} : p ≤ 1
```

**theorem ne\_iff**[\[source\]](#)

```
theorem ne_iff {x y : Prob} : (x : ℝ) ≠ (y : ℝ) ↔ x ≠ y
```

**theorem toReal\_mul**[\[source\]](#)

```
@[simp, norm_cast]
theorem toReal_mul (x y : Prob) : (x * y : Prob) = (x : ℝ) * (y : ℝ)
```

**theorem toNNReal\_mk**[\[source\]](#)

```
@[simp]
theorem toNNReal_mk : toNNReal { val := x, property := hx } = { val := x, property := hx.1 }
```

**instance Coe Prob ℝ≥0**[\[source\]](#)

```
instance : Coe Prob ℝ≥0
```

**instance canLiftNN**[\[source\]](#)

```
instance canLiftNN : CanLift ℝ≥0 Prob toNNReal fun r => r ≤ 1
```

**theorem eq\_iff\_nnreal**[\[source\]](#)

```
protected theorem eq_iff_nnreal (n m : Prob) : (n : ℝ≥0) = (m : ℝ≥0) ↔ n = m
```

**theorem toNNReal\_zero**[\[source\]](#)

```
@[simp, norm_cast]
theorem toNNReal_zero : (0 : Prob) = (0 : ℝ≥0)
```

**theorem toNNReal\_one**[\[source\]](#)

```
@[simp, norm_cast]
theorem toNNReal_one : (1 : Prob) = (1 : ℝ≥0)
```

**theorem ofNNReal\_toNNReal**[\[source\]](#)

```
theorem ofNNReal_toNNReal : ENNReal.ofNNReal (toNNReal p) = ENNReal.ofReal (p : ℝ)
```

**def asProb**[\[source\]](#)

```
def NNReal.asProb (p : ℝ≥0) (hp : p ≤ 1) : Prob
```

**def asProb'**[\[source\]](#)

```
def NNReal.asProb' (p : ℝ≥0) (hp : p.1 ≤ 1) : Prob
```

**theorem zero\_lt\_coe**[\[source\]](#)

```
theorem zero_lt_coe {p : Prob} (hp : p ≠ 0) : (0 : ℝ) < p
```

**instance instSub**[\[source\]](#)

*Subtract a probability from another. Truncates to zero, so this is often not great to work with, for the same reason that Nat subtraction is a pain. But, it lets you write  $1 - p$ , which is sufficiently useful on its own that this seems worth having.*

```
instance instSub : Sub Prob where
```

**theorem coe\_sub**[\[source\]](#)

```
theorem coe_sub (p q : Prob) : (p - q : Prob) = (p.val - q.val) ⊔ (0 : ℝ)
```

**theorem coe\_one\_minus**[\[source\]](#)

```
@[simp, norm_cast]
```

```
theorem coe_one_minus (p : Prob) : (1 - p : Prob) = 1 - (p : ℝ)
```

**theorem add\_one\_minus**[\[source\]](#)

```
theorem add_one_minus (p : Prob) : p.val + (1 - p).val = 1
```

**theorem one\_minus\_inv**[\[source\]](#)

```
@[simp]
```

```
theorem one_minus_inv (p : Prob) : 1 - (1 - p) = p
```

**instance OrderTopology Prob**[\[source\]](#)

```
instance : OrderTopology Prob
```

**theorem coe\_iInf**[\[source\]](#)

```
@[simp, norm_cast]
```

```
theorem coe_iInf {ι : Type*} [Nonempty ι] (f : ι → Prob) : ↑(⋂ t, f t) = (⋂ t, f t : ℝ)
```

**instance Nontrivial Prob**[\[source\]](#)

```
instance : Nontrivial Prob where
```

**theorem top\_eq\_one**[\[source\]](#)

```
@[simp]
theorem top_eq_one : (T : Prob) = 1
```

**theorem sub\_zero**[\[source\]](#)

```
@[simp]
theorem sub_zero (p : Prob) : p - 0 = p
```

**theorem toNNReal\_Continuous**[\[source\]](#)

```
@[fun_prop]
theorem toNNReal_Continuous : Continuous Prob.toNNReal
```

**theorem mul\_eq\_one\_iff**[\[source\]](#)

```
@[simp]
theorem mul_eq_one_iff (p q : Prob) : p * q = 1 ↔ p = 1 ∧ q = 1
```

**class Mixable**[\[source\]](#)

A *Mixable*  $T$  typeclass instance gives a compact way of talking about the action of probabilities for forming linear combinations in convex spaces. The notation  $p [x_1 \leftrightarrow x_2]$  means to take a convex combination, equal to  $x_1$  if  $p=1$  and to  $x_2$  if  $p=0$ .

*Mixable* is defined by an "underlying" data type  $U$  with addition and scalar multiplication, and a bijection between the  $T$  and a convex set of  $U$ . For instance, in *Mixable* (*Distribution* (*Fin*  $n$ )),  $U$  is  $n$ -element vectors (which form the probability simplex, degenerate in one dimension). For *QuantumInfo.Finite.MState* density matrices in quantum mechanics, which are PSD matrices of trace 1,  $U$  is the underlying matrix.

Why not just stick with existing notions of *Convex*? *Convex* requires that the type already forms an *AddCommMonoid* and *Module*  $\mathbb{R}$ . But many types, such as *Distribution*, are not: there is no good notion of "multiplying a probability distribution by 0.3" to get another distribution. We can coerce the distribution into, say, a vector or a function, but then we are not doing arithmetic with distributions. Accordingly, the expression  $0.3 * \text{distA} + 0.7 * \text{distB}$  cannot represent a distribution on its own.

```
class Mixable (U : outParam (Type u)) (T : Type v) [AddCommMonoid U] [Module ℝ U] where
  to_U : T → U
  to_U_inj : ∀ {T₁ T₂}, to_U T₁ = to_U T₂ → T₁ = T₂
  convex : Convex ℝ (Set.range to_U)
  mkT : {u : U} → (∃ t, to_U t = u) → { t : T // to_U t = u }
```

**def mix\_ab**[\[source\]](#)

```
@[reducible]
def mix_ab [inst : Mixable U T] {a b : ℝ} (ha : 0 ≤ a) (hb : 0 ≤ b) (hab : a + b = 1) (x₁ x₂ : T) : T
```

**def Mixable.mix**[\[source\]](#)

*Mixable.mix* represents the notion of "convex combination" on the type  $T$ , afforded by the *Mixable* instance. It takes a *Prob*, that is, a *Real* between 0 and 1. For working directly with a *Real*, use *mix\_ab*.

```
def mix [inst : Mixable U T] (p : Prob) (x1 x2 : T) : T
```

**theorem to\_U\_of\_mkT**[\[source\]](#)

```
@[simp]
```

```
theorem to_U_of_mkT [inst : Mixable U T] (u : U) {h} : inst.to_U (mkT (u := u) h).1 = u
```

**theorem mix\_zero**[\[source\]](#)

```
@[simp]
```

```
theorem mix_zero [inst : Mixable U T] (x1 x2 : T) : (0 : Prob) [ x1 ↔ x2 : inst] = x2
```

**instance instUniv**[\[source\]](#)

When  $T$  is the whole space, and  $T$  is a suitable vector space over  $\mathbb{R}$ , we get a *Mixable* instance.

```
instance instUniv [AddCommMonoid T] [Module ℝ T] : Mixable T T where
```

**theorem mkT\_instUniv**[\[source\]](#)

```
@[simp]
```

```
theorem mkT_instUniv [AddCommMonoid T] [Module ℝ T] {t : T} (h : ∃ t', to_U t' = t) : instUniv.  
mkT h = (t, rfl)
```

**theorem to\_U\_instUniv**[\[source\]](#)

```
@[simp]
```

```
theorem to_U_instUniv [AddCommMonoid T] [Module ℝ T] {t : T} : instUniv.to_U t = t
```

**theorem lem\_1**[\[source\]](#)

```
theorem instPi.lem_1 {D : Type*} {T U : D → Type*} [∀ i, AddCommMonoid (U i)] [∀ i, Module ℝ (U  
i)] [inst : ∀ i, Mixable (U i) (T i)]  
{u : (i : D) → U i} (h : ∃ (t : (i : D) → T i), (fun d => to_U (t d)) = u) (d : D) : ∃ (t :  
T d), to_U t = u d
```

**instance instPi**[\[source\]](#)

*Mixable* instance on *Pi* types.

```
instance instPi : Mixable ((i:D) → U i) ((i:D) → T i) where
```

**theorem val\_mkT\_instPi**[\[source\]](#)

```
@[simp]
theorem val_mkT_instPi (D : Type*) [inst : Mixable U T] {u : D → U} (h : ∃ t, to_U t = u) : (
  instPi.mkT h).val =
  fun d ↦ (inst.mkT (instPi.lem_1 h d)).val
```

### theorem to\_U\_instPi

[\[source\]](#)

```
@[simp]
theorem to_U_instPi (D : Type*) [inst : Mixable U T] {t : D → T} : (instPi).to_U t = fun d ↦
  inst.to_U (t d)
```

### def instSubtype

[\[source\]](#)

*Mixable instances on subtypes (of other mixable types), assuming that they have the correct closure properties.*

```
def instSubtype {T : Type*} {P : T → Prop} (inst : Mixable U T)
  (h : ∀{x y:T},
    ∀{a b : ℝ}, (ha : 0 ≤ a) → (hb : 0 ≤ b) → (hab : a + b = 1) →
    P x → P y → P (inst.mix_ab ha hb hab x y))
  : Mixable U { t // P t} where
```

### instance instMixable

[\[source\]](#)

*Probabilities Prob themselves are convex.*

```
instance instMixable : Mixable ℝ Prob where
```

### theorem to\_U\_mixable

[\[source\]](#)

```
@[simp]
theorem to_U_mixable [AddCommMonoid T] [SMul ℝ T] (t : Prob) : instMixable.to_U t = t.val
```

### theorem mkT\_mixable

[\[source\]](#)

```
@[simp]
theorem mkT_mixable (u : ℝ) (h : ∃ t : Prob, Mixable.to_U t = u) : Mixable.mkT h =
  ⟨⟨u,Exists.casesOn h fun t ht ↦ ht ▸ t.2⟩, rfl⟩
```

### abbrev Prob.mix

[\[source\]](#)

*Prob.mix is an alias of Mixable.mix so it can be accessed from a probability with dot notation, e.g. p.mix x y.*

```
abbrev mix [AddCommMonoid U] [Module ℝ U] [inst : Mixable U T] (p : Prob) (x₁ x₂ : T)
```

### def negLog

[\[source\]](#)

*Map a probability  $[0,1]$  to  $[0,+\infty]$  with  $-\log p$ . Special case that 0 maps to  $+\infty$  (not 0, as Real.log does). This makes it Antitone.*

```
noncomputable def negLog : Prob → ENNReal
```

**theorem negLog\_Antitone**[\[source\]](#)

```
theorem negLog_Antitone : Antitone negLog
```

**theorem negLog\_zero**[\[source\]](#)

```
@[simp]
theorem negLog_zero : -log (0 : Prob) = T
```

**theorem negLog\_one**[\[source\]](#)

```
@[simp]
theorem negLog_one : -log 1 = 0
```

**theorem negLog\_eq\_top\_iff**[\[source\]](#)

```
@[simp]
theorem negLog_eq_top_iff {p : Prob} : -log p = T ↔ p = 0
```

**theorem negLog\_pos\_ENNReal**[\[source\]](#)

```
theorem negLog_pos_ENNReal {p : Prob} (hp : p ≠ 0) : -log p = .ofNNReal (-Real.log p,
  Left.nonneg_neg_iff.mpr (Real.log_nonpos p.2.1 p.2.2))
```

**theorem negLog\_pos\_Real**[\[source\]](#)

```
@[simp]
theorem negLog_pos_Real {p : Prob} : (-log p).toReal = -Real.log p
```

**theorem le\_negLog\_of\_le\_exp**[\[source\]](#)

```
theorem le_negLog_of_le_exp {p : Prob} {x : ℝ} (h : p ≤ Real.exp (-x)) : ENNReal.ofReal x ≤ -
  log p
```

**theorem negLog\_ne\_top**[\[source\]](#)

```
@[aesop (rule_sets := [finiteness]) safe apply]
theorem negLog_ne_top {p : Prob} (hp : 0 < p.val) : -log p ≠ ∞
```

**theorem negLog\_eq\_neg\_ENNReal\_log**[\[source\]](#)

```
theorem negLog_eq_neg_ENNReal_log (p : Prob) : -log p = -ENNReal.log p
```

**theorem negLog\_eq\_ofReal\_neg\_log**[\[source\]](#)

```
theorem negLog_eq_ofReal_neg_log {p : Prob} (hp : 0 < p) :
  ENNReal.ofReal (-Real.log p) = -log p
```

**theorem zero\_lt\_negLog**[\[source\]](#)

```
@[simp]
theorem zero_lt_negLog {p : Prob} : 0 < -log p ↔ p ≠ 1
```

**theorem Continuous\_negLog**[\[source\]](#)

```
@[fun_prop]
theorem Continuous_negLog : Continuous negLog
```

**2.25 QuantumInfo.Finite.AxiomatizedEntropy.Defs**

[QuantumInfo/Finite/AxiomatizedEntropy/Defs.lean](#) on GitHub.

**class RelEntropy**[\[source\]](#)

*The axioms to be a well-behaved quantum relative entropy, as given by [Tomamichel](https://www.marcotomamichel.name/masterclass2022.pdf).*

*This simpler class allows for `_trivial_` relative entropies, such as  $-\log \text{tr}(\rho^\circ \sigma)$ . Use `mixing RelEntropy.Nontrivial` to only allow nontrivial relative entropies.*

```
class RelEntropy : Prop where
  DPI {d1 d2 : Type u} [Fintype d1] [DecidableEq d1] [Fintype d2] [DecidableEq d2]
    (ρ σ : MState d1) (Λ : CPTPMap d1 d2) : f (Λ ρ) (Λ σ) ≤ f ρ σ
  of_kron {d1 d2 : Type u} [Fintype d1] [Fintype d2] [DecidableEq d1] [DecidableEq d2] :
    ∀ (ρ1 σ1 : MState d1) (ρ2 σ2 : MState d2), f (ρ1 ⊗ ρ2) (σ1 ⊗ σ2) = f ρ1 σ1 + f ρ2 σ2
  normalized {d : Type u} [fin : Fintype d] [DecidableEq d] [Nonempty d] (i : d) :
    f (.ofClassical (.constant i)) MState.uniform.M =
      some (Real.log fin.card, Real.log_nonneg (mod_cast Fintype.card_pos))
```

**class Nontrivial**[\[source\]](#)

*The axioms to be a well-behaved quantum relative entropy, as given by [Tomamichel](https://www.marcotomamichel.name/masterclass2022.pdf).*

```
class RelEntropy.Nontrivial [RelEntropy f] where
  nontrivial (d) [Fintype d] [DecidableEq d] : ∃ (ρ σ : MState d),
    ρ.M.support = τ ∧ σ.M.support = τ ∧ 0 < f ρ σ
```

**theorem of\_equiv\_eq**[\[source\]](#)

*Relabelling a state with `CPTPMap.of_equiv` leaves relative entropies unchanged.*

```
@[simp]
theorem of_equiv_eq (e : d ≈ d2) (ρ σ : MState d) :
  f (CPTPMap.of_equiv e ρ) (CPTPMap.of_equiv e σ) = f ρ σ
```

**theorem relabel\_eq**[\[source\]](#)

*Relabelling a state with `MState.relabel` leaves relative entropies unchanged.*

```
@[simp]
theorem relabel_eq (e : d2 ≈ d) (ρ σ : MState d) :
  f (ρ.relabel e) (σ.relabel e) = f ρ σ
```



**lemma wrt\_self\_eq\_zero'**

[\[source\]](#)

*The relative entropy is zero between any two states on a 1-D Hilbert space.*

```
private lemma wrt_self_eq_zero' [Unique d] (p σ : MState d) : f p σ = 0
```

**theorem wrt\_self\_eq\_zero**

[\[source\]](#)

*The relative entropy  $D(\rho||\rho) = 0$ .*

```
@[simp]
theorem wrt_self_eq_zero (p : MState d) : f p p.M = 0
```

**theorem faithful**

[\[source\]](#)

*A nontrivial relative entropy is **faithful**, it can distinguish when two states are equal.*

```
theorem faithful (p σ : MState d) : f p σ = 0 ↔ p = σ
```

**def min**

[\[source\]](#)

*Quantum relative min-entropy.*

```
def min (p : MState d) (σ : HermitianMat d ℂ) : ENNReal
```

**theorem min\_eq\_top\_iff**

[\[source\]](#)

```
@[aesop (rule_sets := [finiteness]) simp]
theorem min_eq_top_iff (p : MState d) (σ : HermitianMat d ℂ) :
  (min p σ) = 1 ↔ p.M.support ≤ σ.ker
```

**theorem toReal\_min**

[\[source\]](#)

```
protected theorem toReal_min (p : MState d) (σ : HermitianMat d ℂ) :
  (min p σ).toReal = -Real.log (p.exp_val {0 ≤p σ})
```

**instance RelEntropy min**

[\[source\]](#)

*Min-relative entropy is a valid entropy function, albeit trivial (and not faithful).*

```
instance : RelEntropy min where
```

**theorem not\_Nontrivial\_min**

[\[source\]](#)

```
theorem not_Nontrivial_min : ¬Nontrivial min
```

**theorem min\_le**

[\[source\]](#)

*The relative min-entropy is a lower bound on all relative entropies.*

```
theorem min_le (p σ : MState d) : min p σ ≤ f p σ
```

**def max**[\[source\]](#)*Quantum relative max-entropy.***def** max (p : MState d) (σ : HermitianMat d ℂ) : ENNReal**theorem max\_not\_top**[\[source\]](#)

@[aesop (rule\_sets := [finiteness]) simp]

**protected theorem** max\_not\_top (p : MState d) (σ : HermitianMat d ℂ) :  
(max p σ) ≠ ⊤ ↔ σ.ker ≤ p.M.ker**theorem toReal\_max**[\[source\]](#)**protected theorem** toReal\_max (p : MState d) (σ : HermitianMat d ℂ) :  
(max p σ).toReal = sInf { x : ℝ | p.M ≤ Real.exp x • σ }**theorem le\_max**[\[source\]](#)*The relative max-entropy is a lower bound on all relative entropies.***theorem** le\_max (p σ : MState d) : f p σ ≤ max p σ**class Entropy**[\[source\]](#)**class** Entropy (f : ∀ {d : Type u} [Fintype d] [DecidableEq d], MState d → ℝ≥0) **where**  
of\_const {d : Type u} [Fintype d] [DecidableEq d] (ψ : Ket d) : f (.pure ψ) = 0  
of\_kron {d₁ d₂ : Type u} [Fintype d₁] [Fintype d₂] [DecidableEq d₁] [DecidableEq d₂] :  
∀ (p : MState d₁) (σ : MState d₂), f (p ⊗ σ) = f p + f σ

## 2.26 QuantumInfo.Finite.AxiomatizedEntropy.Renyi

[QuantumInfo/Finite/AxiomatizedEntropy/Renyi.lean](#) on GitHub.**def qRelativeEnt**[\[source\]](#)*The quantum relative entropy  $S(\rho\|\sigma) = \text{Tr}[\rho (\log \rho - \log \sigma)]$ .*

@[irreducible]

**noncomputable def** qRelativeEnt (p : MState d) (σ : HermitianMat d ℂ) : ENNReal**instance RelEntropy qRelativeEnt**[\[source\]](#)**instance** : RelEntropy qRelativeEnt **where****instance RelEntropy.Nontrivial qRelativeEnt**[\[source\]](#)**instance** : RelEntropy.Nontrivial qRelativeEnt **where****theorem qRelativeEnt\_ker**[\[source\]](#)*Quantum relative entropy as  $\text{Tr}[\rho (\log \rho - \log \sigma)]$  when supports are correct.*

```
theorem qRelativeEnt_ker {p σ : MState d} (h : σ.M.ker ≤ p.M.ker) :
  (D(p||σ) : EReal) = p.M.inner (HermitianMat.log p - HermitianMat.log σ)
```

## 2.27 QuantumInfo.Finite.Braket

[QuantumInfo/Finite/Braket.lean](#) on GitHub.

### structure Ket

[\[source\]](#)

*A ket as a vector of unit norm. We follow the convention in **Matrix** of vectors as simple functions from a **Fintype**. Kets are distinctly not a vector space in our notion, as they represent only normalized states and so cannot (in general) be added or scaled.*

```
structure Ket where
  vec : d → ℂ
  normalized' : ∑ x, ||vec x|| ^ 2 = 1
```

### structure Bra

[\[source\]](#)

*A bra is identical in definition to a **Ket**, but are separate to avoid complex conjugation confusion. They can be interconverted with the adjoint: **Ket.to\_bra** and **Bra.to\_ket***

```
structure Bra where
  vec : d → ℂ
  normalized' : ∑ x, ||vec x|| ^ 2 = 1
```

### instance instFunLikeKet

[\[source\]](#)

```
instance instFunLikeKet : FunLike (Ket d) d ℂ where
```

### instance instFunLikeBra

[\[source\]](#)

```
instance instFunLikeBra : FunLike (Bra d) d ℂ where
```

### def dot

[\[source\]](#)

```
def dot (ξ : Bra d) (ψ : Ket d) : ℂ
```

### theorem dot\_eq\_dotProduct

[\[source\]](#)

```
theorem dot_eq_dotProduct (ψ : Bra d) (φ : Ket d) : ⟨ψ||φ⟩ = dotProduct (m := d) ψ φ
```

### theorem Ket.apply

[\[source\]](#)

```
theorem Ket.apply (ψ : Ket d) (i : d) : ψ i = ψ.vec i
```

### theorem Bra.apply

[\[source\]](#)

```
theorem Bra.apply (ψ : Bra d) (i : d) : ψ i = ψ.vec i
```

**theorem Ket.ext**[\[source\]](#)

@[ext]

**theorem** Ket.ext { $\xi \ \psi : \text{Ket } d$ } ( $h : \forall x, \xi x = \psi x$ ) :  $\xi = \psi$ **theorem Bra.ext**[\[source\]](#)

@[ext]

**theorem** Bra.ext { $\xi \ \psi : \text{Bra } d$ } ( $h : \forall x, \xi x = \psi x$ ) :  $\xi = \psi$ **theorem Ket.normalized**[\[source\]](#)**theorem** Ket.normalized ( $\psi : \text{Ket } d$ ) :  $\sum x, \text{Complex.normSq } (\psi x) = 1$ **theorem Bra.normalized**[\[source\]](#)**theorem** Bra.normalized ( $\psi : \text{Bra } d$ ) :  $\sum x, \text{Complex.normSq } (\psi x) = 1$ **def to\_bra**[\[source\]](#)*Any Bra can be turned into a Ket by conjugating the elements.*

@[coe]

**def** Ket.to\_bra ( $\psi : \text{Ket } d$ ) :  $\text{Bra } d$ **def to\_ket**[\[source\]](#)*Any Ket can be turned into a Bra by conjugating the elements.*

@[coe]

**def** Bra.to\_ket ( $\psi : \text{Bra } d$ ) :  $\text{Ket } d$ **instance instBraOfKet**[\[source\]](#)**instance** instBraOfKet :  $\text{Coe } (\text{Ket } d) (\text{Bra } d)$ **instance instKetOfBra**[\[source\]](#)**instance** instKetOfBra :  $\text{Coe } (\text{Bra } d) (\text{Ket } d)$ **theorem eq\_conj**[\[source\]](#)

@[simp]

**theorem** Bra.eq\_conj ( $\psi : \text{Ket } d$ ) ( $x : d$ ) :  $\langle \psi | x = \text{conj } (| \psi \rangle x)$ **theorem apply'**[\[source\]](#)**theorem** Bra.apply' ( $\psi : \text{Ket } d$ ) ( $i : d$ ) :  $\langle \psi | i = \text{conj } (\psi.\text{vec } i)$

**theorem Ket.exists\_ne\_zero**[\[source\]](#)

```
theorem Ket.exists_ne_zero (ψ : Ket d) : ∃ x, ψ x ≠ 0
```

**theorem Bra.exists\_ne\_zero**[\[source\]](#)

```
theorem Bra.exists_ne_zero (ψ : Bra d) : ∃ x, ψ x ≠ 0
```

**def Ket.normalize**[\[source\]](#)

*Create a ket out of a vector given it has a nonzero component*

```
def Ket.normalize (v : d → ℂ) (h : ∃ x, v x ≠ 0) : Ket d
```

**theorem Ket.normalize\_ket\_eq\_self**[\[source\]](#)

*A ket is already normalized*

```
theorem Ket.normalize_ket_eq_self (ψ : Ket d) : Ket.normalize (ψ.vec) (Ket.exists_ne_zero ψ) = ψ
```

**def Bra.normalize**[\[source\]](#)

*Create a bra out of a vector given it has a nonzero component*

```
def Bra.normalize (v : d → ℂ) (h : ∃ x, v x ≠ 0) : Bra d
```

**def Bra.normalize\_ket\_eq\_self**[\[source\]](#)

*A bra is already normalized*

```
def Bra.normalize_ket_eq_self (ψ : Bra d) : Bra.normalize (ψ.vec) (Bra.exists_ne_zero ψ) = ψ
```

**def uniform\_superposition**[\[source\]](#)

*Ket form by the superposition of all elements in d. Commonly denoted by  $|+\rangle$ , especially for qubits*

```
def uniform_superposition [hdne : Nonempty d] : Ket d
```

**instance instInhabited**[\[source\]](#)

*There exists a ket for every nonempty d. Here, we use the uniform superposition*

```
instance instInhabited [Nonempty d] : Inhabited (Ket d) where
```

**def Ket.basis**[\[source\]](#)

*Construct the Ket corresponding to a basis vector, with a +1 phase.*

```
def Ket.basis (i : d) : Ket d
```

**def Bra.basis**[\[source\]](#)

*Construct the Bra corresponding to a basis vector, with a +1 phase.*

```
def Bra.basis (i : d) : Bra d
```

**instance instFunLikeBraket**[\[source\]](#)

*A Bra can be viewed as a function from Ket's to  $\mathbb{C}$ .*

```
instance instFunLikeBraket : FunLike (Bra d) (Ket d)  $\mathbb{C}$  where
```

**theorem dot\_self\_eq\_one**[\[source\]](#)

*The inner product of any state with itself is 1.*

```
@[simp]
```

```
theorem Braket.dot_self_eq_one (ψ : Ket d) :  $\langle \psi | \psi \rangle = 1$ 
```

**def prod**[\[source\]](#)

*The outer product of two kets, creating an unentangled state.*

```
def Ket.prod (ψ1 : Ket d1) (ψ2 : Ket d2) : Ket (d1 × d2) where
```

**def IsProd**[\[source\]](#)

*A Ket is a product if it's Ket.prod of two kets.*

```
def Ket.IsProd (ψ : Ket (d1 × d2)) : Prop
```

**def IsEntangled**[\[source\]](#)

*A Ket is entangled if it's not Ket.prod of two kets.*

```
def Ket.IsEntangled (ψ : Ket (d1 × d2)) : Prop
```

**theorem IsProd\_prod**[\[source\]](#)

*Ket.prod states are product states.*

```
@[simp]
```

```
theorem Ket.IsProd_prod (ψ1 : Ket d1) (ψ2 : Ket d2) : (ψ1.prod ψ2).IsProd
```

**theorem not\_IsEntangled\_prod**[\[source\]](#)

*Ket.prod states are not entangled states.*

```
@[simp]
```

```
theorem Ket.not_IsEntangled_prod (ψ1 : Ket d1) (ψ2 : Ket d2) :  $\neg(\psi_1.prod \psi_2).IsEntangled$ 
```

**theorem IsProd\_iff\_mul\_eq\_mul**[\[source\]](#)

*A ket is a product state iff its components are cross-multiplicative.*

```
theorem Ket.IsProd_iff_mul_eq_mul (ψ : Ket (d1 × d2)) : ψ.IsProd ↔
  ∀ i1 i2 j1 j2, ψ (i1, j1) * ψ (i2, j2) = ψ (i1, j2) * ψ (i2, j1)
```

**def MES**[\[source\]](#)

*The Maximally Entangled State, or MES, on a  $d \times d$  system. In principle there are many, this is specifically the MES with an all-positive phase. For instance on  $d := \text{Fin } 2$ , this is the Bell state.*

```
def Ket.MES (d) [Fintype d] [Nonempty d] : Ket (d × d) where
```

**theorem MES\_isEntangled**[\[source\]](#)

*On any space of dimension at least two, the maximally entangled state MES is entangled.*

```
theorem Ket.MES_isEntangled [Nontrivial d] : (Ket.MES d).IsEntangled
```

**theorem transposeTrick**[\[source\]](#)

*The transpose trick*

```
theorem transposeTrick {d} [Fintype d] [Nonempty d] [DecidableEq d] {M : Matrix d d ℂ} :
  (M ⊗k 1) *v (Ket.MES d).vec = (1 ⊗k M.transpose) *v (Ket.MES d).vec
```

**def PhaseEquiv**[\[source\]](#)

*The equivalence relation on Ket where two kets are equivalent if they are equal up to a global phase, i.e.  $\exists z, \|z\| = 1 \wedge a.\text{vec} = z \bullet b.\text{vec}$*

```
def Ket.PhaseEquiv : Setoid (Ket d) where
```

**def KetUpToPhase**[\[source\]](#)

*The type of Kets up to a global phase equivalence, as given by Ket.PhaseEquiv. In particular, MStates really only care about a KetUpToPhase, and not Kets themselves.*

```
def KetUpToPhase
```

**def mk**[\[source\]](#)

*Construct a KetUpToPhase from a Ket.*

```
def KetUpToPhase.mk (ψ : Ket d) : KetUpToPhase d
```

**def lift**[\[source\]](#)

*Lift a function on Ket d to KetUpToPhase d, given that it respects phase equivalence.*

```
def KetUpToPhase.lift {α : Sort*} (f : Ket d → α)
  (hf : ∀ ψ φ, Ket.PhaseEquiv.r ψ φ → f ψ = f φ) : KetUpToPhase d → α
```

**theorem lift\_mk**[\[source\]](#)

```
@[simp]
theorem KetUpToPhase.lift_mk {α : Sort*} (f : Ket d → α)
  (hf : ∀ ψ φ, Ket.PhaseEquiv.r ψ φ → f ψ = f φ) (ψ : Ket d) :
  KetUpToPhase.lift f hf (KetUpToPhase.mk ψ) = f ψ
```

**theorem ind**[\[source\]](#)

```
theorem KetUpToPhase.ind {p : KetUpToPhase d → Prop}
  (h : ∀ ψ : Ket d, p (KetUpToPhase.mk ψ)) : ∀ q, p q
```

**theorem surjective\_mk**[\[source\]](#)

```
theorem KetUpToPhase.surjective_mk : Function.Surjective (KetUpToPhase.mk (d := d))
```

**2.28 QuantumInfo.Finite.Capacity**

[QuantumInfo/Finite/Capacity.lean](#) on GitHub.

**def Emulates**[\[source\]](#)

*A channel  $\Lambda_1$  Emulates another channel  $\Lambda_2$  if there are  $D$  and  $E$  such that  $D \circ \Lambda_1 \circ E = \Lambda_2$ .*

```
def Emulates (Λ₁ : CPTPMap d₁ d₂) (Λ₂ : CPTPMap d₃ d₄) : Prop
```

**def εApproximates**[\[source\]](#)

*A channel  $A$  εApproximates channel  $B$  of the same dimensions if the for every state  $\rho$ , the fidelity  $F(A(\rho), B(\rho))$  is at least  $1 - \varepsilon$ .*

```
def εApproximates (A B : CPTPMap d₁ d₂) (ε : ℝ) : Prop
```

**def AchievesRate**[\[source\]](#)

*A channel  $A$  AchievesRate  $R : \mathbb{R}$  if for every  $\varepsilon > 0$ , some  $n$  copies of  $A$  emulates a channel  $B$  such that  $\log_2(\text{dimout}(B))/n \geq R$ , and that  $B$  εApproximates the identity channel.*

```
def AchievesRate (A : CPTPMap d₁ d₂) (R : ℝ) : Prop
```

**def quantumCapacity**[\[source\]](#)

```
noncomputable def quantumCapacity (A : CPTPMap d₁ d₂) : ℝ
```

**theorem emulates\_self**[\[source\]](#)

*Every quantum channel emulates itself.*

```
@[refl]
theorem emulates_self (Λ : CPTPMap d₁ d₂) : Λ.Emulates Λ
```



**theorem emulates\_trans**[\[source\]](#)

*If a quantum channel  $A$  emulates  $B$ , and  $B$  emulates  $C$ , then  $A$  emulates  $C$ .*

```
@[trans]
```

```
theorem emulates_trans ( $\Lambda_1$  : CPTPMap  $d_1$   $d_2$ ) ( $\Lambda_2$  : CPTPMap  $d_3$   $d_4$ ) ( $\Lambda_3$  : CPTPMap  $d_5$   $d_6$ )  
  ( $h_{12}$  :  $\Lambda_1$ .Emulates  $\Lambda_2$ ) ( $h_{23}$  :  $\Lambda_2$ .Emulates  $\Lambda_3$ ) :  $\Lambda_1$ .Emulates  $\Lambda_3$ 
```

**theorem  $\epsilon$ Approximates\_self**[\[source\]](#)

*Every quantum channel perfectly approximates itself, that is,  $\epsilon$ Approximates with  $\epsilon = 0$ .*

```
theorem  $\epsilon$ Approximates_self ( $\Lambda$  : CPTPMap  $d_1$   $d_2$ ) :  $\Lambda$ . $\epsilon$ Approximates  $\Lambda$  0
```

**theorem  $\epsilon$ Approximates\_monotone**[\[source\]](#)

*If a quantum channel  $A$  approximates  $B$  with  $\epsilon_0$ , it also approximates  $B$  with all larger  $\epsilon_1$ .*

```
theorem  $\epsilon$ Approximates_monotone { $A$   $B$  : CPTPMap  $d_1$   $d_2$ } { $\epsilon_0$  :  $\mathbb{R}$ } ( $h$  :  $A$ . $\epsilon$ Approximates  $B$   $\epsilon_0$ )  
  { $\epsilon_1$  :  $\mathbb{R}$ } ( $h_2$  :  $\epsilon_0 \leq \epsilon_1$ ) :  $A$ . $\epsilon$ Approximates  $B$   $\epsilon_1$ 
```

**theorem achievesRate\_0**[\[source\]](#)

*Every quantum channel achieves a rate of zero.*

```
theorem achievesRate_0 ( $\Lambda$  : CPTPMap  $d_1$   $d_2$ ) :  $\Lambda$ .AchievesRate 0
```

**theorem id\_achievesRate\_log\_dim**[\[source\]](#)

*The identity channel on  $D$  dimensional space achieves a rate of  $\log_2(D)$ .*

```
theorem id_achievesRate_log_dim : (id ( $d_{In}$  :=  $d_1$ )).AchievesRate (Real.logb 2 (Fintype.card  $d_1$ ))
```

**theorem not\_achievesRate\_gt\_log\_dim\_in**[\[source\]](#)

*A channel cannot achieve a rate greater than  $\log_2(D)$ , where  $D$  is the input dimension.*

```
theorem not_achievesRate_gt_log_dim_in ( $\Lambda$  : CPTPMap  $d_1$   $d_2$ ) { $R$  :  $\mathbb{R}$ } ( $hR$  : Real.logb 2 (Fintype.  
  card  $d_1$ ) <  $R$ ) :  
   $\neg \Lambda$ .AchievesRate  $R$ 
```

**theorem not\_achievesRate\_gt\_log\_dim\_out**[\[source\]](#)

*A channel cannot achieve a rate greater than  $\log_2(D)$ , where  $D$  is the output dimension.*

```
theorem not_achievesRate_gt_log_dim_out ( $\Lambda$  : CPTPMap  $d_1$   $d_2$ ) { $R$  :  $\mathbb{R}$ } ( $hR$  : Real.logb 2 (Fintype.  
  card  $d_2$ ) <  $R$ ) :  $\neg \Lambda$ .AchievesRate  $R$ 
```

**theorem bddAbove\_achievesRate**[\[source\]](#)

*The achievable rates are a bounded set.*

```
theorem bddAbove_achievesRate (Λ : CPTPMap d1 d2) : BddAbove {R | Λ.AchievesRate R}
```

### **theorem zero\_le\_quantumCapacity**

[\[source\]](#)

*Quantum channel capacity is nonnegative.*

```
theorem zero_le_quantumCapacity (Λ : CPTPMap d1 d2) : 0 ≤ Λ.quantumCapacity
```

### **theorem quantumCapacity\_ge\_log\_dim\_in**

[\[source\]](#)

*Quantum channel capacity is at most  $\log_2(D)$ , where  $D$  is the input dimension.*

```
theorem quantumCapacity_ge_log_dim_in (Λ : CPTPMap d1 d2) : Λ.quantumCapacity ≤ Real.logb 2 (
  Fintype.card d1)
```

### **theorem coherentInfo\_le\_quantumCapacity**

[\[source\]](#)

*The LSD (Lloyd-Shor-Devetak) theorem: the quantum capacity is at least as large the single-copy coherent information. The "coherent information" is used in literature to refer to both a function of state and a channel (coherentInfo), or a function of just a channel. In the latter case, the state is implicitly maximized over. Here we use the former definition and state that the lower bound is true for all states.*

```
theorem coherentInfo_le_quantumCapacity (Λ : CPTPMap d1 d2) (ρ : MState d1) : coherentInfo ρ Λ
  ≤ Λ.quantumCapacity
```

### **theorem quantumCapacity\_eq\_piProd\_coherentInfo**

[\[source\]](#)

*The quantum capacity is the limit of the coherent information of  $n$ -copy uses of the channel.*

```
theorem quantumCapacity_eq_piProd_coherentInfo (Λ : CPTPMap d1 d2) : Λ.quantumCapacity =
  sSup { r : ℝ | ∃ n ρ, r = coherentInfo ρ (CPTPMap.piProd (fun (_ : Fin n) ↦ Λ)) }
```

## **2.29 QuantumInfo.Finite.Channel.DegradableOrder**

[QuantumInfo/Finite/Channel/DegradableOrder.lean](#) on GitHub.

### **def DegradablePreorder**

[\[source\]](#)

```
def DegradablePreorder (dIn : Type*) [Fintype dIn] [DecidableEq dIn] : Preorder
  (Σ dOut : (Σ t, Fintype t × DecidableEq t), let _ := dOut.snd.1; CPTPMap dIn dOut.fst)
  where
```

## **2.30 QuantumInfo.Finite.CPTPMap.Bundled**

[QuantumInfo/Finite/CPTPMap/Bundled.lean](#) on GitHub.

### **structure HPMMap**

[\[source\]](#)

*Hermitian-preserving linear maps.*

```
structure HPMMap extends MatrixMap dIn dOut  $\mathbb{k}$  where
  HP : MatrixMap.IsHermitianPreserving toLinearMap
```

### **structure UnitalMap**

[\[source\]](#)

*Unital linear maps.*

```
structure UnitalMap [DecidableEq dIn] [DecidableEq dOut]
  extends MatrixMap dIn dOut R where
  unital : MatrixMap.Unital toLinearMap
```

### **structure TMap**

[\[source\]](#)

*Trace-preserving linear maps.*

```
structure TMap [Fintype dIn] [Fintype dOut] extends MatrixMap dIn dOut R where
  TP : MatrixMap.IsTracePreserving toLinearMap
```

### **structure PMap**

[\[source\]](#)

*Positive linear maps.*

```
structure PMap [Fintype dIn] [Fintype dOut]
  extends HPMMap dIn dOut  $\mathbb{k}$  where
  pos : MatrixMap.IsPositive toLinearMap
  HP := pos.IsHermitianPreserving
```

### **structure CMap**

[\[source\]](#)

*Completely positive linear maps.*

```
structure CMap [Fintype dIn] [Fintype dOut] [DecidableEq dIn]
  extends PMap dIn dOut  $\mathbb{k}$  where
  cp : MatrixMap.IsCompletelyPositive toLinearMap
  pos := cp.IsPositive
```

### **structure PTPMap**

[\[source\]](#)

*Positive trace-preserving linear maps. These includes all channels, but aren't necessarily \*completely\* positive, see CPTPMap.*

```
structure PTPMap [Fintype dIn] [Fintype dOut]
  extends PMap dIn dOut  $\mathbb{k}$ , TMap dIn dOut  $\mathbb{k}$ 
```

### **structure PUMap**

[\[source\]](#)

*Positive unital maps. These are important because they are the dual to PTPMap: they are the most general way to map \*observables\*.*

```
structure PUMap [Fintype dIn] [Fintype dOut] [DecidableEq dIn] [DecidableEq dOut]
  extends PMap dIn dOut  $\mathbb{k}$ , UnitalMap dIn dOut  $\mathbb{k}$ 
```

**structure CPTPMap**[\[source\]](#)

*Completely positive trace-preserving linear maps. This is the most common meaning of "channel", often described as "the most general physically realizable quantum operation".*

```
structure CPTPMap [Fintype dIn] [Fintype dOut] [DecidableEq dIn]
  extends PTPMap dIn dOut ( $\mathbb{k} := \mathbb{k}$ ), CPMMap dIn dOut  $\mathbb{k}$  where
```

**structure CPUMap**[\[source\]](#)

*Completely positive unital maps. These are important because they are the dual to CPTPMap: they are the physically realizable ways to map \*observables\*.*

```
structure CPUMap [Fintype dIn] [Fintype dOut] [DecidableEq dIn] [DecidableEq dOut]
  extends CPMMap dIn dOut  $\mathbb{k}$ , PUMMap dIn dOut  $\mathbb{k}$ 
```

**abbrev map**[\[source\]](#)

```
abbrev map (M : HPMMap dIn dOut  $\mathbb{k}$ ) : MatrixMap dIn dOut  $\mathbb{k}$ 
```

**theorem HPMMap.ext**[\[source\]](#)

```
@[ext]
theorem ext (h :  $\Lambda_1$ .map =  $\Lambda_2$ .map) :  $\Lambda_1 = \Lambda_2$ 
```

**theorem funext\_hermitian**[\[source\]](#)

*Two maps are equal if they agree on all Hermitian inputs.*

```
theorem funext_hermitian (h :  $\forall M : \text{HermitianMat } dIn \mathbb{C}, C\Lambda_1.map M = C\Lambda_2.map M$ ) :
   $C\Lambda_1 = C\Lambda_2$ 
```

**theorem funext\_pos**[\[source\]](#)

*Two maps are equal if they agree on all positive inputs.*

```
theorem funext_pos [Fintype dIn] (h :  $\forall M : \text{HermitianMat } dIn \mathbb{C}, 0 \leq M \rightarrow C\Lambda_1.map M = C\Lambda_2.map M$ ) :
  :
   $C\Lambda_1 = C\Lambda_2$ 
```

**theorem funext\_pos\_trace**[\[source\]](#)

*Two maps are equal if they agree on all positive inputs with trace one*

```
theorem funext_pos_trace [Fintype dIn]
  (h :  $\forall M : \text{HermitianMat } dIn \mathbb{C}, 0 \leq M \rightarrow M.trace = 1 \rightarrow C\Lambda_1.map M = C\Lambda_2.map M$ ) :
   $C\Lambda_1 = C\Lambda_2$ 
```

**theorem funext\_mstate**[\[source\]](#)

*Two maps are equal if they agree on all MStates.*

```
theorem funext_mstate [Fintype dIn] [DecidableEq dIn] {Λ1 Λ2 : HPMat dIn dOut ℂ}
  (h : ∀ ρ : MState dIn, Λ1.map ρ.m = Λ2.map ρ.m) :
    Λ1 = Λ2
```

**instance HPMat.instFunLike**

[\[source\]](#)

*Hermitian-preserving maps are functions from HermitianMats to HermitianMats.*

```
instance instFunLike : FunLike (HPMap dIn dOut ℂ) (HermitianMat dIn ℂ) (HermitianMat dOut ℂ)
  where
```

**instance ContinuousLinearMapClass (HPMap dIn dOut ℂ) ℝ (HermitianMat dIn ℂ) (Her...** [\[source\]](#)

```
instance [Fintype dIn] : ContinuousLinearMapClass
  (HPMap dIn dOut ℂ) ℝ (HermitianMat dIn ℂ) (HermitianMat dOut ℂ) where
```

**theorem PMap.ext**

[\[source\]](#)

```
@[ext]
theorem ext {Λ1 Λ2 : PMap dIn dOut ℤ} (h : Λ1.map = Λ2.map) : Λ1 = Λ2
```

**theorem injective\_toHPMap**

[\[source\]](#)

```
theorem injective_toHPMap : (PMap.toHPMap (dIn := dIn) (dOut := dOut) (ℤ := ℤ)).Injective
```

**instance PMap.instFunLike**

[\[source\]](#)

*Positive maps are functions from HermitianMats to HermitianMats.*

```
instance instFunLike : FunLike (PMap dIn dOut ℂ) (HermitianMat dIn ℂ) (HermitianMat dOut ℂ)
  where
```

**instance PMap.instLinearMapClass**

[\[source\]](#)

```
instance instLinearMapClass : LinearMapClass (PMap dIn dOut ℂ) ℝ (HermitianMat dIn ℂ) (
  HermitianMat dOut ℂ) where
```

**instance instContinuousOrderHomClass**

[\[source\]](#)

```
instance instContinuousOrderHomClass : ContinuousOrderHomClass (PMap dIn dOut ℂ)
  (HermitianMat dIn ℂ) (HermitianMat dOut ℂ) where
```

**theorem PMap.pos\_Hermitian**

[\[source\]](#)

*Positive-preserving maps also preserve positivity on, specifically, Hermitian matrices.*

```
@[simp]
theorem pos_Hermitian (M : PMap dIn dOut ℂ) {x : HermitianMat dIn ℂ} (h : 0 ≤ x) : 0 ≤ M x
```

**def of\_kraus\_CMap**[\[source\]](#)

```
def of_kraus_CMap {κ : Type*} [Fintype κ] [DecidableEq dIn] (M : κ → Matrix dOut dIn ℚ) :
  CMap dIn dOut ℚ where
```

**theorem PTPMap.ext**[\[source\]](#)

```
@[ext]
theorem ext {Λ₁ Λ₂ : PTPMap dIn dOut ℚ} (h : Λ₁.map = Λ₂.map) : Λ₁ = Λ₂
```

**theorem PTPMap.injective\_toPMap**[\[source\]](#)

```
theorem injective_toPMap : (PTPMap.toPMap (dIn := dIn) (dOut := dOut) (ℚ := ℚ)).Injective
```

**instance PTPMap.instFunLike**[\[source\]](#)

*Positive trace-preserving maps are functions from HermitianMats to HermitianMats.*

```
instance instFunLike : FunLike (PTPMap dIn dOut ℂ) (HermitianMat dIn ℂ) (HermitianMat dOut ℂ)
  where
```

**instance PTPMap.instLinearMapClass**[\[source\]](#)

```
instance instLinearMapClass : LinearMapClass (PTPMap dIn dOut ℂ) ℝ (HermitianMat dIn ℂ) (
  HermitianMat dOut ℂ) where
```

**instance PTPMap.instHContinuousOrderHomClass**[\[source\]](#)

```
instance instHContinuousOrderHomClass : ContinuousOrderHomClass (PTPMap dIn dOut ℂ)
  (HermitianMat dIn ℂ) (HermitianMat dOut ℂ) where
```

**theorem PTPMap.pos\_Hermitian**[\[source\]](#)

*PTP maps also preserve positivity on Hermitian matrices.*

```
@[simp]
theorem pos_Hermitian (M : PTPMap dIn dOut ℂ) {x : HermitianMat dIn ℂ} (h : 0 ≤ x) : 0 ≤ M x
```

**instance PTPMap.instMFunLike**[\[source\]](#)

*PTPMaps are functions from MStates to MStates.*

```
instance instMFunLike [DecidableEq dIn] [DecidableEq dOut] :
  FunLike (PTPMap dIn dOut) (MState dIn) (MState dOut) where
```

**instance instMContinuousMapClass**[\[source\]](#)

```
instance instMContinuousMapClass [DecidableEq dIn] [DecidableEq dOut] :
  ContinuousMapClass (PTPMap dIn dOut) (MState dIn) (MState dOut) where
```

**theorem val\_apply\_MState**[\[source\]](#)

```
theorem val_apply_MState [DecidableEq dIn] (M : PTPMap dIn dOut) (ρ : MState dIn) :
  (M ρ : HermitianMat dOut ℂ) = (instFunLike.coe M) ρ
```

**theorem nonemptyOut**[\[source\]](#)

```
theorem nonemptyOut (Λ : PTPMap dIn dOut) [hIn : Nonempty dIn] [DecidableEq dIn] : Nonempty
  dOut
```

**theorem CPTPMap.ext**[\[source\]](#)

*Two CPTPMaps are equal if their MatrixMaps are equal.*

```
@[ext]
```

```
theorem ext {Λ1 Λ2 : CPTPMap dIn dOut ℤ} (h : Λ1.map = Λ2.map) : Λ1 = Λ2
```

**theorem injective\_toPTPMap**[\[source\]](#)

```
theorem injective_toPTPMap : (CPTPMap.toPTPMap (dIn := dIn) (dOut := dOut) (ℤ := ℤ)).Injective
```

**instance CPTPMap.instMFunLike**[\[source\]](#)

*CPTPMaps are functions from MStates to MStates.*

```
instance instMFunLike [DecidableEq dOut] : FunLike (CPTPMap dIn dOut) (MState dIn) (MState dOut)
  ) where
```

**theorem IsTracePreserving**[\[source\]](#)

```
@[simp]
```

```
theorem IsTracePreserving (Λ : CPTPMap dIn dOut ℤ) : Λ.map.IsTracePreserving
```

**def of\_kraus\_CPTPMap**[\[source\]](#)

```
def of_kraus_CPTPMap {κ : Type*} [Fintype κ] [DecidableEq dIn]
  (M : κ → Matrix dOut dIn ℤ)
  (hTP : (∑ k, (M k).conjTranspose * (M k)) = 1) : CPTPMap dIn dOut ℤ where
```

**theorem PUMap.ext**[\[source\]](#)

```
@[ext]
```

```
theorem ext {Λ1 Λ2 : PUMap dIn dOut ℤ} (h : Λ1.map = Λ2.map) : Λ1 = Λ2
```

**theorem PUMap.injective\_toPMap**[\[source\]](#)

```
theorem injective_toPMap : (PUMap.toPMap (dIn := dIn) (dOut := dOut) (ℤ := ℤ)).Injective
```

**instance PUMap.instFunLike**[\[source\]](#)

*PUMaps are functions from HermitianMats to HermitianMats.*

```
instance instFunLike : FunLike (PUMap dIn dOut ℂ) (HermitianMat dIn ℂ) (HermitianMat dOut ℂ)
  where
```

**instance PUMap.instLinearMapClass**

[\[source\]](#)

```
instance instLinearMapClass : LinearMapClass (PUMap dIn dOut ℂ) ℝ (HermitianMat dIn ℂ) (
  HermitianMat dOut ℂ) where
```

**instance PUMap.instHContinuousOrderHomClass**

[\[source\]](#)

```
instance instHContinuousOrderHomClass : ContinuousOrderHomClass (PUMap dIn dOut ℂ)
  (HermitianMat dIn ℂ) (HermitianMat dOut ℂ) where
```

**instance PUMap.instOneHomClass**

[\[source\]](#)

```
instance instOneHomClass : OneHomClass (PUMap dIn dOut ℂ)
  (HermitianMat dIn ℂ) (HermitianMat dOut ℂ) where
```

**theorem PUMap.pos\_Hermitian**

[\[source\]](#)

*CPTP maps also preserve positivity on Hermitian matrices.*

```
@[simp]
theorem pos_Hermitian (M : PUMap dIn dOut ℂ) {x : HermitianMat dIn ℂ} (h : 0 ≤ x) : 0 ≤ M x
```

**theorem CPUMap.ext**

[\[source\]](#)

```
@[ext]
theorem ext {Λ₁ Λ₂ : CPUMap dIn dOut ℤ} (h : Λ₁.map = Λ₂.map) : Λ₁ = Λ₂
```

**theorem CPUMap.injective\_toPMap**

[\[source\]](#)

```
theorem injective_toPMap : (CPMap.toPMap ∘ CPUMap.toCPMap (dIn := dIn) (dOut := dOut) (ℤ := ℤ))
  .Injective
```

**instance CPUMap.instFunLike**

[\[source\]](#)

*CPUMaps are functions from HermitianMats to HermitianMats.*

```
instance instFunLike : FunLike (CPUMap dIn dOut ℂ) (HermitianMat dIn ℂ) (HermitianMat dOut ℂ)
  where
```

**instance CPUMap.instLinearMapClass**

[\[source\]](#)

```
instance instLinearMapClass : LinearMapClass (CPUMap dIn dOut ℂ) ℝ (HermitianMat dIn ℂ) (
  HermitianMat dOut ℂ) where
```

**instance CPUMap.instHContinuousOrderHomClass**

[\[source\]](#)

```
instance instHContinuousOrderHomClass : ContinuousOrderHomClass (CPUMap dIn dOut ℂ)
  (HermitianMat dIn ℂ) (HermitianMat dOut ℂ) where
```



**instance CPUMap.instOneHomClass**[\[source\]](#)

```
instance instOneHomClass : OneHomClass (CPUMap dIn dOut ℂ)
  (HermitianMat dIn ℂ) (HermitianMat dOut ℂ) where
```

**theorem CPUMap.pos\_Hermitian**[\[source\]](#)

*CPTP maps also preserve positivity on Hermitian matrices.*

```
@[simp]
theorem pos_Hermitian (M : CPUMap dIn dOut ℂ) {x : HermitianMat dIn ℂ} (h : 0 ≤ x) : 0 ≤ M x
```

**2.31 QuantumInfo.Finite.CPTPMap.CPTP**

[QuantumInfo/Finite/CPTPMap/CPTP.lean](#) on GitHub.

**def choi**[\[source\]](#)

*The Choi matrix of a CPTPMap.*

```
@[reducible]
def choi
```

**theorem choi\_ext**[\[source\]](#)

*Two CPTPMaps are equal if their Choi matrices are equal.*

```
theorem choi_ext {Λ₁ Λ₂ : CPTPMap dIn dOut} (h : Λ₁.choi = Λ₂.choi) : Λ₁ = Λ₂
```

**theorem choi\_PSD\_of\_CPTP**[\[source\]](#)

*The Choi matrix of a channel is PSD.*

```
theorem choi_PSD_of_CPTP : Λ.map.choi_matrix.PosSemidef
```

**theorem Tr\_of\_choi\_of\_CPTP**[\[source\]](#)

*The trace of a Choi matrix of a CPTP map is the cardinality of the input space.*

```
@[simp]
theorem Tr_of_choi_of_CPTP : Λ.choi.trace =
  (Finset.univ (α := dIn)).card
```

**def CPTP\_of\_choi\_PSD\_Tr**[\[source\]](#)

*Construct a CPTP map from a PSD Choi matrix with correct partial trace.*

```
def CPTP_of_choi_PSD_Tr {M : Matrix (dOut × dIn) (dOut × dIn) ℂ} (h₁ : M.PosSemidef)
  (h₂ : M.traceLeft = 1) : CPTPMap dIn dOut where
```

**theorem choi\_of\_CPTP\_of\_choi**[\[source\]](#)

```
@[simp]
theorem choi_of_CPTP_of_choi (M : Matrix (dOut × dIn) (dOut × dIn) ℂ) {h₁} {h₂} :
  (CPTP_of_choi_PSD_Tr (M := M) h₁ h₂).choi = M
```

**theorem mat\_coe\_eq\_apply\_mat**[\[source\]](#)

```
theorem mat_coe_eq_apply_mat [DecidableEq dOut] (ρ : MState dIn) : (Λ ρ).m = Λ.map ρ.m
```

**theorem funext**[\[source\]](#)

```
@[ext]
```

```
theorem funext [DecidableEq dOut] {Λ1 Λ2 : CPTPMap dIn dOut} (h : ∀ ρ, Λ1 ρ = Λ2 ρ) : Λ1 = Λ2
```

**def compose**[\[source\]](#)

*The composition of CPTPMaps, as a CPTPMap.*

```
def compose (Λ2 : CPTPMap dM dOut) (Λ1 : CPTPMap dIn dM) : CPTPMap dIn dOut where
```

**theorem compose\_eq**[\[source\]](#)

*Composition of CPTPMaps by CPTPMap.compose is compatible with the instFunLike action.*

```
@[simp]
```

```
theorem compose_eq [DecidableEq dOut] {Λ1 : CPTPMap dIn dM} {Λ2 : CPTPMap dM dOut} : ∀ ρ, (Λ2 ◦m Λ1) ρ = Λ2 (Λ1 ρ)
```

**theorem compose\_assoc**[\[source\]](#)

*Composition of CPTPMaps is associative.*

```
theorem compose_assoc [DecidableEq dOut] (Λ3 : CPTPMap dM2 dOut) (Λ2 : CPTPMap dM dM2)  
(Λ1 : CPTPMap dIn dM) : (Λ3 ◦m Λ2) ◦m Λ1 = Λ3 ◦m (Λ2 ◦m Λ1)
```

**instance instMixable**[\[source\]](#)

*CPTPMaps have a convex structure from their Choi matrices.*

```
instance instMixable : Mixable (Matrix (dOut × dIn) (dOut × dIn) ℂ) (CPTPMap dIn dOut) where
```

**def id**[\[source\]](#)

*The identity channel, which leaves the input unchanged.*

```
def id : CPTPMap dIn dIn where
```

**theorem id\_map**[\[source\]](#)

*The map CPTPMap.id leaves any matrix unchanged.*

```
@[simp]
```

```
theorem id_map : (id (dIn := dIn)).map = LinearMap.id
```

**theorem id\_MState**[\[source\]](#)

*The map CPTPMap.id leaves the input state unchanged.*

```
@[simp]
theorem id_MState (ρ : MState dIn) : CPTPMap.id (dIn := dIn) ρ = ρ
```

**theorem id\_compose**[\[source\]](#)

*The map CPTPMap.id composed with any map is the same map.*

```
@[simp]
theorem id_compose [DecidableEq dOut] (Λ : CPTPMap dIn dOut) : id ∘m Λ = Λ
```

**theorem compose\_id**[\[source\]](#)

*Any map composed with CPTPMap.id is the same map.*

```
@[simp]
theorem compose_id (Λ : CPTPMap dIn dOut) : Λ ∘m id = Λ
```

**def ofEquiv**[\[source\]](#)

*Given a equivalence (a bijection) between the types  $d_1$  and  $d_2$ , that is, if they're the same dimension, then there's a CPTP channel for this. This is what we need for defining e.g. the SWAP channel, which is 'unitary' but takes heterogeneous input and outputs types  $(d_1 \times d_2)$  and  $(d_2 \times d_1)$ .*

```
def ofEquiv (σ : dIn ≈ dOut) : CPTPMap dIn dOut where
```

**theorem ofEquiv\_apply**[\[source\]](#)

```
@[simp]
theorem ofEquiv_apply (σ : dIn ≈ dOut) (ρ : MState dIn) :
  ofEquiv σ ρ = ρ.relabel σ.symm
```

**theorem equiv\_inverse**[\[source\]](#)

```
@[simp]
theorem equiv_inverse (σ : dIn ≈ dOut) : (ofEquiv σ) ∘ (ofEquiv σ.symm) = id (dIn := dOut)
```

**def SWAP**[\[source\]](#)

*The SWAP operation, as a channel.*

```
def SWAP : CPTPMap (d1 × d2) (d2 × d1)
```

**def assoc**[\[source\]](#)

*The associator, as a channel.*

```
def assoc : CPTPMap ((d1 × d2) × d3) (d1 × d2 × d3)
```

**def assoc'**[\[source\]](#)*The inverse associator, as a channel.***def** assoc' : CPTPMap ( $d_1 \times d_2 \times d_3$ ) ( $((d_1 \times d_2) \times d_3)$ )**theorem SWAP\_eq\_MState\_SWAP**[\[source\]](#)

@[simp]

**theorem** SWAP\_eq\_MState\_SWAP ( $\rho$  : MState ( $d_1 \times d_2$ )) : SWAP ( $d_1 := d_1$ ) ( $d_2 := d_2$ )  $\rho$  =  $\rho$ .SWAP**theorem assoc\_eq\_MState\_assoc**[\[source\]](#)

@[simp]

**theorem** assoc\_eq\_MState\_assoc ( $\rho$  : MState ( $((d_1 \times d_2) \times d_3)$ )) : assoc ( $d_1 := d_1$ ) ( $d_2 := d_2$ ) ( $d_3 := d_3$ )  $\rho$  =  $\rho$ .assoc**theorem assoc'\_eq\_MState\_assoc'**[\[source\]](#)

@[simp]

**theorem** assoc'\_eq\_MState\_assoc' ( $\rho$  : MState ( $d_1 \times d_2 \times d_3$ )) : assoc' ( $d_1 := d_1$ ) ( $d_2 := d_2$ ) ( $d_3 := d_3$ )  $\rho$  =  $\rho$ .assoc'**theorem assoc\_assoc'**[\[source\]](#)

@[simp]

**theorem** assoc\_assoc' : (assoc ( $d_1 := d_1$ ) ( $d_2 := d_2$ ) ( $d_3 := d_3$ ))  $\circ_m$  assoc' = id**def traceLeft**[\[source\]](#)*Partial tracing out the left, as a CPTP map.*

@[sims]

**def** traceLeft : CPTPMap ( $d_1 \times d_2$ )  $d_2$ **def traceRight**[\[source\]](#)*Partial tracing out the right, as a CPTP map.***def** traceRight : CPTPMap ( $d_1 \times d_2$ )  $d_1$ **theorem traceLeft\_eq\_MState\_traceLeft**[\[source\]](#)

@[simp]

**theorem** traceLeft\_eq\_MState\_traceLeft ( $\rho$  : MState ( $d_1 \times d_2$ )) : traceLeft ( $d_1 := d_1$ ) ( $d_2 := d_2$ )  $\rho$  =  $\rho$ .traceLeft**theorem traceRight\_eq\_MState\_traceRight**[\[source\]](#)

@[simp]

**theorem** traceRight\_eq\_MState\_traceRight ( $\rho$  : MState ( $d_1 \times d_2$ )) : traceRight ( $d_1 := d_1$ ) ( $d_2 := d_2$ )  $\rho$  =  $\rho$ .traceRight

**def replacement**[\[source\]](#)

*The replacement channel that maps all inputs to a given state.*

```
def replacement [Nonempty dIn] [DecidableEq dOut] (ρ : MState dOut) : CPTPMap dIn dOut
```

**theorem replacement\_apply**[\[source\]](#)

*The output of replacement  $\rho$  is always that  $\rho$ .*

```
@[simp]
theorem replacement_apply [Nonempty dIn] [DecidableEq dOut] (ρ : MState dOut) (ρ₀ : MState dIn)
  :
  replacement ρ ρ₀ = ρ
```

**instance Inhabited (CPTPMap dIn dOut)**[\[source\]](#)

```
instance [Nonempty dIn] [Nonempty dOut] [DecidableEq dOut] : Inhabited (CPTPMap dIn dOut)
```

**instance Nonempty (CPTPMap dIn dOut)**[\[source\]](#)

```
instance [Nonempty dIn] [Nonempty dOut] : Nonempty (CPTPMap dIn dOut)
```

**def destroy**[\[source\]](#)

*There is a CPTP map that takes a system of any (nonzero) dimension and outputs the trivial Hilbert space, 1-dimensional, indexed by any Unique type. We can think of this as "destroying" the whole system; tracing out everything.*

```
def destroy [Nonempty dIn] [Unique dOut] : CPTPMap dIn dOut
```

**theorem eq\_if\_output\_unique**[\[source\]](#)

*Two CPTP maps into the same one-dimensional output space must be equal*

```
theorem eq_if_output_unique [Unique dOut] (Λ₁ Λ₂ : CPTPMap dIn dOut) : Λ₁ = Λ₂
```

**instance instUnique**[\[source\]](#)

*There is exactly one CPTPMap to a 1-dimensional space.*

```
instance instUnique [Nonempty dIn] [Unique dOut] : Unique (CPTPMap dIn dOut) where
```

**theorem destroy\_comp**[\[source\]](#)

```
@[simp]
theorem destroy_comp {dOut₂ : Type*} [Unique dOut₂] [DecidableEq dOut] [Nonempty dIn] [Nonempty dOut]
  (Λ : CPTPMap dIn dOut) :
  destroy (dOut := dOut₂) ∘ₘ Λ = destroy
```

**def prod**[\[source\]](#)

*The tensor product of two CPTPMaps.*

```
def prod (Λ1 : CPTPMap dI1 dO1) (Λ2 : CPTPMap dI2 dO2) : CPTPMap (dI1 × dI2) (dO1 × dO2) where
```

**def piProd**[\[source\]](#)

*Finitely-indexed tensor products of CPTPMaps.*

```
def piProd (Λi : (i:ι) → CPTPMap (dI i) (dO i)) : CPTPMap ((i:ι) → dI i) ((i:ι) → dO i) where
```

**theorem fin\_1\_piProd**[\[source\]](#)

```
theorem fin_1_piProd
  {dI : Fin 1 → Type v} [Fintype (dI 0)] [DecidableEq (dI 0)]
  {dO : Fin 1 → Type w} [Fintype (dO 0)] [DecidableEq (dO 0)]
  (Λi : (i : Fin 1) → CPTPMap (dI 0) (dO 0)) :
  piProd Λi = sorry ∘m ((Λi 1) ∘m sorry)
```

**theorem piProd\_comp**[\[source\]](#)

*The tensor product of composed maps is the composition of the tensor products.*

```
theorem piProd_comp
  {d1 d2 d3 : ι → Type*}
  [∀ i, Fintype (d1 i)] [∀ i, DecidableEq (d1 i)]
  [∀ i, Fintype (d2 i)] [∀ i, DecidableEq (d2 i)]
  [∀ i, Fintype (d3 i)] [∀ i, DecidableEq (d3 i)]
  (Λ1 : ∀ i, CPTPMap (d1 i) (d2 i)) (Λ2 : ∀ i, CPTPMap (d2 i) (d3 i)) :
  piProd (fun i => (Λ2 i) ∘m (Λ1 i)) = (piProd Λ2) ∘m (piProd Λ1)
```

**def ofUnitary**[\[source\]](#)

*Conjugating density matrices by a unitary as a channel. This is standard unitary evolution.*

```
def ofUnitary (U : U[dIn]) : CPTPMap dIn dIn where
```

**theorem ofUnitary\_eq\_conj**[\[source\]](#)

*The unitary channel U conjugated by U.*

```
theorem ofUnitary_eq_conj (U : U[dIn]) (ρ : MState dIn) :
  (ofUnitary U) ρ = ρ.U_conj U
```

**def IsUnitary**[\[source\]](#)

*A channel is unitary iff it is ofUnitary U.*

```
def IsUnitary (Λ : CPTPMap dIn dIn) : Prop
```

**theorem IsUnitary\_iff\_U\_conj**[\[source\]](#)

*A channel is unitary iff it can be written as conjugation by a unitary.*

```
theorem IsUnitary_iff_U_conj (Λ : CPTMap dIn dIn) : IsUnitary Λ ↔ ∃ U, ∀ ρ, Λ ρ = ρ.U_conj U
```

**theorem IsUnitary\_equiv**[\[source\]](#)

```
theorem IsUnitary_equiv (σ : dIn ≈ dIn) : IsUnitary (ofEquiv σ)
```

**lemma kraus\_sum\_eq\_one\_of\_TP**[\[source\]](#)

```
private lemma kraus_sum_eq_one_of_TP
  {κ : Type*} [Fintype κ]
  {K₁ K₂ : κ → Matrix dOut dIn ℂ}
  (hTP : (MatrixMap.of_kraus K₁ K₂).IsTracePreserving) :
  ∑ k, (K₂ k).conjTranspose * (K₁ k) = 1
```

**lemma exists\_unitary\_extending\_isometry**[\[source\]](#)

```
private lemma exists_unitary_extending_isometry
  {m n : Type*} [Fintype m] [Fintype n] [DecidableEq m] [DecidableEq n]
  (V : Matrix m n ℂ) (hV : V.conjTranspose * V = 1)
  (emb : n ↪ m) :
  ∃ U : U[m], ∀ i j, U.val i (emb j) = V i j
```

**lemma purify\_isometry\_condition**[\[source\]](#)

*Given Kraus operators  $K$  indexed by  $(dOut \times dIn)$ , define the isometry matrix  $V$  : Matrix  $(dIn \times dOut \times dOut) dIn \mathbb{C}$  by  $V_{\{(a, b, d), a'\}} = (K(b, a))_{\{d, a'\}}$ . Then  $V^\dagger V = 1$ .*

```
private lemma purify_isometry_condition
  {K : (dOut × dIn) → Matrix dOut dIn ℂ}
  (hTP : ∑ k, (K k).conjTranspose * (K k) = 1) :
  let V : Matrix (dIn × dOut × dOut) dIn ℂ
```

**lemma purify\_MState\_pure\_basis\_default\_entry**[\[source\]](#)

```
private lemma purify_MState_pure_basis_default_entry (i j : dOut × dOut) :
  (MState.pure (Ket.basis (default : dOut × dOut))) .m i j =
  if i = default ∧ j = default then 1 else 0
```

**lemma purify\_replacement\_single\_eq**[\[source\]](#)

```
private lemma purify_replacement_single_eq (ρ₀ : MState (dOut × dOut)) (b₁ b₂ : dOut × dOut) :
  ((replacement ρ₀).map (Matrix.single () () 1)) b₁ b₂ = ρ₀.m b₁ b₂
```

**lemma purify\_prep\_append\_entry**[\[source\]](#)

```
private lemma purify_prep_append_entry (X : Matrix dIn dIn ℂ)
  (a₁ : dIn) (b₁c₁ : dOut × dOut) (a₂ : dIn) (b₂c₂ : dOut × dOut) :
  let ρ₀
```

**lemma purify\_conj\_entry**[\[source\]](#)

```
private lemma purify_conj_entry (X : Matrix dIn dIn ℂ) (U : U[dIn × dOut × dOut])
  (i j : dIn × dOut × dOut) :
  let ρ₀
```

**lemma purify\_rhs\_entry**[\[source\]](#)

```
private lemma purify_rhs_entry (X : Matrix dIn dIn ℂ) (d₁ d₂ : dOut)
  (U : U[dIn × dOut × dOut]) :
  let ρ₀
```

**lemma purify\_of\_kraus\_entry**[\[source\]](#)

```
private lemma purify_of_kraus_entry (K : (dOut × dIn) → Matrix dOut dIn ℂ) (X : Matrix dIn dIn
  ℂ) (d₁ d₂ : dOut) :
  (MatrixMap.of_kraus K K) X d₁ d₂ =
  ∑ k : dOut × dIn, ∑ α₁ : dIn, ∑ α₂ : dIn,
  (K k) d₁ α₁ * X α₁ α₂ * starRingEnd ℂ ((K k) d₂ α₂)
```

**theorem exists\_purify**[\[source\]](#)

```
theorem exists_purify (Λ : CPTPMap dIn dOut) :
  ∃ (Λ' : CPTPMap (dIn × dOut × dOut) (dIn × dOut × dOut)),
  Λ'.IsUnitary Λ
  Λ = (
  let zero_prep : CPTPMap Unit (dOut × dOut) := replacement (MState.pure (Ket.basis default
  ))
  let prep := (id ⊗CP zero_prep)
  let append : CPTPMap dIn (dIn × Unit) := CPTPMap.ofEquiv (Equiv.prodPUnit dIn).symm
  CPTPMap.traceLeft ◦m CPTPMap.traceLeft ◦m Λ' ◦m prep ◦m append
  )
```

**def purify**[\[source\]](#)

Every channel can be written as a unitary channel on a larger system. In general, if the original channel was  $A \rightarrow B$ , we may need to go as big as dilating the output system (the environment) by a factor of  $A^*B$ . One way of stating this would be that it forms an isometry from  $A$  to  $(B \times A \times B)$ . So that we can instead talk about the cleaner unitaries, we say that this is a unitary on  $(A \times B \times B)$ . The defining properties that this is a valid purification comes are `purify_IsUnitary` and `purify_trace`. This means the environment always has type  $dIn \times dOut$ .

Furthermore, since we need a canonical "0" state on  $B$  in order to add with the input, we require a typeclass instance `[Inhabited dOut]`.

```
def purify (Λ : CPTPMap dIn dOut) : CPTPMap (dIn × dOut × dOut) (dIn × dOut × dOut)
```

**theorem purify\_IsUnitary**[\[source\]](#)

```
theorem purify_IsUnitary (Λ : CPTPMap dIn dOut) : Λ.purify.IsUnitary
```



**theorem purify\_trace**[\[source\]](#)

*With a channel  $\Lambda : A \rightarrow B$ , a valid purification  $(A \times B \times B) \rightarrow (A \times B \times B)$  is such that:  
 \* Preparing the default  $|0\rangle$  state on two copies of  $B$  \* Appending these to the input \*  
 Applying the purified unitary channel \* Tracing out the two left parts of the output  
 is equivalent to the original channel. This theorem states that the channel output by  
 purify has this property.*

```
theorem purify_trace ( $\Lambda$  : CPTPMap dIn dOut) :  $\Lambda$  = (
  let zero_prep : CPTPMap Unit (dOut  $\times$  dOut) := replacement (MState.pure (Ket.basis default))
  let prep := (id  $\otimes^{cP}$  zero_prep)
  let append : CPTPMap dIn (dIn  $\times$  Unit) := CPTPMap.ofEquiv (Equiv.prodPUnit dIn).symm
  CPTPMap.traceLeft  $\circ_m$  CPTPMap.traceLeft  $\circ_m$   $\Lambda$ .purify  $\circ_m$  prep  $\circ_m$  append
)
```

**def complementary**[\[source\]](#)

*The complementary channel comes from tracing out the other half (the right half) of  
 the purified channel purify.*

```
def complementary ( $\Lambda$  : CPTPMap dIn dOut) : CPTPMap dIn (dIn  $\times$  dOut)
```

**def IsDegradableTo**[\[source\]](#)

*A channel is \*degradable to\* another, if the other can be written as a composition  
 of a \_\_degrading\_\_ channel  $D$  with the original channel.*

```
def IsDegradableTo ( $\Lambda$  : CPTPMap dIn dOut) ( $\Lambda_2$  : CPTPMap dIn dOut2) : Prop
```

**def IsAntidegradableTo**[\[source\]](#)

*A channel is \*antidegradable to\* another, if the other IsDegradableTo this one.*

```
@[reducible]
def IsAntidegradableTo ( $\Lambda$  : CPTPMap dIn dOut) ( $\Lambda_2$  : CPTPMap dIn dOut2) : Prop
```

**def IsDegradable**[\[source\]](#)

*A channel is \*degradable\* if it IsDegradableTo its complementary channel.*

```
def IsDegradable ( $\Lambda$  : CPTPMap dIn dOut) : Prop
```

**def IsAntidegradable**[\[source\]](#)

*A channel is \*antidegradable\* if it IsAntidegradableTo its complementary channel.*

```
@[reducible]
def IsAntidegradable ( $\Lambda$  : CPTPMap dIn dOut) : Prop
```

**instance instTop**[\[source\]](#)

*CPTPMaps inherit a topology from their choi matrices.*

```
instance instTop : TopologicalSpace (CPTPMap dIn dOut)
```

**theorem choi\_IsEmbedding**[\[source\]](#)

*The projection from CPTPMap to the Choi matrix is an embedding*

```
theorem choi_IsEmbedding : Topology.IsEmbedding (CPTPMap.choi (dIn := dIn) (dOut := dOut))
where
```

**instance instT3Space**[\[source\]](#)

```
instance instT3Space : T3Space (CPTPMap dIn dOut)
```

**2.32 QuantumInfo.Finite.CPTPMap.Dual**

[QuantumInfo/Finite/CPTPMap/Dual.lean](#) on GitHub.

**def MatrixMap.dual**[\[source\]](#)

*The dual of a map between matrices, defined by  $\text{Tr}[A M(B)] = \text{Tr}[(\text{dual } M)(A) B]$ .  
Sometimes called the adjoint of the map instead.*

```
@[irreducible]
def dual (M : MatrixMap dIn dOut R) : MatrixMap dOut dIn R
```

**theorem trace\_eq**[\[source\]](#)

*The defining property of a dual map: inner products are preserved on the opposite argument.*

```
theorem Dual.trace_eq (M : MatrixMap dIn dOut R) (A : Matrix dIn dIn R) (B : Matrix dOut dOut R) :
  (M A * B).trace = (A * M.dual B).trace
```

**theorem IsHermitianPreserving.dual**[\[source\]](#)

*The dual of a IsHermitianPreserving map also IsHermitianPreserving.*

```
theorem IsHermitianPreserving.dual (h : M.IsHermitianPreserving) : M.dual.IsHermitianPreserving
```

**theorem trace\_mul\_nonneg**[\[source\]](#)

```
theorem _root_.Matrix.PosSemidef.trace_mul_nonneg {n : Type*} [Fintype n] [DecidableEq n]
  {A B : Matrix n n ℚ} (hA : A.PosSemidef) (hB : B.PosSemidef) :
  0 ≤ (A * B).trace
```

**theorem IsPositive.dual**[\[source\]](#)

*The dual of a IsPositive map also IsPositive.*

```
theorem IsPositive.dual (h : M.IsPositive) : M.dual.IsPositive
```

**theorem dual\_Unital**[\[source\]](#)

*The dual of TracePreserving map is *\*not\** trace-preserving, it's *\*unital\**, that is,  
 $M^*(I) = I$ .*

```
theorem dual_Unital (h : M.IsTracePreserving) : M.dual.Unital
```

### lemma dual\_unique

[\[source\]](#)

*If two matrix maps satisfy the trace duality property, they are equal.*

```
lemma dual_unique
  (M : MatrixMap dIn dOut  $\mathbb{k}$ ) (M' : MatrixMap dOut dIn  $\mathbb{k}$ )
  (h :  $\forall A B, (M A * B).trace = (A * M' B).trace$ ) : M.dual = M'
```

### lemma dual\_choi\_matrix

[\[source\]](#)

*The Choi matrix of the dual map is the transpose of the reindexed Choi matrix of the original map.*

```
lemma dual_choi_matrix (M : MatrixMap dIn dOut  $\mathbb{k}$ ) :
  M.dual.choi_matrix = (M.choi_matrix.transpose).reindex (Equiv.prodComm dOut dIn) (Equiv.
  prodComm dOut dIn)
```

### lemma dual\_choi\_matrix\_posSemidef\_of\_posSemidef

[\[source\]](#)

*If the Choi matrix of a map is positive semidefinite, then the Choi matrix of its dual is also positive semidefinite.*

```
lemma dual_choi_matrix_posSemidef_of_posSemidef (M : MatrixMap dIn dOut  $\mathbb{k}$ ) (h : M.choi_matrix.
  PosSemidef) :
  M.dual.choi_matrix.PosSemidef
```

### lemma dual\_id

[\[source\]](#)

*The dual of the identity map is the identity map.*

```
lemma dual_id : (MatrixMap.id dIn  $\mathbb{k}$ ).dual = MatrixMap.id dIn  $\mathbb{k}$ 
```

### lemma dual\_kron

[\[source\]](#)

*The dual of a Kronecker product of maps is the Kronecker product of their duals.*

```
lemma dual_kron {A B C D : Type*} [Fintype A] [Fintype B] [Fintype C] [Fintype D]
  [DecidableEq A] [DecidableEq B] [DecidableEq C] [DecidableEq D]
  (M : MatrixMap A B  $\mathbb{k}$ ) (N : MatrixMap C D  $\mathbb{k}$ ) :
  (M  $\otimes_{\mathbb{k}}$  N).dual = M.dual  $\otimes_{\mathbb{k}}$  N.dual
```

### theorem IsCompletelyPositive.dual

[\[source\]](#)

```
theorem IsCompletelyPositive.dual (h : M.IsCompletelyPositive) : M.dual.IsCompletelyPositive
```

### lemma dualMap\_toDualEquiv\_symm\_comp\_toDualEquiv

[\[source\]](#)

*The composition of the dual of the inverse of the dual basis isomorphism with the dual basis isomorphism is the evaluation map.*

```

lemma Module.Basis.dualMap_toDualEquiv_symm_comp_toDualEquiv {ι R M : Type*} [Fintype ι] [
  DecidableEq ι] [CommRing R] [AddCommGroup M] [Module R M] [Module.IsReflexive R M] (b :
  Module.Basis ι R M) :
  b.toDualEquiv.symm.toLinearMap.dualMap ∘l b.toDualEquiv.toLinearMap = (Module.evalEquiv R M)
  .toLinearMap

```

**lemma toDualEquiv\_symm\_comp\_dualMap\_toDualEquiv**

[\[source\]](#)

*The composition of the inverse of the dual basis isomorphism with the dual of the dual basis isomorphism is the inverse of the evaluation map.*

```

lemma Module.Basis.toDualEquiv_symm_comp_dualMap_toDualEquiv {ι R M : Type*} [Fintype ι] [
  DecidableEq ι] [CommRing R] [AddCommGroup M] [Module R M] [Module.IsReflexive R M] (b :
  Module.Basis ι R M) :
  b.toDualEquiv.symm.toLinearMap ∘l b.toDualEquiv.toLinearMap.dualMap = (Module.evalEquiv R M)
  .symm.toLinearMap

```

**theorem dual\_dual**

[\[source\]](#)

@[simp]

**theorem** dual\_dual : M.dual.dual = M

**def CPTPMap.dual**

[\[source\]](#)

**def** dual (M : CPTPMap dIn dOut) : CPUMap dOut dIn **where**

**theorem dual\_pos**

[\[source\]](#)

**theorem** dual\_pos (M : CPTPMap dIn dOut) {T : HermitianMat dOut ℂ} (hT : 0 ≤ T) :  
0 ≤ M.dual T

**theorem dual.PTP\_POVM**

[\[source\]](#)

*The dual of a CPTP map preserves POVMs. Stated here just for two-element POVMs, that is, an operator T between 0 and 1.*

**theorem** dual.PTP\_POVM (M : CPTPMap dIn dOut) {T : HermitianMat dOut ℂ} (hT : 0 ≤ T ∧ T ≤ 1) :  
(0 ≤ M.dual T ∧ M.dual T ≤ 1)

**theorem exp\_val\_Dual**

[\[source\]](#)

*The defining property of a dual channel, as specialized to MState.exp\_val.*

**theorem** exp\_val\_Dual (E : CPTPMap dIn dOut) (ρ : MState dIn) (T : HermitianMat dOut ℂ) :  
(E ρ).exp\_val T = ρ.exp\_val (E.dual T)

**def ofHermitianMat**

[\[source\]](#)

**def** HPMat.ofHermitianMat {dOut : Type\*} (f : HermitianMat dIn ℂ →<sub>l</sub> [ℝ] HermitianMat dOut ℂ) :  
HPMap dIn dOut **where**

**theorem linearMap\_ofHermitianMat**[\[source\]](#)

@[simp]

**theorem** HPMat.linearMap\_ofHermitianMat (f : HermitianMat dIn  $\mathbb{C}$   $\rightarrow$   $\mathbb{R}$ ] HermitianMat dOut  $\mathbb{C}$ ) :  
 LinearMapClass.linearMap (HPMap.ofHermitianMat f) = f

**theorem ofHermitianMat\_linearMap**[\[source\]](#)

@[simp]

**theorem** HPMat.ofHermitianMat\_linearMap (f : HPMat dIn dOut  $\mathbb{C}$ ) :  
 ofHermitianMat (LinearMapClass.linearMap f) = f

**def HPMat.hermDual**[\[source\]](#)

**def** HPMat.hermDual : HPMat dOut dIn

**theorem hermDual\_hermDual**[\[source\]](#)

@[simp]

**theorem** HPMat.hermDual\_hermDual : f.hermDual.hermDual = f

**theorem inner\_hermDual**[\[source\]](#)

*The defining property of a dual map: inner products are preserved on the opposite argument.*

**theorem** HPMat.inner\_hermDual (B : HermitianMat dOut  $\mathbb{C}$ ) :  
 $\langle\langle f A, B \rangle\rangle = \langle\langle A, f.hermDual B \rangle\rangle$

**theorem inner\_hermDual'**[\[source\]](#)

*Version of HPMat.inner\_hermDual that uses HermitiaMat.inner directly. TODO cleanup*

**theorem** HPMat.inner\_hermDual' (B : HermitianMat dOut  $\mathbb{C}$ ) :  
 $\langle\langle f A, B \rangle\rangle = \langle\langle A, f.hermDual B \rangle\rangle$

**theorem IsPositive.hermDual**[\[source\]](#)

*The dual of a IsPositive map also IsPositive.*

**theorem** MatrixMap.IsPositive.hermDual (h : MatrixMap.IsPositive f.map) : f.hermDual.map.  
 IsPositive

**theorem hermDual\_Unital**[\[source\]](#)

*The dual of TracePreserving map is *\*not\** trace-preserving, it's *\*unital\**, that is,  
 $M^*(I) = I$ .*

**theorem** HPMat.hermDual\_Unital [DecidableEq dIn] [DecidableEq dOut] (h : MatrixMap.  
 IsTracePreserving f.map) :  
 f.hermDual.map.Unital

**def PTPMap.hermDual**[\[source\]](#)

```
def hermDual (M : PTPMap dIn dOut) : PUMap dOut dIn where
```

**theorem hermDual\_pos**[\[source\]](#)

```
theorem hermDual_pos (M : PTPMap dIn dOut) {T : HermitianMat dOut ℂ} (hT : 0 ≤ T) :
  0 ≤ M.hermDual T
```

**theorem hermDual.PTP\_POVM**[\[source\]](#)

*The dual of a PTP map preserves POVMs. Stated here just for two-element POVMs, that is, an operator  $T$  between 0 and 1.*

```
theorem hermDual.PTP_POVM (M : PTPMap dIn dOut) {T : HermitianMat dOut ℂ} (hT : 0 ≤ T ∧ T ≤ 1) :
  (0 ≤ M.hermDual T ∧ M.hermDual T ≤ 1)
```

**theorem exp\_val\_hermDual**[\[source\]](#)

*The defining property of a dual channel, as specialized to `MState.exp_val`.*

```
theorem exp_val_hermDual (E : PTPMap dIn dOut) (p : MState dIn) (T : HermitianMat dOut ℂ) :
  (E p).exp_val T = p.exp_val (E.hermDual T)
```

**2.33 QuantumInfo.Finite.CPTPMap.MatrixMap**

[QuantumInfo/Finite/CPTPMap/MatrixMap.lean](#) on GitHub.

**abbrev MatrixMap**[\[source\]](#)

*A MatrixMap is a linear map between squares matrices of size  $A$  to size  $B$ , over  $R$ .*

```
abbrev MatrixMap (A B R : Type*) [Semiring R]
```

**def id**[\[source\]](#)

*Alias of `LinearMap.id`, but specifically as a `MatrixMap`.*

```
@[reducible]
def id : MatrixMap A A R
```

**def choi\_matrix**[\[source\]](#)

*Choi matrix of a given linear matrix map. Note that this is defined even for things that aren't CPTP, it's just rarely talked about in those contexts. This is the inverse of `MatrixMap.of_choi_matrix`. Compare with `MatrixMap.toMatrix`, which gives the transfer matrix.*

```
def choi_matrix (M : MatrixMap A B R) : Matrix (B × A) (B × A) R
```

**def of\_choi\_matrix**[\[source\]](#)

*Given the Choi matrix, generate the corresponding  $R$ -linear map between matrices as a `MatrixMap`. This is the inverse of `MatrixMap.choi_matrix`.*

```
def of_choi_matrix (M : Matrix (B × A) (B × A) R) : MatrixMap A B R where
```

**theorem map\_choi\_inv**[\[source\]](#)

*Proves that `MatrixMap.of_choi_matrix` and `MatrixMap.choi_matrix` inverses.*

```
@[simp]
```

```
theorem map_choi_inv (M : Matrix (B × A) (B × A) R) : choi_matrix (of_choi_matrix M) = M
```

**theorem choi\_map\_inv**[\[source\]](#)

*Proves that `MatrixMap.choi_matrix` and `MatrixMap.of_choi_matrix` inverses.*

```
@[simp]
```

```
theorem choi_map_inv (M : MatrixMap A B R) : of_choi_matrix (choi_matrix M) = M
```

**theorem choi\_matrix\_inj**[\[source\]](#)

*The correspondence induced by `MatrixMap.of_choi_matrix` is injective.*

```
theorem choi_matrix_inj : Function.Injective (@choi_matrix A B R _)
```

**def choi\_equiv**[\[source\]](#)

*The linear equivalence between linear maps of matrices, and Choi matrices.*

```
@[sims]
```

```
def choi_equiv : MatrixMap A B R ≈l[R] Matrix (B × A) (B × A) R where
```

**def toMatrix**[\[source\]](#)

*The linear equivalence between `MatrixMap`'s and transfer matrices on a larger space. Compare with `MatrixMap.choi_matrix`, which gives the Choi matrix instead of the transfer matrix.*

```
noncomputable def toMatrix [Fintype B] : MatrixMap A B R ≈l[R] Matrix (B × B) (A × A) R
```

**theorem toMatrix\_comp**[\[source\]](#)

*Multiplication of transfer matrices, `MatrixMap.toMatrix`, is equivalent to composition of maps.*

```
theorem toMatrix_comp [Fintype B] [Fintype C] [DecidableEq B] (M1 : MatrixMap A B R) (M2 : MatrixMap B C R) : toMatrix (M2 ∘l M1) = (toMatrix M2) * (toMatrix M1)
```

**def of\_kraus**[\[source\]](#)

*Construct a matrix map out of families of matrices  $M N : \Sigma \rightarrow \text{Matrix } B A R$  indexed by  $\kappa$  via  $X \mapsto \sum k : \kappa, (M k) * X * (N k)^H$*

```
def of_kraus (M N : κ → Matrix B A R) : MatrixMap A B R
```

**def exists\_kraus**

[\[source\]](#)

```
def exists_kraus (Φ : MatrixMap A B R) : ∃ r : ℕ, ∃ (M N : Fin r → Matrix B A R), Φ = of_kraus
  M N
```

**def submatrix**

[\[source\]](#)

*The MatrixMap corresponding to applying a submatrix operation on each side.*

@[simps]

```
def submatrix (f : B → A) : MatrixMap A B R where
```

**theorem submatrix\_id**

[\[source\]](#)

@[simp]

```
theorem submatrix_id : submatrix R _root_.id = id A R
```

**theorem submatrix\_comp**

[\[source\]](#)

@[simp]

```
theorem submatrix_comp (f : C → B) (g : B → A) :
  submatrix R f ∘l submatrix R g = submatrix R (g ∘ f)
```

**def kron**

[\[source\]](#)

*The Kronecker product of MatrixMaps. Defined here using TensorProduct.map M<sub>1</sub> M<sub>2</sub>, with appropriate reindexing operations and LinearMap.toMatrix/Matrix.toLin. Notation  $\otimes_{km}$ .*

```
noncomputable def kron [CommSemiring R] (M1 : MatrixMap A B R) (M2 : MatrixMap C D R) :
  MatrixMap (A × C) (B × D) R
```

**theorem kron\_def**

[\[source\]](#)

*The extensional definition of the Kronecker product MatrixMap.kron, in terms of the entries of its image.*

```
theorem kron_def [CommSemiring R] (M1 : MatrixMap A B R) (M2 : MatrixMap C D R) (M : Matrix (A
  × C) (A × C) R) :
  (M1 ∘km M2) M (b1, d1) (b2, d2) = ∑ a1, ∑ a2, ∑ c1, ∑ c2,
    (M1 (Matrix.single a1 a2 1) b1 b2) * (M2 (Matrix.single c1 c2 1) d1 d2) * (M (a1, c1) (a
    2, c2))
```

**theorem add\_kron**

[\[source\]](#)

```
theorem add_kron (ML1 ML2 : MatrixMap A B R) (MR : MatrixMap C D R) : (ML1 + ML2) ∘km MR = ML1
  ∘km MR + ML2 ∘km MR
```



**theorem kron\_add**[\[source\]](#)

```
theorem kron_add (ML : MatrixMap A B R) (MR1 MR2 : MatrixMap C D R) : ML  $\otimes_{km}$  (MR1 + MR2) = ML  $\otimes_{km}$  MR1 + ML  $\otimes_{km}$  MR2
```

**theorem smul\_kron**[\[source\]](#)

```
theorem smul_kron (r : R) (ML : MatrixMap A B R) (MR : MatrixMap C D R) : (r • ML)  $\otimes_{km}$  MR = r • (ML  $\otimes_{km}$  MR)
```

**theorem kron\_smul**[\[source\]](#)

```
theorem kron_smul (r : R) (ML : MatrixMap A B R) (MR : MatrixMap C D R) : ML  $\otimes_{km}$  (r • MR) = r • (ML  $\otimes_{km}$  MR)
```

**theorem zero\_kron**[\[source\]](#)

```
@[simp]
theorem zero_kron (MR : MatrixMap C D R) : (0 : MatrixMap A B R)  $\otimes_{km}$  MR = 0
```

**theorem kron\_zero**[\[source\]](#)

```
@[simp]
theorem kron_zero (ML : MatrixMap A B R) : ML  $\otimes_{km}$  (0 : MatrixMap C D R) = 0
```

**theorem kron\_id\_id**[\[source\]](#)

```
theorem kron_id_id : (id A R  $\otimes_{km}$  id B R) = id (A × B) R
```

**theorem kron\_comp\_distrib**[\[source\]](#)

*For maps  $L_1$ ,  $L_2$ ,  $R_1$ , and  $R_2$ , the product  $(L_2 \circ_l L_1) \otimes_{km} (R_2 \circ_l R_1) = (L_2 \otimes_{km} R_2) \circ_l (L_1 \otimes_{km} R_1)$*

```
theorem kron_comp_distrib (L1 : MatrixMap Dl1 Dl2 R) (L2 : MatrixMap Dl2 Dl3 R) (R1 : MatrixMap Dr1 Dr2 R) (R2 : MatrixMap Dr2 Dr3 R) : (L2  $\circ_l$  L1)  $\otimes_{km}$  (R2  $\circ_l$  R1) = (L2  $\otimes_{km}$  R2)  $\circ_l$  (L1  $\otimes_{km}$  R1)
```

**theorem kron\_map\_of\_kron\_state**[\[source\]](#)

*The operational definition of the Kronecker product `MatrixMap.kron`, that it maps a Kronecker product of inputs to the Kronecker product of outputs. It is the unique bilinear map doing so.*

```
theorem kron_map_of_kron_state [CommRing R] (M1 : MatrixMap A B R) (M2 : MatrixMap C D R) (MA : Matrix A A R) (MC : Matrix C C R) : (M1  $\otimes_{km}$  M2) (MA  $\otimes_k$  MC) = (M1 MA)  $\otimes_k$  (M2 MC)
```

**theorem choi\_matrix\_state\_rep**[\[source\]](#)

```
theorem choi_matrix_state_rep {B : Type*} [Fintype B] [Nonempty A] (M : MatrixMap A B  $\mathbb{C}$ ) : M.choi_matrix = (↑(Fintype.card (α := A)) :  $\mathbb{C}$ ) • (M  $\otimes_{km}$  (LinearMap.id : MatrixMap A A  $\mathbb{C}$ )) (MState.pure (Ket.MES A)).m
```

**theorem submatrix\_kron\_submatrix**[\[source\]](#)

```
theorem submatrix_kron_submatrix [CommSemiring R] (f : B → A) (g : D → C) :
  submatrix R f ⊗k m submatrix R g = submatrix R (Prod.map f g)
```

**theorem submatrix\_kron\_id**[\[source\]](#)

```
theorem submatrix_kron_id [CommSemiring R] (f : B → A) :
  submatrix R f ⊗k m id C R = submatrix R (Prod.map f _root_.id)
```

**theorem id\_kron\_submatrix**[\[source\]](#)

```
theorem id_kron_submatrix [CommSemiring R] (f : B → A) :
  id C R ⊗k m submatrix R f = submatrix R (Prod.map _root_.id f)
```

**opaque piTensorProduct**[\[source\]](#)

*Like Basis.tensorProduct, but for PiTensorProduct*

```
noncomputable opaque _root_.Module.Basis.piTensorProduct [Vi, Fintype (L i)]
  (b : (i:ι) → Module.Basis (L i) R (s i)) :
  Module.Basis ((i:ι) → L i) R (PiTensorProduct R s)
```

**def piProd**[\[source\]](#)

*Finite Pi-type tensor product of MatrixMaps. Defined as PiTensorProduct.tprod of the underlying Linear maps. Notation  $\bigotimes_{i \in m} [R] i, f i$ , eventually.*

```
noncomputable def piProd (Λi : ∀ i, MatrixMap (dI i) (dO i) R) : MatrixMap (∀ i, dI i) (∀ i, dO i) R
```

**theorem piProd\_comp**[\[source\]](#)

*Composition of MatrixMap.piProd maps distributes over the tensor product.*

```
theorem piProd_comp
  {d₁ d₂ d₃ : ι → Type*}
  [∀ i, Fintype (d₁ i)] [∀ i, DecidableEq (d₁ i)]
  [∀ i, Fintype (d₂ i)] [∀ i, DecidableEq (d₂ i)]
  [∀ i, Fintype (d₃ i)] [∀ i, DecidableEq (d₃ i)]
  (Λ₁ : ∀ i, MatrixMap (d₁ i) (d₂ i) R) (Λ₂ : ∀ i, MatrixMap (d₂ i) (d₃ i) R) :
  piProd (fun i ↦ (Λ₂ i) ∘l (Λ₁ i)) = (piProd Λ₂) ∘l (piProd Λ₁)
```

**2.34 QuantumInfo.Finite.CPTPMap.Unbundled**

[QuantumInfo/Finite/CPTPMap/Unbundled.lean](#) on GitHub.

**def IsTracePreserving**[\[source\]](#)

*A linear matrix map is \*trace preserving\* if trace of the output equals trace of the input.*

```
def IsTracePreserving (M : MatrixMap A B R) : Prop
```

**theorem IsTracePreserving\_iff\_trace\_choi**[\[source\]](#)

*A map is trace preserving iff the partial trace of the Choi matrix is the identity.*

```
theorem IsTracePreserving_iff_trace_choi [DecidableEq A] (M : MatrixMap A B R) : M.
  IsTracePreserving
  ↔ M.choi_matrix.traceLeft = 1
```

**theorem apply\_trace**[\[source\]](#)

*Simp lemma: the trace of the image of a IsTracePreserving map is the same as the original trace.*

```
@[simp]
theorem apply_trace (h : M.IsTracePreserving) (p : Matrix A A R)
  : (M p).trace = p.trace
```

**theorem trace\_choi**[\[source\]](#)

*The trace of a Choi matrix of a TP map is the cardinality of the input space.*

```
theorem trace_choi [DecidableEq A] (h : M.IsTracePreserving) :
  M.choi_matrix.trace = (Finset.univ (α := A)).card
```

**theorem IsTracePreserving.comp**[\[source\]](#)

*The composition of IsTracePreserving maps is also trace preserving.*

```
theorem comp (h1 : M1.IsTracePreserving) (h2 : M2.IsTracePreserving) :
  IsTracePreserving (M2 ∘l M1)
```

**theorem IsTracePreserving.id**[\[source\]](#)

*The identity MatrixMap IsTracePreserving.*

```
@[simp]
theorem id : (id A R).IsTracePreserving
```

**theorem unit\_linear**[\[source\]](#)

*Unit linear combinations of IsTracePreserving maps are IsTracePreserving.*

```
theorem unit_linear {M1 M2 : MatrixMap A B R} {x y : R}
  (h1 : M1.IsTracePreserving) (h2 : M2.IsTracePreserving) (hxy : x + y = 1) :
  (x • M1 + y • M2).IsTracePreserving
```

**theorem IsTracePreserving.kron**[\[source\]](#)

*The kronecker product of IsTracePreserving maps is also trace preserving.*

```
theorem kron {M1 : MatrixMap A B R} {M2 : MatrixMap C D R} (h1 : M1.IsTracePreserving) (h2 : M2.
  IsTracePreserving) :
  (M1 ⊗k m M2).IsTracePreserving
```

**theorem of\_kraus\_isTracePreserving**[\[source\]](#)

The channel  $X \mapsto \sum k : \kappa, (M k) * X * (N k)^H$  formed by Kraus operators  $M, N : \kappa \rightarrow \text{Matrix } B \ A \ R$  is trace-preserving if  $\sum k : \kappa, (N k)^H * (M k) = 1$

**theorem** of\_kraus\_isTracePreserving

```
(M N : κ → Matrix B A S)
(hTP : (∑ k, (N k).conjTranspose * (M k)) = 1) :
(MatrixMap.of_kraus M N).IsTracePreserving
```

**theorem IsTracePreserving.submatrix**[\[source\]](#)

*MatrixMap.submatrix is trace-preserving when the function is an equivalence.*

**theorem** submatrix (e : A ≈ B) : (MatrixMap.submatrix R e).IsTracePreserving**def Unital**[\[source\]](#)

*A linear matrix map is \*unital\* if it preserves the identity.*

**def** Unital (M : MatrixMap A B R) : Prop**theorem map\_1**[\[source\]](#)

@[simp]

**theorem** map\_1 (h : M.Unital) : M 1 = 1**theorem Unital.id**[\[source\]](#)

*The identity MatrixMap is Unital.*

@[simp]

**theorem** id : (id A R).Unital**def MatrixMap.IsHermitianPreserving**[\[source\]](#)

*A linear matrix map is \*Hermitian preserving\* if it maps IsHermitian matrices to IsHermitian.*

**def** IsHermitianPreserving (M : MatrixMap A B R) : Prop**def MatrixMap.IsPositive**[\[source\]](#)

*A linear matrix map is \*positive\* if it maps PosSemidef matrices to PosSemidef.*

**def** IsPositive [Fintype A] [Fintype B] (M : MatrixMap A B R) : Prop**def IsCompletelyPositive**[\[source\]](#)

*A linear matrix map is \*completely positive\* if, for any integer  $n$ , the tensor product with  $I(n)$  is positive.*

**def** IsCompletelyPositive [Fintype A] [Fintype B] [DecidableEq A] (M : MatrixMap A B R) : Prop

**theorem IsHermitianPreserving.id**[\[source\]](#)*The identity MatrixMap IsHermitianPreserving.*

```
theorem id : (id A R).IsPositive
```

**theorem IsHermitianPreserving.comp**[\[source\]](#)*The composition of IsHermitianPreserving maps is also Hermitian preserving.*

```
theorem comp {M1 : MatrixMap A B R} {M2 : MatrixMap B C R}
  (h1 : M1.IsHermitianPreserving) (h2 : M2.IsHermitianPreserving) : IsHermitianPreserving (M2 ∘l M1)
```

**theorem IsPositive.IsHermitianPreserving**[\[source\]](#)

```
theorem IsHermitianPreserving {M : MatrixMap A B R}
  (hM : IsPositive M) : IsHermitianPreserving M
```

**theorem IsPositive.comp**[\[source\]](#)*The composition of IsPositive maps is also positive.*

```
theorem comp {M1 : MatrixMap A B R} {M2 : MatrixMap B C R} (h1 : M1.IsPositive)
  (h2 : M2.IsPositive) : IsPositive (M2 ∘l M1)
```

**theorem IsPositive.id**[\[source\]](#)*The identity MatrixMap IsPositive.*

```
@[simp]
theorem id : (id A R).IsPositive
```

**theorem IsPositive.add**[\[source\]](#)*Sums of IsPositive maps are IsPositive.*

```
theorem add {M1 M2 : MatrixMap A B R} (h1 : M1.IsPositive) (h2 : M2.IsPositive) :
  (M1 + M2).IsPositive
```

**theorem IsPositive.smul**[\[source\]](#)*Nonnegative scalings of IsPositive maps are IsPositive.*

```
theorem smul {M : MatrixMap A B R} (hM : M.IsPositive) {x : R} (hx : 0 ≤ x) :
  (x • M).IsPositive
```

**theorem of\_Fintype**[\[source\]](#)*Definition of a CP map, but with Fintype T in the definition instead of a Fin n.*

```
theorem of_Fintype {M : MatrixMap A B R} (h : IsCompletelyPositive M)
  (T : Type*) [Fintype T] [DecidableEq T] :
  (M.kron (LinearMap.id : MatrixMap T T R)).IsPositive
```

**theorem IsCompletelyPositive.IsPositive**[\[source\]](#)

```
theorem IsPositive {M : MatrixMap A B R}
  (hM : IsCompletelyPositive M) : IsPositive M
```

**theorem IsCompletelyPositive.comp**[\[source\]](#)

*The composition of IsCompletelyPositive maps is also completely positive.*

```
theorem comp [DecidableEq B] {M1 : MatrixMap A B R} {M2 : MatrixMap B C R} (h1 : M1.
  IsCompletelyPositive)
  (h2 : M2.IsCompletelyPositive) : IsCompletelyPositive (M2 ∘l M1)
```

**theorem IsCompletelyPositive.id**[\[source\]](#)

*The identity MatrixMap IsCompletelyPositive.*

```
@[simp]
theorem id : (id A R).IsCompletelyPositive
```

**theorem IsCompletelyPositive.add**[\[source\]](#)

*Sums of IsCompletelyPositive maps are IsCompletelyPositive.*

```
theorem add {M1 M2 : MatrixMap A B R} (h1 : M1.IsCompletelyPositive) (h2 : M2.
  IsCompletelyPositive) :
  (M1 + M2).IsCompletelyPositive
```

**theorem IsCompletelyPositive.smul**[\[source\]](#)

*Nonnegative scalings of IsCompletelyPositive maps are IsCompletelyPositive.*

```
theorem smul {M : MatrixMap A B R} (hM : M.IsCompletelyPositive) {x : R} (hx : 0 ≤ x) :
  (x • M).IsCompletelyPositive
```

**theorem zero**[\[source\]](#)

*The zero map IsCompletelyPositive.*

```
theorem zero : (0 : MatrixMap A B R).IsCompletelyPositive
```

**theorem finset\_sum**[\[source\]](#)

*A finite sum of completely positive maps is completely positive.*

```
theorem finset_sum {ι : Type*} [Fintype ι] {m : ι → MatrixMap A B R} (hm : ∀ i, (m i).
  IsCompletelyPositive) :
  (∑ i, m i).IsCompletelyPositive
```

**theorem kron\_kronecker\_const**[\[source\]](#)

*The map that takes M and returns  $M \otimes_k C$ , where C is positive semidefinite, is a completely positive map.*

```

theorem kron_kronecker_const {C : Matrix d d R} (h : C.PosSemidef) {h1 h2 : _} :
  IsCompletelyPositive
  (({fun M => M ⊗k C, h1), h2) : MatrixMap A (A × d) R

```

### **theorem choi\_of\_kraus**

[\[source\]](#)

```

theorem choi_of_kraus (K : κ → Matrix B A ℤ) :
  (MatrixMap.of_kraus K K).choi_matrix = ∑ k, Matrix.vecMulVec (fun (x : B × A) => K k x.1 x
  .2) (fun (x : B × A) => star (K k x.1 x.2))

```

### **def conj**

[\[source\]](#)

*The linear map of conjugating a matrix by another,  $x \rightarrow y * x * y^H$ .*

```

@[simps]
def _root_.MatrixMap.conj (y : Matrix B A R) : MatrixMap A B R where

```

### **theorem conj\_isPositive**

[\[source\]](#)

```

theorem conj_isPositive (M : Matrix B A ℤ) : (conj M).IsPositive

```

### **theorem IsPositive\_sum**

[\[source\]](#)

```

theorem IsPositive_sum {ι : Type*} [Fintype ι] (f : ι → MatrixMap A B ℂ) (h : ∀ i, (f i).
  IsPositive) :
  (∑ i, f i).IsPositive

```

### **theorem of\_kraus\_isPositive**

[\[source\]](#)

```

theorem of_kraus_isPositive (K : κ → Matrix B A ℂ) :
  (of_kraus K K).IsPositive

```

### **theorem conj\_kron**

[\[source\]](#)

```

theorem conj_kron (M : Matrix B A ℤ) (N : Matrix D C ℤ) [DecidableEq C] :
  conj M ⊗k conj N = conj (M ⊗k N)

```

### **theorem congruence\_one\_eq\_id**

[\[source\]](#)

```

theorem congruence_one_eq_id : conj (1 : Matrix A A ℂ) = MatrixMap.id A ℂ

```

### **theorem congruence\_CP**

[\[source\]](#)

```

theorem congruence_CP {A B : Type*} [Fintype A] [Fintype B] [DecidableEq A] [DecidableEq B] (M
  : Matrix B A ℤ) : (conj M).IsCompletelyPositive

```

### **theorem IsCompletelyPositive\_sum**

[\[source\]](#)

```

theorem IsCompletelyPositive_sum {ι : Type*} [Fintype ι] (f : ι → MatrixMap A B ℂ) (h : ∀ i, (f
  i).IsCompletelyPositive) :
  (∑ i, f i).IsCompletelyPositive

```

**theorem of\_kraus\_eq\_sum\_conj**[\[source\]](#)

```
theorem of_kraus_eq_sum_conj (K : κ → Matrix B A ℚ) :
  of_kraus K K = ∑ k, conj (K k)
```

**theorem of\_kraus\_CP**[\[source\]](#)

```
theorem of_kraus_CP (K : κ → Matrix B A ℚ) : (of_kraus K K).IsCompletelyPositive
```

**theorem exists\_kraus\_of\_choi\_PSD**[\[source\]](#)

```
theorem exists_kraus_of_choi_PSD
  (C : Matrix (B × A) (B × A) ℚ) (hC : C.PosSemidef) :
  ∃ (K : (B × A) → Matrix B A ℚ), C = (MatrixMap.of_kraus K K).choi_matrix
```

**theorem choi\_matrix\_eq\_map\_proj**[\[source\]](#)

```
theorem choi_matrix_eq_map_proj (M : MatrixMap A B R) :
  M.choi_matrix = (M ⊗κ MatrixMap.id A R) (Matrix.vecMulVec (fun (x : A × A) => if x.1 = x.2
    then 1 else 0) (fun (x : A × A) => star (if x.1 = x.2 then 1 else 0)))
```

**theorem choi\_PSD\_iff\_CP\_map**[\[source\]](#)

*Choi's theorem on completely positive maps: A map IsCompletelyPositive iff its Choi Matrix is PSD.*

```
theorem choi_PSD_iff_CP_map (M : MatrixMap A B R) :
  M.IsCompletelyPositive ↔ M.choi_matrix.PosSemidef
```

**theorem conj\_eq\_mulRightLinearMap\_comp\_mulRightLinearMap**[\[source\]](#)

```
theorem conj_eq_mulRightLinearMap_comp_mulRightLinearMap (y : Matrix B A R) :
  conj y = mulRightLinearMap B R y.conjTranspose ∘l mulLeftLinearMap A R y
```

**theorem conj\_isCompletelyPositive**[\[source\]](#)

*The act of conjugating (not necessarily by a unitary, just by any matrix at all) is completely positive.*

```
theorem conj_isCompletelyPositive (M : Matrix B A R) : (conj M).IsCompletelyPositive
```

**theorem IsCompletelyPositive.submatrix**[\[source\]](#)

*MatrixMap.submatrix is completely positive*

```
theorem IsCompletelyPositive.submatrix (f : B → A) : (MatrixMap.submatrix R f).
  IsCompletelyPositive
```

**theorem of\_kraus\_isCompletelyPositive**[\[source\]](#)

*The channel  $X \mapsto \sum k : \kappa, (M k) * X * (M k)^H$  formed by Kraus operators  $M : \kappa \rightarrow \text{Matrix } B \ A \ R$  is completely positive*



```
theorem of_kraus_isCompletelyPositive (M :  $\kappa \rightarrow \text{Matrix } B \ A \ R$ ) :
  (of_kraus M M).IsCompletelyPositive
```

### **theorem choi\_of\_kraus\_R**

[\[source\]](#)

*The Choi matrix of a map in symmetric Kraus form is a sum of rank-1 projectors.*

```
theorem choi_of_kraus_R [DecidableEq A] (K :  $\kappa \rightarrow \text{Matrix } B \ A \ \mathbb{k}$ ) :
  (of_kraus K K).choi_matrix =  $\sum k$ , Matrix.vecMulVec (fun (x :  $B \times A$ ) => K k x.1 x.2) (fun (x
    :  $B \times A$ ) => star (K k x.1 x.2))
```

### **theorem choi\_eq\_kron\_id\_apply\_choi\_id**

[\[source\]](#)

```
theorem choi_eq_kron_id_apply_choi_id (M : MatrixMap A B R) :
  M.choi_matrix = (M  $\otimes_{\kappa m}$  MatrixMap.id A R) ((MatrixMap.id A R).choi_matrix)
```

### **theorem choi\_id\_is\_PSD**

[\[source\]](#)

```
theorem choi_id_is_PSD {A R : Type*} [Fintype A] [DecidableEq A] [RCLike R] :
  (MatrixMap.id A R).choi_matrix.PosSemidef
```

### **theorem is\_CP\_implies\_choi\_PSD**

[\[source\]](#)

```
theorem is_CP_implies_choi_PSD {A B R : Type*} [Fintype A] [Fintype B] [DecidableEq A] [
  DecidableEq B] [RCLike R] (M : MatrixMap A B R) (hCP : M.IsCompletelyPositive) :
  M.choi_matrix.PosSemidef
```

### **theorem exists\_kraus**

[\[source\]](#)

```
theorem IsCompletelyPositive.exists_kraus ( $\Phi$  : MatrixMap A B R) (hCP :  $\Phi$ .IsCompletelyPositive)
  :
   $\exists (M : (B \times A) \rightarrow \text{Matrix } B \ A \ R), \Phi = \text{of\_kraus } M \ M$ 
```

### **theorem kron\_of\_kraus**

[\[source\]](#)

*The Kronecker product of two Kraus maps is the Kraus map of the Kronecker products of the operators.*

```
theorem kron_of_kraus {A B C D R : Type*} [Fintype A] [Fintype B] [Fintype C] [Fintype D]
  [DecidableEq A] [DecidableEq C] [CommSemiring R] [StarRing R] [SMulCommClass R R R]
  { $\kappa \ \iota$  : Type*} [Fintype  $\kappa$ ] [Fintype  $\iota$ ]
  (M :  $\kappa \rightarrow \text{Matrix } B \ A \ R$ ) (N :  $\iota \rightarrow \text{Matrix } D \ C \ R$ ) :
  of_kraus M M  $\otimes_{\kappa m}$  of_kraus N N =
  of_kraus (fun (k :  $\kappa \times \iota$ ) => M k.1  $\otimes_{\kappa}$  N k.2) (fun k => M k.1  $\otimes_{\kappa}$  N k.2)
```

### **theorem IsCompletelyPositive.kron**

[\[source\]](#)

*The Kronecker product of IsCompletelyPositive maps is also completely positive.*

```
theorem kron {D : Type*} [DecidableEq C] [Fintype D] {M1 : MatrixMap A B R} {M2 : MatrixMap C D
  R}
  (h1 : M1.IsCompletelyPositive) (h2 : M2.IsCompletelyPositive) : IsCompletelyPositive (M1
   $\otimes_{\kappa m}$  M2)
```

**theorem piProd**[\[source\]](#)

*The `MatrixMap.piProd` product of `IsCompletelyPositive` maps is also completely positive.*

```
theorem piProd {Λ i : ∀ i, MatrixMap (dI i) (dO i) ℝ} (h₁ : ∀ i, (Λ i i).IsCompletelyPositive) :
  IsCompletelyPositive (MatrixMap.piProd Λ i)
```

**2.35 QuantumInfo.Finite.Distance.Fidelity**

[QuantumInfo/Finite/Distance/Fidelity.lean](#) on GitHub.

**def fidelity**[\[source\]](#)

*The fidelity of two quantum states. This is the quantum version of the Bhattacharyya coefficient.*

```
def fidelity (ρ σ : MState d) : ℝ
```

**theorem fidelity\_ge\_zero**[\[source\]](#)

```
theorem fidelity_ge_zero : 0 ≤ fidelity ρ σ
```

**theorem fidelity\_le\_one**[\[source\]](#)

```
theorem fidelity_le_one : fidelity ρ σ ≤ 1
```

**def fidelity\_prob**[\[source\]](#)

*The fidelity, as a `Prob` probability with value between 0 and 1.*

```
def fidelity_prob : Prob
```

**theorem fidelity\_self\_eq\_one**[\[source\]](#)

*A state has perfect fidelity with itself.*

```
theorem fidelity_self_eq_one : fidelity ρ ρ = 1
```

**theorem fidelity\_eq\_one\_iff\_self**[\[source\]](#)

*The fidelity is 1 if and only if the two states are the same.*

```
theorem fidelity_eq_one_iff_self : fidelity ρ σ = 1 ↔ ρ = σ
```

**theorem fidelity\_symm**[\[source\]](#)

*The fidelity is a symmetric quantity.*

```
theorem fidelity_symm : fidelity ρ σ = fidelity σ ρ
```

**theorem fidelity\_channel\_nondecreasing**[\[source\]](#)

*The fidelity cannot decrease under the application of a channel.*

```
theorem fidelity_channel_nondecreasing [DecidableEq d₂] (Λ : CPTPMap d d₂) : fidelity (Λ ρ) (Λ σ) ≥ fidelity ρ σ
```

**2.36 QuantumInfo.Finite.Distance.TraceDistance**

[QuantumInfo/Finite/Distance/TraceDistance.lean](#) on GitHub.

**def TrDistance**[\[source\]](#)

*The trace distance between two quantum states: half the trace norm of the difference  $(\rho - \sigma)$ .*

```
def TrDistance (ρ σ : MState d) : ℝ
```

**theorem ge\_zero**[\[source\]](#)

```
theorem ge_zero : 0 ≤ TrDistance ρ σ
```

**theorem le\_one**[\[source\]](#)

```
theorem le_one : TrDistance ρ σ ≤ 1
```

**def prob**[\[source\]](#)

*The trace distance, as a *Prob* probability with value between 0 and 1.*

```
def prob : Prob
```

**theorem symm**[\[source\]](#)

*The trace distance is a symmetric quantity.*

```
theorem symm : TrDistance ρ σ = TrDistance σ ρ
```

**theorem eq\_abs\_eigenvalues**[\[source\]](#)

*The trace distance is equal to half the 1-norm of the eigenvalues of their difference .*

```
theorem eq_abs_eigenvalues : TrDistance ρ σ = (1/2:ℝ) *
  ∑ i, abs ((ρ.Hermitian.sub σ.Hermitian).eigenvalues i)
```

**2.37 QuantumInfo.Finite.Ensemble**

[QuantumInfo/Finite/Ensemble.lean](#) on GitHub.

**abbrev MEnsemble**[\[source\]](#)

*A mixed-state ensemble is a random variable valued in *MState d*. That is, a collection of mixed states  $\text{var} : \alpha \rightarrow \text{MState } d$ , each with their own probability weight described by  $\text{distr} : \text{ProbDistribution } \alpha$ .*

```
abbrev MEnsemble (d : Type*) (α : Type*) [Fintype d] [DecidableEq d] [Fintype α]
```

**abbrev PEnsemble**

[\[source\]](#)

*A pure-state ensemble is a random variable valued in Ket d. That is, a collection of pure states  $\text{var} : \alpha \rightarrow \text{Ket } d$ , each with their own probability weight described by  $\text{distr} : \text{ProbDistribution } \alpha$ .*

```
abbrev PEnsemble (d : Type*) (α : Type*) [Fintype d] [Fintype α]
```

**abbrev MEnsemble.states**

[\[source\]](#)

*Alias for ProbDistribution.var for mixed-state ensembles.*

```
abbrev MEnsemble.states [Fintype α] : MEnsemble d α → (α → MState d)
```

**abbrev PEnsemble.states**

[\[source\]](#)

*Alias for ProbDistribution.var for pure-state ensembles.*

```
abbrev PEnsemble.states [Fintype α] : PEnsemble d α → (α → Ket d)
```

**instance Coe (PEnsemble d α) (MEnsemble d α)**

[\[source\]](#)

*A pure-state ensemble is a mixed-state ensemble if all kets are interpreted as mixed states.*

```
instance : Coe (PEnsemble d α) (MEnsemble d α)
```

**theorem toMEnsemble\_mk**

[\[source\]](#)

```
@[simp]
```

```
theorem toMEnsemble_mk : (toMEnsemble ⟨ps, distr⟩ : MEnsemble d α) = ⟨pure ∘ ps, distr⟩
```

**theorem coe\_PEnsemble\_iff\_pure\_states**

[\[source\]](#)

*A mixed-state ensemble comes from a pure-state ensemble if and only if all states are pure.*

```
theorem coe_PEnsemble_iff_pure_states (me : MEnsemble d α) : (∃ pe : PEnsemble d α, ↑pe = me) ↔
  (∃ ψ : α → Ket d, me.states = MState.pure ∘ ψ)
```

**def mix**

[\[source\]](#)

*The resulting mixed state after mixing the states in an ensemble with their respective probability weights. Note that, generically, a single mixed state has infinitely many ensembles that mixes into it.*

```
def mix (e : MEnsemble d α) : MState d
```

**theorem mix\_of**[\[source\]](#)

```
@[simp]
theorem mix_of (e : MEnsemble d α) : (mix e).m = ∑ i, (e.distr i : ℝ) • (e.states i).m
```

**def congrMEnsemble**[\[source\]](#)

*Two mixed-state ensembles indexed by  $\alpha$  and  $\beta$  are equivalent if  $\alpha \approx \beta$ .*

```
def congrMEnsemble (σ : α ≈ β) : MEnsemble d α ≈ MEnsemble d β
```

**def congrPEnsemble**[\[source\]](#)

*Two pure-state ensembles indexed by  $\alpha$  and  $\beta$  are equivalent if  $\alpha \approx \beta$ .*

```
def congrPEnsemble (σ : α ≈ β) : PEnsemble d α ≈ PEnsemble d β
```

**theorem mix\_congrMEnsemble\_eq\_mix**[\[source\]](#)

*Equivalence of mixed-state ensembles leaves the resulting mixed state invariant*

```
@[simp]
theorem mix_congrMEnsemble_eq_mix (σ : α ≈ β) (e : MEnsemble d α) : mix (congrMEnsemble σ e) =
  mix e
```

**theorem mix\_congrPEnsemble\_eq\_mix**[\[source\]](#)

*Equivalence of pure-state ensembles leaves the resulting mixed state invariant*

```
@[simp]
theorem mix_congrPEnsemble_eq_mix (σ : α ≈ β) (e : PEnsemble d α) : mix (toMEnsemble (
  congrPEnsemble σ e)) = mix (↑e : MEnsemble d α)
```

**def average**[\[source\]](#)

*The average of a function  $f : MState\ d \rightarrow T$ , where  $T$  is of `Mixable U T` instance, on a mixed-state ensemble  $e$  is the expectation value of  $f$  acting on the states of  $e$ , with the corresponding probability weights from  $e.distr$ .*

```
def average {T : Type _} {U : Type*} [AddCommGroup U] [Module ℝ U] [inst : Mixable U T] (f :
  MState d → T) (e : MEnsemble d α) : T
```

**def average\_NNReal**[\[source\]](#)

*A version of `average` conveniently specialized for functions  $f : MState\ d \rightarrow \mathbb{R}_{\geq 0}$  returning nonnegative reals. Notably, the average is also a nonnegative real number.*

```
def average_NNReal {d : Type _} [Fintype d] [DecidableEq d] (f : MState d → NNReal) (e :
  MEnsemble d α) : NNReal
```

**def pure\_average**[\[source\]](#)

The average of a function  $f : \text{Ket } d \rightarrow T$ , where  $T$  is of `Mixable U T` instance, on a pure-state ensemble  $e$  is the expectation value of  $f$  acting on the states of  $e$ , with the corresponding probability weights from  $e.\text{distr}$ .

```
def pure_average {T : Type _} {U : Type*} [AddCommGroup U] [Module ℝ U] [inst : Mixable U T] (f : Ket d → T) (e : PEnsemble d α) : T
```

### def pure\_average\_NNReal

[\[source\]](#)

A version of `average` conveniently specialized for functions  $f : \text{Ket } d \rightarrow \mathbb{R}_{\geq 0}$  returning nonnegative reals. Notably, the average is also a nonnegative real number.

```
def pure_average_NNReal {d : Type _} [Fintype d] (f : Ket d → NNReal) (e : PEnsemble d α) : NNReal
```

### theorem average\_of\_pure\_ensemble

[\[source\]](#)

The average of  $f : \text{MState } d \rightarrow T$  on a coerced pure-state ensemble  $\uparrow e : \text{MEnsemble } d$   $\alpha$  is equal to averaging the restricted function over Kets  $f \circ \text{pure} : \text{Ket } d \rightarrow T$  on  $e$ .

```
theorem average_of_pure_ensemble {T : Type _} {U : Type*} [AddCommGroup U] [Module ℝ U] [inst : Mixable U T] (f : MState d → T) (e : PEnsemble d α) : average f (toMEnsemble e) = pure_average (f ∘ pure) e
```

### theorem distr\_toMEnsemble

[\[source\]](#)

@[simp]

```
theorem distr_toMEnsemble (e : PEnsemble d α) : (toMEnsemble e).distr = e.distr
```

### theorem mix\_pEnsemble\_pure\_iff\_pure

[\[source\]](#)

```
theorem mix_pEnsemble_pure_iff_pure {e : PEnsemble d α} : mix (toMEnsemble e) = MState.pure ψ ↔ ∀ i : α, e.distr i ≠ 0 → MState.pure (e.states i) = MState.pure ψ
```

### theorem mix\_pEnsemble\_pure\_average

[\[source\]](#)

The average of  $f : \text{Ket } d \rightarrow T$  on an ensemble that mixes to a pure state  $\psi$  is  $f \psi$

```
theorem mix_pEnsemble_pure_average {e : PEnsemble d α} {T : Type _} {U : Type*} [AddCommGroup U] [Module ℝ U] [inst : Mixable U T] (f : Ket d → T) (hf : ∀ ψ φ, Ket.PhaseEquiv.r ψ φ → f ψ = f φ) (hmix : mix (toMEnsemble e) = MState.pure ψ) : pure_average f e = f ψ
```

### theorem sum\_prob\_mul\_eq\_one\_iff

[\[source\]](#)

```
theorem sum_prob_mul_eq_one_iff {ι : Type*} [Fintype ι] (p : ι → ℝ) (x : ι → ℝ) (hp : ∀ i, 0 ≤ p i) (hsum : ∑ i, p i = 1) (hx : ∀ i, x i ≤ 1) : (∑ i, p i * x i = 1) ↔ ∀ i, p i ≠ 0 → x i = 1
```

**theorem exp\_val\_pure\_eq\_one\_iff**[\[source\]](#)

```

theorem MState.exp_val_pure_eq_one_iff {d : Type*} [Fintype d] [DecidableEq d] (ρ : MState d) (
  ψ : Ket d) :
  ρ.exp_val (pure ψ) = 1 ↔ ρ = pure ψ

```

**theorem mix\_mEnsemble\_pure\_iff\_pure**[\[source\]](#)

```

theorem mix_mEnsemble_pure_iff_pure {e : MEnsemble d α} :
  mix e = pure ψ ↔ ∀ i : α, e.distr i ≠ 0 → e.states i = MState.pure ψ

```

**theorem mix\_mEnsemble\_pure\_average**[\[source\]](#)

*The average of  $f : MState\ d \rightarrow T$  on an ensemble that mixes to a pure state  $\psi$  is  $f$  (pure  $\psi$ )*

```

theorem mix_mEnsemble_pure_average {e : MEnsemble d α} {T : Type _} {U : Type*} [AddCommGroup U]
  [Module ℝ U] [inst : Mixable U T] (f : MState d → T) (hmix : mix e = pure ψ) :
  average f e = f (pure ψ)

```

**def trivial\_mEnsemble**[\[source\]](#)

*The trivial mixed-state ensemble of  $\rho$  consists of copies of  $\rho$ , with the  $i$ -th one having probability 1.*

```

def trivial_mEnsemble (ρ : MState d) (i : α) : MEnsemble d α

```

**theorem trivial\_mEnsemble\_mix**[\[source\]](#)

*The trivial mixed-state ensemble of  $\rho$  mixes to  $\rho$*

```

theorem trivial_mEnsemble_mix (ρ : MState d) : ∀ i : α, mix (trivial_mEnsemble ρ i) = ρ

```

**theorem trivial\_mEnsemble\_average**[\[source\]](#)

*The average of  $f : MState\ d \rightarrow T$  on a trivial ensemble of  $\rho$  is  $f\ \rho$*

```

theorem trivial_mEnsemble_average {T : Type _} {U : Type*} [AddCommGroup U] [Module ℝ U] [inst
  : Mixable U T] (f : MState d → T) (ρ : MState d):
  ∀ i : α, average f (trivial_mEnsemble ρ i) = f ρ

```

**instance MEnsemble.instInhabited**[\[source\]](#)

```

instance MEnsemble.instInhabited [Nonempty d] [Inhabited α] : Inhabited (MEnsemble d α) where

```

**def trivial\_pEnsemble**[\[source\]](#)

*The trivial pure-state ensemble of  $\psi$  consists of copies of  $\psi$ , with the  $i$ -th one having probability 1.*

```

def trivial_pEnsemble (ψ : Ket d) (i : α) : PEnsemble d α

```

**theorem trivial\_pEnsemble\_mix**[\[source\]](#)

*The trivial pure-state ensemble of  $\psi$  mixes to  $\psi$*

```
theorem trivial_pEnsemble_mix :  $\forall$  i :  $\alpha$ , mix (toMEnsemble (trivial_pEnsemble  $\psi$  i)) = MState.  
pure  $\psi$ 
```

**theorem trivial\_pEnsemble\_average**[\[source\]](#)

*The average of  $f : \text{Ket } d \rightarrow T$  on a trivial ensemble of  $\psi$  is  $f \psi$*

```
theorem trivial_pEnsemble_average {T : Type _} {U : Type*} [AddCommGroup U] [Module  $\mathbb{R}$  U] [inst  
: Mixable U T] (f : Ket d  $\rightarrow$  T) :  
 $\forall$  i :  $\alpha$ , pure_average f (trivial_pEnsemble  $\psi$  i) = f  $\psi$ 
```

**instance PEnsemble.instInhabited**[\[source\]](#)

```
instance PEnsemble.instInhabited [Nonempty d] [Inhabited  $\alpha$ ] : Inhabited (PEnsemble d  $\alpha$ ) where
```

**def spectral\_ensemble**[\[source\]](#)

*The spectral pure-state ensemble of  $\rho$ . The states are its eigenvectors, and the probabilities, eigenvalues.*

```
def spectral_ensemble ( $\rho$  : MState d) : PEnsemble d d where
```

**theorem spectral\_decomposition\_sum**[\[source\]](#)

```
theorem spectral_decomposition_sum {d  $\mathbb{k}$  : Type*} [Fintype d] [DecidableEq d] [RCLike  $\mathbb{k}$ ]  
{A : Matrix d d  $\mathbb{k}$ } (hA : A.IsHermitian) :  
A =  $\sum$  i, (hA.eigenvalues i) • (Matrix.vecMulVec (hA.eigenvectorBasis i) (star (hA.  
eigenvectorBasis i)))
```

**theorem spectral\_ensemble\_mix**[\[source\]](#)

*The spectral pure-state ensemble of  $\rho$  mixes to  $\rho$*

```
theorem spectral_ensemble_mix { $\rho$  : MState d} : mix ( $\uparrow$ (spectral_ensemble  $\rho$ ) : MEnsemble d d) =  $\rho$ 
```

**2.38 QuantumInfo.Finite.Entanglement**

[QuantumInfo/Finite/Entanglement.lean](#) on GitHub.

**def convex\_roof\_ENNReal**[\[source\]](#)

*Convex roof extension of a function  $g : \text{KetUpToPhase } d \rightarrow \mathbb{R}_{\geq 0}$ , defined as the infimum of all pure-state ensembles of a given  $\rho$  of the average of  $g$  in that ensemble.*

*This is valued in the extended nonnegative real numbers  $\mathbb{R}_{\geq 0\infty}$  to have good properties of the infimum, which come from the fact that  $\mathbb{R}_{\geq 0\infty}$  is a complete lattice. For example, it is necessary for `le_iInf` and `iInf_le_of_le`. However, it is also proven in `convex_roof_ne_top` that the convex roof is never  $\infty$ , so the definition `convex_roof` should be used in most applications.*



```
def convex_roof_ENNReal {d : Type _} [Fintype d] [DecidableEq d] (g : KetUpToPhase d → ℝ≥0) :
  MState d → ℝ≥0∞
```

**def mixed\_convex\_roof\_ENNReal**

[\[source\]](#)

*Mixed convex roof extension of a function  $f : MState\ d \rightarrow \mathbb{R}_{\geq 0}$ , defined as the infimum of all mixed-state ensembles of a given  $\rho$  of the average of  $f$  on that ensemble.*

*This is valued in the extended nonnegative real numbers  $\mathbb{R}_{\geq 0\infty}$  to have good properties of the infimum, which come from the fact that  $\mathbb{R}_{\geq 0\infty}$  is a complete lattice (see `ENNReal.instCompleteLinearOrder`). However, it is also proven in `mixed_convex_roof_ne_top` that the mixed convex roof is never  $\infty$ , so the definition `mixed_convex_roof` should be used in most applications.*

```
def mixed_convex_roof_ENNReal {d : Type _} [Fintype d] [DecidableEq d] (f : MState d → ℝ≥0) :
  MState d → ℝ≥0∞
```

**theorem convex\_roof\_ne\_top**

[\[source\]](#)

*The convex roof extension `convex_roof_ENNReal` is never  $\infty$ .*

```
theorem convex_roof_ne_top : ∀ ρ, convex_roof_ENNReal g ρ ≠ ∞
```

**theorem mixed\_convex\_roof\_ne\_top**

[\[source\]](#)

*The convex roof extension `mixed_convex_roof_ENNReal` is never  $\infty$ .*

```
theorem mixed_convex_roof_ne_top : ∀ ρ, mixed_convex_roof_ENNReal f ρ ≠ ∞
```

**def convex\_roof**

[\[source\]](#)

*Convex roof extension of a function  $g : KetUpToPhase\ d \rightarrow \mathbb{R}_{\geq 0}$ , defined as the infimum of all pure-state ensembles of a given  $\rho$  of the average of  $g$  in that ensemble.*

*This is valued in the nonnegative real numbers  $\mathbb{R}_{\geq 0}$  by applying `ENNReal.toNNReal` to `convex_roof_ENNReal`. Hence, it should be used in proofs alongside `convex_roof_ne_top`.*

```
def convex_roof : MState d → ℝ≥0
```

**def mixed\_convex\_roof**

[\[source\]](#)

*Mixed convex roof extension of a function  $f : MState\ d \rightarrow \mathbb{R}_{\geq 0}$ , defined as the infimum of all mixed-state ensembles of a given  $\rho$  of the average of  $f$  on that ensemble.*

*This is valued in the nonnegative real numbers  $\mathbb{R}_{\geq 0}$  by applying `ENNReal.toNNReal` to `mixed_convex_roof_ENNReal`. Hence, it should be used in proofs alongside `mixed_convex_roof_ne_top`.*

```
def mixed_convex_roof : MState d → ℝ≥0
```

**def convex\_roof\_of\_MState\_fun**

[\[source\]](#)

*Auxiliary function. Convex roof of a function  $f : MState\ d \rightarrow \mathbb{R}_{\geq 0}$  defined over mixed states by restricting  $f$  to pure states, via `pureQ : KetUpToPhase\ d → MState\ d`.*

```
def convex_roof_of_MState_fun : MState d → ℝ≥0
```

### theorem le\_mixed\_convex\_roof

[\[source\]](#)

```
theorem le_mixed_convex_roof (p : MState d) :
  (∀ n > 0, ∀ e : MEnsemble d (Fin n), mix e = p → c ≤ average_NNReal f e) → (c ≤
    mixed_convex_roof f p)
```

### theorem le\_convex\_roof

[\[source\]](#)

```
theorem le_convex_roof (p : MState d) :
  (∀ n > 0, ∀ e : PEnsemble d (Fin n), mix (toMEnsemble e) = p → c ≤ pure_average_NNReal (g ∘
    KetUpToPhase.mk) e) → (c ≤ convex_roof g p)
```

### theorem convex\_roof\_le

[\[source\]](#)

```
theorem convex_roof_le (p : MState d):
  (∃ n > 0, ∃ e : PEnsemble d (Fin n), mix (toMEnsemble e) = p ∧ pure_average_NNReal (g ∘
    KetUpToPhase.mk) e ≤ c) → (convex_roof g p ≤ c)
```

### theorem mixed\_convex\_roof\_le

[\[source\]](#)

```
theorem mixed_convex_roof_le (p : MState d):
  (∃ n > 0, ∃ e : MEnsemble d (Fin n), mix e = p ∧ average_NNReal f e ≤ c) → (mixed_convex_roof f
    p ≤ c)
```

### theorem mixed\_convex\_roof\_le\_convex\_roof

[\[source\]](#)

*The mixed convex roof extension of  $f$  is smaller than or equal to its convex roof extension, since the former minimizes over a larger set of ensembles.*

```
theorem mixed_convex_roof_le_convex_roof : mixed_convex_roof f ≤ convex_roof_of_MState_fun f
```

### theorem convex\_roof\_of\_pure

[\[source\]](#)

*The convex roof extension of  $g : \text{KetUpToPhase } d \rightarrow \mathbb{R}_{\geq 0}$  applied to a pure state  $\psi$  is  $g (\text{KetUpToPhase.mk } \psi)$ .*

```
theorem convex_roof_of_pure (ψ : Ket d) : convex_roof g (pure ψ) = g (KetUpToPhase.mk ψ)
```

### theorem mixed\_convex\_roof\_of\_pure

[\[source\]](#)

*The mixed convex roof extension of  $f : \text{MState } d \rightarrow \mathbb{R}_{\geq 0}$  applied to a pure state  $\psi$  is  $f (\text{pure } \psi)$ .*

```
theorem mixed_convex_roof_of_pure (ψ : Ket d) : mixed_convex_roof f (pure ψ) = f (pure ψ)
```

### def EoF

[\[source\]](#)

*Entanglement of Formation of bipartite systems. It is the convex roof extension of the von Neumann entropy of one of the subsystems (here chosen to be the left one, but see `Entropy.Svn_of_partial_eq`).*

*The function  $S_{v^n} \circ \text{traceRight} \circ \text{pure}$  is phase-invariant because  $\text{pure}$  maps phase-equivalent kets to the same mixed state, so it descends to  $\text{KetUpToPhase}$  via  $\text{pureQ}$ .*

```
def EoF : MState (d1 × d2) → ℝ≥0
```

**theorem traceRight\_pure\_MES**

[\[source\]](#)

```
theorem traceRight_pure_MES (d : Type*) [Fintype d] [DecidableEq d] [Nonempty d] :
  (MState.pure (Ket.MES d)).traceRight = MState.uniform
```

**theorem S<sub>v<sup>n</sup></sub>\_eq\_trace\_cfc**

[\[source\]](#)

```
theorem Svn_eq_trace_cfc {d : Type*} [Fintype d] [DecidableEq d] (ρ : MState d) :
  Svn ρ = (HermitianMat.cfc ρ.M Real.negMulLog).trace
```

**theorem S<sub>v<sup>n</sup></sub>\_ofClassical**

[\[source\]](#)

```
theorem Svn_ofClassical {d : Type*} [Fintype d] [DecidableEq d] (dist : ProbDistribution d) :
  Svn (MState.ofClassical dist) = Hs dist
```

**theorem EoF\_of\_MES**

[\[source\]](#)

*The entanglement of formation of the maximally entangled state with on-site dimension is  $\log()$ .*

```
theorem EoF_of_MES : EoF (pure <| Ket.MES d) = Real.log (Finset.card Finset.univ (α := d))
```

## 2.39 QuantumInfo.Finite.Entropy.DPI

[QuantumInfo/Finite/Entropy/DPI.lean](#) on GitHub.

**def weighted\_norm**

[\[source\]](#)

*The weighted norm  $\|X\|_{p, \sigma}$  defined in the paper.*

```
noncomputable def weighted_norm (p : ℝ) (σ : MState d) (X : Matrix d d ℂ) : ℝ
```

**def spectralNorm\_mat**

[\[source\]](#)

*The spectral norm (operator norm) of a matrix.*

```
noncomputable def spectralNorm_mat (A : Matrix d d ℂ) : ℝ
```

**def weighted\_norm\_infty**

[\[source\]](#)

*The weighted norm for  $p = \infty$ .*

```
noncomputable def weighted_norm_infty (_ : MState d) (X : Matrix d d ℂ) : ℝ
```

**def Gamma** [\[source\]](#)

*The map  $\Gamma_\sigma(X) = \sigma^{\{1/2\}} X \sigma^{\{1/2\}}$ .*

```
noncomputable def Gamma (σ : MState d) (X : Matrix d d ℂ) : Matrix d d ℂ
```

**def Gamma\_inv** [\[source\]](#)

*The inverse map  $\Gamma_\sigma^{\{-1\}}(X) = \sigma^{\{-1/2\}} X \sigma^{\{-1/2\}}$ .*

```
noncomputable def Gamma_inv (σ : MState d) (X : Matrix d d ℂ) : Matrix d d ℂ
```

**def T\_op** [\[source\]](#)

*The operator  $T = \Gamma_{\{\Phi(\sigma)\}^{\{-1\}}} \circ \Phi \circ \Gamma_\sigma$ .*

```
noncomputable def T_op (Φ : CPTPMap d d₂) (σ : MState d) (X : Matrix d d ℂ) : Matrix d₂ d₂ ℂ
```

**def induced\_norm** [\[source\]](#)

*The induced norm of a map  $\Psi: M_d \rightarrow M_{d_2}$  with respect to weighted norms.*

```
noncomputable def induced_norm (p : ℝ) (σ : MState d) (Φ : CPTPMap d d₂) (Ψ : Matrix d d ℂ →  
Matrix d₂ d₂ ℂ) : ℝ
```

**def induced\_norm\_infty\_map** [\[source\]](#)

```
noncomputable def induced_norm_infty_map (σ : MState d) (Φ : CPTPMap d d₂) (Ψ : Matrix d d ℂ →  
Matrix d₂ d₂ ℂ) : ℝ
```

**def T\_map** [\[source\]](#)

```
noncomputable def T_map (σ : MState d) (Φ : CPTPMap d d₂) : MatrixMap d d₂ ℂ
```

**def Gamma\_map** [\[source\]](#)

```
noncomputable def Gamma_map (σ : MState d) : MatrixMap d d ℂ
```

**lemma Gamma\_map\_eq** [\[source\]](#)

```
lemma Gamma_map_eq (σ : MState d) (X : Matrix d d ℂ) :  
Gamma_map σ X = Gamma σ X
```

**lemma Gamma\_map\_CP** [\[source\]](#)

```
lemma Gamma_map_CP (σ : MState d) : (Gamma_map σ).IsCompletelyPositive
```

**def Gamma\_inv\_map** [\[source\]](#)

```
noncomputable def Gamma_inv_map (σ : MState d) : MatrixMap d d ℂ
```

**lemma Gamma\_inv\_map\_eq**[\[source\]](#)

```
lemma Gamma_inv_map_eq (σ : MState d) (X : Matrix d d ℂ) :
  Gamma_inv_map σ X = Gamma_inv σ X
```

**def sigma\_inv\_sqrt**[\[source\]](#)

```
noncomputable def sigma_inv_sqrt (σ : MState d) : Matrix d d ℂ
```

**lemma Gamma\_inv\_map\_eq\_conj**[\[source\]](#)

```
lemma Gamma_inv_map_eq_conj (σ : MState d) :
  Gamma_inv_map σ = MatrixMap.conj (sigma_inv_sqrt σ)
```

**lemma Gamma\_inv\_map\_CP**[\[source\]](#)

```
lemma Gamma_inv_map_CP (σ : MState d) : (Gamma_inv_map σ).IsCompletelyPositive
```

**lemma T\_map\_eq\_comp**[\[source\]](#)

```
lemma T_map_eq_comp (σ : MState d) (Φ : CPTPMap d d₂) :
  T_map σ Φ = (Gamma_inv_map (Φ σ)).comp (Φ.map.comp (Gamma_map σ))
```

**lemma T\_is\_CP**[\[source\]](#)

```
lemma T_is_CP (σ : MState d) (Φ : CPTPMap d d₂) :
  (T_map σ Φ).IsCompletelyPositive
```

**lemma T\_is\_positive**[\[source\]](#)

```
lemma T_is_positive (σ : MState d) (Φ : CPTPMap d d₂) :
  (T_map σ Φ).IsPositive
```

**lemma weighted\_norm\_one\_eq\_trace\_norm\_Gamma**[\[source\]](#)

```
lemma weighted_norm_one_eq_trace_norm_Gamma (σ : MState d) (X : Matrix d d ℂ) :
  weighted_norm 1 σ X = schattenNorm (Gamma σ X) 1
```

**def general\_induced\_norm**[\[source\]](#)

```
noncomputable def general_induced_norm
  (p q : ℝ) (σ : MState d) (σ' : MState d₂)
  (Ψ : MatrixMap d d₂ ℂ) : ℝ
```

**lemma cfc\_mul**[\[source\]](#)

```
lemma HermitianMat.cfc_mul {d : Type*} [Fintype d] [DecidableEq d]
  (A : HermitianMat d ℂ) (f g : ℝ → ℝ) :
  (A.cfc f).mat * (A.cfc g).mat = (A.cfc (fun x => f x * g x)).mat
```

**lemma Gamma\_one**[\[source\]](#)

```
lemma Gamma_one (σ : MState d) : Gamma σ 1 = σ.M.mat
```

**lemma Gamma\_inv\_self**[\[source\]](#)

```
lemma Gamma_inv_self (σ : MState d) (hσ : σ.m.PosDef) :
  Gamma_inv σ σ.M.mat = 1
```

**lemma CPTPMap\_apply\_MState\_M**[\[source\]](#)

```
lemma CPTPMap_apply_MState_M (Φ : CPTPMap d d₂) (σ : MState d) :
  (Φ σ).M.mat = Φ.map σ.M.mat
```

**theorem T\_map\_unital**[\[source\]](#)

```
theorem T_map_unital (σ : MState d) (Φ : CPTPMap d d₂) (hΦσ : (Φ σ).m.PosDef) :
  (T_map σ Φ) 1 = 1
```

**theorem T\_map\_is\_CP\_proof**[\[source\]](#)

```
theorem T_map_is_CP_proof (σ : MState d) (Φ : CPTPMap d d₂) :
  (T_map σ Φ).IsCompletelyPositive
```

**lemma Gamma\_Gamma\_inv**[\[source\]](#)

```
lemma Gamma_Gamma_inv (σ : MState d) (hσ : σ.m.PosDef) (X : Matrix d d ℂ) :
  Gamma σ (Gamma_inv σ X) = X
```

**theorem le\_smul\_one\_imp\_eigenvalues\_le**[\[source\]](#)

```
theorem HermitianMat.le_smul_one_imp_eigenvalues_le (A : HermitianMat d ℂ) (M : ℝ)
  (h : A ≤ M • (1 : HermitianMat d ℂ)) (i : d) :
  A.H.eigenvalues i ≤ M
```

**theorem eigenvalues\_le\_imp\_le\_smul\_one**[\[source\]](#)

```
theorem HermitianMat.eigenvalues_le_imp_le_smul_one (A : HermitianMat d ℂ) (M : ℝ)
  (h : ∀ i, A.H.eigenvalues i ≤ M) :
  A ≤ M • (1 : HermitianMat d ℂ)
```

**theorem block\_matrix\_posSemidef**[\[source\]](#)

*The block matrix  $\begin{bmatrix} 1 & X \\ X^\dagger & X^\dagger X \end{bmatrix}$  is positive semidefinite.*

```
theorem block_matrix_posSemidef {m n k : Type*} [Fintype m] [Fintype n] [Fintype k]
  (X : Matrix k n ℂ) (Y : Matrix k m ℂ) :
  (Matrix.fromBlocks (YH * Y) (YH * X) (XH * Y) (XH * X)).PosSemidef
```

**theorem block\_matrix\_one\_posSemidef**[\[source\]](#)

```

theorem block_matrix_one_posSemidef {m n : Type*} [Fintype m] [Fintype n] [DecidableEq m]
  (X : Matrix m n ℂ) :
  (Matrix.fromBlocks 1 X XH (XH * X)).PosSemidef

```

**theorem sandwichedRenyiEntropy\_DPI**[\[source\]](#)

*The Data Processing Inequality for the Sandwiched Renyi relative entropy. Proved in <https://arxiv.org/pdf/1306.5920>. Seems kind of involved.*

```

theorem sandwichedRenyiEntropy_DPI (hα : 1 ≤ α) (ρ σ : MState d) (Φ : CPTMap d dz) :
  D_ α(Φ ρ || Φ σ) ≤ D_ α(ρ || σ)

```

**axiom sandwichedRenyiEntropy\_DPI\_ax**[\[source\]](#)

```

axiom sandwichedRenyiEntropy_DPI_ax : type_of% @sandwichedRenyiEntropy_DPI

```

**2.40 QuantumInfo.Finite.Entropy.Relative**

[QuantumInfo/Finite/Entropy/Relative.lean](#) on GitHub.

**theorem sandwiched\_trace\_pos**[\[source\]](#)

```

private theorem sandwiched_trace_pos (h : σ.M.ker ≤ ρ.M.ker) :
  0 < ((ρ.M.conj (σ.M ^ ((1 - α)/(2 * α))) ).mat) ^ α).trace

```

**lemma weighted\_jensen\_rpow**[\[source\]](#)

*Weighted Jensen inequality: for weights  $w_j \geq 0$  with  $\sum w_j = 1$ , values  $b_j \geq 0$ , and  $q \geq 1$ :  $(\sum_j w_j * b_j)^q \leq \sum_j w_j * b_j^q$ .*

*This is the special case of `Real.rpow_arith_mean_le_arith_mean_rpow` applied to `Finset.univ`*

```

lemma weighted_jensen_rpow (b w : d → ℝ) (q : ℝ)
  (hb : ∀ j, 0 ≤ b j) (hw : ∀ j, 0 ≤ w j) (hsum : ∑ j, w j = 1) (hq : 1 ≤ q) :
  (∑ j, w j * b j) ^ q ≤ ∑ j, w j * b j ^ q

```

**lemma doubly\_stochastic\_holder**[\[source\]](#)

*Doubly stochastic Hölder inequality: for nonneg  $a, b$ , doubly stochastic  $w$ , and conjugate  $p, q > 1$ :  $\sum_{ij} a_i * b_j * w_{ij} \leq (\sum_i a_i^p)^{1/p} * (\sum_j b_j^q)^{1/q}$ .*

```

lemma doubly_stochastic_holder (a b : d → ℝ) (w : d → d → ℝ)
  (ha : ∀ i, 0 ≤ a i) (hb : ∀ j, 0 ≤ b j)
  (hw : ∀ i j, 0 ≤ w i j)
  (hrow : ∀ i, ∑ j, w i j = 1) (hcol : ∀ j, ∑ i, w i j = 1)
  (p q : ℝ) (hp : 1 < p) (hpq : 1/p + 1/q = 1) :
  ∑ i, ∑ j, a i * b j * w i j ≤ (∑ i, a i ^ p) ^ (1/p) * (∑ j, b j ^ q) ^ (1/q)

```

**lemma inner\_le\_trace\_rpow\_mul**[\[source\]](#)

*Hermitian trace Hölder inequality: for PSD  $A, B$  and conjugate exponents  $p, q > 1$ ,  $\langle\langle A, B \rangle\rangle \leq \text{Tr}[A^p]^{1/p} * \text{Tr}[B^q]^{1/q}$ .*

```

lemma HermitianMat.inner_le_trace_rpow_mul
  {A B : HermitianMat d ℂ} (hA : 0 ≤ A) (hB : 0 ≤ B)
  (p q : ℝ) (hp : 1 < p) (hpq : 1/p + 1/q = 1) :
  ⟨⟨A, B⟩⟩_ℝ ≤ (A ^ p).trace ^ (1/p) * (B ^ q).trace ^ (1/q)

```

**lemma rpow\_le\_one'** [\[source\]](#)

```

lemma MState.rpow_le_one' {r : ℝ} (hr : 0 < r) : σ.M ^ r ≤ 1

```

**lemma eigenvalues\_le\_one\_of\_le\_one** [\[source\]](#)

*If  $A \geq 0$  and  $A \leq 1$ , then each eigenvalue of  $A$  is in  $[0, 1]$ .*

```

lemma HermitianMat.eigenvalues_le_one_of_le_one
  (A : HermitianMat d ℂ) (hA1 : A ≤ 1) (i : d) :
  A.H.eigenvalues i ≤ 1

```

**lemma trace\_rpow\_le\_trace\_of\_le\_one** [\[source\]](#)

*For positive  $A \leq 1$  and  $p \geq 1$ ,  $\text{Tr}[A^p] \leq \text{Tr}[A]$ .*

```

lemma HermitianMat.trace_rpow_le_trace_of_le_one
  (A : HermitianMat d ℂ) (hA : 0 ≤ A) (hA1 : A ≤ 1)
  (p : ℝ) (hp : 1 ≤ p) :
  (A ^ p).trace ≤ A.trace

```

**lemma trace\_conj\_rpow\_eq\_inner** [\[source\]](#)

```

private lemma trace_conj_rpow_eq_inner (hα₀ : 0 < α) (hα : α < 1) :
  ((p.M ^ α).conj (σ.M ^ ((1 - α) / (2 * α) * α)).mat).trace = ⟨⟨p.M ^ α, σ.M ^ (1 - α)⟩⟩_ℝ

```

**lemma inner\_rpow\_le\_one** [\[source\]](#)

```

private lemma inner_rpow_le_one (hα₀ : 0 < α) (hα : α < 1) :
  ⟨⟨p.M ^ α, σ.M ^ (1 - α)⟩⟩_ℝ ≤ 1

```

**theorem sandwiched\_trace\_of\_lt\_1** [\[source\]](#)

```

private theorem sandwiched_trace_of_lt_1 (hα₀ : 0 < α) (hα : α < 1) :
  ((p.M.conj (σ.M ^ ((1 - α) / (2 * α))) ).mat) ^ α).trace ≤ 1

```

**lemma rpow\_neg\_mul\_rpow\_eq\_supportProj** [\[source\]](#)

*For PSD  $A$  and  $p \neq 0$ ,  $A^{\{-p\}} * A^p = \text{HermitianMat.supportProj } A$ .*

```

lemma HermitianMat.rpow_neg_mul_rpow_eq_supportProj
  {A : HermitianMat d ℂ} (hA : 0 ≤ A) {p : ℝ} (hp : p ≠ 0) :
  (A ^ (-p)).mat * (A ^ p).mat = A.supportProj.mat

```



**lemma supportProj\_mul\_self**[\[source\]](#)

```
lemma HermitianMat.supportProj_mul_self (A : HermitianMat d ℂ) :
  A.supportProj.mat * A.mat = A.mat
```

**lemma inner\_supportProj\_self**[\[source\]](#)

```
lemma HermitianMat.inner_supportProj_self (A : HermitianMat d ℂ) :
  ⟨⟨A, A.supportProj⟩⟩ = A.trace
```

**lemma mul\_supportProj\_of\_ker\_le**[\[source\]](#)

```
lemma HermitianMat.mul_supportProj_of_ker_le {A B : HermitianMat d ℂ}
  (h : LinearMap.ker B.lin.toLinearMap ≤ LinearMap.ker A.lin.toLinearMap) :
  A.mat * B.supportProj.mat = A.mat
```

**lemma inner\_supportProj\_of\_ker\_le**[\[source\]](#)

```
lemma HermitianMat.inner_supportProj_of_ker_le {A B : HermitianMat d ℂ}
  (h : LinearMap.ker B.lin.toLinearMap ≤ LinearMap.ker A.lin.toLinearMap) :
  ⟨⟨A, B.supportProj⟩⟩ = A.trace
```

**lemma supportProj\_inner\_density**[\[source\]](#)

```
lemma supportProj_inner_density (h : σ.M.ker ≤ p.M.ker) :
  ⟨⟨σ.M.supportProj, p.M⟩⟩_ℝ = 1
```

**lemma sandwiched\_inner\_eq\_one**[\[source\]](#)

```
private lemma sandwiched_inner_eq_one (h : σ.M.ker ≤ p.M.ker) (t : ℝ) :
  ⟨⟨p.M.conj (σ.M ^ t).mat, σ.M ^ (-2 * t)⟩⟩_ℝ = 1
```

**theorem sandwiched\_trace\_of\_gt\_1**[\[source\]](#)

```
private theorem sandwiched_trace_of_gt_1 (h : σ.M.ker ≤ p.M.ker) (hα : α > 1) :
  1 ≤ ((p.M.conj (σ.M ^ ((1 - α)/(2 * α))) ).mat) ^ α).trace
```

**theorem sandwichedRelEntropy\_nonneg\_α\_lt\_1**[\[source\]](#)

```
private theorem sandwichedRelEntropy_nonneg_α_lt_1 (h : σ.M.ker ≤ p.M.ker) (hα0 : 0 < α) (hα :
  α < 1) :
  0 ≤ ((p.M.conj (σ.M ^ ((1 - α)/(2 * α))) ).mat) ^ α).trace.log / (α - 1)
```

**theorem sandwichedRelEntropy\_nonneg\_α\_gt\_1**[\[source\]](#)

```
private theorem sandwichedRelEntropy_nonneg_α_gt_1 (h : σ.M.ker ≤ p.M.ker) (hα : α > 1) :
  0 ≤ ((p.M.conj (σ.M ^ ((1 - α)/(2 * α))) ).mat) ^ α).trace.log / (α - 1)
```

**lemma trace\_at\_one**[\[source\]](#)

```
private lemma sandwichedRelEntropy.trace_at_one (p σ : MState d) :
  ((p.M.conj (σ.M ^ ((1 - (1:ℝ)) / (2 * (1:ℝ))))).mat) ^ (1:ℝ)).trace = 1
```

**lemma hasDerivAt\_trace\_rpow\_at\_one**[\[source\]](#)

```
private lemma hasDerivAt_trace_rpow_at_one (B : HermitianMat d C) (hB : 0 ≤ B) :
  HasDerivAt (fun α : ℝ => (B ^ α).trace) ⟨⟨B, B.log⟩⟩ 1
```

**lemma trace\_conj\_eq\_inner\_rpow**[\[source\]](#)

```
private lemma trace_conj_eq_inner_rpow {p σ : MState d} {t : ℝ} (ht : t ≠ 0) :
  (p.M.conj (σ.M ^ t).mat).trace = ⟨⟨p.M, σ.M ^ (2 * t)⟩⟩
```

**def eigenWeight**[\[source\]](#)

```
private def eigenWeight (p σ : MState d) (i : d) : ℝ
```

**lemma inner\_cfc\_eq\_sum\_eigenWeight**[\[source\]](#)

```
private lemma inner_cfc_eq_sum_eigenWeight (p σ : MState d) (f : ℝ → ℝ) :
  ⟨⟨p.M, σ.M.cfc f⟩⟩ = ∑ i, f (σ.M.H.eigenvalues i) * eigenWeight p σ i
```

**lemma eigenWeight\_nonneg**[\[source\]](#)

```
private lemma eigenWeight_nonneg (p σ : MState d) (i : d) : 0 ≤ eigenWeight p σ i
```

**lemma eigenWeight\_zero\_of\_eigenvalue\_zero**[\[source\]](#)

```
private lemma eigenWeight_zero_of_eigenvalue_zero {i : d} (hσ : σ.M.ker ≤ p.M.ker)
  (hei : σ.M.H.eigenvalues i = 0) :
  eigenWeight p σ i = 0
```

**lemma hasDerivAt\_inner\_rpow\_at\_zero**[\[source\]](#)

```
private lemma hasDerivAt_inner_rpow_at_zero (h : σ.M.ker ≤ p.M.ker) :
  HasDerivAt (fun u : ℝ => ⟨⟨p.M, σ.M ^ u⟩⟩) ⟨⟨p.M, σ.M.log⟩⟩ 0
```

**lemma hasDerivAt\_trace\_conj\_at\_one**[\[source\]](#)

```
private lemma hasDerivAt_trace_conj_at_one {p σ : MState d}
  (h : σ.M.ker ≤ p.M.ker) :
  HasDerivAt
    (fun α : ℝ => ((p.M.conj (σ.M ^ ((1 - α) / (2 * α))).mat)).trace)
    (-⟨⟨p.M, σ.M.log⟩⟩)
    1
```

**lemma scalar\_rpow\_cross\_term**[\[source\]](#)

```
private lemma scalar_rpow_cross_term {b : ℝ → ℝ} {c : ℝ}
  (hb : HasDerivAt b (deriv b 1) 1) (hc : b 1 = c) (hc_pos : 0 < c) :
  HasDerivAt (fun α => b α ^ α - b α) (c * Real.log c) 1
```

**lemma hasDerivAt\_trace\_rpow\_sub\_trace**[\[source\]](#)

For a PSD matrix  $A$ ,  $\text{Tr}[A^s] - \text{Tr}[A]$  has derivative  $\langle\langle A, \log A \rangle\rangle$  at  $s = 1$ . This generalizes `hasDerivAt_trace_rpow_at_one` to give the derivative of the difference  $\text{Tr}[A^s] - \text{Tr}[A]$ , which equals  $\langle\langle A, \log A \rangle\rangle$  since  $d/ds \text{Tr}[A] = 0$ .

```
private lemma hasDerivAt_trace_rpow_sub_trace (A : HermitianMat d ℂ) (hA : 0 ≤ A) :
  HasDerivAt (fun s : ℝ => (A ^ s).trace - A.trace) ⟨⟨A, A.log⟩⟩ 1
```

**abbrev B\_of**[\[source\]](#)

```
private abbrev B_of (ρ σ : MState d) (α : ℝ) : HermitianMat d ℂ
```

**lemma B\_of\_one**[\[source\]](#)

```
private lemma B_of_one (ρ σ : MState d) : B_of ρ σ 1 = ρ.M
```

**lemma B\_of\_nonneg**[\[source\]](#)

```
private lemma B_of_nonneg (ρ σ : MState d) (α : ℝ) : 0 ≤ B_of ρ σ α
```

**lemma trace\_rpow\_sub\_trace\_at\_one**[\[source\]](#)

```
private lemma trace_rpow_sub_trace_at_one (M : HermitianMat d ℂ) :
  (M ^ (1 : ℝ)).trace - M.trace = 0
```

**lemma cross\_term\_at\_one**[\[source\]](#)

```
private lemma cross_term_at_one (ρ σ : MState d) :
  ((B_of ρ σ 1) ^ (1 : ℝ)).trace - (B_of ρ σ 1).trace
  - (ρ.M ^ (1 : ℝ)).trace + 1 = 0
```

**lemma scalar\_rpow\_cross\_term\_of\_continuous**[\[source\]](#)

```
private lemma scalar_rpow_cross_term_of_continuous {b : ℝ → ℝ} {c : ℝ}
  (hb_cont : ContinuousAt b 1) (hc : b 1 = c) (hc_pos : 0 < c)
  (hb_pos : ∀f α in nhds 1, 0 < b α) :
  HasDerivAt (fun α => b α ^ α - b α) (c * Real.log c) 1
```

**lemma scalar\_rpow\_cross\_term\_of\_continuous\_zero**[\[source\]](#)

```
private lemma scalar_rpow_cross_term_of_continuous_zero {b : ℝ → ℝ}
  (hb_cont : ContinuousAt b 1) (hc : b 1 = 0)
  (hb_nonneg : ∀f α in nhds 1, 0 ≤ b α) :
  HasDerivAt (fun α => b α ^ α - b α) 0 1
```

**lemma conj\_supportProj\_eq\_of\_ker\_le**[\[source\]](#)

If  $\ker A \leq \ker \rho M$ , then conjugating  $\rho M$  by the support projection of  $A$  gives back  $\rho M$ . This is because  $\rho M$  is supported entirely on the support (= range) of  $A$ .

```
private lemma conj_supportProj_eq_of_ker_le (A pM : HermitianMat d ℂ) (hker : A.ker ≤ pM.ker) :
  pM.conj (A.supportProj).mat = pM
```

### lemma rpow\_tendsto\_supportProj

[\[source\]](#)

For a PSD matrix  $A$ , the function  $r \mapsto A^{\wedge r}$  converges to  $A.\text{supportProj}$  as  $r \rightarrow 0$  through nonzero values. On positive eigenvalues  $\lambda$ ,  $\lambda^{\wedge r} \rightarrow 1$ . On zero eigenvalues,  $0^{\wedge r} = 0$  for  $r \neq 0$ . So the limit is the support projection (indicator of nonzero eigenvalues).

```
private lemma rpow_tendsto_supportProj
  (A : HermitianMat d ℂ) :
  Filter.Tendsto (fun r : ℝ => A ^ r) (nhdsWithin 0 {(0 : ℝ)}c) (nhds A.supportProj)
```

### lemma conj\_rpow\_continuousAt\_zero

[\[source\]](#)

For PSD matrices  $A, \rho$  with  $A.\text{ker} \leq \rho.\text{ker}$ , the function  $r \mapsto \rho.\text{conj} (A^{\wedge r}).\text{mat}$  is continuous at  $r = 0$ . Even though  $A^{\wedge r}$  is discontinuous at  $r = 0$  when  $A$  has zero eigenvalues, the kernel condition ensures the conj "kills" the discontinuity.

```
private lemma conj_rpow_continuousAt_zero
  (A pM : HermitianMat d ℂ)
  (hker : A.ker ≤ pM.ker) :
  ContinuousAt (fun r : ℝ => pM.conj (A ^ r).mat) 0
```

### lemma B\_of\_continuousAt

[\[source\]](#)

```
private lemma B_of_continuousAt (p σ : MState d) (h : σ.M.ker ≤ p.M.ker) :
  ContinuousAt (B_of p σ) 1
```

### lemma trace\_cfc\_sub\_le

[\[source\]](#)

```
private lemma trace_cfc_sub_le (A : HermitianMat d ℂ) (f g : ℝ → ℝ) :
  |(A.cfc f).trace - (A.cfc g).trace| ≤
  (Fintype.card d : ℝ) * (⊔ i, |f (A.H.eigenvalues i) - g (A.H.eigenvalues i)|)
```

### lemma eigenvalues\_bounded\_near

[\[source\]](#)

Eigenvalues of  $M(\alpha)$  are uniformly bounded near  $\alpha = 1$ .

```
private lemma eigenvalues_bounded_near {M : ℝ → HermitianMat d ℂ}
  (hM_nonneg : ∀f α in nhds 1, 0 ≤ M α)
  (hM_cont : ContinuousAt M 1) :
  ∃ K > 0, ∀f α in nhds 1, ∀ i, 0 ≤ (M α).H.eigenvalues i ∧ (M α).H.eigenvalues i ≤ K
```

### lemma rpow\_slope\_tendsto\_uniformly

[\[source\]](#)

```
private lemma rpow_slope_tendsto_uniformly (K : ℝ) :
  ∀ ε > 0, ∃ δ > 0, ∀ h : ℝ, 0 < |h| → |h| < δ →
  ∀ x ∈ Set.Icc 0 K, |(x ^ (1 + h) - x) / h - x * Real.log x| < ε
```

**lemma trace\_cfc\_tendsto\_of\_tendsto**[\[source\]](#)

```
private lemma trace_cfc_tendsto_of_tendsto (f : ℝ → ℝ)
  (hf : ContinuousOn f (Set.Ici 0))
  {M : ℝ → HermitianMat d ℂ}
  (hM_cont : ContinuousAt M 1) (hM_nonneg : ∀f α in nhds 1, 0 ≤ M α)
  (hM_one : M 1 = p.M) :
  Filter.Tendsto (fun α => ((M α).cfc f).trace) (nhds 1) (nhds ((p.M.cfc f).trace))
```

**lemma cross\_term\_slope\_tendsto\_zero**[\[source\]](#)

```
private lemma cross_term_slope_tendsto_zero
  {M : ℝ → HermitianMat d ℂ}
  (hM_nonneg : ∀f α in nhds 1, 0 ≤ M α)
  (hM_cont : ContinuousAt M 1)
  (hM_one : M 1 = p.M) :
  Filter.Tendsto
    (fun h : ℝ => ((M (1 + h) ^ (1 + h)).trace - (M (1 + h)).trace
      - (p.M ^ (1 + h)).trace + p.M.trace) / h)
    (nhdsWithin 0 {0}ᶜ)
    (nhds 0)
```

**lemma hasDerivAt\_trace\_rpow\_sub\_trace\_variable\_base**[\[source\]](#)

For a differentiable family of PSD matrices  $M(\alpha)$  with  $M(1)$  having eigenvalues  $p_i$ , the function  $\alpha \mapsto \text{Tr}[M(\alpha)^\alpha] - \text{Tr}[M(\alpha)]$  has derivative  $\langle\langle M(1), M(1).\log \rangle\rangle$  at  $\alpha = 1$ . This is because at  $\alpha = 1$ , the function  $x^\alpha - x$  has zero  $x$ -derivative (since  $d/dx(x^1) = 1$ ), so only the  $s$ -derivative contributes, giving the same answer as for fixed eigenvalues.

```
private lemma hasDerivAt_trace_rpow_sub_trace_variable_base
  {M : ℝ → HermitianMat d ℂ}
  (hM_nonneg : ∀f α in nhds 1, 0 ≤ M α)
  (hM_cont : ContinuousAt M 1)
  (hM_one : M 1 = p.M) :
  HasDerivAt (fun α : ℝ => (M α ^ α).trace - (M α).trace) ⟨⟨p.M, p.M.log⟩⟩ 1
```

**lemma rpow\_trace\_cross\_term\_vanishes**[\[source\]](#)

The cross term in the derivative decomposition vanishes: the function  $\alpha \mapsto \text{Tr}[B(\alpha)^\alpha] - \text{Tr}[B(\alpha)] - \text{Tr}[p^\alpha] + 1$  has derivative 0 at  $\alpha = 1$ . This is because at  $\alpha=1$ ,  $B^1 = B$ , so  $\partial/\partial B \text{Tr}[B^\alpha] = \text{Tr}[\cdot]$  (the trace is linear), making the cross term (variation in  $B$  times variation in  $\alpha$ ) vanish.

```
private lemma rpow_trace_cross_term_vanishes {p σ : MState d}
  (h : σ.M.ker ≤ p.M.ker) :
  HasDerivAt
    (fun α : ℝ => ((p.M.conj (σ.M ^ ((1 - α) / (2 * α))).mat) ^ α).trace
      - (p.M.conj (σ.M ^ ((1 - α) / (2 * α))).mat).trace
      - (p.M ^ α).trace + 1)
    0
    1
```

**theorem hasDerivAt\_trace\_at\_one**[\[source\]](#)

```

private theorem sandwichedRelEntropy.hasDerivAt_trace_at_one {p σ : MState d}
  (h : σ.M.ker ≤ p.M.ker) :
  HasDerivAt
    ((fun α : ℝ => ((p.M.conj (σ.M ^ ((1 - α) / (2 * α))))).mat) ^ α).trace)
    ⟨⟨p.M, p.M.log - σ.M.log⟩⟩
    1

```

### theorem limit\_at\_one

[\[source\]](#)

*The key limit: as  $\alpha \rightarrow 1$ ,  $\log(\text{Tr}[(\rho.\text{conj } \sigma^t)^\alpha]) / (\alpha-1) \rightarrow \langle\langle \rho, \log \rho - \log \sigma \rangle\rangle$ , where  $t = (1-\alpha)/(2\alpha)$ . Derived from `hasDerivAt_trace_at_one` via L'Hôpital (or equivalently,  $\log(1+x)/x \rightarrow 1$  and  $(f(\alpha)-1)/(\alpha-1) \rightarrow f'(1)$ ).*

```

private theorem sandwichedRelEntropy.limit_at_one (p σ : MState d)
  (h : σ.M.ker ≤ p.M.ker) :
  Filter.Tendsto
    ((fun α : ℝ => ((p.M.conj (σ.M ^ ((1 - α) / (2 * α))))).mat) ^ α).trace.log / (α - 1))
    (nhdsWithin 1 (Set.Ioi 0 \ {1}))
    (nhds ⟨⟨p.M, p.M.log - σ.M.log⟩⟩)

```

### theorem inner\_log\_sub\_log\_nonneg

[\[source\]](#)

```

theorem inner_log_sub_log_nonneg (h : σ.M.ker ≤ p.M.ker) :
  0 ≤ ⟨⟨p.M, p.M.log - σ.M.log⟩⟩

```

### theorem sandwichedRelEntropy\_nonneg

[\[source\]](#)

```

theorem sandwichedRelEntropy_nonneg {α : ℝ} (hα : 0 < α) (h : σ.M.ker ≤ p.M.ker) :
  0 ≤ if α = 1 then ⟨⟨p.M, p.M.log - σ.M.log⟩⟩
    else ((p.M.conj (σ.M ^ ((1 - α)/(2 * α))) ).mat) ^ α).trace.log / (α - 1)

```

### lemma ker\_kron\_le\_of\_le

[\[source\]](#)

*If the kernels of the components are contained, then the kernel of the Kronecker product is contained.*

```

lemma ker_kron_le_of_le {d1 d2 : Type*} [Fintype d1] [Fintype d2] [DecidableEq d1] [DecidableEq d2]
  (A C : Matrix d1 d1 ℂ) (B D : Matrix d2 d2 ℂ)
  (hA : LinearMap.ker A.toEuclideanLin ≤ LinearMap.ker C.toEuclideanLin)
  (hB : LinearMap.ker B.toEuclideanLin ≤ LinearMap.ker D.toEuclideanLin) :
  LinearMap.ker (A.kronecker B).toEuclideanLin ≤ LinearMap.ker (C.kronecker D).toEuclideanLin

```

### lemma ker\_le\_of\_ker\_kron\_le\_left

[\[source\]](#)

*If the kernel of a product state is contained in another, the left component kernel is contained.*

```

lemma ker_le_of_ker_kron_le_left (p1 σ1 : MState d1) (p2 σ2 : MState d2)
  (h : (σ1 ⊗M σ2).M.ker ≤ (p1 ⊗M p2).M.ker) :
  σ1.M.ker ≤ p1.M.ker

```

**lemma ker\_le\_of\_ker\_kron\_le\_right**[\[source\]](#)

*If the kernel of a product state is contained in another, the right component kernel is contained.*

```
lemma ker_le_of_ker_kron_le_right (p₁ σ₁ : MState d₁) (p₂ σ₂ : MState d₂)
  (h : (σ₁ ⊗M σ₂).M.ker ≤ (p₁ ⊗M p₂).M.ker) :
  σ₂.M.ker ≤ p₂.M.ker
```

**lemma ker\_prod\_le\_iff**[\[source\]](#)

*The kernel of a product state is contained in another product state's kernel iff the individual kernels are contained.*

```
lemma ker_prod_le_iff (p₁ σ₁ : MState d₁) (p₂ σ₂ : MState d₂) :
  (σ₁ ⊗M σ₂).M.ker ≤ (p₁ ⊗M p₂).M.ker ↔ σ₁.M.ker ≤ p₁.M.ker ∧ σ₂.M.ker ≤ p₂.M.ker
```

**lemma inner\_kron**[\[source\]](#)

```
lemma HermitianMat.inner_kron
  (A : HermitianMat d₁ ℂ) (B : HermitianMat d₂ ℂ) (C : HermitianMat d₁ ℂ) (D : HermitianMat d₂ ℂ) :
  ⟨⟨A ⊗k B, C ⊗k D⟩⟩ = ⟨⟨A, C⟩⟩ * ⟨⟨B, D⟩⟩
```

**lemma continuousOn\_rpow\_uniform**[\[source\]](#)

```
lemma continuousOn_rpow_uniform {K : Set ℝ} (hK : IsCompact K) :
  ContinuousOn (fun r : ℝ ↦ UniformOnFun.ofFun {K} (fun t : ℝ ↦ t ^ r)) (Set.Ioi 0)
```

**theorem sandwichedRelEntropy\_additive\_alpha\_one\_aux**[\[source\]](#)

```
theorem sandwichedRelEntropy_additive_alpha_one_aux (p₁ σ₁ : MState d₁) (p₂ σ₂ : MState d₂)
  (h1 : σ₁.M.ker ≤ p₁.M.ker) (h2 : σ₂.M.ker ≤ p₂.M.ker) :
  ⟨⟨(p₁ ⊗M p₂).M, (p₁ ⊗M p₂).M.log - (σ₁ ⊗M σ₂).M.log⟩⟩ =
  ⟨⟨p₁.M, p₁.M.log - σ₁.M.log⟩⟩_ℝ + ⟨⟨p₂.M, p₂.M.log - σ₂.M.log⟩⟩
```

**def SandwichedRelEntropy**[\[source\]](#)

*The Sandwiched Renyi Relative Entropy, defined with  $\ln$  (nits). Note that at  $\alpha = 1$  this definition switch to the standard Relative Entropy, for continuity. For  $\alpha \leq 0$ , this gives junk value 0. (There is no conventional value for  $\alpha < 0$ ; there is a continuous limit at  $\alpha = 0$ , but it is complicated and unneeded at the moment.)*

```
def SandwichedRelEntropy (α : ℝ) (p σ : MState d) : ENNReal
```

**def qRelativeEnt**[\[source\]](#)

*The quantum relative entropy  $D(p||\sigma) := \text{Tr}[p (\log p - \log \sigma)]$ . Also called the Umegaki quantum relative entropy, when it's necessary to distinguish from other relative entropies.*

```
def qRelativeEnt (p σ : MState d) : ENNReal
```

**theorem sandwichedRelEntropy\_additive\_alpha\_one**[\[source\]](#)

*The Sandwiched Renyi Relative entropy is additive for  $\alpha=1$  (standard relative entropy).*

```
private theorem sandwichedRelEntropy_additive_alpha_one (p₁ σ₁ : MState d₁) (p₂ σ₂ : MState d₂) :
  D_1(p₁ ⊗ p₂ || σ₁ ⊗ σ₂) = D_1(p₁ || σ₁) + D_1(p₂ || σ₂)
```

**lemma sandwiched\_term\_product**[\[source\]](#)

```
lemma sandwiched_term_product (p₁ σ₁ : MState d₁) (p₂ σ₂ : MState d₂) (α β : ℝ) :
  (((p₁ ⊗ p₂).M.conj ((σ₁ ⊗ σ₂).M ^ β).mat) ^ α).trace =
  ((p₁.M.conj (σ₁.M ^ β).mat) ^ α).trace * ((p₂.M.conj (σ₂.M ^ β).mat) ^ α).trace
```

**theorem sandwichedRelEntropy\_additive\_alpha\_ne\_one**[\[source\]](#)

```
theorem sandwichedRelEntropy_additive_alpha_ne_one {α : ℝ} (hα : α ≠ 1) (p₁ σ₁ : MState d₁) (
  p₂ σ₂ : MState d₂) :
  D_α(p₁ ⊗ p₂ || σ₁ ⊗ σ₂) = D_α(p₁ || σ₁) + D_α(p₂ || σ₂)
```

**theorem sandwichedRelEntropy\_additive**[\[source\]](#)

*The Sandwiched Renyi Relative entropy is additive when the inputs are product states*

```
@[simp]
theorem sandwichedRelEntropy_additive (α) (p₁ σ₁ : MState d₁) (p₂ σ₂ : MState d₂) :
  D_α(p₁ ⊗ p₂ || σ₁ ⊗ σ₂) = D_α(p₁ || σ₁) + D_α(p₂ || σ₂)
```

**theorem qRelativeEnt\_additive**[\[source\]](#)

*The quantum relative entropy is additive when the inputs are product states*

```
@[simp]
theorem qRelativeEnt_additive (p₁ σ₁ : MState d₁) (p₂ σ₂ : MState d₂) :
  D(p₁ ⊗ p₂ || σ₁ ⊗ σ₂) = D(p₁ || σ₁) + D(p₂ || σ₂)
```

**theorem sandwichedRelEntropy\_relabel**[\[source\]](#)

```
@[simp]
theorem sandwichedRelEntropy_relabel (p σ : MState d) (e : d₂ = d) :
  D_α(p.relabel e || σ.relabel e) = D_α(p || σ)
```

**theorem sandwichedRelEntropy\_self**[\[source\]](#)

```
@[simp]
theorem sandwichedRelEntropy_self (hα : 0 < α) (p : MState d) :
```

**theorem sandwichedRelEntropy\_ne\_top**[\[source\]](#)

```
@[aesop (rule_sets := [finiteness]) unsafe apply]
theorem sandwichedRelEntropy_ne_top {p σ : MState d} [σ.M.NonSingular] : D_α(p || σ) ≠ τ
```



**lemma continuous0n\_exponent**[\[source\]](#)

```
@[fun_prop]
lemma continuous0n_exponent : Continuous0n (fun  $\alpha : \mathbb{R} \Rightarrow (1 - \alpha) / (2 * \alpha)$ ) (Set.Ioi 0)
```

**lemma continuous0n\_cpow\_const\_Ioi**[\[source\]](#)

```
@[fun_prop]
lemma Complex.continuous0n_cpow_const_Ioi (z :  $\mathbb{C}$ ) :
  Continuous0n (fun  $r : \mathbb{R} \Rightarrow z ^ (r : \mathbb{C})$ ) (Set.Ioi 0)
```

**lemma maps\_to\_Iio\_of\_Ioi\_1**[\[source\]](#)

*The function  $\alpha \mapsto (1 - \alpha) / (2 * \alpha)$  maps the interval  $(1, \infty)$  to  $(-\infty, 0)$ .*

```
lemma maps_to_Iio_of_Ioi_1 : Set.MapsTo (fun  $\alpha : \mathbb{R} \Rightarrow (1 - \alpha) / (2 * \alpha)$ ) (Set.Ioi 1) (Set.Iio 0)
```

**theorem frontier\_singleton**[\[source\]](#)

```
@[simp]
theorem frontier_singleton {X : Type*} [TopologicalSpace X] [T1Space X] [PerfectSpace X]
  (p : X) : frontier {p} = {p}
```

**theorem continuous0n\_Ioi\_1\_aux**[\[source\]](#)

```
private theorem sandwichedRelEntropy.continuous0n_Ioi_1_aux (p  $\sigma$  : MState d) :
  Continuous0n (fun ( $\alpha : \mathbb{R}$ )  $\mapsto ((\text{HermitianMat.conj } (\sigma.M ^ ((1 - \alpha) / (2 * \alpha))).mat) p.M ^ \alpha)$ )
  (Set.Ioi 1)
```

**theorem continuous0n\_Ioi\_1**[\[source\]](#)

```
private theorem sandwichedRelEntropy.continuous0n_Ioi_1 (p  $\sigma$  : MState d) :
  Continuous0n (fun  $\alpha \Rightarrow D_{\sim} \alpha(\rho||\sigma)$ ) (Set.Ioi 1)
```

**theorem continuous0n\_Ioo\_0\_1\_aux**[\[source\]](#)

```
private theorem sandwichedRelEntropy.continuous0n_Ioo_0_1_aux (p  $\sigma$  : MState d) :
  Continuous0n (fun ( $\alpha : \mathbb{R}$ )  $\mapsto ((\text{HermitianMat.conj } (\sigma.M ^ ((1 - \alpha) / (2 * \alpha))).mat) p.M ^ \alpha)$ )
  (Set.Ioo 0 1)
```

**theorem continuous0n\_Ioo\_0\_1**[\[source\]](#)

*Continuity on  $(0,1)$ : the sandwich relative Rényi entropy is continuous in  $\alpha$  on  $(0,1)$ .*

```
private theorem sandwichedRelEntropy.continuous0n_Ioo_0_1 (p  $\sigma$  : MState d) :
  Continuous0n (fun  $\alpha \Rightarrow D_{\sim} \alpha(\rho||\sigma)$ ) (Set.Ioo 0 1)
```

**theorem continuousAt\_1**[\[source\]](#)

*Continuity at 1: the sandwich relative Rényi entropy is continuous at  $\alpha = 1$ .*

```
private theorem sandwichedRelEntropy.continuousAt_1 (p σ : MState d) :
  ContinuousWithinAt (fun α => D_ α(p||σ)) (Set.Ioi 0) 1
```

### theorem continuous0n

[\[source\]](#)

```
@[fun_prop]
theorem sandwichedRelEntropy.continuous0n (p σ : MState d) :
  Continuous0n (fun α => D_ α(p||σ)) (Set.Ioi 0)
```

### theorem qRelativeEnt\_ker

[\[source\]](#)

*Quantum relative entropy as  $\text{Tr}[\rho (\log \rho - \log \sigma)]$  when supports are contained.*

```
theorem qRelativeEnt_ker {p σ : MState d} (h : σ.M.ker ≤ p.M.ker) :
  D(p||σ).toEReal = ⟨⟨p.M, p.M.log - σ.M.log⟩⟩
```

### theorem qRelativeEnt\_eq\_neg\_S\_vn\_add

[\[source\]](#)

```
theorem qRelativeEnt_eq_neg_S_vn_add (p σ : MState d) :
  (qRelativeEnt p σ).toEReal = -(S_vn p : EReal) +
  if σ.M.ker ≤ p.M.ker then (-⟨⟨p.M, σ.M.log⟩⟩ : EReal) else (τ : EReal)
```

### theorem qRelativeEnt\_relabel

[\[source\]](#)

*The quantum relative entropy is unchanged by `MState.relabel`*

```
@[simp]
theorem qRelativeEnt_relabel (p σ : MState d) (e : d₂ ≈ d) :
  D(p.relabel e||σ.relabel e) = D(p||σ)
```

### theorem sandwichedRelEntropy\_of\_unique

[\[source\]](#)

```
@[simp]
theorem sandwichedRelEntropy_of_unique [Unique d] (p σ : MState d) :
  D_α(p||σ) = 0
```

### theorem qRelEntropy\_of\_unique

[\[source\]](#)

```
@[simp]
theorem qRelEntropy_of_unique [Unique d] (p σ : MState d) :
  D(p||σ) = 0
```

### theorem sandwichedRelEntropy\_heq\_congr

[\[source\]](#)

```
theorem sandwichedRelEntropy_heq_congr
  {d₁ d₂ : Type u} [Fintype d₁] [DecidableEq d₁] [Fintype d₂] [DecidableEq d₂]
  {ρ₁ σ₁ : MState d₁} {ρ₂ σ₂ : MState d₂} (hd : d₁ = d₂) (hp : ρ₁ ≈ ρ₂) (hσ : σ₁ ≈ σ₂) :
  D_α(ρ₁||σ₁) = D_α(ρ₂||σ₂)
```

**theorem sandwichedRelEntropy\_congr**[\[source\]](#)

```
@[gcongr]
theorem sandwichedRelEntropy_congr {α : ℝ}
  {d₁ d₂ : Type u} [Fintype d₁] [DecidableEq d₁] [Fintype d₂] [DecidableEq d₂]
  {ρ₁ σ₁ : MState d₁} {ρ₂ σ₂ : MState d₂} (hd : d₁ = d₂)
  (hp : ρ₁ = ρ₂.relabel (Equiv.cast hd)) (hσ : σ₁ = σ₂.relabel (Equiv.cast hd)) :
  D_ α(ρ₁||σ₁) = D_ α(ρ₂||σ₂)
```

**theorem qRelEntropy\_heq\_congr**[\[source\]](#)

```
@[gcongr]
theorem qRelEntropy_heq_congr {d₁ d₂ : Type u} [Fintype d₁] [DecidableEq d₁] [Fintype d₂] [
  DecidableEq d₂]
  {ρ₁ σ₁ : MState d₁} {ρ₂ σ₂ : MState d₂} (hd : d₁ = d₂) (hp : ρ₁ ≈ ρ₂) (hσ : σ₁ ≈ σ₂) :
  D(ρ₁||σ₁) = D(ρ₂||σ₂)
```

**theorem qRelativeEnt\_rank**[\[source\]](#)

*Quantum relative entropy when  $\sigma$  has full rank*

```
theorem qRelativeEnt_rank {ρ σ : MState d} [σ.M.NonSingular] :
  (D(ρ||σ) : EReal) = ⟨⟨ρ.M, ρ.M.log - σ.M.log⟩⟩
```

**def approxLog**[\[source\]](#)

```
private def approxLog (N : ℕ) : ℝ → ℝ
```

**lemma approxLog\_continuous**[\[source\]](#)

```
private lemma approxLog_continuous (N : ℕ) : Continuous (approxLog N)
```

**lemma approxLog\_ge\_log\_pos**[\[source\]](#)

```
private lemma approxLog_ge_log_pos {t : ℝ} (ht : 0 < t) (N : ℕ) :
  Real.log t ≤ approxLog N t
```

**lemma continuous\_inner\_cfc\_approxLog**[\[source\]](#)

```
private lemma continuous_inner_cfc_approxLog (ρ : MState d) (N : ℕ) :
  Continuous (fun σ : MState d => ⟨⟨ρ.M, σ.M.cfc (approxLog N)⟩⟩)
```

**lemma approxLog\_tendsto\_at\_pos**[\[source\]](#)

```
private lemma approxLog_tendsto_at_pos {t : ℝ} (ht : 0 < t) :
  Filter.Tendsto (fun N : ℕ => approxLog N t) Filter.atTop (nhds (Real.log t))
```

**lemma inner\_cfc\_approxLog\_ge**[\[source\]](#)

```
private lemma inner_cfc_approxLog_ge (ρ σ : MState d) (N : ℕ) (hσ : σ.M.ker ≤ ρ.M.ker) :
  ⟨⟨ρ.M, σ.M.log⟩⟩ ≤ ⟨⟨ρ.M, σ.M.cfc (approxLog N)⟩⟩
```

**lemma tendsto\_inner\_cfc\_approxLog**[\[source\]](#)

```
private lemma tendsto_inner_cfc_approxLog (p x : MState d) (hx : x.M.ker ≤ p.M.ker) :
  Filter.Tendsto (fun N : ℕ => ⟨⟨p.M, x.M.cfc (approxLog N)⟩⟩)
    Filter.atTop (nhds ⟨⟨p.M, x.M.log⟩⟩)
```

**lemma inner\_log\_bounded\_near**[\[source\]](#)

```
lemma inner_log_bounded_near (hx : σ.M.ker ≤ p.M.ker) {y : ℝ} (hy : ⟨⟨p.M, σ.M.log⟩⟩ < y) :
  ∀f x in nhds σ, x.M.ker ≤ p.M.ker → ⟨⟨p.M, x.M.log⟩⟩ < y
```

**lemma eigenWeight\_eq\_zero\_iff**[\[source\]](#)

```
private lemma eigenWeight_eq_zero_iff (p x : MState d) (i : d) :
  eigenWeight p x i = 0 ↔ (x.M.H.eigenvectorBasis i : EuclideanSpace ℂ d) ∈ p.M.ker
```

**lemma ker\_le\_iff\_eigenWeight\_zero**[\[source\]](#)

```
private lemma ker_le_iff_eigenWeight_zero (p x : MState d) :
  x.M.ker ≤ p.M.ker ↔ ∀ i, x.M.H.eigenvalues i = 0 → eigenWeight p x i = 0
```

**lemma neg\_ker\_exists\_eigenWeight\_pos**[\[source\]](#)

```
private lemma neg_ker_exists_eigenWeight_pos (p x : MState d) (hx : ¬(x.M.ker ≤ p.M.ker)) :
  ∃ i, x.M.H.eigenvalues i = 0 ∧ 0 < eigenWeight p x i
```

**lemma approxLog\_at\_zero**[\[source\]](#)

```
private lemma approxLog_at_zero (N : ℕ) : approxLog N 0 = -(N : ℝ)
```

**lemma inner\_cfc\_approxLog\_tendsto\_bot**[\[source\]](#)

```
private lemma inner_cfc_approxLog_tendsto_bot (p x : MState d) (hx : ¬(x.M.ker ≤ p.M.ker)) :
  Filter.Tendsto (fun N : ℕ => ⟨⟨p.M, x.M.cfc (approxLog N)⟩⟩) Filter.atTop Filter.atBot
```

**theorem qRelativeEnt\_lowerSemicontinuous\_2**[\[source\]](#)

```
theorem qRelativeEnt_lowerSemicontinuous_2 (p x : MState d) (hx : ¬(x.M.ker ≤ p.M.ker)) (y :
  ENNReal) (hy : y < τ) :
  ∀f (x' : MState d) in nhds x,
    y < (if x'.M.ker ≤ p.M.ker then ⟨⟨p.M, p.M.log - x'.M.log⟩⟩ else τ : EReal)
```

**theorem lowerSemicontinuous**[\[source\]](#)

```
@[fun_prop]
theorem qRelativeEnt_lowerSemicontinuous (p : MState d) : LowerSemicontinuous fun σ => D(p||σ)
```

**theorem qRelativeEnt\_joint\_convexity**[\[source\]](#)

*Joint convexity of Quantum relative entropy. We can't state this with `ConvexOn` because that requires an `AddCommMonoid`, which `MStates` are not. Instead we state it with `Mixable`.*

*TODO: \* Add the `Mixable` instance that infers from the `Coe` so that the right hand side can be written as  $p [D(\rho_1||\sigma_1) \leftrightarrow D(\rho_2||\sigma_2)]$  \* Define (joint) convexity as its own thing - a `ConvexOn` for `Mixable` types. \* Maybe, more broadly, find a way to make `ConvexOn` work with the subset of `Matrix` that corresponds to `MState`.*

```
theorem qRelativeEnt_joint_convexity :
  ∀ (p₁ p₂ σ₁ σ₂ : MState d), ∀ (p : Prob),
    D(p [p₁ ↔ p₂] || p [σ₁ ↔ σ₂]) ≤ p * D(p₁ || σ₁) + (1 - p) * D(p₂ || σ₂)
```

**theorem qRelEntropy\_self**

[\[source\]](#)

```
@[simp]
theorem qRelEntropy_self (p : MState d) : D(p || p) = 0
```

**theorem qRelativeEnt\_ne\_top**

[\[source\]](#)

```
@[aesop (rule_sets := [finiteness]) unsafe apply]
theorem qRelativeEnt_ne_top {p σ : MState d} [σ.M.NonSingular] : D(p || σ) ≠ τ
```

**theorem qMutualInfo\_as\_qRelativeEnt**

[\[source\]](#)

$$I(A:B) = D(\rho^{AB} || \rho^A \otimes \rho^B)$$

```
theorem qMutualInfo_as_qRelativeEnt (p : MState (dA × dB)) :
  qMutualInfo p = (D(p || p.traceRight * p.traceLeft) : EReal)
```

**lemma pos\_of\_MState\_le\_smul**

[\[source\]](#)

```
private lemma pos_of_MState_le_smul {σ₁ σ₂ : MState d} (hσ : σ₂.M ≤ α • σ₁.M) : 0 < α
```

**lemma inner\_log\_mono\_of\_posDef\_of\_le**

[\[source\]](#)

```
private lemma HermitianMat.inner_log_mono_of_posDef_of_le {A B C : HermitianMat d ℤ}
  (hC : 0 ≤ C) (hA : A.mat.PosDef) (hAB : A ≤ B) :
  ⟨⟨C, A.log⟩⟩ ≤ ⟨⟨C, B.log⟩⟩
```

**lemma posDef\_add\_eps**

[\[source\]](#)

```
private lemma posDef_add_eps {A : HermitianMat d ℂ} (hA : 0 ≤ A) {ε : ℝ} (hε : 0 < ε) :
  (A + ε • 1).mat.PosDef
```

**lemma log\_add\_eps\_eq\_cfc**

[\[source\]](#)

```
private lemma log_add_eps_eq_cfc (A : HermitianMat d ℤ) (ε : ℝ) :
  (A + ε • 1).log = A.cfc (Real.log <| · + ε)
```

**lemma inner\_cfc\_eq\_sum**[\[source\]](#)

```
private lemma inner_cfc_eq_sum (A C : HermitianMat d ℤ) (f : ℝ → ℝ) :
  ⟨⟨C, A.cfc f⟩⟩ = ∑ i, f (A.H.eigenvalues i) *
    RCLike.re ((C.mat * (A.H.eigenvectorUnitary.val * Matrix.single i i 1 * A.H.
      eigenvectorUnitary.val.conjTranspose)).trace)
```

**lemma eigenproj\_coeff\_zero\_of\_ker\_le**[\[source\]](#)

```
private lemma eigenproj_coeff_zero_of_ker_le {A C : HermitianMat d ℤ}
  (hker : A.ker ≤ C.ker) {i : d} (hi : A.H.eigenvalues i = 0) :
  RCLike.re ((C.mat * (A.H.eigenvectorUnitary.val * Matrix.single i i 1 * A.H.
    eigenvectorUnitary.val.conjTranspose)).trace) = 0
```

**lemma inner\_log\_shift\_tendsto**[\[source\]](#)

```
private lemma inner_log_shift_tendsto {A C : HermitianMat d ℤ} (hker : A.ker ≤ C.ker) :
  (ℕ[>] 0).Tendsto (fun (ε : ℝ) ↦ ⟨⟨C, (A + ε • 1).log⟩⟩) (ℕ ⟨⟨C, A.log⟩⟩)
```

**lemma inner\_log\_mono\_of\_psd\_of\_le**[\[source\]](#)

```
private lemma HermitianMat.inner_log_mono_of_psd_of_le {A B C : HermitianMat d ℤ}
  (hA : 0 ≤ A) (hAB : A ≤ B) (hC : 0 ≤ C) (hker : A.ker ≤ C.ker) :
  ⟨⟨C, A.log⟩⟩ ≤ ⟨⟨C, B.log⟩⟩
```

**lemma inner\_log\_sub\_le\_log\_alpha**[\[source\]](#)

```
private lemma HermitianMat.inner_log_sub_le_log_alpha (ρ : MState d) {σ₁ σ₂ : MState d} {α : ℝ}
  (hσ : σ₂.M ≤ α • σ₁.M)
  (hker₁ : σ₁.M.ker ≤ ρ.M.ker) (hker₂ : σ₂.M.ker ≤ ρ.M.ker) :
  ⟨⟨ρ.M, σ₂.M.log - σ₁.M.log⟩⟩ ≤ Real.log α
```

**theorem qRelEntropy\_le\_add\_of\_le\_smul**[\[source\]](#)

```
theorem qRelEntropy_le_add_of_le_smul (ρ : MState d) {σ₁ σ₂ : MState d} (hσ : σ₂.M ≤ α • σ₁.M)
  :
  D(ρ||σ₁) ≤ D(ρ||σ₂) + ENNReal.ofReal (Real.log α)
```

**theorem qRelativeEnt\_op\_le**[\[source\]](#)

```
theorem qRelativeEnt_op_le {ρ σ : MState d} (h : ρ.M ≤ α • σ.M) :
  D(ρ||σ) ≤ .ofReal (Real.log α)
```

**2.41 QuantumInfo.Finite.Entropy.SSA**

[QuantumInfo/Finite/Entropy/SSA.lean](#) on GitHub.

**def opNorm**[\[source\]](#)

*The operator norm of a matrix, with respect to the Euclidean norm (l2 norm) on the domain and codomain.*

```
noncomputable def Matrix.opNorm [RCLike ℤ] (A : Matrix m n ℤ) : ℝ
```

**theorem isometry\_preserves\_norm**[\[source\]](#)

```
theorem Matrix.isometry_preserves_norm (A : Matrix n m ℤ) (hA : A.Isometry) (x : EuclideanSpace
  ℤ m) :
  ||Matrix.toEuclideanLin A x|| = ||x||
```

**theorem opNorm\_isometry**[\[source\]](#)

```
theorem Matrix.opNorm_isometry [Nonempty m] (A : Matrix n m ℤ) (hA : A.Isometry) : Matrix.
  opNorm A = 1
```

**def map\_to\_tensor\_MES**[\[source\]](#)

*The matrix representation of the map  $v \mapsto v \otimes \sum_k |k\rangle\langle k|$ .  
The output index is  $(d_1 \times d_2) \times d_2$ .*

```
def map_to_tensor_MES : Matrix ((d1 × d2) × d2) d1 ℂ
```

**theorem map\_to\_tensor\_MES\_prop**[\[source\]](#)

```
theorem map_to_tensor_MES_prop (A : Matrix (d1 × d2) (d1 × d2) ℂ) :
  (map_to_tensor_MES d1 d2).conjTranspose * (Matrix.kronecker A (1 : Matrix d2 d2 ℂ)) * (
    map_to_tensor_MES d1 d2) =
  A.traceRight
```

**def V\_rho**[\[source\]](#)

*The isometry  $V_{\rho}$  from the paper.*

```
noncomputable def V_rho (ρAB : HermitianMat (dA × dB) ℂ) : Matrix ((dA × dB) × dB) dA ℂ
```

**def V\_sigma**[\[source\]](#)

*The isometry  $V_{\sigma}$  from the paper.*

```
noncomputable def V_sigma (σBC : HermitianMat (dB × dC) ℂ) : Matrix (dB × (dB × dC)) dC ℂ
```

**lemma V\_rho\_conj\_mul\_self\_eq**[\[source\]](#)

*$V_{\rho}^H * V_{\rho}$  simplifies to sandwiching the traceRight by the inverse square root.*

```
lemma V_rho_conj_mul_self_eq (ρAB : HermitianMat (dA × dB) ℂ) (hp : ρAB.mat.PosDef) :
  let ρA
```

**lemma PosDef\_traceRight**[\[source\]](#)

*The partial trace (left) of a positive definite matrix is positive definite.*

```
lemma PosDef_traceRight [Nonempty dB] (A : HermitianMat (dA × dB) ℂ) (hA : A.mat.PosDef) :
  A.traceRight.mat.PosDef
```

**lemma PosDef\_traceLeft**[\[source\]](#)

```
lemma PosDef_traceLeft [Nonempty dA] (A : HermitianMat (dA × dB) ℂ) (hA : A.mat.PosDef) :
  A.traceLeft.mat.PosDef
```

**theorem V\_rho\_isometry**[\[source\]](#)

*V\_rho is an isometry.*

```
theorem V_rho_isometry [Nonempty dB] (pAB : HermitianMat (dA × dB) ℂ) (hp : pAB.mat.PosDef) :
  (V_rho pAB)H * (V_rho pAB) = 1
```

**theorem V\_sigma\_isometry**[\[source\]](#)

*V\_sigma is an isometry.*

```
theorem V_sigma_isometry [Nonempty dB] (oBC : HermitianMat (dB × dC) ℂ) (ho : oBC.mat.PosDef) :
  (V_sigma oBC).conjTranspose * (V_sigma oBC) = 1
```

**def W\_mat**[\[source\]](#)

*The operator W from the paper, defined as a matrix product.*

```
noncomputable def W_mat (pAB : HermitianMat (dA × dB) ℂ) (oBC : HermitianMat (dB × dC) ℂ) :
  Matrix ((dA × dB) × dC) ((dA × dB) × dC) ℂ
```

**theorem opNorm\_mul\_le**[\[source\]](#)

*The operator norm of a matrix product is at most the product of the operator norms.*

```
theorem Matrix.opNorm_mul_le {l m n k : Type*} [Fintype l] [Fintype m] [Fintype n]
  [DecidableEq l] [DecidableEq m] [DecidableEq n] [RCLike k]
  (A : Matrix l m k) (B : Matrix m n k) :
  Matrix.opNorm (A * B) ≤ Matrix.opNorm A * Matrix.opNorm B
```

**theorem opNorm\_reindex\_proven**[\[source\]](#)

```
theorem Matrix.opNorm_reindex_proven {l m n p : Type*} [Fintype l] [Fintype m] [Fintype n] [
  Fintype p]
  [DecidableEq l] [DecidableEq m] [DecidableEq n] [DecidableEq p]
  (e : m ≈ l) (f : n ≈ p) (A : Matrix m n k) :
  Matrix.opNorm (A.reindex e f) = Matrix.opNorm A
```

**def U\_rho**[\[source\]](#)

*Define U\_rho as the Kronecker product of V\_rho and the identity.*

```
noncomputable def U_rho (pAB : HermitianMat (dA × dB) ℂ) : Matrix (((dA × dB) × dB) × dC) (dA
  × dC) ℂ
```

**def U\_sigma**[\[source\]](#)

*Define U\_sigma as the Kronecker product of the identity and V\_sigma.*



```
noncomputable def U_sigma (oBC : HermitianMat (dB × dC) ℂ) : Matrix (dA × (dB × (dB × dC))) (dA × dC) ℂ
```

### **theorem opNorm\_conjTranspose\_eq\_opNorm**

[\[source\]](#)

*The operator norm of the conjugate transpose is equal to the operator norm.*

```
theorem Matrix.opNorm_conjTranspose_eq_opNorm {m n : Type*} [Fintype m] [Fintype n]
  [DecidableEq m] [DecidableEq n] (A : Matrix m n ℝ) :
  Matrix.opNorm AH = Matrix.opNorm A
```

### **theorem isometry\_mul\_conjTranspose\_le\_one**

[\[source\]](#)

```
theorem isometry_mul_conjTranspose_le_one {m n : Type*} [Fintype m] [Fintype n]
  [DecidableEq m] [DecidableEq n]
  (V : Matrix m n ℂ) (hV : V.conjTranspose * V = 1) :
  V * V.conjTranspose ≤ 1
```

### **theorem conjTranspose\_isometry\_mul\_isometry\_le\_one**

[\[source\]](#)

```
theorem conjTranspose_isometry_mul_isometry_le_one {m n k : Type*}
  [Fintype m] [Fintype n] [Fintype k] [DecidableEq m] [DecidableEq n] [DecidableEq k]
  (A : Matrix k m ℂ) (B : Matrix k n ℂ)
  (hA : A.conjTranspose * A = 1) (hB : B.conjTranspose * B = 1) :
  (A.conjTranspose * B).conjTranspose * (A.conjTranspose * B) ≤ 1
```

### **theorem reindex\_le\_reindex\_iff**

[\[source\]](#)

```
theorem HermitianMat.reindex_le_reindex_iff {d d₂ : Type*} [Fintype d] [DecidableEq d]
  [Fintype d₂] [DecidableEq d₂] (e : d ≈ d₂) (A B : HermitianMat d ℂ) :
  A.reindex e ≤ B.reindex e ↔ A ≤ B
```

### **theorem inv\_kronecker**

[\[source\]](#)

```
theorem HermitianMat.inv_kronecker {m n : Type*} [Fintype m] [DecidableEq m]
  [Fintype n] [DecidableEq n] [Nonempty m] [Nonempty n]
  (A : HermitianMat m ℂ) (B : HermitianMat n ℂ)
  [A.NonSingular] [B.NonSingular] :
  (A ⊗ℂ B)-1 = A-1 ⊗ℂ B-1
```

### **theorem inv\_reindex**

[\[source\]](#)

```
theorem HermitianMat.inv_reindex {d d₂ : Type*} [Fintype d] [DecidableEq d]
  [Fintype d₂] [DecidableEq d₂] (A : HermitianMat d ℂ) (e : d ≈ d₂) :
  (A.reindex e)-1 = A-1.reindex e
```

### **theorem PosDef\_kronecker**

[\[source\]](#)

```
theorem HermitianMat.PosDef_kronecker {m n : Type*} [Fintype m] [DecidableEq m]
  [Fintype n] [DecidableEq n]
  (A : HermitianMat m ℂ) (B : HermitianMat n ℂ)
  (hA : A.mat.PosDef) (hB : B.mat.PosDef) :
```

```
(A ⊗k B).mat.PosDef
```

**theorem PosDef\_reindex**[\[source\]](#)

```
theorem HermitianMat.PosDef_reindex {d d₂ : Type*} [Fintype d] [DecidableEq d]
  [Fintype d₂] [DecidableEq d₂] (A : HermitianMat d ℂ) (e : d ≈ d₂)
  (hA : A.mat.PosDef) :
  (A.reindex e).mat.PosDef
```

**def S\_mat**[\[source\]](#)

*The sandwich matrix  $S$  used in the proof of `intermediate_ineq`. This is derived from `W_mat_sq_le_one` by algebraic manipulation (conjugation and simplification).*

```
private noncomputable def S_mat (ρAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC)
  ℂ) :
  Matrix ((dA × dB) × dC) ((dA × dB) × dC) ℂ
```

**lemma W\_mat\_sq\_eq\_conj**[\[source\]](#)

```
private lemma W_mat_sq_eq_conj [Nonempty dA] [Nonempty dB] [Nonempty dC]
  (ρAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ)
  (hp : ρAB.mat.PosDef) (hσ : σBC.mat.PosDef) :
  (W_mat ρAB σBC)H * (W_mat ρAB σBC) =
  S_mat ρAB σBC * (ρAB ⊗k (σBC.traceLeft)-1).mat * S_mat ρAB σBC
```

**lemma S\_mat\_conj\_rhs\_eq\_one**[\[source\]](#)

```
private lemma S_mat_conj_rhs_eq_one [Nonempty dA] [Nonempty dB] [Nonempty dC]
  (ρAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ)
  (hp : ρAB.mat.PosDef) (hσ : σBC.mat.PosDef) :
  S_mat ρAB σBC * ((ρAB.traceRight ⊗k σBC-1).reindex (Equiv.prodAssoc dA dB dC).symm).mat *
  S_mat ρAB σBC = 1
```

**lemma W\_mat\_eq\_factored**[\[source\]](#)

```
private lemma W_mat_eq_factored
  (ρAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ) :
  W_mat ρAB σBC =
  let F
```

**lemma F\_tensor\_MES\_eq\_V\_rho**[\[source\]](#)

```
private lemma F_tensor_MES_eq_V_rho
  (ρAB : HermitianMat (dA × dB) ℂ) :
  let F
```

**abbrev BigIdx**[\[source\]](#)

```
abbrev BigIdx (dA dB dC : Type*)
```

**abbrev SmallIdx**[\[source\]](#)

```
abbrev SmallIdx (dA dB dC : Type*)
```

**abbrev MidIdx**[\[source\]](#)

```
abbrev MidIdx (dA dB dC : Type*)
```

**def assoc\_equiv**[\[source\]](#)

*The associator equivalence (no swap needed). Maps  $((dA \times dB) \times dB) \times (dB \times dC)$  to  $((dA \times dB) \times (dB \times dC))$ .*

```
private def assoc_equiv (dA dB dC : Type*) :  
  BigIdx dA dB dC ≈ MidIdx dA dB dC
```

**def T<sub>1</sub>\_mat**[\[source\]](#)

```
private def T1_mat (pAB : HermitianMat (dA × dB) ℂ) :  
  Matrix (BigIdx dA dB dC) (SmallIdx dA dB dC) ℂ
```

**def T<sub>2</sub>\_mat**[\[source\]](#)

```
private def T2_mat (oBC : HermitianMat (dB × dC) ℂ) :  
  Matrix (SmallIdx dA dB dC) (MidIdx dA dB dC) ℂ
```

**def PERM\_mat**[\[source\]](#)

```
private def PERM_mat (dA dB dC : Type*) [Fintype dA] [Fintype dB] [Fintype dC]  
  [DecidableEq dA] [DecidableEq dB] [DecidableEq dC] :  
  Matrix (MidIdx dA dB dC) (BigIdx dA dB dC) ℂ
```

**lemma T<sub>1</sub>\_isometry**[\[source\]](#)

```
private lemma T1_isometry [Nonempty dB]  
  (pAB : HermitianMat (dA × dB) ℂ) (hp : pAB.mat.PosDef) :  
  (T1_mat dA dB dC pAB)H * (T1_mat dA dB dC pAB) = 1
```

**lemma T<sub>2</sub>\_sq\_le\_one**[\[source\]](#)

```
private lemma T2_sq_le_one [Nonempty dB]  
  (oBC : HermitianMat (dB × dC) ℂ) (ho : oBC.mat.PosDef) :  
  (T2_mat dA dB dC oBC)H * (T2_mat dA dB dC oBC) ≤ 1
```

**lemma PERM\_isometry**[\[source\]](#)

```
private lemma PERM_isometry : (PERM_mat dA dB dC)H * PERM_mat dA dB dC = 1
```

**lemma W\_mat\_entry**[\[source\]](#)

*Element-wise identity:  $W\_mat = \sum_{\rho} \{b^*\}_{\rho} V_{\rho} * V_{\sigma}^\dagger$ . This is the key computation from Eq. (6) of Lin-Kim-Hsieh.*

```

private lemma W_mat_entry (pAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ)
  (i j : SmallIdx dA dB dC) :
  W_mat pAB σBC i j =
    ∑ b_star : dB,
      V_rho pAB ((i.1, b_star)) j.1.1 *
      (V_sigma σBC)H i.2 (b_star, (j.1.2, j.2))

```

**lemma RHS\_entry**[\[source\]](#)

*Element-wise identity:  $RHS = \sum_{b^*} \{b^*\} V\_rho * V\_sigma^\dagger$ .*

```

private lemma RHS_entry (pAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ)
  (i j : SmallIdx dA dB dC) :
  (T2_mat dA dB dC σBC * PERM_mat dA dB dC * T1_mat dA dB dC pAB) i j =
    ∑ b_star : dB,
      V_rho pAB ((i.1, b_star)) j.1.1 *
      (V_sigma σBC)H i.2 (b_star, (j.1.2, j.2))

```

**lemma W\_mat\_eq\_three\_factors**[\[source\]](#)

```

private lemma W_mat_eq_three_factors [Nonempty dA] [Nonempty dB] [Nonempty dC]
  (pAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ) :
  W_mat pAB σBC =
    T2_mat dA dB dC σBC * PERM_mat dA dB dC * T1_mat dA dB dC pAB

```

**theorem W\_mat\_sq\_le\_one**[\[source\]](#)

*Core inequality:  $W^\dagger W \leq I$ . This is the key step, following from the isometry argument:  $V\_rho \otimes I\_C$  and  $I\_A \otimes V\_sigma$  are isometries, their cross product has norm  $\leq 1$ , and the result can be related to  $W\_mat$  through the MES computation (Eq. 6 in Lin-Kim-Hsieh).*

```

theorem W_mat_sq_le_one [Nonempty dA] [Nonempty dB] [Nonempty dC]
  (pAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ)
  (hp : pAB.mat.PosDef) (hσ : σBC.mat.PosDef) :
  (W_mat pAB σBC)H * (W_mat pAB σBC) ≤ 1

```

**lemma S\_mat\_isUnit**[\[source\]](#)

```

private lemma S_mat_isUnit [Nonempty dA] [Nonempty dB] [Nonempty dC]
  (pAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ)
  (hp : pAB.mat.PosDef) (hσ : σBC.mat.PosDef) :
  IsUnit (S_mat pAB σBC)

```

**theorem intermediate\_ineq**[\[source\]](#)

*The intermediate operator inequality:  $\rho_{AB} \otimes \sigma_C^{-1} \leq (\rho_A \otimes \sigma_{BC}^{-1}).reindex(assoc^{-1})$ . This is derived from  $W\_mat\_sq\_le\_one$  by algebraic manipulation (conjugation and simplification).*

```

theorem intermediate_ineq [Nonempty dA] [Nonempty dB] [Nonempty dC]
  (pAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ)
  (hp : pAB.mat.PosDef) (hσ : σBC.mat.PosDef) :
  pAB ⊗k (σBC.traceLeft)-1 ≤

```

```
(pAB.traceRight ⊗k σBC-1).reindex (Equiv.prodAssoc dA dB dC).symm
```

**theorem operator\_ineq\_SSA**[\[source\]](#)

*\*\*Operator extension of SSA\*\* (Main result of Lin-Kim-Hsieh). For positive definite  $\rho_{AB}$  and  $\sigma_{BC}$ :  $\rho_{A^{-1}} \otimes \sigma_{BC} \leq \rho_{AB^{-1}} \otimes \sigma_C$  where  $\rho_A = \text{Tr}_B(\rho_{AB})$  and  $\sigma_C = \text{Tr}_B(\sigma_{BC})$ , and the RHS is reindexed via the associator  $(dA \times dB) \times dC \approx dA \times (dB \times dC)$ .*

```
theorem operator_ineq_SSA [Nonempty dA] [Nonempty dB] [Nonempty dC]
  (pAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ)
  (hp : pAB.mat.PosDef) (hσ : σBC.mat.PosDef) :
  pAB.traceRight-1 ⊗k σBC ≤
    (pAB-1 ⊗k σBC.traceLeft).reindex (Equiv.prodAssoc dA dB dC)
```

**lemma inner\_kron\_one\_eq\_inner\_traceRight**[\[source\]](#)

```
private lemma inner_kron_one_eq_inner_traceRight
  (A : HermitianMat d1 ℂ) (M : HermitianMat (d1 × d2) ℂ) :
  ⟨⟨A ⊗k (1 : HermitianMat d2 ℂ), M⟩⟩ = ⟨⟨A, M.traceRight⟩⟩
```

**lemma inner\_one\_kron\_eq\_inner\_traceLeft**[\[source\]](#)

```
private lemma inner_one_kron_eq_inner_traceLeft
  (B : HermitianMat d2 ℂ) (M : HermitianMat (d1 × d2) ℂ) :
  ⟨⟨(1 : HermitianMat d1 ℂ) ⊗k B, M⟩⟩ = ⟨⟨B, M.traceLeft⟩⟩
```

**lemma hermitianMat\_log\_inv\_eq\_neg**[\[source\]](#)

```
private lemma hermitianMat_log_inv_eq_neg
  (A : HermitianMat d1 ℂ) [A.NonSingular] : A-1.log = -A.log
```

**lemma PosDef\_assoc'\_traceRight**[\[source\]](#)

```
private lemma PosDef_assoc'_traceRight
  (ρ : MState (d1 × d2 × d3)) (hp : ρ.M.mat.PosDef) :
  ρ.assoc'.traceRight.M.mat.PosDef
```

**lemma wm\_inner\_lhs**[\[source\]](#)

```
private lemma wm_inner_lhs [Nonempty d1] [Nonempty d2] [Nonempty d3]
  (ρ : MState (d1 × d2 × d3)) :
  ⟨⟨(-ρ.assoc'.traceRight.M.traceRight.log) ⊗k (1 : HermitianMat (d2 × d3) ℂ) +
    (1 : HermitianMat d1 ℂ) ⊗k ρ.traceLeft.M.log, ρ.M⟩⟩ =
  Svn ρ.traceRight - Svn ρ.traceLeft
```

**lemma wm\_inner\_rhs**[\[source\]](#)

```
private lemma wm_inner_rhs [Nonempty d1] [Nonempty d2] [Nonempty d3]
  (ρ : MState (d1 × d2 × d3)) :
  ⟨⟨((-ρ.assoc'.traceRight.M.log) ⊗k (1 : HermitianMat d3 ℂ) +
    (1 : HermitianMat d1 × d2) ℂ) ⊗k ρ.traceLeft.M.traceLeft.log).reindex
    (Equiv.prodAssoc d1 d2 d3), ρ.M⟩⟩ =
```

$$S_{v\ n} \ p.\text{assoc}'.\text{traceRight} - S_{v\ n} \ p.\text{traceLeft}.\text{traceLeft}$$
**lemma  $S_{v\ n\_wm\_pd}$** [\[source\]](#)

*Weak monotonicity (form 2) for positive definite states:  $S(\rho_{12}) + S(\rho_{23}) \geq S(\rho_1) + S(\rho_3)$ . Proved by applying `operator_ineq_SSA`, taking log, then inner product with  $\rho$ .*

```
private lemma S_vn_wm_pd [Nonempty d1] [Nonempty d2] [Nonempty d3]
  (p : MState (d1 × d2 × d3)) (hp : p.M.mat.PosDef) :
  S_vn p.traceRight + S_vn p.traceLeft.traceLeft ≤
  S_vn p.assoc'.traceRight + S_vn p.traceLeft
```

**lemma `approx_by_pd`**[\[source\]](#)

```
private lemma MState.approx_by_pd
  (p : MState d1) :
  ∃ (pn : ℕ → MState d1), (∀ n, (pn n).M.mat.PosDef) ∧
  Filter.Tendsto pn Filter.atTop (nhds p)
```

**lemma `traceLeft_continuous`**[\[source\]](#)

```
@[fun_prop]
private lemma MState.traceLeft_continuous :
  Continuous (MState.traceLeft : MState (d1 × d2) → MState d2)
```

**lemma `traceRight_continuous`**[\[source\]](#)

```
@[fun_prop]
private lemma MState.traceRight_continuous :
  Continuous (MState.traceRight : MState (d1 × d2) → MState d1)
```

**lemma `assoc'_continuous`**[\[source\]](#)

```
@[fun_prop]
private lemma MState.assoc'_continuous :
  Continuous (MState.assoc' : MState (d1 × d2 × d3) → MState ((d1 × d2) × d3))
```

**lemma  $S_{v\ n\_wm}$** [\[source\]](#)

*Weak monotonicity, version with partial traces.*

```
lemma S_vn_wm (p : MState (d1 × d2 × d3)) :
  S_vn p.traceRight + S_vn p.traceLeft.traceLeft ≤
  S_vn p.assoc'.traceRight + S_vn p.traceLeft
```

**def `perm_A_BCR'`**[\[source\]](#)

*Permutation to relabel  $(A \times B \times C) \times R$  as  $A \times (B \times C \times R)$ .*

```
private def perm_A_BCR' (dA dB dC : Type*) :
  (dA × dB × dC) × (dA × dB × dC) ≈ dA × (dB × dC × (dA × dB × dC)) where
```

**def  $\rho$ BCR**[\[source\]](#)

*The BCR state: trace out A from the purification of  $\rho_{ABC}$ .*

```
private def  $\rho$ BCR ( $\rho$  : MState (dA  $\times$  dB  $\times$  dC)) : MState (dB  $\times$  dC  $\times$  (dA  $\times$  dB  $\times$  dC))
```

**lemma S\_BC\_of\_BCR\_eq**[\[source\]](#)

```
private lemma S_BC_of_BCR_eq ( $\rho$  : MState (dA  $\times$  dB  $\times$  dC)) :  
  Sv n ( $\rho$ BCR  $\rho$ ).assoc'.traceRight = Sv n  $\rho$ .traceLeft
```

**def perm\_AB\_CR'**[\[source\]](#)

*Equivalence to relabel the purification as  $(dA \times dB) \times (dC \times R)$ .*

```
private def perm_AB_CR' (dA dB dC : Type*) :  
  (dA  $\times$  dB  $\times$  dC)  $\times$  (dA  $\times$  dB  $\times$  dC)  $\approx$  (dA  $\times$  dB)  $\times$  (dC  $\times$  (dA  $\times$  dB  $\times$  dC)) where
```

**lemma BCR\_traceLeft\_eq\_purify\_traceLeft**[\[source\]](#)

```
private lemma BCR_traceLeft_eq_purify_traceLeft ( $\rho$  : MState (dA  $\times$  dB  $\times$  dC)) :  
  ( $\rho$ BCR  $\rho$ ).traceLeft =  
  ((MState.pure  $\rho$ .purify).relabel (perm_AB_CR' dA dB dC).symm).traceLeft
```

**lemma purify\_AB\_traceRight\_eq**[\[source\]](#)

```
private lemma purify_AB_traceRight_eq ( $\rho$  : MState (dA  $\times$  dB  $\times$  dC)) :  
  Sv n ((MState.pure  $\rho$ .purify).relabel (perm_AB_CR' dA dB dC).symm).traceRight =  
  Sv n  $\rho$ .assoc'.traceRight
```

**lemma S\_CR\_of\_BCR\_eq**[\[source\]](#)

*The CR-marginal of  $\rho$ BCR has the same entropy as the AB-marginal of  $\rho$ .*

```
private lemma S_CR_of_BCR_eq ( $\rho$  : MState (dA  $\times$  dB  $\times$  dC)) :  
  Sv n ( $\rho$ BCR  $\rho$ ).traceLeft = Sv n  $\rho$ .assoc'.traceRight
```

**lemma S\_B\_of\_BCR\_eq**[\[source\]](#)

```
private lemma S_B_of_BCR_eq ( $\rho$  : MState (dA  $\times$  dB  $\times$  dC)) :  
  Sv n ( $\rho$ BCR  $\rho$ ).traceRight = Sv n  $\rho$ .traceLeft.traceRight
```

**lemma S\_R\_of\_BCR\_eq**[\[source\]](#)

```
private lemma S_R_of_BCR_eq ( $\rho$  : MState (dA  $\times$  dB  $\times$  dC)) :  
  Sv n ( $\rho$ BCR  $\rho$ ).traceLeft.traceLeft = Sv n  $\rho$ 
```

**theorem S<sub>v n</sub>\_strong\_subadditivity**[\[source\]](#)

*Strong subadditivity on a tripartite system*

```
theorem Sv n_strong_subadditivity ( $\rho_{123}$  : MState (d1  $\times$  d2  $\times$  d3)) :  
  let  $\rho_{12}$ 
```

**theorem  $S_{v,n}$ \_subadditivity**[\[source\]](#)*"Ordinary" subadditivity of von Neumann entropy*

```

theorem  $S_{v,n}$ _subadditivity ( $\rho$  : MState ( $d_1 \times d_2$ )) :
   $S_{v,n} \rho \leq S_{v,n} \rho.\text{traceRight} + S_{v,n} \rho.\text{traceLeft}$ 

```

**theorem  $S_{v,n}$ \_pure\_tripartite\_triangle**[\[source\]](#)*Triangle inequality for pure tripartite states:  $S(A) \leq S(B) + S(C)$ .*

```

theorem  $S_{v,n}$ _pure_tripartite_triangle ( $\psi$  : Ket (( $d_1 \times d_2$ )  $\times$   $d_3$ )) :
   $S_{v,n} (\text{MState.pure } \psi).\text{traceRight}.\text{traceRight} \leq$ 
   $S_{v,n} (\text{MState.pure } \psi).\text{traceRight}.\text{traceLeft} + S_{v,n} (\text{MState.pure } \psi).\text{traceLeft}$ 

```

**theorem  $S_{v,n}$ \_triangle\_ineq\_one\_way**[\[source\]](#)*One direction of the Araki-Lieb triangle inequality:  $S(A) \leq S(B) + S(AB)$ .*

```

theorem  $S_{v,n}$ _triangle_ineq_one_way ( $\rho$  : MState ( $d_1 \times d_2$ )) :  $S_{v,n} \rho.\text{traceRight} \leq S_{v,n} \rho.\text{traceLeft} +$ 
   $S_{v,n} \rho$ 

```

**theorem  $S_{v,n}$ \_triangle\_subadditivity**[\[source\]](#)*Araki-Lieb triangle inequality on von Neumann entropy*

```

theorem  $S_{v,n}$ _triangle_subadditivity ( $\rho$  : MState ( $d_1 \times d_2$ )) :
  abs ( $S_{v,n} \rho.\text{traceRight} - S_{v,n} \rho.\text{traceLeft}$ )  $\leq S_{v,n} \rho$ 

```

**theorem  $S_{v,n}$ \_weak\_monotonicity**[\[source\]](#)*Weak monotonicity of quantum conditional entropy:  $S(A/B) + S(A/C) \geq 0$ .*

```

theorem  $S_{v,n}$ _weak_monotonicity ( $\rho$  : MState ( $d_A \times d_B \times d_C$ )) :
  let  $\rho_{AB}$ 

```

**theorem qConditionalEnt\_strong\_subadditivity**[\[source\]](#)*Strong subadditivity, stated in terms of conditional entropies. Also called the data processing inequality.  $H(A/BC) \leq H(A/B)$ .*

```

theorem qConditionalEnt_strong_subadditivity ( $\rho_{123}$  : MState ( $d_1 \times d_2 \times d_3$ )) :
  qConditionalEnt  $\rho_{123} \leq \text{qConditionalEnt } \rho_{123}.\text{assoc'}.\text{traceRight}$ 

```

**theorem qMutualInfo\_strong\_subadditivity**[\[source\]](#)*Strong subadditivity, stated in terms of quantum mutual information.  $I(A;BC) \geq I(A;B)$ .*

```

theorem qMutualInfo_strong_subadditivity ( $\rho_{123}$  : MState ( $d_1 \times d_2 \times d_3$ )) :
  qMutualInfo  $\rho_{123} \geq \text{qMutualInfo } \rho_{123}.\text{assoc'}.\text{traceRight}$ 

```



**theorem qcmi\_nonneg**[\[source\]](#)

*The quantum conditional mutual information QCMI is nonnegative.*

```
theorem qcmi_nonneg (p : MState (dA × dB × dC)) :
  0 ≤ qcmi p
```

**theorem qcmi\_le\_2\_log\_dim**[\[source\]](#)

*The quantum conditional mutual information QCMI  $\rho_{ABC}$  is at most  $2 \log dA$ .*

```
theorem qcmi_le_2_log_dim (p : MState (dA × dB × dC)) :
  qcmi p ≤ 2 * Real.log (Fintype.card dA)
```

**theorem qcmi\_le\_2\_log\_dim'**[\[source\]](#)

*The quantum conditional mutual information QCMI  $\rho_{ABC}$  is at most  $2 \log dC$ .*

```
theorem qcmi_le_2_log_dim' (p : MState (dA × dB × dC)) :
  qcmi p ≤ 2 * Real.log (Fintype.card dC)
```

**2.42 QuantumInfo.Finite.Entropy.VonNeumann**

[QuantumInfo/Finite/Entropy/VonNeumann.lean](#) on GitHub.

**def  $S_{v,n}$** [\[source\]](#)

*Von Neumann entropy of a mixed state.*

```
def  $S_{v,n}$  (p : MState d) : ℝ
```

**def qConditionalEnt**[\[source\]](#)

*The Quantum Conditional Entropy  $S(\rho^A/\rho^B)$  is given by  $S(\rho^{AB}) - S(\rho^B)$ .*

```
def qConditionalEnt (p : MState (dA × dB)) : ℝ
```

**def qMutualInfo**[\[source\]](#)

*The Quantum Mutual Information  $I(A:B)$  is given by  $S(\rho^A) + S(\rho^B) - S(\rho^{AB})$ .*

```
def qMutualInfo (p : MState (dA × dB)) : ℝ
```

**def coherentInfo**[\[source\]](#)

*The Coherent Information of a state  $\rho$  passing through a channel  $\Lambda$  is the negative conditional entropy of the image under  $\Lambda$  of the purification of  $\rho$ .*

```
def coherentInfo (p : MState d1) (Λ : CPTMap d1 d2) : ℝ
```

**def qcmi**[\[source\]](#)

*The Quantum Conditional Mutual Information,  $I(A;C/B) = S(A/B) - S(A/BC)$ .*

```
def qcmi (p : MState (dA × dB × dC)) : ℝ
```

**theorem  $S_{v^n\_nonneg}$** [\[source\]](#)*von Neuman entropy is nonnegative.*

```
theorem S_v^n_nonneg (p : MState d) : 0 ≤ S_v^n p
```

**theorem  $S_{v^n\_le\_log\_d}$** [\[source\]](#)*von Neuman entropy is at most  $\log d$ .*

```
theorem S_v^n_le_log_d (p : MState d) : S_v^n p ≤ Real.log (Finset.card Finset.univ (α := d))
```

**theorem  $S_{v^n\_of\_pure\_zero}$** [\[source\]](#)*von Neuman entropy of pure states is zero.*`@[simp]`

```
theorem S_v^n_of_pure_zero (ψ : Ket d) : S_v^n (MState.pure ψ) = 0
```

**theorem  $S_{v^n\_eq\_neg\_trace\_log}$** [\[source\]](#)

```
theorem S_v^n_eq_neg_trace_log (p : MState d) : S_v^n p = -⟨⟨p.M.log, p.M⟩⟩
```

**theorem  $S_{v^n\_eq\_trace\_cfc\_negMulLog}$** [\[source\]](#)*Von Neumann entropy is the trace of the matrix function  $x \mapsto -x \log x$ .*

```
theorem S_v^n_eq_trace_cfc_negMulLog (p : MState d) :  
  S_v^n p = (p.M.cfc Real.negMulLog).trace
```

**theorem  $S_{v^n\_unit\_zero}$** [\[source\]](#)`@[simp]`

```
theorem S_v^n_unit_zero [Unique d] (p : MState d) : S_v^n p = 0
```

**theorem  $S_{v^n\_relabel}$** [\[source\]](#)*Von Neumann entropy is invariant under relabeling of the basis.*`@[simp]`

```
theorem S_v^n_relabel (p : MState d₁) (e : d₂ ≈ d₁) :  
  S_v^n (p.relabel e) = S_v^n p
```

**theorem  $S_{v^n\_of\_SWAP\_eq}$** [\[source\]](#)*Von Neumann entropy is unchanged under SWAP. TODO: All unitaries*`@[simp]`

```
theorem S_v^n_of_SWAP_eq (p : MState (d₁ × d₂)) : S_v^n p.SWAP = S_v^n p
```

**theorem  $S_{v^n\_of\_assoc\_eq}$** [\[source\]](#)*Von Neumann entropy is unchanged under assoc.*

```
@[simp]
theorem Svn_of_assoc_eq (ρ : MState ((d1 × d2) × d3)) : Svn ρ.assoc = Svn ρ
```

**theorem S<sub>v<sub>n</sub></sub>\_of\_assoc'\_eq**

[\[source\]](#)

*Von Neumann entropy is unchanged under assoc'.*

```
@[simp]
theorem Svn_of_assoc'_eq (ρ : MState (d1 × (d2 × d3))) : Svn ρ.assoc' = Svn ρ
```

**theorem selfAdjointMap\_Continuous**

[\[source\]](#)

```
@[fun_prop]
theorem selfAdjointMap_Continuous {ℓ : Type*} [RCLike ℓ] :
  Continuous (IsMaximalSelfAdjoint.selfadjMap : ℓ →+ ℝ)
```

**theorem trace\_Continuous**

[\[source\]](#)

```
@[fun_prop]
theorem HermitianMat.trace_Continuous {d ℓ : Type*} [Fintype d] [RCLike ℓ] :
  Continuous (HermitianMat.trace : HermitianMat d ℓ → ℝ)
```

**theorem S<sub>v<sub>n</sub></sub>\_continuous**

[\[source\]](#)

```
@[fun_prop]
theorem Svn_continuous : Continuous (Svn (d := d))
```

**def vecToMat**

[\[source\]](#)

*Convert a vector on a product space to a matrix.*

```
private def vecToMat {d1 d2 : Type*} [Fintype d1] [Fintype d2] (v : d1 × d2 → ℂ) : Matrix d1 d2 ℂ
```

**lemma traceRight\_eq\_mul\_conjTranspose**

[\[source\]](#)

*The right partial trace of a pure state is the product of the matrix representation and its conjugate transpose.*

```
private lemma traceRight_eq_mul_conjTranspose (ψ : Ket (d1 × d2)) :
  (MState.pure ψ).traceRight.M.val = (vecToMat ψ.vec) * (vecToMat ψ.vec).conjTranspose
```

**lemma traceLeft\_eq\_transpose\_mul\_conj**

[\[source\]](#)

```
lemma traceLeft_eq_transpose_mul_conj (ψ : Ket (d1 × d2)) :
  (MState.pure ψ).traceLeft.M.val = (vecToMat ψ.vec).transpose * (vecToMat ψ.vec).map star
```

**lemma traceLeft\_eq\_transpose\_conjTranspose\_mul**

[\[source\]](#)

*The left partial trace of vecToMat is the transpose of  $M^H * M$ .*

```
private lemma traceLeft_eq_transpose_conjTranspose_mul (ψ : Ket (d₁ × d₂)) :
  (MState.pure ψ).traceLeft.M.val =
  ((vecToMat ψ.vec).conjTranspose * (vecToMat ψ.vec)).transpose
```

**lemma  $H_s$ \_eq\_of\_nonzero\_multiset\_eq**

[\[source\]](#)

*Shannon entropy is determined by the multiset of non-zero probabilities.*

```
private lemma H_s_eq_of_nonzero_multiset_eq {α β : Type*} [Fintype α] [Fintype β]
  (d₁ : ProbDistribution α) (d₂ : ProbDistribution β)
  (h : (Finset.univ.val.map d₁.prob).filter (· ≠ 0) = (Finset.univ.val.map d₂.prob).filter (·
    ≠ 0)) :
  H_s d₁ = H_s d₂
```

**lemma nonzero\_roots\_eq\_of\_charpoly\_eq\_X\_pow**

[\[source\]](#)

*If two polynomials differ by a factor of  $X^k$ , their non-zero roots are the same.*

```
private lemma nonzero_roots_eq_of_charpoly_eq_X_pow {R : Type*} [CommRing R] [IsDomain R]
  [DecidableEq R] (p q : Polynomial R) (k : ℕ) (h : p = Polynomial.X ^ k * q) :
  p.roots.filter (· ≠ 0) = q.roots.filter (· ≠ 0)
```

**lemma nonzero\_eigenvalues\_eq\_of\_mul\_comm**

[\[source\]](#)

*The non-zero eigenvalues of  $AB$  and  $BA$  are the same.*

```
private lemma nonzero_eigenvalues_eq_of_mul_comm {n m : Type*} [Fintype n] [Fintype m]
  [DecidableEq n] [DecidableEq m] (A : Matrix n m ℂ) (B : Matrix m n ℂ) :
  (A * B).charpoly.roots.filter (· ≠ 0) = (B * A).charpoly.roots.filter (· ≠ 0)
```

**lemma  $S_{v^n}$ \_eq\_of\_nonzero\_eigenvalues\_eq**

[\[source\]](#)

*If two states have the same non-zero eigenvalues (spectrum probabilities), they have the same von Neumann entropy.*

```
private lemma S_vn_eq_of_nonzero_eigenvalues_eq (ρ₁ : MState d₁) (ρ₂ : MState d₂)
  (h : (Finset.univ.val.map ρ₁.spectrum.prob).filter (· ≠ 0) =
    (Finset.univ.val.map ρ₂.spectrum.prob).filter (· ≠ 0)) :
  S_vn ρ₁ = S_vn ρ₂
```

**lemma multiset\_filter\_map\_ofReal\_eq**

[\[source\]](#)

*Filtering non-zero elements commutes with mapping  $RCLike.ofReal$  for multisets.*

```
private lemma multiset_filter_map_ofReal_eq {R : Type*} [RCLike R] (M : Multiset ℝ) :
  (M.map (RCLike.ofReal : ℝ → R)).filter (· ≠ 0) = (M.filter (· ≠ 0)).map RCLike.ofReal
```

**lemma charpoly\_roots\_filter\_ne\_zero\_eq\_eigenvalues\_filter\_ne\_zero**

[\[source\]](#)

```
private lemma charpoly_roots_filter_ne_zero_eq_eigenvalues_filter_ne_zero {d : Type*} [Fintype
  d] [DecidableEq d] (A : Matrix d d ℂ) (hA : A.IsHermitian) :
  A.charpoly.roots.filter (· ≠ 0) = ((Finset.univ.val.map hA.eigenvalues).filter (· ≠ 0)).map
  RCLike.ofReal
```

**lemma multiset\_map\_ofReal\_injective**[\[source\]](#)

*Mapping  $RCLike.ofReal$  over a multiset is injective.*

```
private lemma multiset_map_ofReal_injective {R : Type*} [RCLike R] {M N : Multiset ℝ} :
  M.map (RCLike.ofReal : ℝ → R) = N.map RCLike.ofReal ↔ M = N
```

**lemma  $S_{v^n}$ \_eq\_of\_charpoly\_roots\_eq**[\[source\]](#)

*If the non-zero roots of the characteristic polynomials of two states are equal, their von Neumann entropies are equal.*

```
private lemma  $S_{v^n}$ _eq_of_charpoly_roots_eq (p1 : MState d1) (p2 : MState d2)
  (h : (p1.M.l.charpoly.roots.filter (· ≠ 0)) = (p2.M.l.charpoly.roots.filter (· ≠ 0))) :
   $S_{v^n}$  p1 =  $S_{v^n}$  p2
```

**theorem  $S_{v^n}$ \_of\_partial\_eq**[\[source\]](#)

*von Neuman entropies of the left- and right- partial trace of pure states are equal.*

```
theorem  $S_{v^n}$ _of_partial_eq (ψ : Ket (d1 × d2)) :
   $S_{v^n}$  (MState.pure ψ).traceLeft =  $S_{v^n}$  (MState.pure ψ).traceRight
```

**theorem qConditionalEnt\_of\_pure\_symm**[\[source\]](#)

*Quantum conditional entropy is symmetric for pure states.*

```
@[simp]
theorem qConditionalEnt_of_pure_symm (ψ : Ket (d1 × d2)) :
  qConditionalEnt (MState.pure ψ).SWAP = qConditionalEnt (MState.pure ψ)
```

**theorem qMutualInfo\_symm**[\[source\]](#)

*Quantum mutual information is symmetric.*

```
@[simp]
theorem qMutualInfo_symm (p : MState (d1 × d2)) :
  qMutualInfo p.SWAP = qMutualInfo p
```

**theorem  $S_{v^n}$ \_pure\_complement**[\[source\]](#)

*For a pure state, the entropy of one subsystem equals the entropy of its complement, even after relabeling.*

```
@[simp]
theorem  $S_{v^n}$ _pure_complement (ψ : Ket d1) (e : d2 × d3 ≈ d1) :
   $S_{v^n}$  ((MState.pure ψ).relabel e).traceLeft =  $S_{v^n}$  ((MState.pure ψ).relabel e).traceRight
```

**2.43 QuantumInfo.Finite.MState**

[QuantumInfo/Finite/MState.lean](#) on GitHub.

**structure MState**[\[source\]](#)

*A **\*\*mixed quantum state\*\*** is a PSD matrix with trace 1.*

*We don't extend  $(M : \text{HermitianMat } d \ \mathbb{C})$  because that gives an annoying thing where  $M$  is actually a Subtype, which means  $p.M.foo$  notation doesn't work.*

```
@[ext]
```

```
structure MState (d : Type*) [Fintype d] [DecidableEq d] where
  M : HermitianMat d  $\mathbb{C}$ 
  nonneg :  $0 \leq M$ 
  tr : M.trace = 1
```

**instance instCoe**[\[source\]](#)

```
instance instCoe : Coe (MState d) (HermitianMat d  $\mathbb{C}$ )
```

**def m**[\[source\]](#)

*The underlying Matrix in an MState. Prefer MState.M for the HermitianMat.*

```
def m (p : MState d) : Matrix d d  $\mathbb{C}$ 
```

**theorem mat\_M**[\[source\]](#)

```
@[simp]
```

```
theorem mat_M : p.M.mat = p.m
```

**theorem pos**[\[source\]](#)

```
theorem pos (p : MState d) :  $0 < p.M$ 
```

**def evalMStateM**[\[source\]](#)

*Positivity extension for MState.M: it is always positive ( $0 < p.M$ ). Note: we must not call whnfR on e because MState.M is a structure projection (reducible), so whnfR would reduce it and destroy the pattern.*

```
@[positivity MState.M _]
```

```
def evalMStateM : PositivityExt where eval {_u _ $\alpha$ } _z $\alpha$  _p $\alpha$  e
```

**theorem psd**[\[source\]](#)

```
theorem psd : p.m.PosSemidef
```

**theorem Hermitian**[\[source\]](#)

*Every mixed state is Hermitian.*

```
theorem Hermitian : p.m.IsHermitian
```

**theorem tr'** [\[source\]](#)

```
@[simp]
theorem tr' : p.m.trace = 1
```

**theorem ext\_m** [\[source\]](#)

```
theorem ext_m {p₁ p₂ : MState d} (h : p₁.m = p₂.m) : p₁ = p₂
```

**theorem m\_inj** [\[source\]](#)

*The map from mixed states to their matrices is injective*

```
theorem m_inj : (MState.m (d := d)).Injective
```

**theorem M\_Injective** [\[source\]](#)

```
theorem M_Injective : Function.Injective (MState.M (d := d))
```

**theorem convex** [\[source\]](#)

*The matrices corresponding to MStates are Convex  $\mathbb{R}$*

```
theorem convex : Convex  $\mathbb{R}$  (Set.range (MState.M (d := d)))
```

**instance instMixable** [\[source\]](#)

```
instance instMixable : Mixable (HermitianMat d  $\mathbb{C}$ ) (MState d) where
```

**instance nonempty** [\[source\]](#)

```
instance nonempty : Nonempty d
```

**theorem eigenvalue\_nonneg** [\[source\]](#)

```
theorem eigenvalue_nonneg :  $\forall i, 0 \leq \rho.Hermitian.eigenvalues i$ 
```

**theorem eigenvalue\_le\_one** [\[source\]](#)

```
theorem eigenvalue_le_one :  $\forall i, \rho.Hermitian.eigenvalues i \leq 1$ 
```

**theorem le\_one** [\[source\]](#)

```
theorem le_one :  $\rho.M \leq 1$ 
```

**instance Inner Prob (MState d)** [\[source\]](#)

*The inner product of two MState's, as a real number between 0 and 1.*

```
scoped instance : Inner Prob (MState d) where
```

**theorem inner\_def**[\[source\]](#)

```

theorem inner_def :  $\langle\langle \rho, \sigma \rangle\rangle_{\text{Prob}} = \langle\langle \rho.M, \sigma.M \rangle\rangle,$ 
  inner_ge_zero  $\rho.\text{nonneg } \sigma.\text{nonneg},$ 
  (inner_le_mul_trace  $\rho.\text{nonneg } \sigma.\text{nonneg}$ ).trans (by simp))

```

**theorem val\_inner**[\[source\]](#)

```

theorem val_inner : ( $\langle\langle \rho, \sigma \rangle\rangle_{\text{Prob}} : \mathbb{R}$ ) =  $\langle\langle \rho.M, \sigma.M \rangle\rangle$ 

```

**def exp\_val\_ℂ**[\[source\]](#)

```

def exp_val_ℂ (T : Matrix d d ℂ) : ℂ

```

**def exp\_val**[\[source\]](#)

*The **\*\*expectation value\*\*** of an operator on a quantum state.*

```

def exp_val (T : HermitianMat d ℂ) : ℝ

```

**theorem exp\_val\_nonneg**[\[source\]](#)

```

theorem exp_val_nonneg {T : HermitianMat d ℂ} (h :  $0 \leq T$ ) :  $0 \leq \rho.\text{exp\_val } T$ 

```

**theorem exp\_val\_zero**[\[source\]](#)

```

@[simp]
theorem exp_val_zero :  $\rho.\text{exp\_val } 0 = 0$ 

```

**theorem exp\_val\_one**[\[source\]](#)

```

@[simp]
theorem exp_val_one :  $\rho.\text{exp\_val } 1 = 1$ 

```

**theorem exp\_val\_le\_one**[\[source\]](#)

```

theorem exp_val_le_one {T : HermitianMat d ℂ} (h :  $T \leq 1$ ) :  $\rho.\text{exp\_val } T \leq 1$ 

```

**theorem exp\_val\_prob**[\[source\]](#)

```

theorem exp_val_prob {T : HermitianMat d ℂ} (h :  $0 \leq T \wedge T \leq 1$ ) :
   $0 \leq \rho.\text{exp\_val } T \wedge \rho.\text{exp\_val } T \leq 1$ 

```

**theorem exp\_val\_sub**[\[source\]](#)

```

theorem exp_val_sub (A B : HermitianMat d ℂ) :
   $\rho.\text{exp\_val } (A - B) = \rho.\text{exp\_val } A - \rho.\text{exp\_val } B$ 

```



**theorem exp\_val\_eq\_zero\_iff**[\[source\]](#)

*If a PSD observable  $A$  has expectation value of 0 on a state  $\rho$ , it must entirely contain the support of  $\rho$  in its kernel.*

```
theorem exp_val_eq_zero_iff {A : HermitianMat d ℂ} (hA1 : 0 ≤ A) :
  ρ.exp_val A = 0 ↔ ρ.M.support ≤ A.ker
```

**theorem exp\_val\_eq\_one\_iff**[\[source\]](#)

*If an observable  $A$  has expectation value of 1 on a state  $\rho$ , it must entirely contain the support of  $\rho$  in its 1-eigenspace.*

```
theorem exp_val_eq_one_iff {A : HermitianMat d ℂ} (hA2 : A ≤ 1) :
  ρ.exp_val A = 1 ↔ ρ.M.support ≤ (1 - A).ker
```

**theorem exp\_val\_add**[\[source\]](#)

```
theorem exp_val_add (A B : HermitianMat d ℂ) :
  ρ.exp_val (A + B) = ρ.exp_val A + ρ.exp_val B
```

**theorem exp\_val\_smul**[\[source\]](#)

```
@[simp]
theorem exp_val_smul (r : ℝ) (A : HermitianMat d ℂ) :
  ρ.exp_val (r • A) = r * ρ.exp_val A
```

**theorem exp\_val\_le\_exp\_val**[\[source\]](#)

```
@[gcongr]
theorem exp_val_le_exp_val (ρ : MState d) {A B : HermitianMat d ℂ} (h : A ≤ B) :
  ρ.exp_val A ≤ ρ.exp_val B
```

**def pure**[\[source\]](#)

*A mixed state can be constructed as a pure state arising from a ket.*

```
def pure (ψ : Ket d) : MState d where
```

**theorem pure\_apply**[\[source\]](#)

```
@[simp]
theorem pure_apply {i j : d} : (pure ψ).m i j = (ψ i) * conj (ψ j)
```

**theorem pure\_mul\_self**[\[source\]](#)

```
theorem pure_mul_self : (pure ψ).m * (pure ψ).m = (pure ψ : Matrix d d ℂ)
```

**def purity**[\[source\]](#)

*The purity of a state is  $\text{Tr}[\rho^2]$ . This is a *Prob*, because it is always between zero and one.*

```
def purity (p : MState d) : Prob
```

```
def spectrum
```

[\[source\]](#)

*The eigenvalue spectrum of a mixed quantum state, as a Distribution.*

```
def spectrum (p : MState d) : ProbDistribution d
```

```
theorem spectrum_pure_eq_constant
```

[\[source\]](#)

*The spectrum of a pure state is (1,0,0,...), i.e. a constant distribution.*

```
theorem spectrum_pure_eq_constant :  
  ∃ i, (pure ψ).spectrum = ProbDistribution.constant i
```

```
theorem pure_of_constant_spectrum
```

[\[source\]](#)

*If the spectrum of a mixed state is (1,0,0,...) i.e. a constant distribution, it is a pure state.*

```
theorem pure_of_constant_spectrum (h : ∃ i, p.spectrum = ProbDistribution.constant i) :  
  ∃ ψ, p = pure ψ
```

```
theorem pure_iff_constant_spectrum
```

[\[source\]](#)

*A state  $\rho$  is pure iff its spectrum is (1,0,0,...) i.e. a constant distribution.*

```
theorem pure_iff_constant_spectrum : (∃ ψ, p = pure ψ) ↔  
  ∃ i, p.spectrum = ProbDistribution.constant i
```

```
theorem pure_iff_purity_one
```

[\[source\]](#)

```
theorem pure_iff_purity_one : (∃ ψ, p = pure ψ) ↔ p.purity = 1
```

```
theorem spectralDecomposition
```

[\[source\]](#)

```
theorem spectralDecomposition (p : MState d) :  
  ∃ (ψs : d → Ket d), p.M = ∑ i, (p.spectrum i : ℝ) • (MState.pure (ψs i)).M
```

```
def prod
```

[\[source\]](#)

```
def prod (p1 : MState d1) (p2 : MState d2) : MState (d1 × d2) where
```

```
theorem prod_inner_prod
```

[\[source\]](#)

```
theorem prod_inner_prod (ξ1 ψ1 : MState d1) (ξ2 ψ2 : MState d2) :  
  ⟨⟨ξ1 ⊗ ξ2, ψ1 ⊗ ψ2⟩⟩_Prob = ⟨⟨ξ1, ψ1⟩⟩_Prob * ⟨⟨ξ2, ψ2⟩⟩_Prob
```

**theorem pure\_prod\_pure**[\[source\]](#)

*The product of pure states is a pure product state , Ket.prod.*

```
theorem pure_prod_pure (ψ1 : Ket d1) (ψ2 : Ket d2) : pure (ψ1 ⊗ψ ψ2) = (pure ψ1) ⊗M (pure ψ2)
```

**def ofClassical**[\[source\]](#)

*A representation of a classical distribution as a quantum state, diagonal in the given basis.*

```
def ofClassical (dist : ProbDistribution d) : MState d where
```

**theorem coe\_ofClassical**[\[source\]](#)

```
@[simp]
```

```
theorem coe_ofClassical (dist : ProbDistribution d) :  
  (ofClassical dist).M = diagonal C (dist .)
```

**theorem ofClassical\_pow**[\[source\]](#)

```
theorem ofClassical_pow (dist : ProbDistribution d) (p : ℝ) :  
  (ofClassical dist).M ^ p = diagonal C (fun i ↦ (dist i) ^ p)
```

**def uniform**[\[source\]](#)

*The maximally mixed state.*

```
def uniform [Nonempty d] : MState d
```

**instance instUnique**[\[source\]](#)

*There is exactly one state on a dimension-1 system.*

```
instance instUnique [Unique d] : Unique (MState d) where
```

**instance instInhabited**[\[source\]](#)

*There exists a mixed state for every nonempty d. Here, the maximally mixed one is chosen.*

```
instance instInhabited [Nonempty d] : Inhabited (MState d) where
```

**theorem M\_default**[\[source\]](#)

```
@[simp]
```

```
theorem M_default [Unique d] : (default : MState d).M = 1
```

**def traceLeft**[\[source\]](#)

*Partial tracing out the left half of a system.*

```
@[sims]
```

```
def traceLeft (p : MState (d1 × d2)) : MState d2 where
```

**def traceRight**[\[source\]](#)

*Partial tracing out the right half of a system.*

```
@[simps]
def traceRight (p : MState (d1 × d2)) : MState d1 where
```

**theorem traceLeft\_prod\_eq**[\[source\]](#)

*Taking the direct product on the left and tracing it back out gives the same state.*

```
@[simp]
theorem traceLeft_prod_eq (p1 : MState d1) (p2 : MState d2) : (p1 ⊗M p2).traceLeft = p2
```

**theorem traceRight\_prod\_eq**[\[source\]](#)

*Taking the direct product on the right and tracing it back out gives the same state.*

```
@[simp]
theorem traceRight_prod_eq (p1 : MState d1) (p2 : MState d2) : (p1 ⊗M p2).traceRight = p1
```

**theorem spectrum\_prod**[\[source\]](#)

*Spectrum of direct product. There is a permutation  $\sigma$  so that the spectrum of the direct product of  $\rho_1$  and  $\rho_2$ , as permuted under  $\sigma$ , is the pairwise products of the spectra of  $\rho_1$  and  $\rho_2$ .*

```
theorem spectrum_prod (p1 : MState d1) (p2 : MState d2) : ∃ (σ : d1 × d2 ≈ d1 × d2),
  ∀ i, ∀ j, (p1 ⊗M p2).spectrum (σ (i, j)) = (p1.spectrum i) * (p2.spectrum j)
```

**theorem sInf\_spectrum\_prod**[\[source\]](#)

```
theorem sInf_spectrum_prod (p : MState d) (σ : MState d2) :
  sInf (_root_.spectrum ℝ (p ⊗M σ).m) = sInf (_root_.spectrum ℝ p.m) * sInf (_root_.spectrum
    ℝ σ.m)
```

**def IsSeparable**[\[source\]](#)

*A mixed state is separable iff it can be written as a convex combination of product mixed states.*

```
def IsSeparable (p : MState (d1 × d2)) : Prop
```

**theorem IsSeparable\_prod**[\[source\]](#)

*A product state `MState.prod` is separable.*

```
theorem IsSeparable_prod (p1 : MState d1) (p2 : MState d2) : IsSeparable (p1 ⊗M p2)
```

**theorem eq\_of\_sum\_eq\_pure**[\[source\]](#)

```
theorem eq_of_sum_eq_pure {d : Type*} [Fintype d] [DecidableEq d]
  {ι : Type*} {s : Finset ι} {p : ι → ℝ} {ps : ι → MState d}
  {ρ : MState d} (h_pure : ρ.purity = 1) (h_sum : ρ.M = ∑ i ∈ s, p i • (ps i).M)
```

```
(hp_nonneg : ∀ i ∈ s, 0 ≤ p i) (hp_sum : ∑ i ∈ s, p i = 1) (i : ι) (hi : i ∈ s) (hpi : 0 < p i) :
ps i = p
```

**theorem purity\_prod**[\[source\]](#)

```
theorem purity_prod {d₁ d₂ : Type*} [Fintype d₁] [Fintype d₂] [DecidableEq d₁] [DecidableEq d₂]
(ρ₁ : MState d₁) (ρ₂ : MState d₂) : (ρ₁ ⊗M ρ₂).purity = ρ₁.purity * ρ₂.purity
```

**theorem pure\_eq\_pure\_iff**[\[source\]](#)

```
theorem pure_eq_pure_iff {d : Type*} [Fintype d] [DecidableEq d] (ψ φ : Ket d) :
pure ψ = pure φ ↔ ∃ z : ℂ, ||z|| = 1 ∧ ψ.vec = z • φ.vec
```

**theorem PhaseEquiv\_iff\_pure\_eq**[\[source\]](#)

*Two kets are phase-equivalent if and only if their pure states are equal.*

```
theorem PhaseEquiv_iff_pure_eq {d : Type*} [Fintype d] [DecidableEq d] (ψ φ : Ket d) :
Ket.PhaseEquiv.r ψ φ ↔ MState.pure ψ = MState.pure φ
```

**def pureQ**[\[source\]](#)

*MState.pure descends to the quotient KetUpToPhase.*

```
def pureQ {d : Type*} [Fintype d] [DecidableEq d] : KetUpToPhase d → MState d
```

**theorem pureQ\_mk**[\[source\]](#)

```
@[simp]
theorem pureQ_mk {d : Type*} [Fintype d] [DecidableEq d] (ψ : Ket d) :
pureQ (Quotient.mk Ket.PhaseEquiv ψ) = MState.pure ψ
```

**theorem pureQ\_injective**[\[source\]](#)

```
theorem pureQ_injective {d : Type*} [Fintype d] [DecidableEq d] : Function.Injective (pureQ (d
:= d))
```

**theorem pure\_separable\_imp\_IsProd**[\[source\]](#)

```
theorem pure_separable_imp_IsProd {d₁ d₂ : Type*} [Fintype d₁] [Fintype d₂] [DecidableEq d₁] [
DecidableEq d₂]
(ψ : Ket (d₁ × d₂)) (h : IsSeparable (pure ψ)) : ψ.IsProd
```

**theorem pure\_separable\_iff\_IsProd**[\[source\]](#)

*A pure state is separable iff the ket is a product state.*

```
theorem pure_separable_iff_IsProd (ψ : Ket (d₁ × d₂)) :
IsSeparable (pure ψ) ↔ ψ.IsProd
```

**theorem pure\_iff\_rank\_eq\_one**[\[source\]](#)

*A mixed state is pure if and only if its rank is 1.*

```
theorem pure_iff_rank_eq_one {d : Type*} [Fintype d] [DecidableEq d] (p : MState d) :
  (∃ ψ, p = pure ψ) ↔ p.m.rank = 1
```

**theorem IsProd\_iff\_rank\_eq\_one**[\[source\]](#)

*A ket on a product space is a product state if and only if its coefficient matrix has rank 1.*

```
theorem Ket.IsProd_iff_rank_eq_one {d₁ d₂ : Type*} [Fintype d₁] [Fintype d₂] [DecidableEq d₁] [
  DecidableEq d₂]
  (ψ : Ket (d₁ × d₂)) :
  ψ.IsProd ↔ (Matrix.of (fun i j => ψ (i, j))).rank = 1
```

**theorem pure\_separable\_iff\_traceLeft\_pure**[\[source\]](#)

*A pure state is separable iff the partial trace on the left is pure.*

```
theorem pure_separable_iff_traceLeft_pure (ψ : Ket (d₁ × d₂)) : IsSeparable (pure ψ) ↔
  ∃ ψ₁, pure ψ₁ = (pure ψ).traceLeft
```

**def purify**[\[source\]](#)

*The purification of a mixed state. Always uses the full dimension of the Hilbert space ( $d$ ) to purify, so e.g. an existing pure state with  $d=4$  still becomes  $d=16$  in the purification. The defining property is `MState.traceRight_of_purify`; see also `MState.purify` for the bundled version.*

```
def purify (p : MState d) : Ket (d × d) where
```

**theorem purify\_spec**[\[source\]](#)

*The defining property of purification, that tracing out the purifying system gives the original mixed state.*

```
@[simp]
theorem purify_spec (p : MState d) : (pure p.purify).traceRight = p
```

**def purifyX**[\[source\]](#)

*`MState.purify` bundled with its defining property `MState.traceRight_of_purify`.*

```
def purifyX (p : MState d) : { ψ : Ket (d × d) // (pure ψ).traceRight = p }
```

**def MState.relabel**[\[source\]](#)

```
@[simps]
def relabel (p : MState d₁) (e : d₂ ≈ d₁) : MState d₂ where
```

**theorem relabel\_m**[\[source\]](#)

```
@[simp]
theorem relabel_m (ρ : MState d₁) (e : d₂ ≈ d₁) :
  (ρ.relabel e).m = ρ.m.submatrix e e
```

**theorem relabel\_refl**[\[source\]](#)

```
@[simp]
theorem relabel_refl {d : Type*} [Fintype d] [DecidableEq d] (ρ : MState d) :
  ρ.relabel (Equiv.refl d) = ρ
```

**theorem relabel\_pure\_exists**[\[source\]](#)

*Relabeling a pure state by a bijection yields another pure state.*

```
theorem relabel_pure_exists (ψ : Ket d₁) (e : d₂ ≈ d₁) :
  ∃ ψ' : Ket d₂, (pure ψ).relabel e = pure ψ'
```

**theorem relabel\_relabel**[\[source\]](#)

```
@[simp]
theorem relabel_relabel {d d₂ d₃ : Type*}
  [Fintype d] [DecidableEq d] [Fintype d₂] [DecidableEq d₂] [Fintype d₃] [DecidableEq d₃]
  (ρ : MState d) (e : d₂ ≈ d) (e₂ : d₃ ≈ d₂) : (ρ.relabel e).relabel e₂ = ρ.relabel (e₂.trans e)
```

**theorem eq\_relabel\_iff**[\[source\]](#)

```
theorem eq_relabel_iff {d₁ d₂ : Type u} [Fintype d₁] [DecidableEq d₁] [Fintype d₂] [DecidableEq d₂]
  (ρ : MState d₁) (σ : MState d₂) (h : d₁ ≈ d₂) :
  ρ = σ.relabel h ↔ ρ.relabel h.symm = σ
```

**theorem relabel\_comp**[\[source\]](#)

```
theorem relabel_comp {d₁ d₂ d₃ : Type*} [Fintype d₁] [DecidableEq d₁] [Fintype d₂] [DecidableEq d₂]
  [Fintype d₃] [DecidableEq d₃] (ρ : MState d₁) (e : d₂ ≈ d₁) (f : d₃ ≈ d₂) :
  (ρ.relabel e).relabel f = ρ.relabel (f.trans e)
```

**theorem relabel\_cast**[\[source\]](#)

```
theorem relabel_cast {d₁ d₂ : Type u} [Fintype d₁] [DecidableEq d₁]
  [Fintype d₂] [DecidableEq d₂]
  (ρ : MState d₁) (e : d₂ ≈ d₁) :
  ρ.relabel (Equiv.cast e) = cast (by have := e.symm; congr <=> (apply Subsingleton.helim;
  congr)) ρ
```

**theorem spectrum\_relabel**[\[source\]](#)

```
@[simp]
theorem spectrum_relabel {ρ : MState d} (e : d₂ ≈ d) :
```

```
_root_.spectrum ℝ (p.relabel e).m = _root_.spectrum ℝ p.m
```

**theorem purity\_relabel**[\[source\]](#)

*The purity of a state is invariant under relabeling of the basis.*

```
@[simp]
```

```
theorem purity_relabel (p : MState d1) (e : d2 ≈ d1) : (p.relabel e).purity = p.purity
```

**def SWAP**[\[source\]](#)

*The heterogeneous SWAP gate that exchanges the left and right halves of a quantum system. This can apply even when the two "halves" are of different types, as opposed to (say) the SWAP gate on quantum circuits that leaves the qubit dimensions unchanged. Notably, it is not unitary.*

```
def SWAP (p : MState (d1 × d2)) : MState (d2 × d1)
```

**lemma multiset\_spectrum\_relabel\_eq**[\[source\]](#)

*The multiset of values in the spectrum of a relabeled state is the same as the multiset of values in the spectrum of the original state.*

```
lemma multiset_spectrum_relabel_eq {d1 d2 : Type*} [Fintype d1] [DecidableEq d1] [Fintype d2] [DecidableEq d2]
  (p : MState d1) (e : d2 ≈ d1) :
  Multiset.map (p.relabel e).spectrum Finset.univ.val = Multiset.map p.spectrum Finset.univ.val
```

**def spectrum\_SWAP**[\[source\]](#)

```
def spectrum_SWAP (p : MState (d1 × d2)) : ∃ e, p.SWAP.spectrum.relabel e = p.spectrum
```

**theorem SWAP\_SWAP**[\[source\]](#)

```
@[simp]
```

```
theorem SWAP_SWAP (p : MState (d1 × d2)) : p.SWAP.SWAP = p
```

**theorem traceLeft\_SWAP**[\[source\]](#)

```
@[simp]
```

```
theorem traceLeft_SWAP (p : MState (d1 × d2)) : p.SWAP.traceLeft = p.traceRight
```

**theorem traceRight\_SWAP**[\[source\]](#)

```
@[simp]
```

```
theorem traceRight_SWAP (p : MState (d1 × d2)) : p.SWAP.traceRight = p.traceLeft
```

**def assoc**[\[source\]](#)

*The associator that re-clusters the parts of a quantum system.*



```
def assoc (p : MState ((d1 × d2) × d3)) : MState (d1 × d2 × d3)
```

**def assoc'**

[\[source\]](#)

*The associator that re-clusters the parts of a quantum system.*

```
def assoc' (p : MState (d1 × d2 × d3)) : MState ((d1 × d2) × d3)
```

**theorem assoc\_assoc'**

[\[source\]](#)

@[simp]

```
theorem assoc_assoc' (p : MState (d1 × d2 × d3)) : p.assoc'.assoc = p
```

**theorem assoc'\_assoc**

[\[source\]](#)

@[simp]

```
theorem assoc'_assoc (p : MState ((d1 × d2) × d3)) : p.assoc.assoc' = p
```

**theorem traceLeft\_right\_assoc**

[\[source\]](#)

@[simp]

```
theorem traceLeft_right_assoc (p : MState ((d1 × d2) × d3)) :  
  p.assoc.traceLeft.traceRight = p.traceRight.traceLeft
```

**theorem traceRight\_left\_assoc'**

[\[source\]](#)

@[simp]

```
theorem traceRight_left_assoc' (p : MState (d1 × d2 × d3)) :  
  p.assoc'.traceRight.traceLeft = p.traceLeft.traceRight
```

**theorem traceRight\_assoc**

[\[source\]](#)

@[simp]

```
theorem traceRight_assoc (p : MState ((d1 × d2) × d3)) :  
  p.assoc.traceRight = p.traceRight.traceRight
```

**theorem traceLeft\_assoc'**

[\[source\]](#)

@[simp]

```
theorem traceLeft_assoc' (p : MState (d1 × d2 × d3)) :  
  p.assoc'.traceLeft = p.traceLeft.traceLeft
```

**theorem traceLeft\_left\_assoc**

[\[source\]](#)

@[simp]

```
theorem traceLeft_left_assoc (p : MState ((d1 × d2) × d3)) :  
  p.assoc.traceLeft.traceLeft = p.traceLeft
```

**theorem traceRight\_right\_assoc'**[\[source\]](#)

```
@[simp]
theorem traceRight_right_assoc' (ρ : MState (d1 × d2 × d3)) :
  ρ.assoc'.traceRight.traceRight = ρ.traceRight
```

**theorem traceNorm\_eq\_1**[\[source\]](#)

```
@[simp]
theorem traceNorm_eq_1 (ρ : MState d) : ρ.m.traceNorm = 1
```

**theorem relabel\_kron**[\[source\]](#)

```
theorem relabel_kron (ρ : MState d1) (σ : MState d2) (e : d3 ≈ d1) :
  ((ρ.relabel e) ⊗M σ) = (ρ ⊗M σ).relabel (e.prodCongr (Equiv.refl d2))
```

**theorem kron\_relabel**[\[source\]](#)

```
theorem kron_relabel (ρ : MState d1) (σ : MState d2) (e : d3 ≈ d2) :
  (ρ ⊗M σ.relabel e) = (ρ ⊗M σ).relabel ((Equiv.refl d1).prodCongr e)
```

**theorem prod\_assoc**[\[source\]](#)

```
theorem prod_assoc (ρ : MState d1) (σ : MState d2) (τ : MState d3) :
  (ρ ⊗M (σ ⊗M τ)) = (ρ ⊗M σ ⊗M τ).relabel (Equiv.prodAssoc d1 d2 d3).symm
```

**instance TopologicalSpace (MState d)**[\[source\]](#)

*Mixed states inherit the subspace topology from matrices*

```
instance : TopologicalSpace (MState d)
```

**theorem toMat\_IsEmbedding**[\[source\]](#)

*The projection from mixed states to their Hermitian matrices is an embedding*

```
theorem toMat_IsEmbedding : Topology.IsEmbedding (MState.M (d := d)) where
```

**instance T3Space (MState d)**[\[source\]](#)

```
instance : T3Space (MState d)
```

**instance CompactSpace (MState d)**[\[source\]](#)

```
instance : CompactSpace (MState d)
```

**instance MetricSpace (MState d)**[\[source\]](#)

```
noncomputable instance : MetricSpace (MState d)
```

**theorem dist\_eq**[\[source\]](#)

```
theorem dist_eq (x y : MState d) : dist x y = dist x.M y.M
```

**instance BoundedSpace (MState d)**[\[source\]](#)

```
instance : BoundedSpace (MState d) where
```

**theorem Continuous\_HermitianMat**[\[source\]](#)

```
@[fun_prop]
```

```
theorem Continuous_HermitianMat : Continuous (MState.M (d := d))
```

**theorem Continuous\_Matrix**[\[source\]](#)

```
@[fun_prop]
```

```
theorem Continuous_Matrix : Continuous (MState.m (d := d))
```

**theorem image\_M\_isBounded**[\[source\]](#)

```
theorem image_M_isBounded (S : Set (MState d)) : Bornology.IsBounded (MState.M '' S)
```

**def piProd**[\[source\]](#)

```
def piProd (pi : (i:ι) → MState (dI i)) : MState ((i:ι) → dI i) where
```

**def npow**[\[source\]](#)

*The  $n$ -copy "power" of a mixed state, with the standard basis indexed by  $\pi$  types.*

```
def npow (p : MState d) (n : ℕ) : MState (Fin n → d)
```

**theorem kron**[\[source\]](#)

```
theorem PosDef.kron {d1 d2 : Type*} [Fintype d1] [DecidableEq d1] [Fintype d2] [DecidableEq d2]
  {σ1 : MState d1} {σ2 : MState d2} (hσ1 : σ1.m.PosDef) (hσ2 : σ2.m.PosDef) : (σ1 ⊗m σ2).m.
  PosDef
```

**theorem PosDef.relabel**[\[source\]](#)

```
theorem PosDef.relabel {d1 d2 : Type*} [Fintype d1] [DecidableEq d1] [Fintype d2] [DecidableEq d2]
  {p : MState d1} (hp : p.m.PosDef) (e : d2 ≈ d1) : (p.relabel e).m.PosDef
```

**theorem PosDef\_mix**[\[source\]](#)

*If both states positive definite, so is their mixture.*

```
theorem PosDef_mix {d : Type*} [Fintype d] [DecidableEq d] {σ1 σ2 : MState d}
  (hσ1 : σ1.m.PosDef) (hσ2 : σ2.m.PosDef) (p : Prob) : (p [σ1 ↔ σ2]).m.PosDef
```

**theorem PosDef\_mix\_of\_ne\_zero**[\[source\]](#)

*If one state is positive definite and the mixture is nondegenerate, their mixture is also positive definite.*

```
theorem PosDef_mix_of_ne_zero {d : Type*} [Fintype d] [DecidableEq d] {σ₁ σ₂ : MState d}
  (hσ₁ : σ₁.m.PosDef) (p : Prob) (hp : p ≠ 0) : (p [σ₁ ↔ σ₂]).m.PosDef
```

**theorem PosDef\_mix\_of\_ne\_one**[\[source\]](#)

*If the second state is positive definite and the mixture is nondegenerate, their mixture is also positive definite.*

```
theorem PosDef_mix_of_ne_one {d : Type*} [Fintype d] [DecidableEq d] {σ₁ σ₂ : MState d}
  (hσ₂ : σ₂.m.PosDef) (p : Prob) (hp : p ≠ 1) : (p [σ₁ ↔ σ₂]).m.PosDef
```

**theorem uniform\_posDef**[\[source\]](#)

```
theorem uniform_posDef {d : Type*} [Nonempty d] [Fintype d] [DecidableEq d] :
  (uniform (d := d)).m.PosDef
```

**theorem posDef\_of\_unique**[\[source\]](#)

```
theorem posDef_of_unique {d : Type*} [Fintype d] [DecidableEq d] (p : MState d) [Unique d] : p.
  m.PosDef
```

**2.44 QuantumInfo.Finite.Pinching**

[QuantumInfo/Finite/Pinching.lean](#) on GitHub.

**def pinching\_kraus**[\[source\]](#)

```
def pinching_kraus (p : MState d) : spectrum ℝ p.m → HermitianMat d ℂ
```

**theorem pinching\_kraus\_commutes**[\[source\]](#)

```
theorem pinching_kraus_commutes (p : MState d) (i : spectrum ℝ p.m) :
  Commute (pinching_kraus p i).mat p.m
```

**theorem pinching\_kraus\_mul\_self**[\[source\]](#)

```
theorem pinching_kraus_mul_self (p : MState d) (i : spectrum ℝ p.m) :
  (pinching_kraus p i).mat * p.m = i.val • pinching_kraus p i
```

**instance finite\_spectrum\_inst**[\[source\]](#)

```
instance finite_spectrum_inst (p : MState d) : Fintype (spectrum ℝ p.m)
```

**theorem pinching\_kraus\_orthogonal**[\[source\]](#)

```
theorem pinching_kraus_orthogonal (p : MState d) {i j : spectrum ℝ p.m} (h : i ≠ j) :
  (pinching_kraus p i).mat * (pinching_kraus p j).mat = 0
```

**theorem pinching\_sq\_eq\_self**[\[source\]](#)

*The Kraus operators of the pinching channel are projectors: they square to themselves.*

```
@[simp]
theorem pinching_sq_eq_self (ρ : MState d) (k) : (pinching_kraus ρ k) ^ 2 = pinching_kraus ρ k
```

**theorem pinching\_kraus\_ortho**[\[source\]](#)

*The Kraus operators of the pinching channel are orthogonal projectors.*

```
theorem pinching_kraus_ortho (ρ : MState d) (i j : spectrum ℝ ρ.m) :
  (pinching_kraus ρ i).mat * (pinching_kraus ρ j).mat = if i = j then (pinching_kraus ρ i).
  mat else 0
```

**theorem pinching\_sum**[\[source\]](#)

```
theorem pinching_sum (ρ : MState d) : ∑ k, pinching_kraus ρ k = 1
```

**def pinching\_map**[\[source\]](#)

```
def pinching_map (ρ : MState d) : CPTPMap d d ℂ
```

**theorem pinchingMap\_apply\_M**[\[source\]](#)

```
theorem pinchingMap_apply_M (σ ρ : MState d) : (pinching_map σ ρ).M =
  (_, (MatrixMap.of_kraus_isCompletelyPositive
    (HermitianMat.mat ∘ pinching_kraus σ)).IsPositive.IsHermitianPreserving ρ.M.H)
```

**theorem pinching\_eq\_sum\_conj**[\[source\]](#)

```
theorem pinching_eq_sum_conj (σ ρ : MState d) : (pinching_map σ ρ).M =
  ∑ k, (pinching_kraus σ k).mat * ρ.M * (pinching_kraus σ k).mat
```

**theorem pinching\_commutates\_kraus**[\[source\]](#)

```
theorem pinching_commutates_kraus (σ ρ : MState d) (i : spectrum ℝ σ.m) :
  Commute (pinching_map σ ρ).m (pinching_kraus σ i).mat
```

**theorem pinching\_commutates**[\[source\]](#)

```
theorem pinching_commutates (ρ σ : MState d) :
  Commute (pinching_map σ ρ).m σ.m
```

**theorem pinching\_self**[\[source\]](#)

```
@[simp]
theorem pinching_self (ρ : MState d) : pinching_map ρ ρ = ρ
```

**theorem pinching\_bound**[\[source\]](#)

*Lemma 3.10 of Hayashi's book "Quantum Information Theory - Mathematical Foundations". Also, Lemma 5 in <https://arxiv.org/pdf/quant-ph/0107004>. – Used in (S60)*

```
theorem pinching_bound (ρ σ : MState d) : ρ.M ≤ (↑(Fintype.card (spectrum ℝ σ.m)) : ℝ) • (
  pinching_map σ ρ).M
```

**theorem ker\_le\_ker\_pinching\_of\_PosDef**[\[source\]](#)

```
theorem ker_le_ker_pinching_of_PosDef (ρ σ : MState d) (hpos : σ.m.PosDef) : σ.M.ker ≤ (
  pinching_map σ ρ).M.ker
```

**theorem pinching\_idempotent**[\[source\]](#)

```
theorem pinching_idempotent (ρ σ : MState d) :
  (pinching_map σ) (pinching_map σ ρ) = (pinching_map σ ρ)
```

**theorem inner\_cfc\_pinching**[\[source\]](#)

```
theorem inner_cfc_pinching (ρ σ : MState d) (f : ℝ → ℝ) :
  ⟨⟨ρ.M, (pinching_map σ ρ).M.cfc f⟩⟩ = ⟨⟨(pinching_map σ ρ).M, (pinching_map σ ρ).M.cfc f⟩⟩
```

**theorem inner\_cfc\_pinching\_right**[\[source\]](#)

```
theorem inner_cfc_pinching_right (ρ σ : MState d) (f : ℝ → ℝ) :
  ⟨⟨(pinching_map σ ρ).M, σ.M.cfc f⟩⟩ = ⟨⟨ρ.M, σ.M.cfc f⟩⟩
```

**theorem pinching\_map\_eq\_sum\_conj\_hermitian**[\[source\]](#)

```
theorem pinching_map_eq_sum_conj_hermitian (σ ρ : MState d) :
  (pinching_map σ ρ).M = ∑ k, ρ.M.conj (pinching_kraus σ k).mat
```

**theorem pinching\_map\_ker\_le**[\[source\]](#)

```
theorem pinching_map_ker_le (ρ σ : MState d) : (pinching_map σ ρ).M.ker ≤ ρ.M.ker
```

**theorem pinching\_kraus\_ker\_of\_ne\_zero**[\[source\]](#)

```
theorem pinching_kraus_ker_of_ne_zero {d : Type*} [Fintype d] [DecidableEq d]
  (σ : MState d) (v : d → ℂ) (hv : σ.m.mulVec v = 0)
  (k : spectrum ℝ σ.m) (hk : k.val ≠ 0) :
  (pinching_kraus σ k).mat *v v = 0
```

**theorem ker\_le\_ker\_pinching\_map\_ker**[\[source\]](#)

```
theorem ker_le_ker_pinching_map_ker (ρ σ : MState d) (h : σ.M.ker ≤ ρ.M.ker) :
  σ.M.ker ≤ (pinching_map σ ρ).M.ker
```

**theorem pinching\_pythagoras**[\[source\]](#)

*Exercise 2.8 of Hayashi's book "A group theoretic approach to Quantum Information".*

```
theorem pinching_pythagoras (ρ σ : MState d) :
  D(ρ||σ) = D(ρ||pinching_map σ ρ) + D(pinching_map σ ρ||σ)
```

**2.45 QuantumInfo.Finite.POVm**

[QuantumInfo/Finite/POVM.lean](#) on GitHub.

**structure POVM**[\[source\]](#)

*A POVM is a (finite) collection of PSD matrices on the same Hilbert space that sum to the identity. Here  $X$  indexes the matrices, and  $d$  is the space dimension.*

*Applied to an `MState` on that space with `POVM.measure`, this produces a distribution of outcomes indexed by the same type as the collection.*

*This measurement action can be composed with `MState.of_classical`, in which case it is equal to a CPTP map `measurement_map`.*

```
structure POVM (X : Type*) (d : Type*) [Fintype X] [Fintype d] [DecidableEq d] where
  mats : X → HermitianMat d ℂ
  nonneg : ∀ x, 0 ≤ (mats x)
  normalized : ∑ x, mats x = 1
```

**def measurementMap**[\[source\]](#)

*The act of measuring is a quantum channel, that maps a  $d$ -dimensional quantum state to an  $d \times X$ -dimensional quantum-classical state.*

```
def measurementMap (Λ : POVM X d) : CPTPMap d (d × X) where
```

**theorem measurementMap\_apply\_matrix**[\[source\]](#)

```
theorem measurementMap_apply_matrix (Λ : POVM X d) (m : Matrix d d ℂ) :
  Λ.measurementMap.map m = ∑ x : X,
    (((Λ.mats x) ^ (1/2:ℝ)).mat * m * ((Λ.mats x)^(1/2:ℝ)).mat) ⊗k Matrix.single x x 1
```

**theorem measurementMap\_apply\_hermitianMat**[\[source\]](#)

```
theorem measurementMap_apply_hermitianMat (Λ : POVM X d) (m : HermitianMat d ℂ) :
  Λ.measurementMap.toHPMap m = ∑ x : X,
```

**def measure**[\[source\]](#)

*A POVM leads to a distribution of outcomes on any given mixed state  $\rho$ .*

```
def measure (Λ : POVM X d) (ρ : MState d) : ProbDistribution X
```

**theorem traceLeft\_measurementMap\_eq\_measure**[\[source\]](#)

*The quantum-classical `POVM.measurement_map`, gives a marginal on the right equal to `POVM.measure`.*

```
theorem traceLeft_measurementMap_eq_measure ( $\Lambda$  : POVM X d) (p : MState d) :
  ( $\Lambda$ .measurementMap p).traceLeft = MState.ofClassical ( $\Lambda$ .measure p)
```

**def measureDiscard**

[\[source\]](#)

*The action of measuring a state with the POVM  $\Lambda$ , discarding the resulting state, and keeping the mixed state recording the outcome. This resulting state is purely diagonal, as given in `POVM.measureDiscard_apply`.*

```
noncomputable def measureDiscard ( $\Lambda$  : POVM X d) : CPTPMap d X
```

**theorem measureDiscard\_apply**

[\[source\]](#)

```
theorem measureDiscard_apply ( $\Lambda$  : POVM X d) (p : MState d) :
   $\Lambda$ .measureDiscard p = MState.ofClassical ( $\Lambda$ .measure p)
```

**def measureForget**

[\[source\]](#)

*The action of measuring a state with the POVM  $\Lambda$ , forgetting the measurement outcome, and keeping the disturbed state.*

```
noncomputable def measureForget ( $\Lambda$  : POVM X d) : CPTPMap d d
```

## 2.46 QuantumInfo.Finite.Qubit.Basic

[QuantumInfo/Finite/Qubit/Basic.lean](#) on GitHub.

**abbrev Qubit**

[\[source\]](#)

```
abbrev Qubit
```

**def Z**

[\[source\]](#)

*The Pauli Z gate on a qubit.*

```
def Z : U[Qubit]
```

**def X**

[\[source\]](#)

*The Pauli X gate on a qubit.*

```
def X : U[Qubit]
```

**def Y**

[\[source\]](#)

*The Pauli Y gate on a qubit.*

```
def Y : U[Qubit]
```



**def H** [\[source\]](#)

*The H gate, a Hadamard gate, on a qubit.*

```
noncomputable def H : U[Qubit]
```

**def S** [\[source\]](#)

*The S gate, or  $Rz(\pi/2)$  rotation on a qubit.*

```
def S : U[Qubit]
```

**def T** [\[source\]](#)

*The T gate, or  $Rz(\pi/4)$  rotation on a qubit.*

```
noncomputable def T : U[Qubit]
```

**theorem Z\_sq** [\[source\]](#)

```
@[simp]
theorem Z_sq : Z * Z = 1
```

**theorem X\_sq** [\[source\]](#)

```
@[simp]
theorem X_sq : X * X = 1
```

**theorem Y\_sq** [\[source\]](#)

```
@[simp]
theorem Y_sq : Y * Y = 1
```

**theorem H\_sq** [\[source\]](#)

```
@[simp]
theorem H_sq : H * H = 1
```

**theorem S\_sq** [\[source\]](#)

```
@[simp]
theorem S_sq : S * S = Z
```

**theorem T\_sq** [\[source\]](#)

```
@[simp]
theorem T_sq : T * T = S
```

**theorem X\_Y\_anticom** [\[source\]](#)

*The anticommutator  $\{X, Y\}$  is zero. Marked simp as to put Pauli products in a canonical Y-X-Z order.*

```
@[simp]
theorem X_Y_anticom : X * Y = -Y * X
```

**theorem Y\_Z\_anticom** [\[source\]](#)

*The anticommutator  $\{Y, Z\}$  is zero.*

```
theorem Y_Z_anticom : Z * Y = -Y * Z
```

**theorem Z\_X\_anticom** [\[source\]](#)

*The anticommutator  $\{Z, X\}$  is zero.*

```
theorem Z_X_anticom : Z * X = -X * Z
```

**theorem H\_mul\_X\_eq\_Z\_mul\_H** [\[source\]](#)

```
@[simp]
theorem H_mul_X_eq_Z_mul_H : H * X = Z * H
```

**theorem H\_mul\_Z\_eq\_X\_mul\_H** [\[source\]](#)

```
@[simp]
theorem H_mul_Z_eq_X_mul_H : H * Z = X * H
```

**theorem S\_Z\_comm** [\[source\]](#)

```
@[simp]
theorem S_Z_comm : Z * S = S * Z
```

**theorem T\_Z\_comm** [\[source\]](#)

```
@[simp]
theorem T_Z_comm : Z * T = T * Z
```

**theorem S\_T\_comm** [\[source\]](#)

```
@[simp]
theorem S_T_comm : S * T = T * S
```

**def controllize** [\[source\]](#)

*Given a unitary  $U$  on some Hilbert space  $k$ , we have the controllized version that acts on  $\text{Fin } 2 \otimes k$  where  $U$  is conditionally applied if the first qubit is 1.*

```
def controllize (g : U[k]) : U[Qubit × k]
```

**def CNOT** [\[source\]](#)

*Controlled-NOT gate on two qubits. The first qubit is the control and the second qubit is the target.*

```
def CNOT : U[Fin 2 × Fin 2]
```

**lemma CNOT\_matrix**

[\[source\]](#)

*The matrix representation of CNOT is the standard  $4 \times 4$  permutation matrix.*

```
lemma CNOT_matrix :
  Matrix.reindex finProdFinEquiv finProdFinEquiv CNOT.val =
    ![[!(1:C), 0, 0, 0],
      ![0, 1, 0, 0],
      ![0, 0, 0, 1],
      ![0, 0, 1, 0]]
```

**theorem controllize\_apply\_zero\_zero**

[\[source\]](#)

```
@[simp]
theorem controllize_apply_zero_zero : C[g] (0, j₁) (0, j₂) = (1 : U[k]) j₁ j₂
```

**theorem controllize\_apply\_zero\_one**

[\[source\]](#)

```
@[simp]
theorem controllize_apply_zero_one : C[g] (0, j₁) (1, j₂) = 0
```

**theorem controllize\_apply\_one\_zero**

[\[source\]](#)

```
@[simp]
theorem controllize_apply_one_zero : C[g] (1, j₁) (0, j₂) = 0
```

**theorem controllize\_apply\_one\_one**

[\[source\]](#)

```
@[simp]
theorem controllize_apply_one_one : C[g] (1, j₁) (1, j₂) = g j₁ j₂
```

**theorem controllize\_mul**

[\[source\]](#)

```
@[simp]
theorem controllize_mul (g₁ g₂ : U[k]) : C[g₁] * C[g₂] = C[g₁ * g₂]
```

**theorem controllize\_one**

[\[source\]](#)

```
@[simp]
theorem controllize_one : C[(1 : U[k])] = 1
```

**theorem controllize\_mul\_inv**

[\[source\]](#)

```
@[simp]
theorem controllize_mul_inv : C[g] * C[g⁻¹] = 1
```

**theorem X\_controllize\_X**[\[source\]](#)

@[simp]

**theorem** X\_controllize\_X :  $(X \otimes_u 1) * C[g] * (X \otimes_u 1) = (1 \otimes_u g) * C[g^{-1}]$ **2.47 QuantumInfo.Finite.ResourceTheory.FreeState**[QuantumInfo/Finite/ResourceTheory/FreeState.lean](#) on GitHub.**class ResourcePretheory**[\[source\]](#)

A *ResourcePretheory* is a family of Hilbert spaces closed under tensor products, with an instance of *Fintype* and *DecidableEq* for each. It forms a pre-structure then on which to discuss resource theories. For instance, to talk about "two-party scenarios", we could write *ResourcePretheory*  $(\mathbb{N} \times \mathbb{N})$ , with  $H(a, b) := (\text{Fin } a) \times (\text{Fin } b)$ .

The *Semigroup*  $\iota$  structure means we have a way to take products of our labels of Hilbert spaces in a way that is associative (with actual equality). The *prodEquiv* lets us reinterpret between a product-labelled Hilbert spaces, and an actual pair of Hilbert spaces.

```
class ResourcePretheory ( $\iota$  : Type*) extends Semigroup  $\iota$  where
  H :  $\iota \rightarrow$  Type*
  [FinH :  $\forall$  i, Fintype (H i)]
  [DecEqH :  $\forall$  i, DecidableEq (H i)]
  [NonemptyH :  $\forall$  i, Nonempty (H i)]
  prodEquiv i j : H (i * j)  $\approx$  (H i)  $\times$  (H j)
  hAssoc i j k :
    ((prodEquiv (i * j) k).trans <|
      ((prodEquiv i j).prodCongr (Equiv.refl (H k))).trans <|
      (Equiv.prodAssoc _ _).trans <|
      ((Equiv.refl (H i)).prodCongr ((prodEquiv j k).symm))).trans
      (prodEquiv i (j * k)).symm
    )
    = Equiv.cast (congrArg H <| mul_assoc i j k)
```

**def prodRelabel**[\[source\]](#)

The *prod* operation of *ResourcePretheory* gives the natural product operation on *MStates* that puts us in a new Hilbert space of the category. Accessible by the notation  $\rho_1 \otimes_r \rho_2$ .

```
noncomputable def prodRelabel ( $\rho_1$  : MState (H i)) ( $\rho_2$  : MState (H j)) : MState (H (i * j))
```

**theorem prodRelabel\_assoc**[\[source\]](#)

```
theorem prodRelabel_assoc ( $\rho_1$  : MState (H i)) ( $\rho_2$  : MState (H j)) ( $\rho_3$  : MState (H k)) :
   $\rho_1 \otimes_r \rho_2 \otimes_r \rho_3 \approx \rho_1 \otimes_r (\rho_2 \otimes_r \rho_3)$ 
```

**theorem prodRelabel\_relabel\_cast\_prod**[\[source\]](#)

A *MState.relabel* can be distributed across a *prodRelabel*, if you have proofs that the factors correspond correctly.

```

theorem prodRelabel_relabel_cast_prod
  (ρ1 : MState (H i)) (ρ2 : MState (H j))
  (h : H (k * l) = H (i * j)) (hik : k = i) (hlj : l = j) :
  (ρ1 ⊗r ρ2).relabel (Equiv.cast h) =
  (ρ1.relabel (Equiv.cast (congrArg H hik))) ⊗r (ρ2.relabel (Equiv.cast (congrArg H hlj)))

```

### def prodCPTPMap

[\[source\]](#)

*The prod operation of ResourcePretheory gives the natural product operation on CPTPMaps. Accessible by the notation  $M_1 \otimes^{CP} M_2$ .*

```

noncomputable def prodCPTPMap (M1 : CPTPMap (H i) (H j)) (M2 : CPTPMap (H k) (H l)) :
  CPTPMap (H (i * k)) (H (j * l))

```

### theorem prod

[\[source\]](#)

```

theorem PosDef.prod {ρ : MState (H i)} {σ : MState (H j)} (hp : ρ.m.PosDef) (hσ : σ.m.PosDef)
  : (ρ ⊗r σ).m.PosDef

```

### theorem qRelEntropy\_prodRelabel

[\[source\]](#)

```

@[simp]
theorem qRelEntropy_prodRelabel (ρ1 ρ2 : MState (H i)) (σ1 σ2 : MState (H j)) :
  D(ρ1 ⊗r σ1 || ρ2 ⊗r σ2) = D(ρ1 || ρ2) + D(σ1 || σ2)

```

### theorem sandwichedRelEntropy\_prodRelabel

[\[source\]](#)

```

@[simp]
theorem sandwichedRelEntropy_prodRelabel {α : ℝ} (ρ1 ρ2 : MState (H i)) (σ1 σ2 : MState (H j))
  :
  D̃α(ρ1 ⊗r σ1 || ρ2 ⊗r σ2) = D̃α(ρ1 || ρ2) + D̃α(σ1 || σ2)

```

### class UnitalPretheory

[\[source\]](#)

*A ResourcePretheory is Unital if it has a Hilbert space of size 1, i.e.  $\mathbb{C}$ .*

```

class UnitalPretheory (ι : Type*) extends ResourcePretheory ι, MulOneClass ι, Unique (H 1)
  where
  prod_default {i} (ρ : MState (H i)) :
    (toResourcePretheory.prodRelabel ρ (Inhabited.default : MState (H 1))) ≈ ρ
  default_prod {i} (ρ : MState (H i)) :
    (toResourcePretheory.prodRelabel (Inhabited.default : MState (H 1)) ρ) ≈ ρ

```

### instance Monoid ι

[\[source\]](#)

```

instance : Monoid ι where

```

### def spacePow

[\[source\]](#)

*Powers of spaces.*

*We define it for Nat in a UnitalPretheory. In principal this could be done for any ResourcePretheory and be defined for PNat so that we don't have zeroth powers. In*

*anticipation that we might some day want that, and that we might do everything with a non-equality associator, we keep this as its own definition and keep our own names for rewriting theorems where possible.*

```
noncomputable def spacePow (i : ι) (n : ℕ) : ι
```

**theorem spacePow\_zero**

[\[source\]](#)

```
@[simp]
```

```
theorem spacePow_zero (i : ι) : i ^ 0 = 1
```

**theorem spacePow\_one**

[\[source\]](#)

```
@[simp]
```

```
theorem spacePow_one (i : ι) : i ^ 1 = i
```

**theorem spacePow\_succ**

[\[source\]](#)

```
theorem spacePow_succ (i : ι) (n : ℕ) : i ^ (n + 1) = (i ^ n) * i
```

**theorem spacePow\_add**

[\[source\]](#)

```
theorem spacePow_add (m n : ℕ) :
```

```
  i ^ (m + n) = (i ^ m) * (i ^ n)
```

**theorem spacePow\_mul**

[\[source\]](#)

```
theorem spacePow_mul (m n : ℕ) :
```

```
  i ^ (m * n) = (i ^ m) ^ n
```

**def statePow**

[\[source\]](#)

*Powers of states, using the resource theory's notion of product. Accessible via the notation  $\rho \otimes_r^n$ .*

```
noncomputable def statePow (ρ : MState (H i)) (n : ℕ) : MState (H (i ^ n))
```

**theorem statePow\_zero**

[\[source\]](#)

```
@[simp]
```

```
theorem statePow_zero (ρ : MState (H i)) : ρ ⊗r^[0] = default
```

**theorem statePow\_one**

[\[source\]](#)

```
@[simp]
```

```
theorem statePow_one (ρ : MState (H i)) : ρ ⊗r^[1] ≈ ρ
```

**theorem statePow\_succ**

[\[source\]](#)

```
theorem statePow_succ (ρ : MState (H i)) (n : ℕ) : ρ ⊗r^[n + 1] = ρ ⊗r^[n] ⊗r ρ
```

**theorem statePow\_add**[\[source\]](#)

```
theorem statePow_add (p : MState (H i)) (m n : ℕ) : p ⊗r[m + n] ≈ p ⊗r[m] ⊗r p ⊗r[n]
```

**theorem statePow\_add\_relabel**[\[source\]](#)

```
theorem statePow_add_relabel (p : MState (H i)) (m n : ℕ) :
  p ⊗r[m + n] = (p ⊗r[m] ⊗r p ⊗r[n]).relabel (Equiv.cast (by congr; exact pow_add i m n))
```

**theorem statePow\_mul**[\[source\]](#)

```
theorem statePow_mul (p : MState (H i)) (m n : ℕ) : p ⊗r[m * n] ≈ (p ⊗r[m]) ⊗r[n]
```

**theorem statePow\_mul\_relabel**[\[source\]](#)

```
theorem statePow_mul_relabel {i : ι} (p : MState (H i)) (m n : ℕ) :
  p ⊗r[m * n] = (p ⊗r[m]) ⊗r[n].relabel (Equiv.cast (congrArg H (pow_mul i m n)))
```

**theorem PosDef.npow**[\[source\]](#)

```
theorem PosDef.npow {p : MState (H i)} (hp : p.m.PosDef) (n : ℕ)
  : (p ⊗r[n]).m.PosDef
```

**theorem statePow\_rw**[\[source\]](#)

```
theorem statePow_rw {n m : ℕ} (h : n = m) (p : MState (H i)) :
  p ⊗r[n] = (p ⊗r[m]).relabel (Equiv.cast (by congr))
```

**theorem qRelEntropy\_statePow**[\[source\]](#)

```
@[simp]
theorem qRelEntropy_statePow (p σ : MState (H i)) (n : ℕ) :
  D(p ⊗r[n] || σ ⊗r[n]) = n * D(p || σ)
```

**theorem sInf\_spectrum\_rprod**[\[source\]](#)

```
theorem sInf_spectrum_rprod {j : ι} (p : MState (H i)) (σ : MState (H j)) :
  sInf (spectrum ℝ (p ⊗r σ).m) = sInf (spectrum ℝ p.m) * sInf (spectrum ℝ σ.m)
```

**lemma sInf\_spectrum\_spacePow**[\[source\]](#)

```
lemma sInf_spectrum_spacePow (σ : MState (H i)) (n : ℕ) :
  sInf (spectrum ℝ (σ ⊗r[n]).m) = sInf (spectrum ℝ σ.m) ^ n
```

**theorem sandwichedRelEntropy\_statePow**[\[source\]](#)

```
@[simp]
theorem sandwichedRelEntropy_statePow {α : ℝ} (p σ : MState (H i)) (n : ℕ) :
  D_α(p ⊗r[n] || σ ⊗r[n]) = n * D_α(p || σ)
```

**class FreeStateTheory**[\[source\]](#)

*A FreeStateTheory is a collection of mixed states (MStates) in a ResourcePretheory that obeys some necessary axioms: \* For each Hilbert space  $H$   $i$ , the free states are a closed, convex set \* For each Hilbert space  $H$   $i$ , there is a free state of full rank \* If  $\rho_i \in H$   $i$  and  $\rho_j \in H$   $j$  are free, then  $\rho_i \otimes \rho_j$  is free in  $H$  (prod  $i$   $j$ ).*

```
class FreeStateTheory (ι : Type*) extends ResourcePretheory ι where
  IsFree : Set (MState (toResourcePretheory.H i))
  free_closed : IsClosed (@IsFree i)
  free_convex : Convex ℝ (MState.M '' (@IsFree i))
  free_prod {ρ₁ : MState (H i)} {ρ₂ : MState (H j)} (h₁ : IsFree ρ₁) (h₂ : IsFree ρ₂) : IsFree
    (ρ₁ ⊗r ρ₂)
  free_fullRank (i : ι) : open ComplexOrder in ∃ (p : MState (H i)), p.m.PosDef ∧ IsFree p
```

**instance Inhabited\_IsFree**[\[source\]](#)

```
noncomputable instance Inhabited_IsFree : Inhabited (IsFree (i := i))
```

**theorem of\_unique**[\[source\]](#)

```
theorem IsFree.of_unique [Unique (H i)] (p : MState (H i)) : p ∈ IsFree
```

**theorem IsCompact\_IsFree**[\[source\]](#)

*The set of free states is compact because it's a closed subset of a compact space.*

```
theorem IsCompact_IsFree : IsCompact (IsFree (i := i))
```

**theorem mix**[\[source\]](#)

```
theorem IsFree.mix {ι : Type*} [FreeStateTheory ι] {i : ι} {σ₁ σ₂ : MState (H i)}
  (hσ₁ : IsFree σ₁) (hσ₂ : IsFree σ₂) (p : Prob) : IsFree (p [σ₁ ↔ σ₂])
```

**class UnitalFreeStateTheory**[\[source\]](#)

```
class UnitalFreeStateTheory (ι : Type*) extends FreeStateTheory ι, UnitalPretheory ι
```

**theorem IsFree.npow**[\[source\]](#)

```
theorem _root_.FreeStateTheory.IsFree.npow {i : ι} {ρ : MState (H i)}
  (hp : IsFree ρ) (n : ℕ) : IsFree (ρ ⊗r^[n])
```

**theorem relabel\_cast\_isFree**[\[source\]](#)

```
@[simp]
theorem relabel_cast_isFree {i j : ι} (p : MState (H i)) (h : j = i) {h' : H j = H i}:
  p.relabel (Equiv.cast h') ∈ IsFree ↔ p ∈ IsFree
```

**lemma relativeEntResource\_ne\_top**[\[source\]](#)

*In a FreeStateTheory, we have free states of full rank, therefore the minimum relative entropy of any state  $\rho$  to a free state is finite.*



```
@[aesop (rule_sets := [finiteness]) safe]
lemma relativeEntResource_ne_top (ρ : MState (H i)) :  $\bigcap \sigma \in \text{IsFree}, \mathbf{D}(\rho \parallel \sigma) \neq \top$ 
```

**def RelativeEntResource**[\[source\]](#)

```
noncomputable def RelativeEntResource : MState (H i) → ℝ≥0
```

**theorem exists\_isFree\_relativeEntResource**[\[source\]](#)

```
theorem exists_isFree_relativeEntResource (ρ : MState (H i)) :
  ∃ σ ∈ IsFree,  $\mathbf{D}(\rho \parallel \sigma) = R_r \rho$ 
```

**theorem Subadditive**[\[source\]](#)

```
theorem RelativeEntResource.Subadditive (ρ : MState (H i)) : Subadditive fun n ↦  $R_r (\rho \otimes_r^n)$ 
```

**def RegularizedRelativeEntResource**[\[source\]](#)

```
noncomputable def RegularizedRelativeEntResource (ρ : MState (H i)) : ℝ≥0
```

**theorem tendsto**[\[source\]](#)

*Lemma 5*

```
theorem RelativeEntResource.tendsto (ρ : MState (H i)) :
  Filter.atTop.Tendsto (fun n ↦  $R_r (\rho \otimes_r^n)$  / n) (ℳ (Rr∞ ρ))
```

**theorem tendsto\_ennreal**[\[source\]](#)

*Alternate version of Lemma 5 which states the convergence with the ENNReal expression for RelativeEntResource, as opposed its untop-ped NNReal value.*

```
theorem RelativeEntResource.tendsto_ennreal (ρ : MState (H i)) :
  Filter.atTop.Tendsto (fun n ↦ ( $\bigcap \sigma \in \text{IsFree}, \mathbf{D}(\rho \otimes_r^n \parallel \sigma)$ ) / ↑n) (ℳ (Rr∞ ρ))
```

**def GlobalRobustness**[\[source\]](#)

```
noncomputable def GlobalRobustness {i : ι} : MState (H i) → ℝ≥0
```

**def IsAsymptoticallyNongenerating**[\[source\]](#)

*A sequence of operations  $f_n$  is asymptotically nongenerating if  $\lim_{n \rightarrow \infty} RG(f_n(\rho_n)) = 0$ , where  $RG$  is GlobalRobustness and  $\rho_n$  is any sequence of free states. Equivalently, we can take the  $\max$  ( over operations and states) on the left-hand side inside the limit.*

```
def IsAsymptoticallyNongenerating (dI dO : ι) (f : (n : ℕ) → CPTPMap (H (dI ⊗H [n])) (H (dO ⊗H [n]))) : Prop
```

## 2.48 QuantumInfo.Finite.ResourceTheory.HypothesisTesting

[QuantumInfo/Finite/ResourceTheory/HypothesisTesting.lean](#) on GitHub.

**def OptimalHypothesisRate** [\[source\]](#)

*The optimal hypothesis testing rate, for a tolerance  $\varepsilon$ : given a state  $\rho$  and a set of states  $S$ , the optimum distinguishing rate that allows a probability  $\varepsilon$  of errors.*

```
noncomputable def OptimalHypothesisRate (ρ : MState d) (ε : Prob) (S : Set (MState d)) : Prob
```

**instance iInf\_Inhabited** [\[source\]](#)

*The space of strategies  $T$  in `OptimalHypothesisRate` is inhabited, we always have some valid strategy.*

```
instance iInf_Inhabited (ρ : MState d) (ε : Prob) :  
  Inhabited { m // ρ.exp_val (1 - m) ≤ ε ∧ 0 ≤ m ∧ m ≤ 1 }
```

**instance Inhabited {m | ρ.exp\_val (1 - m) ≤ ε ∧ 0 ≤ m ∧ m ≤ 1}** [\[source\]](#)

```
instance (ρ : MState d) (ε : Prob) : Inhabited {m | ρ.exp_val (1 - m) ≤ ε ∧ 0 ≤ m ∧ m ≤ 1}
```

**theorem iInf\_IsCompact** [\[source\]](#)

*The space of strategies  $T$  in `OptimalHypothesisRate` is compact.*

```
theorem iInf_IsCompact (ρ : MState d) (ε : Prob) : IsCompact { m | ρ.exp_val (1 - m) ≤ ε ∧ 0 ≤ m ∧ m ≤ 1 }
```

**theorem iInf\_IsConvex** [\[source\]](#)

*The space of strategies  $T$  in `OptimalHypothesisRate` is convex.*

```
theorem iInf_IsConvex (ρ : MState d) (ε : Prob) : Convex ℝ { m | ρ.exp_val (1 - m) ≤ ε ∧ 0 ≤ m ∧ m ≤ 1 }
```

**theorem of\_empty** [\[source\]](#)

*When  $S$  is empty, the optimal hypothesis testing rate is zero.*

```
@[simp]  
theorem of_empty {ρ : MState d} (ε : Prob) : β_ ε(ρ||∅) = 0
```

**theorem le\_sup\_exp\_val** [\[source\]](#)

```
theorem le_sup_exp_val {ρ : MState d} (ε : Prob) {S : Set (MState d)}  
  (m : HermitianMat d ℂ) (hExp : ρ.exp_val (1 - m) ≤ ε) (hm : 0 ≤ m ∧ m ≤ 1) :  
  β_ ε(ρ||S) ≤ ⨆ σ ∈ S, (⟦_, σ.exp_val_prob hm)
```

**theorem le\_of\_subset** [\[source\]](#)

```
theorem le_of_subset (ρ : MState d) (ε : Prob) {S1 S2 : Set (MState d)} (h : S1 ⊆ S2) :  
  β_ ε(ρ||S1) ≤ β_ ε(ρ||S2)
```

**theorem of\_singleton**[\[source\]](#)

```

theorem of_singleton {p σ : MState d} {ε : Prob} :
  β_ε(p||{σ}) =
    ⌊ T : { m : HermitianMat d ℂ // p.exp_val (1 - m) ≤ ε ∧ 0 ≤ m ∧ m ≤ 1 },
    (⌊, σ.exp_val_prob T.2.right)

```

**theorem negLog\_le\_singleton**[\[source\]](#)

```

theorem negLog_le_singleton (p : MState d) (ε : Prob) (S : Set (MState d))
  (σ : MState d) (h : σ ∈ S) : -log β_ε(p||S) ≤ -log β_ε(p||{σ})

```

**theorem singleton\_le\_exp\_val**[\[source\]](#)

```

theorem singleton_le_exp_val {p σ : MState d} {ε : Prob} (m : HermitianMat d ℂ)
  (hExp : p.exp_val (1 - m) ≤ ε) (hm : 0 ≤ m ∧ m ≤ 1) :
  β_ε(p||{σ}) ≤ (⌊, σ.exp_val_prob hm)

```

**theorem exists\_min'**[\[source\]](#)

*There exists an optimal  $T$  for the hypothesis testing, that is, it's a minimum and not just an infimum. This states we have  $1 - \varepsilon \leq p.\text{exp\_val } T$ , but we can always "worsen"  $T$  to make that bound tight, which is `exists_min`.*

```

theorem exists_min' (p : MState d) (ε : Prob) (S : Set (MState d)) :
  ∃ (T : { m : HermitianMat d ℂ // p.exp_val (1 - m) ≤ ε ∧ 0 ≤ m ∧ m ≤ 1 }),
    (⌊ σ ∈ S, (⌊, σ.exp_val_prob T.prop.right) = β_ε(p||S))
    ∧ 1 - ε ≤ p.exp_val T

```

**theorem exists\_min**[\[source\]](#)

*There exists an optimal  $T$  for the hypothesis testing, that is, it's a minimum and not just an infimum. This tightens the  $T$  from `exists_min'` to a  $\langle\langle p, T \rangle\rangle = 1 - \varepsilon$  bound.*

```

theorem exists_min (p : MState d) (ε : Prob) (S : Set (MState d)) :
  ∃ (T : { m : HermitianMat d ℂ // p.exp_val (1 - m) ≤ ε ∧ 0 ≤ m ∧ m ≤ 1 }),
    (⌊ σ ∈ S, (⌊, σ.exp_val_prob T.prop.right) = β_ε(p||S))
    ∧ p.exp_val T = 1 - ε

```

**theorem pos\_of\_lt\_one**[\[source\]](#)

*When the allowed Type I error  $\varepsilon$  is less than 1 (so, we have some limit on our errors), and the kernel of the state  $p$  contains the kernel of some element in  $S$ , then the optimal hypothesis rate is positive - there is some lower bound on the type II errors we'll see. In other words, under these conditions, we cannot completely avoid type II errors.*

```

theorem pos_of_lt_one {p : MState d} (S : Set (MState d))
  (hp : ∃ σ ∈ S, σ.M.ker ≤ p.M.ker)
  {ε : Prob} (hε : ε < 1) : 0 < β_ε(p||S)

```

**theorem Lemma3**[\[source\]](#)

```
theorem Lemma3 {p : MState d} (ε : Prob) {S : Set (MState d)} (hS1 : IsCompact S)
(hS2 : Convex ℝ (MState.M '' S)) :  $\bigcup \sigma \in S, \beta_{- \varepsilon}(\rho || \{\sigma\}) = \beta_{- \varepsilon}(\rho || S)$ 
```

### **theorem ker\_diagonal\_prob\_eq\_bot**

[\[source\]](#)

```
theorem ker_diagonal_prob_eq_bot {q : Prob} (hq1 : 0 < q) (hq2 : q < 1) :
HermitianMat.ker (.diagonal ℂ (ProbDistribution.coin q .)) = ⊥
```

### **theorem optimalHypothesisRate\_antitone**

[\[source\]](#)

*Lemma S1*

```
theorem optimalHypothesisRate_antitone (p σ : MState d) (ℰ : CPTPMap d d2) (ε : Prob) :
β- ε(ρ || {σ}) ≤ β- ε(ℰ ρ || {ℰ σ})
```

### **theorem Ref81Lem5**

[\[source\]](#)

*This is from [Strong converse exponents for a quantum channel discrimination problem and quantum-feedback-assisted communication](<https://doi.org/10.1007/s00220-016-2645-4>), Lemma 5.*

*It seems like this is actually true for all  $0 < \alpha$  (with appropriate modifications at  $\alpha = 1$ ), but we only need it for the case of  $1 < \alpha$ .*

```
theorem Ref81Lem5 (p σ : MState d) (ε : Prob) (hε : ε < 1) (α : ℝ) (hα : 1 < α) :
- log β- ε(ρ || {σ}) ≤ D- α(ρ || σ) + -log (1 - ε) *
(.ofNNReal (α, zero_le_one.trans hα.le)) / (.ofNNReal (α - 1, sub_nonneg_of_le hα.le))
```

### **theorem rate\_pos\_of\_smul\_pos**

[\[source\]](#)

```
theorem rate_pos_of_smul_pos {ε : Prob} {d : Type*} [Fintype d] [DecidableEq d] {ρ σ1 σ2 :
MState d}
(hσ2 : 0 < β- ε(ρ || {σ2})) {c : ℝ} (hc : 0 < c) (hσ : c • σ2 ≤ σ1.M) : 0 < β- ε(ρ || {σ1})
```

### **theorem rate\_Continuous\_singleton**

[\[source\]](#)

@[fun\_prop]

```
theorem rate_Continuous_singleton {ε : Prob} {d : Type*} [Fintype d] [DecidableEq d] (ρ :
MState d) :
Continuous fun σ ↦ β- ε(ρ || {σ})
```

### **theorem optimalHypothesisRate\_unique**

[\[source\]](#)

*On the 1D Hilbert space, the optimal hypothesis testing rate is simply  $1 - \varepsilon$ , since there's nothing to learn. (More generally this would hold whenever  $\rho = \sigma$ .)*

```
theorem optimalHypothesisRate_unique {d : Type*} [Fintype d] [DecidableEq d]
(ε : Prob) (p σ : MState d) [Unique d] : β- ε(ρ || {σ}) = 1 - ε
```

## **2.49 QuantumInfo.Finite.ResourceTheory.ResourceTheory**

[QuantumInfo/Finite/ResourceTheory/ResourceTheory.lean](#) on GitHub.

**class ResourceTheory**[\[source\]](#)

A quantum resource theory extends a *FreeStateTheory* with a collection of free operations. It is required that any state we can always prepare for free must be free, and this is all then the resource theory is "minimal", but we can have a more restricted set of operations.

```
class ResourceTheory (ι : Type*) extends FreeStateTheory ι where
  freeOps i j : Set (CPTPMap (H i) (H j))
  nongenerating {i j : ι} {f} (h : f ∈ freeOps i j) : ∀ ρ, IsFree ρ → IsFree (f ρ)
  free_id i : CPTPMap.id ∈ freeOps i i
  free_comp {i j k} (Y : freeOps j k) (X : freeOps i j) : Y.1.compose X.1 ∈ freeOps i k
```

**def maximal**[\[source\]](#)

Given a *FreeStateTheory*, there is a maximal set of free operations compatible with the free states. That is the set of all operations that don't generate non-free states from free states. We call this the maximal resource theory.

```
def maximal [FreeStateTheory ι] : ResourceTheory ι where
```

**def IsMaximal**[\[source\]](#)

A resource theory *IsMaximal* if it includes all non-generating operations.

```
def IsMaximal (r : ResourceTheory ι) : Prop
```

**structure IsTensorial**[\[source\]](#)

A resource theory *IsTensorial* if it includes tensor products of operations, creating free states, and discarding. This implies that includes a unit object.

```
structure IsTensorial [UnitalPretheory ι] : Prop where
  prod : ∀ {i j k l : ι} {f g}, f ∈ freeOps i k → g ∈ freeOps j l → (f ⊗CPr g) ∈ freeOps (prod i j) (prod k l)
  create : ∀ {i : ι} (ρ), IsFree ρ → CPTPMap.replacement ρ ∈ freeOps Unital.unit i
  destroy : ∀ (i : ι), CPTPMap.destroy ∈ freeOps i Unital.unit
```

**theorem maximal\_IsMaximal**[\[source\]](#)

The theory *ResourceTheory.maximal* always *IsMaximal*.

```
theorem maximal_IsMaximal [FreeStateTheory ι] : IsMaximal (maximal (ι := ι))
```

**2.50 QuantumInfo.Finite.ResourceTheory.SteinsLemma**

[QuantumInfo/Finite/ResourceTheory/SteinsLemma.lean](#) on GitHub.

**def Lemma6\_on**[\[source\]](#)

The  $\{\tilde{\sigma}\}_n$  defined in Lemma 6, also in equation (S40) in Lemma 7.

```
noncomputable def Lemma6_on (m : ℕ) (σf : MState (H i)) (σm : MState (H (i ^ m))) : (n : ℕ) →
  MState (H (i ^ n))
```

**theorem Lemma6\_on\_IsFree**[\[source\]](#)

```

theorem Lemma6_on_IsFree {σ1 : MState (H i)} {σm : (m : ℕ) → MState (H (i ^ m))} (hσ1_free :
  IsFree σ1)
  (hσm : ∀ (m : ℕ), σm m ∈ IsFree) (m n : ℕ) : Lemma6_on m σ1 (σm m) n ∈ IsFree

```

**theorem Lemma6**[\[source\]](#)

*Lemma 6 from the paper. We did end up doing the version that "works also in the case of  $\varepsilon = 0$ ", which is nice.*

```

private theorem Lemma6 {m : ℕ} (hm : 0 < m) (p of : MState (H i)) (σm : MState (H (i ^ m)))
  [of.M.NonSingular] {ε : Prob} (hε : ε < 1) :
  Filter.atTop.limsup (fun n ↦ -log β_ ε(p ⊗r^[n]||{Lemma6_on m of σm n}) / n) ≤ D(p ⊗r^[m]||σm)
    / m

```

**theorem LemmaS2liminf**[\[source\]](#)

```

theorem LemmaS2liminf {ε3 : Prob} {ε4 : ℝ≥0} (hε4 : 0 < ε4)
  {d : ℕ → Type*} [∀ n, Fintype (d n)] [∀ n, DecidableEq (d n)] (p : (n : ℕ) → MState (d n)) (
  σ : (n : ℕ) → MState (d n))
  {Rinf : ℝ≥0} (hRinf : Rinf ≥ Filter.atTop.liminf (fun (n : ℕ) ↦ -log β_ ε3(p n||{σ n}) / n))
  :
  (Filter.atTop.liminf (fun (n : ℕ) ↦ ⟨{ (p n).M ≥p (Real.exp (n * (Rinf + ε4))) • (σ n).M }, (p
    n).M⟩) ≤ 1 - ε3)

```

**theorem LemmaS2limsup**[\[source\]](#)

```

theorem LemmaS2limsup {ε3 : Prob} {ε4 : ℝ≥0} (hε4 : 0 < ε4)
  {d : ℕ → Type*} [∀ n, Fintype (d n)] [∀ n, DecidableEq (d n)] (p : (n : ℕ) → MState (d n)) (
  σ : (n : ℕ) → MState (d n))
  {Rsup : ℝ≥0} (hRsup : Rsup ≥ Filter.atTop.limsup (fun (n : ℕ) ↦ -log β_ ε3(p n||{σ n}) / n))
  :
  (Filter.atTop.limsup (fun (n : ℕ) ↦ ⟨{ (p n) ≥p (Real.exp (n * (Rsup + ε4))) • (σ n).M }, p n⟩)
    ≤ 1 - ε3)

```

**theorem LemmaS3\_helper**[\[source\]](#)

```

private theorem LemmaS3_helper {ε : Prob} {d : ℕ → Type*} [∀ n, Fintype (d n)] [∀ n,
  DecidableEq (d n)]
  (p σ1 σ2 : (n : ℕ) → MState (d n))
  (f : ℕ → ℝ≥0) (hσ : ∀ (i : ℕ), Real.exp (-f i) • (σ2 i).M ≤ (σ1 i).M) (n : ℕ) :
  -log β_ ε(p n||{σ1 n}) ≤ -log β_ ε(p n||{σ2 n}) + f n

```

**theorem LemmaS3\_inf**[\[source\]](#)

*Lemma S3 from the paper. What they denote as  $\sigma_n$  and  $\sigma_n'$ , we denote as  $\sigma_1$  and  $\sigma_2$ . The  $\exp(-o(n))$  we express as a function  $f : \mathbb{N} \rightarrow \mathbb{R}$ , together with the fact that  $f$  is little- $o$  of  $n$  (i.e. that  $f = o[.atTop] id$ ), and then writing  $\exp(-f)$ . We also split LemmaS3 into two parts, the  $\liminf$  part and the  $\limsup$  part. The theorem as written is true for any  $f$ , but we can restrict to nonnegative  $f$  (so,  $\mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ ) which is easier to work with and more natural in the subsequent proofs.*

```

private theorem LemmaS3_inf {ε : Prob}

```

```

{d : ℕ → Type*} [∀ n, Fintype (d n)] [∀ n, DecidableEq (d n)]
(ρ σ₁ σ₂ : (n : ℕ) → MState (d n))
(f : ℕ → ℝ≥0) (hf : (f · : ℕ → ℝ) =o[atTop] (· : ℕ → ℝ))
(hσ : ∀ i, Real.exp (-f i) • (σ₂ i).M ≤ σ₁ i)
:
Filter.atTop.liminf (fun (n : ℕ) ↦ -log β_ε(ρ n||{σ₁ n}) / n) ≤
  Filter.atTop.liminf (fun (n : ℕ) ↦ -log β_ε(ρ n||{σ₂ n}) / n)

```

**theorem LemmaS3\_sup**[\[source\]](#)

```

private theorem LemmaS3_sup {ε : Prob}
{d : ℕ → Type*} [∀ n, Fintype (d n)] [∀ n, DecidableEq (d n)]
(ρ σ₁ σ₂ : (n : ℕ) → MState (d n))
(f : ℕ → ℝ≥0) (hf : (f · : ℕ → ℝ) =o[atTop] (· : ℕ → ℝ))
(hσ : ∀ i, Real.exp (-f i) • (σ₂ i).M ≤ σ₁ i)
:
Filter.atTop.limsup (fun (n : ℕ) ↦ -log β_ε(ρ n||{σ₁ n}) / n) ≤
  Filter.atTop.limsup (fun (n : ℕ) ↦ -log β_ε(ρ n||{σ₂ n}) / n)

```

**def R1**[\[source\]](#)

```
private noncomputable def R1 (ρ : MState (H i)) (ε : Prob) : ℝ≥0∞
```

**def R2**[\[source\]](#)

```
private noncomputable def R2 (ρ : MState (H i)) : ((n : ℕ) → IsFree (i := i ^ n)) → ℝ≥0∞
```

**lemma commute\_aux**[\[source\]](#)

```

private lemma commute_aux (n : ℕ) {x : ℝ}
{E ℰ σ : HermitianMat d ℂ} (hℰσ : Commute ℰ.mat σ.mat)
(hE : E = 1 - {Real.exp (n * x) • σ ≤_p ℰ})
: Commute ((1 / n : ℝ) • E).mat (ℰ.log - σ.log).mat

```

**lemma rexp\_mul\_smul\_proj\_lt\_mul\_sub\_le\_mul\_sub**[\[source\]](#)

```

private lemma rexp_mul_smul_proj_lt_mul_sub_le_mul_sub {n : ℕ} {x : ℝ}
{E ℰ σ : HermitianMat d ℂ} (hℰσ : Commute ℰ.mat σ.mat) (hx : 0 < x)
(hℰ : ℰ.mat.PosSemidef) (hσ : σ.mat.PosDef)
(hE : E = 1 - {Real.exp (n * x) • σ ≤_p ℰ})
: ⟨⟨ℰ, ((1 / n : ℝ) • E) * (ℰ.log - σ.log)⟩⟩ ≤ ⟨⟨ℰ, x • E⟩⟩

```

**lemma rexp\_mul\_smul\_proj\_lt\_mul\_sub\_le\_mul\_sub'**[\[source\]](#)

```

private lemma rexp_mul_smul_proj_lt_mul_sub_le_mul_sub' {n : ℕ} {x : ℝ} {y : ℝ}
{E ℰ σ : HermitianMat d ℂ} (hℰσ : Commute ℰ.mat σ.mat)
(hℰ : ℰ.mat.PosSemidef) (hσ : σ.mat.PosDef)
(hE : E = {Real.exp (n * y) • σ ≤_p ℰ} - {Real.exp (n * x) • σ ≤_p ℰ})
: (1 / n : ℝ) • E.mat * (ℰ.log.mat - σ.log.mat) ≤ x • E

```

**lemma f\_image\_bound**[\[source\]](#)

```
private lemma f_image_bound (mineig : ℝ) (n : ℕ) (h : 0 < mineig) (hn : 0 < n) :
  let c : ℕ → ℝ
```

lemma c'\_bounded

[\[source\]](#)

```
private lemma c'_bounded {mineig : ℝ} {ε2 : ℕ → ℝ≥0}
  (hε2 : ∀ (n : ℕ), ε2 n < 1) (o : ℝ) :
  let c : ℕ → ℝ
```

def σ<sub>1</sub>

[\[source\]](#)

```
private def σ1 : MState (H i)
```

def σ<sub>1</sub>\_mineig

[\[source\]](#)

```
private def σ1_mineig
```

def σ<sub>1</sub>\_c

[\[source\]](#)

*The sequence  $c_n$  given in (S44). In order to handle when  $c = 0$ , we've replaced the  $(\text{Real.log } 3) / n$  term with  $(\text{Real.log } 3) / (\max n 1)$ .*

```
private def σ1_c (n : ℕ) : ℝ
```

def f\_map

[\[source\]](#)

*The function  $f_n(\lambda)$  in (S45).*

```
private def f_map (n : ℕ) (lam : ℝ) : ℝ
```

def «σ»

[\[source\]](#)

```
private def «σ» :
  (n : ℕ) → MState (H (i ^ n))
```

def «σ\*»

[\[source\]](#)

```
private def «σ*» : (n : ℕ) → MState (H (i ^ n))
```

def σ'

[\[source\]](#)

```
private def σ' : (n : ℕ) → MState (H (i ^ n))
```

def σ''\_unnormalized

[\[source\]](#)

```
private def σ''_unnormalized : (n : ℕ) → HermitianMat (H (i ^ n)) ℂ
```

theorem σ<sub>1</sub>\_pos

[\[source\]](#)

```
private theorem σ1_pos : (σ1 i).m.PosDef
```



**theorem  $\sigma_1$ \_isFree**[\[source\]](#)

```
private theorem  $\sigma_1$ _isFree : IsFree ( $\sigma_1$  i)
```

**theorem mineig\_pos**[\[source\]](#)

```
private theorem mineig_pos :  $0 < \sigma_1$ _mineig i
```

**theorem mineig\_le\_one**[\[source\]](#)

```
private theorem mineig_le_one :  $\sigma_1$ _mineig i  $\leq$  1
```

**theorem  $\sigma_1$ \_c\_pos**[\[source\]](#)

```
private theorem  $\sigma_1$ _c_pos (n) :  $0 < \sigma_1$ _c i n
```

**theorem  $\sigma_1$ \_c\_div\_lim**[\[source\]](#)

```
private theorem  $\sigma_1$ _c_div_lim : Filter.atTop.Tendsto (fun n  $\mapsto$  ( $\sigma_1$ _c i n) / n) ( $\mathcal{N}$  0)
```

**lemma  $\sigma_1$ \_c\_identity**[\[source\]](#)

```
private lemma  $\sigma_1$ _c_identity {n :  $\mathbb{N}$ } (hn :  $0 < n$ ) :  
  Real.exp (- $\sigma_1$ _c i n) * (1 / 3) *  $\sigma_1$ _mineig i ^ n =  
  Real.exp (-n * ( $\sigma_1$ _c i n +  $\sigma_1$ _c i n / n))
```

**theorem  $\sigma_1$ \_c\_little0**[\[source\]](#)

```
theorem  $\sigma_1$ _c_little0 : (fun n :  $\mathbb{N}$   $\mapsto$   $\sigma_1$ _c i n + Real.log 3) =o[Filter.atTop] (fun x  $\mapsto$  (x :  $\mathbb{R}$ ))
```

**theorem log\_le\_f**[\[source\]](#)

(S46), part 1

```
private theorem log_le_f (n :  $\mathbb{N}$ ) (lam :  $\mathbb{R}$ ) : Real.log lam  $\leq$  f_map i n lam
```

**theorem f\_le\_log**[\[source\]](#)

(S46), part 2

```
private theorem f_le_log (n :  $\mathbb{N}$ ) (lam :  $\mathbb{R}$ ) : f_map i n lam < Real.log lam +  $\sigma_1$ _c i n
```

**theorem le\_exp\_f**[\[source\]](#)

```
private theorem le_exp_f (n :  $\mathbb{N}$ ) (x :  $\mathbb{R}$ ) (hx :  $0 < x$ ) : x  $\leq$  Real.exp (f_map i n x)
```

**theorem exp\_f\_le**[\[source\]](#)

```
private theorem exp_f_le (n :  $\mathbb{N}$ ) (x :  $\mathbb{R}$ ) (hx :  $0 < x$ ) : Real.exp (f_map i n x) < Real.exp ( $\sigma_1$ _c  
  i n) * x
```

**theorem** «σ~isFree»[\[source\]](#)**private theorem** «σ~isFree» : IsFree («σ~ m σ n)**theorem** «σ\*\_free»[\[source\]](#)**private theorem** «σ\*\_free» : IsFree («σ\*» ρ ∈ n)**theorem** «σ\*\_max»[\[source\]](#)**private theorem** «σ\*\_max» :  
IsMax0n (fun σ ↦ β\_ ε(ρ ⊗<sub>r</sub>^[n]||{σ})) IsFree («σ\*» ρ ∈ n)**theorem** σ'\_free[\[source\]](#)**private theorem** σ'\_free : IsFree (σ' ρ ∈ m σ n)**theorem** σ'\_posdef[\[source\]](#)**private theorem** σ'\_posdef : (σ' ρ ∈ m σ n).m.PosDef**theorem** hσ'n\_eq\_sum\_third[\[source\]](#)**private theorem** hσ'n\_eq\_sum\_third : (σ' ρ ∈ m σ n).M =  
(1 / 3 : ℝ) • («σ~ m σ n) + (1 / 3 : ℝ) • («σ\*» ρ ∈ n) + (1 / 3 : ℝ) • ((σ<sub>1</sub> i) ⊗<sub>r</sub>^[n])**theorem** hσ<sub>1</sub>\_le\_σ'[\[source\]](#)**private theorem** hσ<sub>1</sub>\_le\_σ' : (1 / 3 : ℝ) • ((σ<sub>1</sub> i) ⊗<sub>r</sub>^[n]).M ≤ (σ' ρ ∈ m σ n).M**theorem** σ''\_unnormalized\_PosDef[\[source\]](#)**private theorem** σ''\_unnormalized\_PosDef : Matrix.PosDef (σ''\_unnormalized ρ ∈ m σ n).mat**theorem** σ''\_tr\_bounds[\[source\]](#)**private theorem** σ''\_tr\_bounds : 1 ≤ (σ''\_unnormalized ρ ∈ m σ n).trace ∧  
(σ''\_unnormalized ρ ∈ m σ n).trace < Real.exp (σ<sub>1</sub>\_c i n)**def** σ''[\[source\]](#)**private def** σ'' : (n : ℕ) → MState (H (i ^ n))**lemma** σ''\_posdef[\[source\]](#)**private lemma** σ''\_posdef n : (σ'' ρ ∈ m σ n).M.mat.PosDef**lemma** σ'\_le\_σ''[\[source\]](#)**private lemma** σ'\_le\_σ'' (n) : Real.exp (-σ<sub>1</sub>\_c i n) • (σ' ρ ∈ m σ n).M ≤ σ'' ρ ∈ m σ n

**lemma  $\sigma''\_le\_ \sigma'$**

[\[source\]](#)

```
private lemma  $\sigma''\_le\_ \sigma'$  (n) :  $\sigma'' \rho \in m \sigma n \leq \text{Real.exp } (\sigma_1\_c \ i \ n) \cdot (\sigma' \rho \in m \sigma n).M$ 
```

**theorem  $\ll \sigma''\_ge\_ \sigma^* \gg$**

[\[source\]](#)

```
private theorem  $\ll \sigma''\_ge\_ \sigma^* \gg$  n :  $\sigma'' \rho \in m \sigma n \geq (\text{Real.exp } (-\sigma_1\_c \ i \ n) / 3) \cdot (\ll \sigma^* \gg \rho \in n).M$ 
```

**theorem  $\ll \sigma''\_ge\_ \sigma^{\sim} \gg$**

[\[source\]](#)

```
private theorem  $\ll \sigma''\_ge\_ \sigma^{\sim} \gg$  n :  $\sigma'' \rho \in m \sigma n \geq (\text{Real.exp } (-\sigma_1\_c \ i \ n) / 3) \cdot (\ll \sigma^{\sim} m \sigma n \gg).M$ 
```

**theorem  $\sigma''\_ge\_ \sigma_1$**

[\[source\]](#)

```
private theorem  $\sigma''\_ge\_ \sigma_1$  n :  $\sigma'' \rho \in m \sigma n \geq (\text{Real.exp } (-\sigma_1\_c \ i \ n) / 3) \cdot ((\sigma_1 \ i) \otimes_r^n [n]).M$ 
```

**abbrev  $\varepsilon_0\_func$**

[\[source\]](#)

```
private abbrev  $\varepsilon_0\_func$  ( $\varepsilon'$  : Prob) :  $\mathbb{R}$ 
```

**theorem EquationS88**

[\[source\]](#)

```
private theorem EquationS88 ( $\rho$  : MState (H i)) ( $\sigma$  : (n :  $\mathbb{N}$ )  $\rightarrow$   $\uparrow$ IsFree) { $\varepsilon \ \varepsilon'$  : Prob}
  (hR1R2 :  $R1 \rho \varepsilon < R2 \rho \sigma$ ) (hR1 :  $R1 \rho \varepsilon \neq \top$ ) (hR2 :  $R2 \rho \sigma \neq \top$ ) (h $\varepsilon_0\_1$  :  $0 < \varepsilon_0\_func \rho \varepsilon \sigma$ 
     $\varepsilon'$ ) (m :  $\mathbb{N}$ )
  :
  let  $\mathcal{E}$ 
```

**theorem EquationS62**

[\[source\]](#)

```
private theorem EquationS62
  ( $\rho$  : MState (H i)) ( $\sigma$  : (n :  $\mathbb{N}$ )  $\rightarrow$  IsFree (i := i ^ n))
  { $\varepsilon \ \varepsilon'$  : Prob} (h $\varepsilon'_1$  :  $0 < \varepsilon'$ ) (h $\varepsilon'_2$  :  $\varepsilon' < \varepsilon$ ) (h $\varepsilon$  :  $\varepsilon < 1$ )
  (hR1R2 :  $R1 \rho \varepsilon < R2 \rho \sigma$ ) (hR1 :  $R1 \rho \varepsilon \neq \top$ ) (hR2 :  $R2 \rho \sigma \neq \top$ )
  (h $\varepsilon_0$  :  $0 < \varepsilon_0\_func \rho \varepsilon \sigma \varepsilon'$ ) (h $\varepsilon_0'$  :  $(R1 \rho \varepsilon).toReal \leq (R2 \rho \sigma).toReal + \varepsilon_0\_func \rho \varepsilon \sigma \varepsilon'$ )
  (m :  $\mathbb{N}$ ) (hm :  $m \geq 1 \wedge \mathbf{D}(\rho \otimes_r^m [m]) \upharpoonright (\sigma m) / \upharpoonright m < R2 \rho \sigma + (.ofNNReal (\varepsilon_0\_func \rho \varepsilon \sigma \varepsilon'), h\mathbf{e}_0.
    le)))$ 
```

**theorem Lemma7**

[\[source\]](#)

*Lemma 7 from the paper. We write  $\varepsilon'$  for their  $\tilde{\varepsilon}$ .*

```
private theorem Lemma7 ( $\rho$  : MState (H i)) { $\varepsilon$  : Prob} (h $\varepsilon$  :  $0 < \varepsilon \wedge \varepsilon < 1$ ) ( $\sigma$  : (n :  $\mathbb{N}$ )  $\rightarrow$  IsFree
  (i := i ^ n)) :
  ( $R2 \rho \sigma \geq R1 \rho \varepsilon$ )  $\rightarrow$ 
   $\forall \varepsilon' : \text{Prob}, (h\mathbf{e}' : 0 < \varepsilon' \wedge \varepsilon' < \varepsilon) \rightarrow$ 
   $\exists \sigma' : (n : \mathbb{N}) \rightarrow \text{IsFree } (i := i ^ n),$ 
   $R2 \rho \sigma' - R1 \rho \varepsilon \leq .ofNNReal (1 - \varepsilon' : \text{Prob}) * (R2 \rho \sigma - R1 \rho \varepsilon)$ 
```

**def Lemma7\_improver**[\[source\]](#)

*Lemma 7 gives us a way to repeatedly "improve" a sequence  $\sigma$  to one with a smaller gap between  $R2$  and  $R1$ . The paper paints this as pretty much immediate from Lemma7, but we need to handle the case where  $R2$  is below  $R1$ .*

```
private noncomputable def Lemma7_improver (p : MState (H i)) {ε : Prob} (hε : 0 < ε ∧ ε < 1) {
  ε' : Prob} (hε' : 0 < ε' ∧ ε' < ε) :
```

**theorem Lemma7\_gap**[\[source\]](#)

*The Lemma7\_improver does its job at shrinking the gap.*

```
theorem Lemma7_gap (p : MState (H i)) {ε : Prob} (hε : 0 < ε ∧ ε < 1) {ε' : Prob} (hε' : 0 < ε'
  ∧ ε' < ε) :
  ∀ σ,
    letI σ'
```

**theorem GeneralizedQSteinsLemma**[\[source\]](#)

*Theorem 1 in <https://arxiv.org/pdf/2408.02722v3>*

```
theorem GeneralizedQSteinsLemma {i : ι} (p : MState (H i)) {ε : Prob} (hε : 0 < ε ∧ ε < 1) :
  Filter.atTop.Tendsto (fun n ↦ -log β_ε(p ⊗_r^[n]||IsFree) / n) (N (R_∞ p))
```

**theorem limit\_hypotesting\_eq\_limit\_rel\_entropy**[\[source\]](#)

*Theorem 4, which is also called the Generalized quantum Stein's lemma in Hayashi & Yamasaki. What they state as an equality of limits, which don't exist per se in Mathlib, we state as the existence of a number (which happens to be `RegularizedRelativeEntResource`) to which both sides converge.*

```
theorem limit_hypotesting_eq_limit_rel_entropy (p : MState (H i)) (ε : Prob) (hε : 0 < ε ∧ ε <
  1) :
  ∃ d : ℝ≥0,
    Filter.atTop.Tendsto (fun n ↦ -log β_ε(p ⊗_r^[n]||IsFree) / n) (N d)
    ∧
    Filter.atTop.Tendsto (fun n ↦ (⌊ ⌋ σ ∈ IsFree, D(p ⊗_r^[n]||σ)) / n) (N d)
```

**2.51 QuantumInfo.Finite.Unitary**

[QuantumInfo/Finite/Unitary.lean](#) on GitHub.

**theorem trace\_conj\_unitary**[\[source\]](#)

```
@[simp]
theorem trace_conj_unitary : (conj U.val A).trace = A.trace
```

**theorem le\_conj\_unitary**[\[source\]](#)

```
@[simp]
theorem le_conj_unitary : A.conj U.val ≤ B.conj U ↔ A ≤ B
```

**theorem inner\_conj\_unitary**[\[source\]](#)

@[simp]

**theorem** inner\_conj\_unitary :  $\langle\langle A.\text{conj } U.\text{val}, B.\text{conj } U.\text{val} \rangle\rangle = \langle\langle A, B \rangle\rangle$ **theorem eigenvalues\_conj**[\[source\]](#)

*The eigenvalues of a Hermitian matrix conjugated by a unitary matrix are the same as the eigenvalues of the original matrix.*

@[simp]

**theorem** eigenvalues\_conj :  $(A.\text{conj } U.\text{val}).H.\text{eigenvalues} = A.H.\text{eigenvalues}$ **def U\_conj**[\[source\]](#)

*Conjugate a state by a unitary matrix (applying the unitary as an evolution).*

**def** U\_conj (p : MState d) (U : U[d]) : MState d **where****theorem U\_conj\_spectrum\_eq**[\[source\]](#)

*You might think this should only be true up to permutation, so that it would read like  $\exists \sigma : \text{Equiv.Perm } d, (\rho.U_{\text{conj}} U).\text{spectrum} = \rho.\text{spectrum}.\text{relabel } \sigma$ . But since eigenvalues of a matrix are always canonically sorted, this is actually an equality.*

@[simp]

**theorem** U\_conj\_spectrum\_eq (p : MState d) (U : U[d]) :  
 $(\rho.U_{\text{conj}} U).\text{spectrum} = \rho.\text{spectrum}$ **theorem inner\_U\_conj**[\[source\]](#)

@[simp]

**theorem** inner\_U\_conj (p σ : MState d) (U : U[d]) :  $\langle\langle U \triangleleft \rho, U \triangleleft \sigma \rangle\rangle_{\text{Prob}} = \langle\langle \rho, \sigma \rangle\rangle_{\text{Prob}}$ **theorem no\_cloning**[\[source\]](#)

*The **No-cloning theorem**, saying that if states  $\psi$  and  $\phi$  can both be perfectly cloned using a unitary  $U$  and a fiducial state  $f$ , and they aren't identical (their inner product is less than 1), then the two states must be orthogonal to begin with. In short: only orthogonal states can be simultaneously cloned.*

**theorem** no\_cloning {U : U[d × d]}(hψ : U  $\triangleleft$  pure (ψ  $\otimes^\phi$  f) = pure (ψ  $\otimes^\phi$  ψ))(hφ : U  $\triangleleft$  pure (φ  $\otimes^\phi$  f) = pure (φ  $\otimes^\phi$  φ))(H :  $\langle\langle \text{pure } \psi, \text{pure } \phi \rangle\rangle_{\text{Prob}} < (1 : \mathbb{R})$ ) : $\langle\langle \text{pure } \psi, \text{pure } \phi \rangle\rangle_{\text{Prob}} = (0 : \mathbb{R})$ **2.52 QuantumInfo.ForMathlib.ContinuousLinearMap**

QuantumInfo/ForMathlib/ContinuousLinearMap.lean on GitHub.

**theorem range\_zero**[\[source\]](#)

```
@[simp]
theorem range_zero [RingHomSurjective σ] : (θ : M →SL[σ] M₂).range = ⊥
```

**theorem ker\_zero** [\[source\]](#)

```
@[simp]
theorem ker_zero : (θ : M →SL[σ] M₂).ker = ⊤
```

**theorem ker\_mk** [\[source\]](#)

```
theorem ker_mk (f : M →SL[σ] M₂) (hf : Continuous f.toFun) :
  (ContinuousLinearMap.mk f hf).ker = LinearMap.ker f
```

**theorem support\_eq\_sup\_eigenspace\_nonzero** [\[source\]](#)

*The support of a Hermitian matrix is the sum of its nonzero eigenspaces.*

```
theorem support_eq_sup_eigenspace_nonzero (A : EuclideanSpace ℤ n →L[ℤ] EuclideanSpace ℤ n)
  (hA : A.IsSymmetric) : A.range = ⋃ μ ≠ 0, Module.End.eigenspace A μ
```

## 2.53 QuantumInfo.ForMathlib.ContinuousSup

[QuantumInfo/ForMathlib/ContinuousSup.lean](#) on GitHub.

**theorem sSup\_image\_eq\_sSup\_image\_closure** [\[source\]](#)

```
theorem sSup_image_eq_sSup_image_closure {f : β → α}
  (hS : IsCompact (closure S)) (hf : Continuous f) :
  sSup (f '' S) = sSup (f '' closure S)
```

**theorem sInf\_image\_eq\_sInf\_image\_closure** [\[source\]](#)

```
theorem sInf_image_eq_sInf_image_closure {f : β → α} (hS : IsCompact (closure S)) (hf :
  Continuous f) :
  sInf (f '' S) = sInf (f '' closure S)
```

**theorem closure\_continuous\_sSup** [\[source\]](#)

*A version of `IsCompact.continuous_sSup` with a slightly weaker hypothesis on the set  $K$ , that its closure is compact (but the set itself need not be).*

```
theorem closure_continuous_sSup (hS : IsCompact (closure S)) (hf : Continuous ↑f) :
  Continuous fun x ↦ sSup (f x '' S)
```

**theorem closure\_continuous\_sInf** [\[source\]](#)

*A version of `IsCompact.continuous_sInf` with a slightly weaker hypothesis on the set  $K$ , that its closure is compact (but the set itself need not be).*

```
theorem closure_continuous_sInf (hS : IsCompact (closure S)) (hf : Continuous ↑f) :
  Continuous fun x ↦ sInf (f x '' S)
```

**theorem IsCompact.continuous\_iSup**[\[source\]](#)

```
theorem continuous_iSup (hS : IsCompact (closure S)) (hf : Continuous ↑f) :
  Continuous fun x ↦ ⨆ y : S, f x y
```

**theorem IsCompact.continuous\_iInf**[\[source\]](#)

```
theorem continuous_iInf (hS : IsCompact (closure S)) (hf : Continuous ↑f) :
  Continuous fun x ↦ ⨅ y : S, f x y
```

**theorem continuous\_sSup**[\[source\]](#)

*Similar to IsCompact.continuous\_sSup, but taking a bounded set in the bornology instead of a compact set.*

```
theorem continuous_sSup (hS : Bornology.IsBounded S) (hf : Continuous ↑f) :
  Continuous fun x ↦ sSup (f x '' S)
```

**theorem continuous\_sInf**[\[source\]](#)

*Similar to IsCompact.continuous\_sInf, but taking a bounded set in the bornology instead of a compact set.*

```
theorem continuous_sInf (hS : Bornology.IsBounded S) (hf : Continuous ↑f) :
  Continuous fun x ↦ sInf (f x '' S)
```

**theorem IsBounded.continuous\_iSup**[\[source\]](#)

```
theorem continuous_iSup (hS : Bornology.IsBounded S) (hf : Continuous ↑f) :
  Continuous fun x ↦ ⨆ y : S, f x y
```

**theorem IsBounded.continuous\_iInf**[\[source\]](#)

```
theorem continuous_iInf (hS : Bornology.IsBounded S) (hf : Continuous ↑f) :
  Continuous fun x ↦ ⨅ y : S, f x y
```

**theorem LinearMap.continuous\_iSup**[\[source\]](#)

*For bilinear maps in suitably well-behaved spaces with IsModuleTopology, taking the supremum in one argument is still Continuous, by Bornology.IsBounded.continuous\_iSup.*

```
theorem continuous_iSup {E F ℚ : Type*}
  [CommRing ℚ] [TopologicalSpace ℚ] [IsTopologicalRing ℚ] [ConditionallyCompleteLinearOrder ℚ]
  [OrderTopology ℚ]
  [AddCommGroup E] [TopologicalSpace E] [Module ℚ E] [IsModuleTopology ℚ E]
  [AddCommGroup F] [Module ℚ F] [PseudoMetricSpace F] [ProperSpace F] [Module.Finite ℚ F] [
    IsModuleTopology ℚ F]
  (f : E →ℓ[ℚ] F →ℓ[ℚ] ℚ) (hS : Bornology.IsBounded S) :
  Continuous fun x ↦ ⨆ y : S, f x y
```

**theorem LinearMap.continuous\_iInf**[\[source\]](#)

*For bilinear maps in suitably well-behaved spaces with IsModuleTopology, taking the infimum in one argument is still Continuous, by Bornology.IsBounded.continuous\_iInf.*

```

theorem continuous_iInf {E F  $\mathbb{k}$  : Type*}
  [CommRing  $\mathbb{k}$ ] [TopologicalSpace  $\mathbb{k}$ ] [IsTopologicalRing  $\mathbb{k}$ ] [ConditionallyCompleteLinearOrder  $\mathbb{k}$ ]
  [OrderTopology  $\mathbb{k}$ ]
  [AddCommGroup E] [TopologicalSpace E] [Module  $\mathbb{k}$  E] [IsModuleTopology  $\mathbb{k}$  E]
  [AddCommGroup F] [Module  $\mathbb{k}$  F] [PseudoMetricSpace F] [ProperSpace F] [Module.Finite  $\mathbb{k}$  F] [
    IsModuleTopology  $\mathbb{k}$  F]
  (f : E  $\rightarrow_l$  [ $\mathbb{k}$ ] F  $\rightarrow_l$  [ $\mathbb{k}$ ]  $\mathbb{k}$ ) {S : Set F} (hS : Bornology.IsBounded S) :
    Continuous fun x  $\mapsto \bigcap y : S, f \times y$ 

```

**theorem continuous\_iSup'**

[\[source\]](#)

*A specialization of `LinearMap.continuous_iSup` to finite dimensional spaces, in place of requiring a (non-instance) `IsModuleTopology`.*

```

theorem continuous_iSup' {E F  $\mathbb{k}$  : Type*}
  [NontriviallyNormedField  $\mathbb{k}$ ] [ConditionallyCompleteLinearOrder  $\mathbb{k}$ ] [OrderTopology  $\mathbb{k}$ ] [
    CompleteSpace  $\mathbb{k}$ ]
  [AddCommGroup E] [TopologicalSpace E] [IsTopologicalAddGroup E] [T2Space E]
  [Module  $\mathbb{k}$  E] [ContinuousSMul  $\mathbb{k}$  E] [FiniteDimensional  $\mathbb{k}$  E]
  [PseudoMetricSpace F] [ProperSpace F] [AddCommGroup F] [IsTopologicalAddGroup F] [T2Space F]
  [Module  $\mathbb{k}$  F] [ContinuousSMul  $\mathbb{k}$  F] [FiniteDimensional  $\mathbb{k}$  F]
  (f : E  $\rightarrow_l$  [ $\mathbb{k}$ ] F  $\rightarrow_l$  [ $\mathbb{k}$ ]  $\mathbb{k}$ ) {S : Set F} (hS : Bornology.IsBounded S) :
    Continuous fun x  $\mapsto \bigcup y : S, f \times y$ 

```

**theorem continuous\_iInf'**

[\[source\]](#)

*A specialization of `LinearMap.continuous_iInf` to finite dimensional spaces, in place of requiring a (non-instance) `IsModuleTopology`.*

```

theorem continuous_iInf' {E F  $\mathbb{k}$  : Type*}
  [NontriviallyNormedField  $\mathbb{k}$ ] [ConditionallyCompleteLinearOrder  $\mathbb{k}$ ] [OrderTopology  $\mathbb{k}$ ] [
    CompleteSpace  $\mathbb{k}$ ]
  [AddCommGroup E] [TopologicalSpace E] [IsTopologicalAddGroup E] [T2Space E]
  [Module  $\mathbb{k}$  E] [ContinuousSMul  $\mathbb{k}$  E] [FiniteDimensional  $\mathbb{k}$  E]
  [PseudoMetricSpace F] [ProperSpace F] [AddCommGroup F] [IsTopologicalAddGroup F] [T2Space F]
  [Module  $\mathbb{k}$  F] [ContinuousSMul  $\mathbb{k}$  F] [FiniteDimensional  $\mathbb{k}$  F]
  (f : E  $\rightarrow_l$  [ $\mathbb{k}$ ] F  $\rightarrow_l$  [ $\mathbb{k}$ ]  $\mathbb{k}$ ) {S : Set F} (hS : Bornology.IsBounded S) :
    Continuous fun x  $\mapsto \bigcap y : S, f \times y$ 

```

**theorem BilinForm.continuous\_iSup**

[\[source\]](#)

*Alias of `LinearMap.continuous_iSup` that takes `LinearMap.BilinForm`.*

```

theorem BilinForm.continuous_iSup { $\mathbb{k}$  E : Type*}
  [RCLike  $\mathbb{k}$ ] [ConditionallyCompleteLinearOrder  $\mathbb{k}$ ] [OrderTopology  $\mathbb{k}$ ]
  [NormedAddCommGroup E] [InnerProductSpace  $\mathbb{k}$  E] [FiniteDimensional  $\mathbb{k}$  E] [ProperSpace E]
  (f : LinearMap.BilinForm  $\mathbb{k}$  E) {S : Set E} (hS : Bornology.IsBounded S) :
    Continuous fun x  $\mapsto \bigcup y : S, f \times y$ 

```

**theorem BilinForm.continuous\_iInf**

[\[source\]](#)

*Alias of `LinearMap.continuous_iInf` that takes `LinearMap.BilinForm`.*

```

theorem BilinForm.continuous_iInf { $\mathbb{k}$  E : Type*}
  [RCLike  $\mathbb{k}$ ] [ConditionallyCompleteLinearOrder  $\mathbb{k}$ ] [OrderTopology  $\mathbb{k}$ ]

```



```
[NormedAddCommGroup E] [InnerProductSpace  $\mathbb{k}$  E] [FiniteDimensional  $\mathbb{k}$  E] [ProperSpace E]
(f : LinearMap.BilinForm  $\mathbb{k}$  E) {S : Set E} (hS : Bornology.IsBounded S) :
Continuous fun x  $\mapsto \bigsqcup y : S, f \ x \ y$ 
```

### theorem ContinuousLinearMap.continuous\_iSup

[\[source\]](#)

*A specialization of Bornology.IsBounded.continuous\_iSup\_bilinear to ContinuousLinearMap.*

```
theorem continuous_iSup
(f : F  $\rightarrow$  SL[ $\sigma_{12}$ ] E  $\rightarrow$  L[ $\mathbb{k}$ ] G) {S : Set E} (hS : Bornology.IsBounded S) :
Continuous fun x  $\mapsto \bigsqcup y : S, f \ x \ y$ 
```

### theorem ContinuousLinearMap.continuous\_iInf

[\[source\]](#)

```
theorem continuous_iInf
(f : F  $\rightarrow$  SL[ $\sigma_{12}$ ] E  $\rightarrow$  L[ $\mathbb{k}$ ] G) {S : Set E} (hS : Bornology.IsBounded S) :
Continuous fun x  $\mapsto \bigsqcap y : S, f \ x \ y$ 
```

### theorem continuous\_iSup\_fst

[\[source\]](#)

```
theorem LinearMap.BilinForm.continuous_iSup_fst
{E : Type*} [NormedAddCommGroup E] [InnerProductSpace  $\mathbb{R}$  E] [FiniteDimensional  $\mathbb{R}$  E]
(f : LinearMap.BilinForm  $\mathbb{R}$  E) {S : Set E} (hS : Bornology.IsBounded S) :
Continuous fun x  $\mapsto \bigsqcup y : S, f \ y \ x$ 
```

### theorem continuous\_iInf\_fst

[\[source\]](#)

```
theorem LinearMap.BilinForm.continuous_iInf_fst
{E : Type*} [NormedAddCommGroup E] [InnerProductSpace  $\mathbb{R}$  E] [FiniteDimensional  $\mathbb{R}$  E]
(f : LinearMap.BilinForm  $\mathbb{R}$  E) {S : Set E} (hS : Bornology.IsBounded S) :
Continuous fun x  $\mapsto \bigsqcap y : S, f \ y \ x$ 
```

## 2.54 QuantumInfo.ForMathlib.Filter

[QuantumInfo/ForMathlib/Filter.lean](#) on GitHub.

### theorem Tendsto\_inv\_nat\_mul\_div\_real

[\[source\]](#)

```
theorem Filter.Tendsto_inv_nat_mul_div_real (m :  $\mathbb{N}$ )
: Filter.Tendsto (fun (x :  $\mathbb{N}$ ) => (( $\uparrow$ x) $^{-1}$  *  $\uparrow$ (x / m) :  $\mathbb{R}$ )) Filter.atTop ( $\mathcal{N}$  (1 /  $\uparrow$ m))
```

### theorem tendsto\_toReal\_iff\_of\_eventually\_ne\_top

[\[source\]](#)

```
theorem ENNReal.tendsto_toReal_iff_of_eventually_ne_top
{ $\iota$ } {fi : Filter  $\iota$ } {f :  $\iota \rightarrow \mathbb{R}_{\geq 0\infty}$ } (hf :  $\forall^f i$  in fi, f i  $\neq \infty$ ) {x :  $\mathbb{R}_{\geq 0\infty}$ }
(hx : x  $\neq \infty$ ) : Tendsto (fun n => (f n).toReal) fi ( $\mathcal{N}$  x.toReal)  $\leftrightarrow$  Tendsto f fi ( $\mathcal{N}$  x)
```

## 2.55 QuantumInfo.ForMathlib.HermitianMat.Basic

[QuantumInfo/ForMathlib/HermitianMat/Basic.lean](#) on GitHub.

**def HermitianMat**[\[source\]](#)

*The type of Hermitian matrices, as a Subtype. Equivalent to a `Matrix n n α` bundled with the fact that `Matrix.IsHermitian`.*

```
def HermitianMat (n : Type*) (α : Type*) [AddGroup α] [StarAddMonoid α]
```

**theorem eq\_IsHermitian**[\[source\]](#)

```
theorem eq_IsHermitian : HermitianMat n α = { m : Matrix n n α // m.IsHermitian }
```

**instance Coe (HermitianMat n α) (Matrix n n α)**[\[source\]](#)

```
instance : Coe (HermitianMat n α) (Matrix n n α)
```

**theorem val\_eq\_coe**[\[source\]](#)

```
@[simp]
theorem val_eq_coe (A : HermitianMat n α) : A.val = A
```

**theorem mat\_mk**[\[source\]](#)

```
@[simp]
theorem mat_mk (x : Matrix n n α) (h) : mat ⟨x, h⟩ = x
```

**theorem mk\_mat**[\[source\]](#)

```
@[simp]
theorem mk_mat {A : HermitianMat n α} (h : A.mat.IsHermitian) : ⟨A.mat, h⟩ = A
```

**theorem H**[\[source\]](#)

*Alias for `HermitianMat.property` or `HermitianMat.2`, this gets the fact that the value is actually `IsHermitian`.*

```
theorem H (A : HermitianMat n α) : A.mat.IsHermitian
```

**instance instFun**[\[source\]](#)

```
instance instFun : FunLike (HermitianMat n α) n (n → α) where
```

**theorem mat\_apply**[\[source\]](#)

```
@[simp]
theorem mat_apply {A : HermitianMat n α} {i j : n} : A.mat i j = A i j
```

**theorem conjTranspose\_mat**[\[source\]](#)

```
@[simp]
theorem conjTranspose_mat (A : HermitianMat n α) :
  A.mat.conjTranspose = A.mat
```

**instance AddGroup (HermitianMat n  $\alpha$ )**

[\[source\]](#)

**instance** : AddGroup (HermitianMat n  $\alpha$ )

**instance Unique (HermitianMat n  $\alpha$ )**

[\[source\]](#)

**instance** [IsEmpty n] : Unique (HermitianMat n  $\alpha$ ) **where**

**theorem mat\_zero**

[\[source\]](#)

@[simp, norm\_cast]

**theorem** mat\_zero : ( $0$  : HermitianMat n  $\alpha$ ).mat =  $0$

**theorem mk\_zero**

[\[source\]](#)

@[simp]

**theorem** mk\_zero (h : ( $0$  : Matrix n n  $\alpha$ ).IsHermitian) : ( $0$ , h) = ( $0$  : HermitianMat n  $\alpha$ )

**theorem zero\_apply**

[\[source\]](#)

@[simp]

**theorem** zero\_apply (i j : n) : ( $0$  : HermitianMat n  $\mathbb{k}$ ) i j =  $0$

**theorem mat\_add**

[\[source\]](#)

@[simp, norm\_cast]

**theorem** mat\_add (A B : HermitianMat n  $\alpha$ ) :  
(A + B).mat = A.mat + B.mat

**theorem mat\_sub**

[\[source\]](#)

@[simp, norm\_cast]

**theorem** mat\_sub (A B : HermitianMat n  $\alpha$ ) :  
(A - B).mat = A.mat - B.mat

**theorem mat\_neg**

[\[source\]](#)

@[simp, norm\_cast]

**theorem** mat\_neg (A : HermitianMat n  $\alpha$ ) :  
(-A).mat = -A.mat

**instance SMul R (HermitianMat n  $\alpha$ )**

[\[source\]](#)

**instance** : SMul R (HermitianMat n  $\alpha$ )

**theorem mat\_smul**

[\[source\]](#)

@[simp, norm\_cast]

**theorem** mat\_smul (c : R) (A : HermitianMat n  $\alpha$ ) :  
(c • A).mat = c • A.mat

**theorem smul\_apply**[\[source\]](#)

```
@[simp]
theorem smul_apply (c : R) (A : HermitianMat n  $\alpha$ ) (i j : n) :
  (c • A) i j = c • A i j
```

**instance TopologicalSpace (HermitianMat n  $\alpha$ )**[\[source\]](#)

```
instance : TopologicalSpace (HermitianMat n  $\alpha$ )
```

**theorem continuous\_mat**[\[source\]](#)

```
@[fun_prop]
theorem continuous_mat : Continuous (HermitianMat.mat : HermitianMat n  $\alpha$  → Matrix n n  $\alpha$ )
```

**lemma continuousOn\_iff\_coe**[\[source\]](#)

```
lemma continuousOn_iff_coe {X : Type*} [TopologicalSpace X] {s : Set X}
  (f : X → HermitianMat n  $\alpha$ ) :
  ContinuousOn f s ↔ ContinuousOn (fun x => (f x).mat) s
```

**instance ContinuousAdd (HermitianMat n  $\alpha$ )**[\[source\]](#)

```
instance : ContinuousAdd (HermitianMat n  $\alpha$ )
```

**instance ContinuousNeg (HermitianMat n  $\alpha$ )**[\[source\]](#)

```
instance : ContinuousNeg (HermitianMat n  $\alpha$ )
```

**instance IsTopologicalAddGroup (HermitianMat n  $\alpha$ )**[\[source\]](#)

```
instance : IsTopologicalAddGroup (HermitianMat n  $\alpha$ ) where
```

**instance ContinuousSMul R (HermitianMat n  $\alpha$ )**[\[source\]](#)

```
instance : ContinuousSMul R (HermitianMat n  $\alpha$ ) where
```

**instance IsTopologicalAddGroup (HermitianMat n  $\mathbb{k}$ )**[\[source\]](#)

```
instance : IsTopologicalAddGroup (HermitianMat n  $\mathbb{k}$ )
```

**instance ContinuousSMul  $\mathbb{R}$  (HermitianMat n  $\mathbb{C}$ )**[\[source\]](#)

```
instance : ContinuousSMul  $\mathbb{R}$  (HermitianMat n  $\mathbb{C}$ )
```

**instance T3Space (HermitianMat n  $\mathbb{k}$ )**[\[source\]](#)

```
instance : T3Space (HermitianMat n  $\mathbb{k}$ )
```

**instance MulAction R (HermitianMat n  $\alpha$ )**

[\[source\]](#)

**instance** : MulAction R (HermitianMat n  $\alpha$ )

**instance AddCommGroup (HermitianMat n  $\alpha$ )**

[\[source\]](#)

**instance** : AddCommGroup (HermitianMat n  $\alpha$ )

**theorem mat\_finset\_sum**

[\[source\]](#)

@[simp, norm\_cast]

**theorem** mat\_finset\_sum (f :  $\iota \rightarrow$  HermitianMat n  $\alpha$ ) (s : Finset  $\iota$ ) :  
 $(\sum i \in s, f i).mat = \sum i \in s, (f i).mat$

**instance Module R (HermitianMat n  $\alpha$ )**

[\[source\]](#)

**instance** : Module R (HermitianMat n  $\alpha$ )

**def** mat <sub>$\iota$</sub>

[\[source\]](#)

*The projection from HermitianMat to Matrix, as a continuous linear map.*

@[simps]

**def** mat <sub>$\iota$</sub>  : HermitianMat n  $\alpha \rightarrow$  L[R] Matrix n n  $\alpha$  **where**

**instance One (HermitianMat n  $\alpha$ )**

[\[source\]](#)

**instance** : One (HermitianMat n  $\alpha$ )

**theorem mat\_one**

[\[source\]](#)

@[simp, norm\_cast]

**theorem** mat\_one : (1 : HermitianMat n  $\alpha$ ).mat = 1

**theorem mk\_one**

[\[source\]](#)

@[simp]

**theorem** mk\_one (h : (1 : Matrix n n  $\alpha$ ).IsHermitian) : (1, h) = (1 : HermitianMat n  $\alpha$ )

**theorem one\_apply**

[\[source\]](#)

@[simp]

**theorem** one\_apply (i j : n) : (1 : HermitianMat n  $\alpha$ ) i j = (1 : Matrix n n  $\alpha$ ) i j

**instance AddCommMonoidWithOne (HermitianMat n  $\mathbb{k}$ )**

[\[source\]](#)

**noncomputable instance** : AddCommMonoidWithOne (HermitianMat n  $\mathbb{k}$ ) **where**

**instance NeZero (1 : HermitianMat n  $\mathbb{k}$ )**

[\[source\]](#)

**instance** [i : Nonempty n] : NeZero (1 : HermitianMat n  $\mathbb{k}$ )

**instance instInv**[\[source\]](#)

```
noncomputable instance instInv : Inv (HermitianMat m α)
```

**theorem mat\_inv**[\[source\]](#)

```
@[simp, norm_cast]
theorem mat_inv : (A-1).mat = A.mat-1
```

**theorem zero\_inv**[\[source\]](#)

```
@[simp]
theorem zero_inv : ((0 : HermitianMat m α)-1) = 0
```

**theorem one\_inv**[\[source\]](#)

```
@[simp]
theorem one_inv : ((1 : HermitianMat m α)-1) = 1
```

**instance instPow**[\[source\]](#)

```
noncomputable instance instPow : Pow (HermitianMat m α) ℕ
```

**theorem mat\_pow**[\[source\]](#)

```
@[simp, norm_cast]
theorem mat_pow (n : ℕ) : (A ^ n).mat = A.mat ^ n
```

**theorem pow\_zero**[\[source\]](#)

```
@[simp]
theorem pow_zero : A ^ 0 = 1
```

**theorem zero\_pow**[\[source\]](#)

```
@[simp]
theorem zero_pow (hn : n ≠ 0): (0 : HermitianMat m α) ^ n = 0
```

**theorem one\_pow**[\[source\]](#)

```
@[simp]
theorem one_pow : ((1 : HermitianMat m α) ^ n) = 1
```

**instance instZPow**[\[source\]](#)

```
noncomputable instance instZPow : Pow (HermitianMat m α) ℤ
```

**theorem mat\_zpow**[\[source\]](#)

```
@[simp]
theorem mat_zpow (z : ℤ) : (A ^ z).mat = A.mat ^ z
```

**theorem zpow\_natCast**[\[source\]](#)

```
@[simp, norm_cast]
theorem zpow_natCast : A ^ (n : ℤ) = A ^ n
```

**theorem zpow\_zero**[\[source\]](#)

```
@[simp]
theorem zpow_zero : A ^ (0 : ℤ) = 1
```

**theorem zpow\_one**[\[source\]](#)

```
@[simp]
theorem zpow_one : A ^ (1 : ℤ) = A
```

**theorem one\_zpow**[\[source\]](#)

```
@[simp]
theorem one_zpow : ((1 : HermitianMat m α) ^ z) = 1
```

**theorem zpow\_neg\_one**[\[source\]](#)

```
@[simp]
theorem zpow_neg_one : A ^ (-1) = A-1
```

**theorem inv\_zpow**[\[source\]](#)

```
@[simp]
theorem inv_zpow : A-1 ^ z = (A ^ z)-1
```

**theorem inv\_commute**[\[source\]](#)

```
@[aesop safe apply (rule_sets := [Commutes])]
theorem _root_.Matrix.inv_commute {α : Type*} {A : Matrix m m α} [CommRing α] : Commute A-1 A
```

**theorem commute\_inv\_self**[\[source\]](#)

```
@[aesop safe apply (rule_sets := [Commutes])]
theorem commute_inv_self : Commute A-1.mat A.mat
```

**theorem commute\_self\_inv**[\[source\]](#)

```
@[aesop safe apply (rule_sets := [Commutes])]
theorem commute_self_inv : Commute A.mat A-1.mat
```

**instance FiniteDimensional**[\[source\]](#)

```
instance FiniteDimensional : FiniteDimensional  $\mathbb{R}$  (HermitianMat n  $\mathbb{K}$ )
```

**theorem im\_diag\_eq\_zero**[\[source\]](#)

```
@[simp]
theorem im_diag_eq_zero (A : HermitianMat n  $\mathbb{K}$ ) (x : n) :
  RCLike.im (A x x) = 0
```

**theorem complex\_im\_eq\_zero**[\[source\]](#)

```
@[simp]
theorem complex_im_eq_zero (A : HermitianMat n  $\mathbb{C}$ ) (x : n) :
  (A x x).im = 0
```

**def conj**[\[source\]](#)

*The Hermitian matrix given by conjugating by a (possibly rectangular) Matrix. If we required  $B$  to be square, this would apply to any `Semigroup+StarMul` (as proved by `IsSelfAdjoint.conjugate`). But this lets us conjugate to other sizes too, as is done in e.g. Kraus operators. That is, it's a `__heterogeneous__` conjugation.*

```
def conj {m} (B : Matrix m n  $\alpha$ ) : HermitianMat n  $\alpha$   $\rightarrow$  HermitianMat m  $\alpha$  where
```

**theorem conj\_apply**[\[source\]](#)

```
theorem conj_apply (B : Matrix m n  $\alpha$ ) (A : HermitianMat n  $\alpha$ ) :
  conj B A = (B * A.mat * B.conjTranspose, (conj B A).2)
```

**theorem conj\_apply\_mat**[\[source\]](#)

```
@[simp]
theorem conj_apply_mat (B : Matrix m n  $\alpha$ ) (A : HermitianMat n  $\alpha$ ) :
  (A.conj B).mat = B * A.mat * B.conjTranspose
```

**theorem conj\_conj**[\[source\]](#)

```
theorem conj_conj {m l} [Fintype m] (B : Matrix m n  $\alpha$ ) (C : Matrix l m  $\alpha$ ) :
  (A.conj B).conj C = A.conj (C * B)
```

**theorem conj\_zero**[\[source\]](#)

```
@[simp]
theorem conj_zero [DecidableEq n] : A.conj (0 : Matrix m n  $\alpha$ ) = 0
```

**theorem conj\_one**[\[source\]](#)

```
@[simp]
theorem conj_one [DecidableEq n] : A.conj 1 = A
```



**lemma conj\_one\_unitary**[\[source\]](#)

```
@[simp]
lemma conj_one_unitary [DecidableEq n] (U : Matrix.unitaryGroup n α) :
  conj U.val 1 = 1
```

**def conjLinear**[\[source\]](#)

*HermitianMat.conj as an  $R$ -linear map, where  $R$  is the ring of relevant reals.*

```
def conjLinear {m} (B : Matrix m n α) : HermitianMat n α →l[R] HermitianMat m α where
```

**theorem conjLinear\_apply**[\[source\]](#)

```
@[simp]
theorem conjLinear_apply (B : Matrix m n α) : conjLinear R B A = conj B A
```

**lemma continuous\_conj**[\[source\]](#)

```
@[fun_prop]
lemma continuous_conj (p : HermitianMat n ℚ) : Continuous (p.conj (m := m) ·)
```

**instance FaithfulSMul ℝ (HermitianMat n ℚ)**[\[source\]](#)

```
instance [i : Nonempty n] : FaithfulSMul ℝ (HermitianMat n ℚ) where
```

**def lin**[\[source\]](#)

*The continuous linear map associated with a Hermitian matrix.*

```
def lin : EuclideanSpace ℚ n →L[ℚ] EuclideanSpace ℚ n where
```

**theorem isSymmetric**[\[source\]](#)

```
@[simp]
theorem isSymmetric : A.lin.IsSymmetric
```

**theorem lin\_zero**[\[source\]](#)

```
@[simp]
theorem lin_zero : (0 : HermitianMat n ℚ).lin = 0
```

**theorem lin\_one**[\[source\]](#)

```
@[simp]
theorem lin_one : (1 : HermitianMat n ℚ).lin = 1
```

**def eigenspace**[\[source\]](#)

```
noncomputable def eigenspace (μ : ℚ) : Submodule ℚ (EuclideanSpace ℚ n)
```

**def ker** [\[source\]](#)

*The kernel of a Hermitian matrix  $A$  as a submodule of Euclidean space, defined by `LinearMap.ker A.toMat.toEuclideanLin`. Equivalently, the zero-eigenspace.*

```
def ker : Submodule ℚ (EuclideanSpace ℚ n)
```

**theorem mem\_ker\_iff\_mulVec\_zero** [\[source\]](#)

```
theorem mem_ker_iff_mulVec_zero (x : EuclideanSpace ℚ n) : x ∈ A.ker ↔ A.mat.mulVec x = 0
```

**theorem ker\_eq\_eigenspace\_zero** [\[source\]](#)

*The kernel of a Hermitian matrix is its zero eigenspace.*

```
theorem ker_eq_eigenspace_zero : A.ker = A.eigenspace 0
```

**theorem ker\_zero** [\[source\]](#)

```
@[simp]
theorem ker_zero : (0 : HermitianMat n ℚ).ker = 0
```

**theorem ker\_one** [\[source\]](#)

```
@[simp]
theorem ker_one : (1 : HermitianMat n ℚ).ker = 1
```

**theorem ker\_pos\_smul** [\[source\]](#)

```
theorem ker_pos_smul {c : ℝ} (hc : c ≠ 0) : (c • A).ker = A.ker
```

**def support** [\[source\]](#)

*The support of a Hermitian matrix  $A$  as a submodule of Euclidean space, defined by `LinearMap.range A.toMat.toEuclideanLin`. Equivalently, the sum of all nonzero eigenspaces.*

```
def support : Submodule ℚ (EuclideanSpace ℚ n)
```

**theorem support\_eq\_sup\_eigenspace\_nonzero** [\[source\]](#)

*The support of a Hermitian matrix is the sum of its nonzero eigenspaces.*

```
theorem support_eq_sup_eigenspace_nonzero : A.support = ⨆ μ ≠ 0, A.eigenspace μ
```

**theorem support\_zero** [\[source\]](#)

```
@[simp]
theorem support_zero : (0 : HermitianMat n ℚ).support = 0
```

**theorem support\_one**[\[source\]](#)

```
@[simp]
theorem support_one : (1 : HermitianMat n  $\mathbb{k}$ ).support =  $\tau$ 
```

**theorem ker\_orthogonal\_eq\_support**[\[source\]](#)

```
@[simp]
theorem ker_orthogonal_eq_support : A.ker⊥ = A.support
```

**theorem support\_orthogonal\_eq\_range**[\[source\]](#)

```
@[simp]
theorem support_orthogonal_eq_range : A.support⊥ = A.ker
```

**def diagonal**[\[source\]](#)

```
def diagonal (f : n →  $\mathbb{R}$ ) : HermitianMat n  $\mathbb{k}$ 
```

**theorem diagonal\_mat**[\[source\]](#)

```
@[simp]
theorem diagonal_mat : (diagonal  $\mathbb{k}$  f).mat = Matrix.diagonal (f · : n →  $\mathbb{k}$ )
```

**theorem diagonal\_zero**[\[source\]](#)

```
@[simp]
theorem diagonal_zero : (diagonal  $\mathbb{k}$  0) = (0 : HermitianMat n  $\mathbb{k}$ )
```

**theorem diagonal\_one**[\[source\]](#)

```
@[simp]
theorem diagonal_one : (diagonal  $\mathbb{k}$  1) = (1 : HermitianMat n  $\mathbb{k}$ )
```

**lemma diagonal\_add**[\[source\]](#)

```
lemma diagonal_add : diagonal  $\mathbb{k}$  (f + g) = diagonal  $\mathbb{k}$  f + diagonal  $\mathbb{k}$  g
```

**lemma diagonal\_add\_apply**[\[source\]](#)

```
lemma diagonal_add_apply : diagonal  $\mathbb{k}$  (fun x ↦ f x + g x) = diagonal  $\mathbb{k}$  f + diagonal  $\mathbb{k}$  g
```

**lemma diagonal\_sub**[\[source\]](#)

```
lemma diagonal_sub : diagonal  $\mathbb{k}$  (f - g) = diagonal  $\mathbb{k}$  f - diagonal  $\mathbb{k}$  g
```

**theorem diagonal\_mul**[\[source\]](#)

```
theorem diagonal_mul (c :  $\mathbb{R}$ ) : diagonal  $\mathbb{k}$  (fun x ↦ c * f x) = c • diagonal  $\mathbb{k}$  f
```

**theorem diagonal\_conj\_diagonal**[\[source\]](#)

```

theorem diagonal_conj_diagonal [Fintype n] :
  (diagonal ℤ f).conj (diagonal ℤ g) = diagonal ℤ (fun i ↦ f i * (g i)^2)

```

**lemma eq\_conj\_diagonal**[\[source\]](#)

*A Hermitian matrix is equal to its diagonalization conjugated by its eigenvector unitary matrix.*

```

lemma eq_conj_diagonal [Fintype n] (A : HermitianMat n ℤ) :
  A = (diagonal ℤ A.H.eigenvalues).conj A.H.eigenvectorUnitary

```

**def kronecker**[\[source\]](#)

*The kronecker product of two HermitianMats, see Matrix.kroneckerMap.*

```

def kronecker (A : HermitianMat m α) (B : HermitianMat n α) : HermitianMat (m × n) α where

```

**theorem kronecker\_mat**[\[source\]](#)

```

@[simp, norm_cast]
theorem kronecker_mat (A : HermitianMat m α) (B : HermitianMat n α) :
  (A ⊗ℤ B).mat = A.mat ⊗ℤ B.mat

```

**theorem zero\_kronecker**[\[source\]](#)

```

@[simp]
theorem zero_kronecker (A : HermitianMat m α) : (0 : HermitianMat n α) ⊗ℤ A = 0

```

**theorem kronecker\_zero**[\[source\]](#)

```

@[simp]
theorem kronecker_zero (A : HermitianMat m α) : A ⊗ℤ (0 : HermitianMat n α) = 0

```

**theorem kronecker\_one\_one**[\[source\]](#)

```

@[simp]
theorem kronecker_one_one : (1 : HermitianMat m α) ⊗ℤ (1 : HermitianMat n α) = 1

```

**theorem add\_kronecker**[\[source\]](#)

```

theorem add_kronecker : (A + B) ⊗ℤ C = A ⊗ℤ C + B ⊗ℤ C

```

**theorem kronecker\_add**[\[source\]](#)

```

theorem kronecker_add : A ⊗ℤ (B + C) = A ⊗ℤ B + A ⊗ℤ C

```

**lemma kronecker\_diagonal**[\[source\]](#)

```

lemma kronecker_diagonal [DecidableEq m] [DecidableEq n] (d₁ : m → ℝ) (d₂ : n → ℝ) :
  (diagonal ℤ d₁ ⊗ℤ diagonal ℤ d₂) = diagonal ℤ (fun (i : m × n) => d₁ i.1 * d₂ i.2)

```

**theorem kron\_commute**[\[source\]](#)

$A \otimes_k B$  always commutes with  $C \otimes_k D$  if the pairs commute.

```
@[aesop safe apply (rule_sets := [Commutes])]
theorem kron_commute [Fintype m] [Fintype n] [DecidableEq m] [DecidableEq n]
  {A C : HermitianMat m α} {B D : HermitianMat n α}
  (hAC : Commute A.mat C.mat) (hBD : Commute B.mat D.mat):
  Commute (A ⊗k B).mat (C ⊗k D).mat
```

**theorem kron\_id\_commute\_id\_kro**[\[source\]](#)

$A \otimes_k 1$  always commutes with  $1 \otimes_k B$

```
@[aesop safe apply (rule_sets := [Commutes])]
theorem kron_id_commute_id_kro [Fintype m] [Fintype n] [DecidableEq m] [DecidableEq n]
  (A : HermitianMat m α) (B : HermitianMat n α) :
  Commute (A ⊗k (1 : HermitianMat n α)).mat ((1 : HermitianMat m α) ⊗k B).mat
```

**lemma kronecker\_conj**[\[source\]](#)

```
lemma kronecker_conj [Fintype m] [Fintype n]
  (A : HermitianMat m α) (B : HermitianMat n α) (C : Matrix p m α) (D : Matrix q n α) :
  (A ⊗k B).conj (C ⊗k D) = (A.conj C) ⊗k (B.conj D)
```

**theorem range\_le\_ker\_imp\_zero**[\[source\]](#)

```
theorem range_le_ker_imp_zero {A : HermitianMat d ℤ}
  (h : LinearMap.range A.mat.toEuclideanLin ≤ LinearMap.ker A.mat.toEuclideanLin) : A = 0
```

**theorem range\_mul\_conjTranspose\_of\_ker\_le\_ker**[\[source\]](#)

If  $\ker M \subseteq \ker A$ , then  $\text{range}(A M^H) = \text{range } A$ .

```
theorem _root_.Matrix.range_mul_conjTranspose_of_ker_le_ker {A : Matrix d d ℤ} {M : Matrix d2 d
  ℤ}
  (h : LinearMap.ker M.toEuclideanLin ≤ LinearMap.ker A.toEuclideanLin) :
  LinearMap.range (A * M.conjTranspose).toEuclideanLin = LinearMap.range A.toEuclideanLin
```

**theorem conj\_ne\_zero**[\[source\]](#)

```
theorem conj_ne_zero {A : HermitianMat d ℤ} {M : Matrix d2 d ℤ} (hA : A ≠ 0)
  (h : LinearMap.ker M.toEuclideanLin ≤ A.ker) : A.conj M ≠ 0
```

**theorem conj\_ne\_zero\_iff**[\[source\]](#)

```
theorem conj_ne_zero_iff {A : HermitianMat d ℤ} {M : Matrix d2 d ℤ}
  (h : LinearMap.ker M.toEuclideanLin ≤ A.ker) : A.conj M ≠ 0 ↔ A ≠ 0
```

**theorem IsHermitian.spectrum\_rcLike**[\[source\]](#)

```
theorem _root_.Matrix.IsHermitian.spectrum_rcLike {A : Matrix n n ℤ} (hA : A.IsHermitian) :
  RCLike.ofReal '' spectrum ℝ A = spectrum ℤ A
```

**theorem HermitianMat.spectrum\_rcLike**[\[source\]](#)

*We fix a simp-normal form that, for HermitianMat, we always work in terms of the real spectrum.*

```
@[simp]
theorem spectrum_rcLike (A : HermitianMat n ℚ) :
  spectrum ℚ A.mat = RCLike.ofReal '' spectrum ℝ A.mat
```

**theorem ne\_zero\_iff\_ne\_zero\_spectrum**[\[source\]](#)

```
theorem ne_zero_iff_ne_zero_spectrum (A : HermitianMat n ℚ) :
  A ≠ 0 ↔ ∃ x ∈ spectrum ℝ A.mat, x ≠ 0
```

**theorem spectrum\_prod**[\[source\]](#)

```
theorem spectrum_prod
  {A : HermitianMat m ℚ} {B : HermitianMat n ℚ} :
  spectrum ℝ (A ⊗ₖ B).mat = spectrum ℝ A.mat * spectrum ℝ B.mat
```

**instance AddCommMonoid (HermitianMat d ℂ)**[\[source\]](#)

```
noncomputable instance : AddCommMonoid (HermitianMat d ℂ)
```

**2.56 QuantumInfo.ForMathlib.HermitianMat.CFC**

[QuantumInfo/ForMathlib/HermitianMat/CFC.lean](#) on GitHub.

**theorem isSelfAdjoint**[\[source\]](#)

```
@[aesop safe apply (rule_sets := [CStarAlgebra])]
theorem isSelfAdjoint : IsSelfAdjoint A.mat
```

**theorem continuousOn\_finite**[\[source\]](#)

```
@[fun_prop]
theorem continuousOn_finite {α β : Type*} (f : α → β) (S : Set α)
  [TopologicalSpace α] [TopologicalSpace β] [T1Space α] [Finite S] : ContinuousOn f S
```

**theorem conjTranspose\_cfc**[\[source\]](#)

```
@[simp]
theorem conjTranspose_cfc : (cfc f A.mat).conjTranspose = cfc f A.mat
```

**def cfc**[\[source\]](#)

```
protected def cfc : HermitianMat d ℚ
```

**theorem cfc\_eq**[\[source\]](#)

```
theorem cfc_eq : A.cfc f = (cfc f A.mat, cfc_predicate f A.mat)
```

**theorem mat\_cfc**[\[source\]](#)

```
@[simp]
theorem mat_cfc : (A.cfc f).mat = _root_.cfc f A.mat
```

**theorem cfc\_eq\_cfc\_iff\_eq0n**[\[source\]](#)

```
theorem cfc_eq_cfc_iff_eq0n (f g :  $\mathbb{R} \rightarrow \mathbb{R}$ ) :
  A.cfc f = A.cfc g  $\leftrightarrow$  Set.Eq0n f g (spectrum  $\mathbb{R}$  A.mat)
```

**theorem cfc\_congr**[\[source\]](#)

```
@[gcongr]
nonrec theorem cfc_congr (hfg : Set.Eq0n f g (spectrum  $\mathbb{R}$  A.mat)) :
  A.cfc f = A.cfc g
```

**theorem cfc\_congr\_of\_nonneg**[\[source\]](#)

*Version of cfc\_congr specialized to PSD matrices.*

```
nonrec theorem cfc_congr_of_nonneg (hA :  $0 \leq A$ ) (hfg : Set.Eq0n f g (Set.Ici 0)) :
  A.cfc f = A.cfc g
```

**theorem cfc\_congr\_of\_posDef**[\[source\]](#)

*Version of cfc\_congr specialized to positive definite matrices.*

```
nonrec theorem cfc_congr_of_posDef (hA : A.mat.PosDef) (hfg : Set.Eq0n f g (Set.Ioi 0)) :
  A.cfc f = A.cfc g
```

**theorem cfc\_left**[\[source\]](#)

```
@[aesop unsafe apply 50% (rule_sets := [Commutes])]
theorem _root_.Commute.cfc_left (hAB : Commute A.mat B.mat) :
  Commute (A.cfc f).mat B.mat
```

**theorem cfc\_right**[\[source\]](#)

```
@[aesop unsafe apply 50% (rule_sets := [Commutes])]
theorem _root_.Commute.cfc_right (hAB : Commute A.mat B.mat) :
  Commute A.mat (B.cfc f).mat
```

**theorem cfc\_commute**[\[source\]](#)

```
theorem cfc_commute (f g :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (hAB : Commute A.mat B.mat) :
  Commute (A.cfc f).mat (B.cfc g).mat
```

**theorem cfc\_self\_commute**[\[source\]](#)

```
@[aesop safe apply (rule_sets := [Commutes])]
theorem cfc_self_commute (A : HermitianMat d  $\mathbb{k}$ ) (f g :  $\mathbb{R} \rightarrow \mathbb{R}$ ) :
  Commute (A.cfc f).mat (A.cfc g).mat
```

**theorem cfc\_reindex**[\[source\]](#)*Reindexing a matrix commutes with applying the CFC.*

@[simp]

**theorem** cfc\_reindex (e : d ≈ d<sub>2</sub>) : (A.reindex e).cfc f = (A.cfc f).reindex e**theorem spectrum\_cfc\_eq\_image**[\[source\]](#)**theorem** spectrum\_cfc\_eq\_image (A : HermitianMat d ℤ) (f : ℝ → ℝ) :  
spectrum ℝ (A.cfc f).mat = f '' (spectrum ℝ A.mat)**theorem cfc\_toMat\_eq\_sum\_smul\_proj**[\[source\]](#)*Spectral decomposition of A.cfc f as a sum of scaled projections (matrix version).***theorem** cfc\_toMat\_eq\_sum\_smul\_proj : (A.cfc f).mat =  
∑ i, f (A.H.eigenvalues i) • (A.H.eigenvectorUnitary.val \* (Matrix.single i i 1) \* A.H.  
eigenvectorUnitary.val.conjTranspose)**theorem cfc\_eigenvalues**[\[source\]](#)**theorem** cfc\_eigenvalues (A : HermitianMat d ℤ) :  
∃ (e : d ≈ d), (A.cfc f).H.eigenvalues = f ∘ A.H.eigenvalues ∘ e**theorem cfc\_id**[\[source\]](#)

@[simp]

nonrec **theorem** cfc\_id : A.cfc id = A**theorem cfc\_id'**[\[source\]](#)

@[simp]

nonrec **theorem** cfc\_id' : A.cfc (·) = A**theorem cfc\_add**[\[source\]](#)nonrec **theorem** cfc\_add : A.cfc (f + g) = A.cfc f + A.cfc g**theorem cfc\_add\_apply**[\[source\]](#)**theorem** cfc\_add\_apply : A.cfc (fun x ↦ f x + g x) = A.cfc f + A.cfc g**theorem cfc\_sub**[\[source\]](#)nonrec **theorem** cfc\_sub : A.cfc (f - g) = A.cfc f - A.cfc g**theorem cfc\_sub\_apply**[\[source\]](#)**theorem** cfc\_sub\_apply : A.cfc (fun x ↦ f x - g x) = A.cfc f - A.cfc g



**theorem cfc\_neg**[\[source\]](#)

```
nonrec theorem cfc_neg : A.cfc (-f) = -A.cfc f
```

**theorem cfc\_neg\_apply**[\[source\]](#)

```
theorem cfc_neg_apply : A.cfc (fun x ↦ -f x) = -A.cfc f
```

**theorem mat\_cfc\_mul**[\[source\]](#)

*We don't have a direct analog of `cfc_mul`, since we can't generally multiply to `Hermi-tianMat`'s to get another one, so the theorem statement wouldn't be well-typed. But, we can say that the matrices are always equal. See `cfc_conj` for the coe-free analog to multiplication.*

```
theorem mat_cfc_mul : (A.cfc (f * g)).mat = A.cfc f * A.cfc g
```

**theorem mat\_cfc\_mul\_apply**[\[source\]](#)

```
theorem mat_cfc_mul_apply : (A.cfc (fun x ↦ f x * g x)).mat = A.cfc f * A.cfc g
```

**theorem cfc\_comp**[\[source\]](#)

```
nonrec theorem cfc_comp : A.cfc (g ∘ f) = (A.cfc f).cfc g
```

**theorem cfc\_comp\_apply**[\[source\]](#)

```
theorem cfc_comp_apply : A.cfc (fun x ↦ g (f x)) = (A.cfc f).cfc g
```

**theorem cfc\_conj**[\[source\]](#)

```
nonrec theorem cfc_conj : (A.cfc f).conj (A.cfc g) = A.cfc (f * g^2)
```

**theorem cfc\_diagonal**[\[source\]](#)

```
@[simp]
```

```
theorem cfc_diagonal (g : d → ℝ) : (diagonal ℤ g).cfc f = diagonal ℤ (f ∘ g)
```

**theorem cfc\_conj\_unitary**[\[source\]](#)

```
theorem cfc_conj_unitary (U : Matrix.unitaryGroup d ℤ) :  
  (A.conj U.val).cfc f = (A.cfc f).conj U
```

**theorem cfc\_const**[\[source\]](#)

```
@[simp]
```

```
nonrec theorem cfc_const : (A.cfc (fun _ ↦ r)) = r • 1
```

**theorem cfc\_const\_mul\_id**[\[source\]](#)

```
@[simp]
nonrec theorem cfc_const_mul_id : A.cfc (fun x ↦ r * x) = r • A
```

**theorem cfc\_const\_mul**[\[source\]](#)

```
@[simp]
nonrec theorem cfc_const_mul : A.cfc (fun x ↦ r * f x) = r • A.cfc f
```

**theorem cfc\_apply\_zero**[\[source\]](#)

```
@[simp]
nonrec theorem cfc_apply_zero : (0 : HermitianMat d ℤ).cfc f = f 0 • 1
```

**theorem cfc\_apply\_one**[\[source\]](#)

```
@[simp]
nonrec theorem cfc_apply_one : (1 : HermitianMat d ℤ).cfc f = f 1 • 1
```

**theorem cfc\_pow**[\[source\]](#)

```
theorem cfc_pow {n : ℕ} : A.cfc (· ^ n) = A ^ n
```

**theorem cfc\_nonneg\_iff**[\[source\]](#)

```
theorem cfc_nonneg_iff : 0 ≤ A.cfc f ↔ ∀ i, 0 ≤ f (A.H.eigenvalues i)
```

**theorem cfc\_posDef**[\[source\]](#)

```
theorem cfc_posDef : (A.cfc f).mat.PosDef ↔ ∀ i, 0 < f (A.H.eigenvalues i)
```

**theorem cfc\_nonneg\_of\_nonneg**[\[source\]](#)

*If a rael function preserves nonnegativity, the CFC preserves PSDness.*

```
theorem cfc_nonneg_of_nonneg (hA : 0 ≤ A) (hf : ∀ i ≥ 0, 0 ≤ f i) :
  0 ≤ A.cfc f
```

**theorem cfc\_nonSingular**[\[source\]](#)

```
theorem cfc_nonSingular (hf : ∀ i, f (A.H.eigenvalues i) ≠ 0) : NonSingular (A.cfc f)
```

**theorem trace\_mul\_cfc**[\[source\]](#)

```
theorem trace_mul_cfc (A : HermitianMat d ℤ) (f : ℝ → ℝ) :
  (A.mat * (A.cfc f).mat).trace = ∑ i, A.H.eigenvalues i * f (A.H.eigenvalues i)
```

**theorem norm\_eq\_sum\_eigenvalues\_sq**[\[source\]](#)

```
theorem norm_eq_sum_eigenvalues_sq (A : HermitianMat d ℤ) :
  ||A|| ^ 2 = ∑ i, (A.H.eigenvalues i)^2
```

**theorem lt\_smul\_of\_norm\_lt**[\[source\]](#)

```
theorem lt_smul_of_norm_lt {r : ℝ} (h : ||A|| ≤ r) : A ≤ r • 1
```

**theorem ball\_subset\_Icc**[\[source\]](#)

```
theorem ball_subset_Icc : Metric.ball A r ⊆ Set.Icc (A - r • 1) (A + r • 1)
```

**theorem spectrum\_subset\_of\_mem\_Icc**[\[source\]](#)

```
theorem spectrum_subset_of_mem_Icc (A B : HermitianMat d ℤ) :
  ∃ a b, ∀ x, A ≤ x ∧ x ≤ B → spectrum ℝ x.mat ⊆ Set.Icc a b
```

**theorem cfc\_continuous**[\[source\]](#)

```
@[fun_prop]
protected theorem cfc_continuous {f : ℝ → ℝ} (hf : Continuous f) :
  Continuous (HermitianMat.cfc · f : HermitianMat d ℤ → HermitianMat d ℤ)
```

**theorem spectrum\_subset\_Ioi**[\[source\]](#)

```
theorem Matrix.PosDef.spectrum_subset_Ioi {d ℤ : Type*} [Fintype d] [DecidableEq d] [RCLike ℤ]
  {A : Matrix d d ℤ} (hA : A.PosDef) : spectrum ℝ A ⊆ Set.Ioi 0
```

**theorem continuous\_cfc\_fun**[\[source\]](#)

*If  $f$  is a continuous family of functions parameterized by  $x$ , then  $(\text{fun } x => A.\text{cfc } (f\ x))$  is also continuous.*

```
@[fun_prop]
theorem continuous_cfc_fun {f : X → ℝ → ℝ} (hf : ∀ i, Continuous (f · i)) :
  Continuous (fun x => A.cfc (f x))
```

**theorem continuousOn\_cfc\_fun**[\[source\]](#)

*ContinuousOn variant for when all the matrices  $(A\ x)$  have a spectrum in a set  $T$ , and  $f$  is continuous on a set  $S$ .*

```
@[fun_prop]
theorem continuousOn_cfc_fun {T : Set ℝ}
  (hf : ∀ i ∈ T, ContinuousOn (f · i) S) (hA : spectrum ℝ A.mat ⊆ T) :
  ContinuousOn (fun x => A.cfc (f x)) S
```

**lemma norm\_cfc\_le\_sqrt\_card\_mul\_bound**[\[source\]](#)

*Bound the Frobenius norm of a functional calculus application.*

```

lemma norm_cfc_le_sqrt_card_mul_bound {A : HermitianMat d ℂ} {f : ℝ → ℝ} {C : ℝ}
  (hC : 0 ≤ C) (hf : ∀ x ∈ spectrum ℝ A.mat, ||f x|| ≤ C) :
  ||A.cfc f|| ≤ Real.sqrt (Fintype.card d) * C

```

**lemma norm\_cfc\_sub\_cfc\_le\_sqrt\_card**

[\[source\]](#)

```

lemma norm_cfc_sub_cfc_le_sqrt_card {A : HermitianMat d ℂ} {f g : ℝ → ℝ} :
  ||A.cfc f - A.cfc g|| ≤ Real.sqrt (Fintype.card d) * ⨆ x ∈ spectrum ℝ A.mat, ||f x - g x||

```

**lemma norm\_cfc\_sub\_le\_of\_sup\_le**

[\[source\]](#)

```

lemma norm_cfc_sub_le_of_sup_le {A : HermitianMat d ℂ} {f g : ℝ → ℝ} {T : Set ℝ} {ε : ℝ}
  (hT : spectrum ℝ A.mat ⊆ T) (hε : 0 ≤ ε) (h_sup : ∀ x ∈ T, ||f x - g x|| ≤ ε) :
  ||A.cfc f - A.cfc g|| ≤ Real.sqrt (Fintype.card d) * ε

```

**lemma dist\_lt\_of\_continuous'**

[\[source\]](#)

*If  $f$  is jointly continuous on  $S \times T$  and  $T$  is compact, then  $x \mapsto f(x, \cdot)$  is continuous into the space of bounded functions on  $T$  with the uniform norm.*

```

lemma dist_lt_of_continuous' {X : Type*} [TopologicalSpace X]
  {f : X → ℝ → ℝ} {S : Set X} {T : Set ℝ}
  (hT : IsCompact T)
  (hf : ContinuousOn (fun (p : X × ℝ) ↦ f p.1 p.2) (S ×s T))
  {x₀ : X} (hx₀ : x₀ ∈ S) {ε : ℝ} (hε : 0 < ε) :
  ∃ U ∈ nhds x₀, ∀ x ∈ U ∩ S, ∀ t ∈ T, ||f x t - f x₀ t|| < ε

```

**lemma continuous0n\_cfc\_of\_compact**

[\[source\]](#)

*The functional calculus is continuous on matrices with spectrum in a compact set.*

```

lemma continuous0n_cfc_of_compact {K : Set ℝ} {g : ℝ → ℝ} (hK : IsCompact K) (hg : ContinuousOn
  g K) :
  ContinuousOn (fun (A : HermitianMat d ℂ) ↦ A.cfc g) {A | spectrum ℝ A.mat ⊆ K}

```

**theorem continuous\_cfc\_joint\_compact**

[\[source\]](#)

```

theorem continuous_cfc_joint_compact {X d : Type*} [TopologicalSpace X] [Fintype d] [
  DecidableEq d]
  {f : X → ℝ → ℝ} {A : X → HermitianMat d ℂ} {S : Set X} {T : Set ℝ}
  (hT : IsCompact T)
  (hf : ContinuousOn (fun (p : X × ℝ) ↦ f p.1 p.2) (S ×s T))
  (hA₁ : ∀ x ∈ S, spectrum ℝ (A x).mat ⊆ T)
  (hA₂ : ContinuousOn (fun x ↦ A x) S) :
  ContinuousOn (fun x ↦ (A x).cfc (f x)) S

```

**lemma eigenvalue\_norm\_le**

[\[source\]](#)

```

lemma eigenvalue_norm_le (A : HermitianMat d ℂ) (i : d) :
  |A.H.eigenvalues i| ≤ ||A||

```

**lemma spectrum\_subset\_closedBall**[\[source\]](#)

*The spectrum of a HermitianMat is contained in the closed ball of radius  $\|A\|$  around 0.*

```
lemma spectrum_subset_closedBall (A : HermitianMat d ℂ) :
  spectrum ℝ A.mat ⊆ Metric.closedBall (0 : ℝ) ‖A‖
```

**lemma spectrum\_subset\_of\_isOpen**[\[source\]](#)

```
lemma spectrum_subset_of_isOpen (A₀ : HermitianMat d ℂ) (U : Set ℝ)
  (hU : IsOpen U) (hAU : spectrum ℝ A₀.mat ⊆ U) :
  ∀ᶠ B in nhds A₀, spectrum ℝ B.mat ⊆ U
```

**lemma continuousWithinAt\_cfc\_of\_continuousOn**[\[source\]](#)

```
lemma continuousWithinAt_cfc_of_continuousOn {T : Set ℝ} {g : ℝ → ℝ}
  {A₀ : HermitianMat d ℂ}
  (hg : ContinuousOn g T) (hA₀ : spectrum ℝ A₀.mat ⊆ T) :
  ContinuousWithinAt (fun B ↦ B.cfc g) {B | spectrum ℝ B.mat ⊆ T} A₀
```

**lemma dist\_lt\_of\_continuous\_spectrum**[\[source\]](#)

```
lemma dist_lt_of_continuous_spectrum {X : Type*} [TopologicalSpace X]
  {f : X → ℝ → ℝ} {A : X → HermitianMat d ℂ} {S : Set X} {T : Set ℝ}
  (hf : ContinuousOn (fun (p : X × ℝ) ↦ f p.1 p.2) (S ×ˢ T))
  (hA₁ : ∀ x ∈ S, spectrum ℝ (A x).mat ⊆ T)
  (hA₂ : ContinuousOn (fun x ↦ A x) S)
  {x₀ : X} (hx₀ : x₀ ∈ S) {ε : ℝ} (hε : 0 < ε) :
  ∃ U ∈ nhds x₀, ∀ y ∈ U ∩ S, ∀ t ∈ spectrum ℝ (A y).mat, ‖f y t - f x₀ t‖ < ε
```

**theorem continuous\_cfc\_joint**[\[source\]](#)

```
@[fun_prop]
theorem continuous_cfc_joint {X d : Type*} [TopologicalSpace X] [Fintype d] [DecidableEq d]
  {f : X → ℝ → ℝ} {A : X → HermitianMat d ℂ} {S : Set X} {T : Set ℝ}
  (hf : ContinuousOn (fun (p : X × ℝ) ↦ f p.1 p.2) (S ×ˢ T))
  (hA₁ : ∀ x ∈ S, spectrum ℝ (A x).mat ⊆ T)
  (hA₂ : ContinuousOn (fun x ↦ A x) S) :
  ContinuousOn (fun x ↦ (A x).cfc (f x)) S
```

**theorem continuousOn\_cfc\_fun\_nonsingular**[\[source\]](#)

*Specialization of continuousOn\_cfc\_fun for nonsingular matrices.*

```
@[fun_prop]
theorem continuousOn_cfc_fun_nonsingular {f : X → ℝ → ℝ} {S : Set X}
  (hf : ∀ i ≠ 0, ContinuousOn (f · i) S) [NonSingular A] :
  ContinuousOn (fun x ↦ A.cfc (f x)) S
```

**theorem continuousOn\_cfc\_fun\_nonneg**[\[source\]](#)

*Specialization of continuousOn\_cfc\_fun for positive semidefinite matrices.*

```
@[fun_prop]
theorem continuous0n_cfc_fun_nonneg {f : X → ℝ → ℝ} {S : Set X}
  (hf : ∀ i ≥ 0, Continuous0n (f · i) S) (hA : 0 ≤ A) :
  Continuous0n (fun x ↦ A.cfc (f x)) S
```

**theorem continuous0n\_cfc\_fun\_posDef**

[\[source\]](#)

*Specialization of continuous0n\_cfc\_fun for positive definite matrices.*

```
@[fun_prop]
theorem continuous0n_cfc_fun_posDef {f : X → ℝ → ℝ} {S : Set X}
  (hf : ∀ i ≥ 0, Continuous0n (f · i) S) (hA : A.mat.PosDef) :
  Continuous0n (fun x ↦ A.cfc (f x)) S
```

**lemma inv\_cfc\_eq\_cfc\_inv**

[\[source\]](#)

*The inverse of the CFC is the CFC of the inverse function.*

```
lemma inv_cfc_eq_cfc_inv (hf : ∀ i, f (A.H.eigenvalues i) ≠ 0) :
  (A.cfc f)-1 = A.cfc (fun u ↦ (f u)-1)
```

**theorem cfc\_inv**

[\[source\]](#)

```
theorem cfc_inv [NonSingular A] : A.cfc (fun u ↦ u-1) = A-1
```

**lemma integral\_toMat**

[\[source\]](#)

*The integral of a Hermitian matrix function commutes with toMat.*

```
lemma integral_toMat (A : ℝ → HermitianMat d ℤ) (T1 T2 : ℝ) {μ : Measure ℝ}
  (hA : IntervalIntegrable A μ T1 T2) :
  (∫ t in T1..T2, A t ∂μ).mat = ∫ t in T1..T2, (A t).mat ∂μ
```

**lemma intervalIntegrable\_sum\_smul\_const**

[\[source\]](#)

*A sum of scaled constant matrices is integrable if the scalar functions are integrable.*

```
lemma intervalIntegrable_sum_smul_const (T1 T2 : ℝ) {μ : Measure ℝ} (g : ℝ → d → ℝ)
  (P : d → Matrix d d ℤ) (hg : ∀ i, IntervalIntegrable (fun t ↦ g t i) μ T1 T2) :
  IntervalIntegrable (fun t ↦ ∑ i, g t i • P i) μ T1 T2
```

**lemma intervalIntegrable\_toMat\_iff**

[\[source\]](#)

*A function to Hermitian matrices is integrable iff its matrix values are integrable.*

```
lemma intervalIntegrable_toMat_iff (A : ℝ → HermitianMat d ℤ) (T1 T2 : ℝ) {μ : Measure ℝ} :
  IntervalIntegrable (fun t ↦ (A t).mat) μ T1 T2 ↔ IntervalIntegrable A μ T1 T2
```

**lemma integrable\_cfc**

[\[source\]](#)

*The CFC of an integrable function family is integrable.*

```

lemma integrable_cfc (T1 T2 : ℝ) (f : ℝ → ℝ → ℝ) {μ : Measure ℝ}
  (hf : ∀ i, IntervalIntegrable (fun t ↦ f t (A.H.eigenvalues i)) μ T1 T2) :
  IntervalIntegrable (fun t ↦ A.cfc (f t)) μ T1 T2

```

**lemma integral\_cfc\_eq\_cfc\_integral**

[\[source\]](#)

*The integral of the CFC is the CFC of the integral.*

```

lemma integral_cfc_eq_cfc_integral (T1 T2 : ℝ) {μ : Measure ℝ} (f : ℝ → ℝ → ℝ)
  (hf : ∀ i, IntervalIntegrable (fun t ↦ f t (A.H.eigenvalues i)) μ T1 T2) :
  ∫ t in T1..T2, A.cfc (f t) ∂μ = A.cfc (fun u ↦ ∫ t in T1..T2, f t u ∂μ)

```

**theorem cfc\_pos\_of\_pos**

[\[source\]](#)

```

theorem cfc_pos_of_pos {A : HermitianMat d ℤ} {f : ℝ → ℝ} (hA : 0 < A)
  (hf : ∀ i > 0, 0 < f i) (hf2 : 0 ≤ f 0) : 0 < A.cfc f

```

**theorem exists\_HermitianMat\_cfc**

[\[source\]](#)

*If two matrices A and B commute, then they is a common matrix with which they are both CFCs of. This is a variant of the common theorem that "commuting matrices can be simultaneously diagonalized."*

```

theorem _root_.Commute.exists_HermitianMat_cfc (hAB : Commute A.mat B.mat) :
  ∃ C : HermitianMat d ℤ, (∃ f : ℝ → ℝ, A = C.cfc f) ∧ (∃ g : ℝ → ℝ, B = C.cfc g)

```

**theorem cfc\_le\_cfc\_of\_PosDef**

[\[source\]](#)

```

theorem cfc_le_cfc_of_PosDef (hfg : ∀ i, 0 < i → f i ≤ g i) (hA : A.mat.PosDef) :
  A.cfc f ≤ A.cfc g

```

**theorem cfc\_le\_cfc\_of\_commute\_mono0n**

[\[source\]](#)

```

theorem cfc_le_cfc_of_commute_mono0n (hf : Monotone0n f (Set.Ioi 0))
  (hAB1 : Commute A.mat B.mat) (hAB2 : A ≤ B) (hA : A.mat.PosDef) (hB : B.mat.PosDef) :
  A.cfc f ≤ B.cfc f

```

**theorem cfc\_le\_cfc\_of\_commute**

[\[source\]](#)

*TODO: See above*

```

theorem cfc_le_cfc_of_commute (hf : Monotone f) (hAB1 : Commute A.mat B.mat) (hAB2 : A ≤ B) :
  A.cfc f ≤ B.cfc f

```

**theorem inv\_ge\_one\_of\_le\_one**

[\[source\]](#)

```

theorem inv_ge_one_of_le_one (hA : A.mat.PosDef) (h : A ≤ 1) : 1 ≤ A-1

```

**lemma trace\_cfc\_eq**

[\[source\]](#)

*The trace of cfc(f, A) equals the sum of f applied to eigenvalues.*

```
lemma trace_cfc_eq (A : HermitianMat d ℂ) (f : ℝ → ℝ) :
  (A.cfc f).trace = ∑ i, f (A.H.eigenvalues i)
```

**lemma mulVec\_eq\_zero\_iff\_inner\_eigenvector\_zero**

[\[source\]](#)

```
lemma mulVec_eq_zero_iff_inner_eigenvector_zero
  (A : HermitianMat d ℂ) (x : EuclideanSpace ℂ d) :
  A.mat.mulVec x = 0 ↔ ∀ i, A.H.eigenvalues i ≠ 0 → inner ℂ (A.H.eigenvectorBasis i) x = 0
```

**lemma cfc\_mulVec\_expansion**

[\[source\]](#)

```
lemma cfc_mulVec_expansion (A : HermitianMat d ℂ) (f : ℝ → ℝ) (x : EuclideanSpace ℂ d) :
  (A.cfc f).mat.mulVec x = ∑ i, (f (A.H.eigenvalues i) : ℂ) • inner ℂ (A.H.eigenvectorBasis i)
    x • A.H.eigenvectorBasis i
```

**lemma ker\_cfc\_le\_ker\_on\_set**

[\[source\]](#)

```
lemma ker_cfc_le_ker_on_set
  (hs : spectrum ℝ A.mat ⊆ s)
  (h : ∀ i ∈ s, f i = 0 → i = 0) :
  (A.cfc f).ker ≤ A.ker
```

**lemma ker\_cfc\_le\_ker**

[\[source\]](#)

```
lemma ker_cfc_le_ker (h : ∀ i, f i = 0 → i = 0) :
  (A.cfc f).ker ≤ A.ker
```

**lemma ker\_cfc\_le\_ker\_nonneg**

[\[source\]](#)

```
lemma ker_cfc_le_ker_nonneg (hA : 0 ≤ A) (h : ∀ i ≥ 0, f i = 0 → i = 0) :
  (A.cfc f).ker ≤ A.ker
```

**lemma ker\_le\_ker\_cfc\_on\_set**

[\[source\]](#)

```
lemma ker_le_ker_cfc_on_set (hs : spectrum ℝ A.mat ⊆ s) (h : ∀ i ∈ s, i = 0 → f i = 0) :
  A.ker ≤ (A.cfc f).ker
```

**lemma ker\_le\_ker\_cfc**

[\[source\]](#)

```
lemma ker_le_ker_cfc (h : ∀ i, i = 0 → f i = 0) :
  A.ker ≤ (A.cfc f).ker
```

**lemma ker\_le\_ker\_cfc\_nonneg**

[\[source\]](#)

```
lemma ker_le_ker_cfc_nonneg (hA : 0 ≤ A) (h : ∀ i ≥ 0, i = 0 → f i = 0) :
  A.ker ≤ (A.cfc f).ker
```

**theorem ker\_cfc\_eq\_ker**

[\[source\]](#)



```
theorem ker_cfc_eq_ker (h :  $\forall i, f\ i = 0 \leftrightarrow i = 0$ ) :
  (A.cfc f).ker = A.ker
```

**theorem ker\_cfc\_eq\_ker\_nonneg**

[\[source\]](#)

```
theorem ker_cfc_eq_ker_nonneg (hA :  $0 \leq A$ ) (h :  $\forall i \geq 0, f\ i = 0 \leftrightarrow i = 0$ ) :
  (A.cfc f).ker = A.ker
```

## 2.57 QuantumInfo.ForMathlib.HermitianMat.Inner

[QuantumInfo/ForMathlib/HermitianMat/Inner.lean](#) on GitHub.

**instance Inner R (HermitianMat n  $\alpha$ )**

[\[source\]](#)

*The Hermitian inner product,  $\text{Tr}[AB]$ . This is equal to `Matrix.trace (A * B)`, but gives real values when the matrices are complex, using `IsMaximalSelfAdjoint`.*

```
instance : Inner R (HermitianMat n  $\alpha$ ) where
```

**theorem inner\_def**

[\[source\]](#)

```
theorem inner_def (A B : HermitianMat n  $\alpha$ ) :
   $\langle\langle A, B \rangle\rangle_R = \text{selfadjMap } (A.\text{mat} * B.\text{mat}).\text{trace}$ 
```

**theorem inner\_add\_right**

[\[source\]](#)

```
protected theorem inner_add_right :  $\langle\langle A, B + C \rangle\rangle_R = \langle\langle A, B \rangle\rangle_R + \langle\langle A, C \rangle\rangle_R$ 
```

**theorem inner\_add\_left**

[\[source\]](#)

```
protected theorem inner_add_left :  $\langle\langle A + B, C \rangle\rangle_R = \langle\langle A, C \rangle\rangle_R + \langle\langle B, C \rangle\rangle_R$ 
```

**theorem inner\_zero\_right**

[\[source\]](#)

```
@[simp]
protected theorem inner_zero_right :  $\langle\langle A, 0 \rangle\rangle_R = 0$ 
```

**theorem inner\_zero\_left**

[\[source\]](#)

```
@[simp]
protected theorem inner_zero_left :  $\langle\langle 0, A \rangle\rangle_R = 0$ 
```

**theorem inner\_neg\_left**

[\[source\]](#)

```
@[simp]
protected theorem inner_neg_left :  $\langle\langle -A, B \rangle\rangle_R = -\langle\langle A, B \rangle\rangle_R$ 
```

**theorem inner\_neg\_right**[\[source\]](#)

@[simp]

**protected theorem** inner\_neg\_right :  $\langle\langle A, -B \rangle\rangle_R = -\langle\langle A, B \rangle\rangle_R$ **theorem inner\_sub\_left**[\[source\]](#)**protected theorem** inner\_sub\_left :  $\langle\langle A, B - C \rangle\rangle_R = \langle\langle A, B \rangle\rangle_R - \langle\langle A, C \rangle\rangle_R$ **theorem inner\_sub\_right**[\[source\]](#)**protected theorem** inner\_sub\_right :  $\langle\langle A - B, C \rangle\rangle_R = \langle\langle A, C \rangle\rangle_R - \langle\langle B, C \rangle\rangle_R$ **theorem inner\_smul\_left**[\[source\]](#)

@[simp]

**protected theorem** inner\_smul\_left (r : R) :  $\langle\langle r \bullet A, B \rangle\rangle_R = r * \langle\langle A, B \rangle\rangle_R$ **theorem inner\_smul\_right**[\[source\]](#)

@[simp]

**protected theorem** inner\_smul\_right (r : R) :  $\langle\langle A, r \bullet B \rangle\rangle_R = r * \langle\langle A, B \rangle\rangle_R$ **def inner<sub>l</sub>**[\[source\]](#)

*The Hermitian inner product as bilinear form. Compare with inner<sub>l</sub> (in the root namespace) which requires an InnerProductSpace instance.*

**protected def** inner<sub>l</sub> : LinearMap.BilinForm R (HermitianMat n α) **where****theorem inner\_one**[\[source\]](#)

@[simp]

**theorem** inner\_one :  $\langle\langle A, 1 \rangle\rangle_R = A.\text{trace}$ **theorem one\_inner**[\[source\]](#)

@[simp]

**theorem** one\_inner :  $\langle\langle 1, A \rangle\rangle_R = A.\text{trace}$ **theorem inner\_eq\_trace\_mul**[\[source\]](#)

*The inner product for Hermtian matrices is equal to the trace of the product.*

**theorem** inner\_eq\_trace\_mul : algebraMap R α  $\langle\langle A, B \rangle\rangle_R = (A.\text{mat} * B.\text{mat}).\text{trace}$ **theorem inner\_comm**[\[source\]](#)**theorem** inner\_comm :  $\langle\langle A, B \rangle\rangle_R = \langle\langle B, A \rangle\rangle_R$

**theorem inner\_eq\_trace\_trivial**[\[source\]](#)

*HermitianMat.inner reduces to Matrix.trace (A \* B) when the elements are a TrivialStar.*

```
theorem inner_eq_trace_trivial : ⟨⟨A, B⟩⟩_α = (A.mat * B.mat).trace
```

**theorem inner\_eq\_re\_trace**[\[source\]](#)

```
theorem inner_eq_re_trace : ⟨⟨A, B⟩⟩ = RCLike.re (A.mat * B.mat).trace
```

**theorem inner\_eq\_trace\_rc**[\[source\]](#)

```
theorem inner_eq_trace_rc : ⟨⟨A, B⟩⟩ = (A.mat * B.mat).trace
```

**theorem inner\_self\_nonneg**[\[source\]](#)

```
theorem inner_self_nonneg: 0 ≤ ⟨⟨A, A⟩⟩
```

**theorem inner\_mul\_nonneg**[\[source\]](#)

```
theorem inner_mul_nonneg (h : 0 ≤ A.mat * B.mat) : 0 ≤ ⟨⟨A, B⟩⟩
```

**theorem inner\_ge\_zero**[\[source\]](#)

*The inner product for PSD matrices is nonnegative.*

```
theorem inner_ge_zero (hA : 0 ≤ A) (hB : 0 ≤ B) : 0 ≤ ⟨⟨A, B⟩⟩
```

**theorem inner\_mono**[\[source\]](#)

```
theorem inner_mono (hA : 0 ≤ A) : B ≤ C → ⟨⟨A, B⟩⟩ ≤ ⟨⟨A, C⟩⟩
```

**theorem inner\_mono'**[\[source\]](#)

```
theorem inner_mono' (hA : 0 ≤ A) : B ≤ C → ⟨⟨B, A⟩⟩ ≤ ⟨⟨C, A⟩⟩
```

**theorem inner\_le\_mul\_trace**[\[source\]](#)

*The inner product for PSD matrices is at most the product of their traces.*

```
theorem inner_le_mul_trace (hA : 0 ≤ A) (hB : 0 ≤ B) : ⟨⟨A, B⟩⟩ ≤ A.trace * B.trace
```

**theorem inner\_zero\_iff\_aux\_lemma**[\[source\]](#)

```
private theorem inner_zero_iff_aux_lemma [DecidableEq n] (hA₁ : A.mat.PosSemidef) (hB₁ : B.mat.
  PosSemidef) :
  RCLike.re (A.val * B.val).trace = 0 ↔
  LinearMap.range (Matrix.toEuclideanLin A.val) ≤
  LinearMap.ker (Matrix.toEuclideanLin B.val)
```

**theorem inner\_zero\_iff**[\[source\]](#)

*The inner product of two PSD matrices is zero iff they have disjoint support, i.e., each lives entirely in the other's kernel.*

```
theorem inner_zero_iff [DecidableEq n] (hA1 : 0 ≤ A) (hB1 : 0 ≤ B)
  : ⟨⟨A, B⟩⟩ = 0 ↔ A.support ≤ B.ker
```

**theorem reindex\_inner**[\[source\]](#)

```
@[simp]
theorem reindex_inner (e : d ≈ d2) (B : HermitianMat d2 ℤ) :
  ⟨⟨A.reindex e, B⟩⟩ = ⟨⟨A, B.reindex e.symm⟩⟩
```

**theorem inner\_continuous**[\[source\]](#)

```
@[fun_prop]
theorem inner_continuous : Continuous (Inner.inner ℝ (E := HermitianMat d ℤ))
```

**theorem inner\_bilinForm\_Continuous**[\[source\]](#)

```
@[fun_prop]
theorem inner_bilinForm_Continuous (A : HermitianMat d ℤ) : Continuous ↑(HermitianMat.innerℓ A)
```

**def InnerProductCore**[\[source\]](#)

*We define the Hermitian inner product as our "canonical" inner product, which does induce a norm. This disagrees slightly with Mathlib convention on the Matrix type, which avoids asserting one norm as there are several reasonable ones; for Hermitian matrices, though, this seem to be the right choice.*

```
noncomputable def InnerProductCore : InnerProductSpace.Core ℝ (HermitianMat d ℤ)
```

**instance instNormedGroup**[\[source\]](#)

*The HermitianMat type inherits the Frobenius necessarily, since it's going to need the Hermitian inner product, and in Mathlib an InnerProductSpace always carries the corresponding norm.*

```
noncomputable instance instNormedGroup : NormedAddCommGroup (HermitianMat d ℤ)
```

**theorem norm\_eq\_frobenius**[\[source\]](#)

```
theorem norm_eq_frobenius (A : HermitianMat d ℤ) :
  ||A|| = (∑ i : d, ∑ j : d, ||A i j|| ^ 2) ^ (1 / 2 : ℝ)
```

**theorem norm\_eq\_sqrt\_inner\_self**[\[source\]](#)

```
theorem norm_eq_sqrt_inner_self (A : HermitianMat d ℤ) : ||A|| = √(⟨⟨A, A⟩⟩)
```

**instance instNormedSpace**[\[source\]](#)

```
noncomputable instance instNormedSpace : NormedSpace ℝ (HermitianMat d ℚ) where
```

**instance instInnerProductSpace**[\[source\]](#)

```
noncomputable instance instInnerProductSpace : InnerProductSpace ℝ (HermitianMat d ℚ)
```

**instance CompleteSpace (HermitianMat d ℚ)**[\[source\]](#)

```
instance : CompleteSpace (HermitianMat d ℚ)
```

**instance NormedAddCommGroup (HermitianMat d ℝ)**[\[source\]](#)

```
noncomputable instance : NormedAddCommGroup (HermitianMat d ℝ)
```

**instance NormedAddCommGroup (HermitianMat d ℂ)**[\[source\]](#)

```
noncomputable instance : NormedAddCommGroup (HermitianMat d ℂ)
```

**def instOrderClosed**[\[source\]](#)

```
def _root_.RCLike.instOrderClosed : OrderClosedTopology ℚ where
```

**theorem dist\_le\_of\_mem\_Icc**[\[source\]](#)

```
theorem dist_le_of_mem_Icc (x : HermitianMat d ℚ) (hA : A ≤ x) (hB : x ≤ B) :  
  ||x - A|| ≤ ||B - A||
```

**theorem IsHermitian\_isClosed**[\[source\]](#)

```
theorem Matrix.IsHermitian_isClosed : IsClosed { A : Matrix n n ℚ | A.IsHermitian }
```

**theorem PosSemiDef\_isClosed**[\[source\]](#)

```
theorem Matrix.PosSemiDef_isClosed : IsClosed { A : Matrix n n ℚ | A.PosSemidef }
```

**theorem isClosed\_nonneg**[\[source\]](#)

```
theorem isClosed_nonneg : IsClosed { A : HermitianMat n ℚ | 0 ≤ A }
```

**instance OrderClosedTopology (HermitianMat d ℚ)**[\[source\]](#)

```
instance : OrderClosedTopology (HermitianMat d ℚ) where
```

**instance CompactIccSpace (HermitianMat d ℚ)**[\[source\]](#)

*Equivalently: the matrices  $X$  such that  $X - A$  is PSD and  $B - X$  is PSD, form a compact set.*

```
instance : CompactIccSpace (HermitianMat d ℤ) where
```

### theorem unitInterval\_IsCompact

[\[source\]](#)

The PSD matrices that are  $\leq 1$  are a compact set. More generally, this is true of any closed interval, but stating that is a bit different because of how numerals are treated. The  $0$  and  $1$  here are already directly matrices, putting in an  $(a : \mathbb{R}) \cdot 1 \leq m \wedge m \leq (b : \mathbb{R}) \cdot 1$  involves casts. But that theorem should follow easily from this. More generally  $A \leq m \wedge m \leq B$  is compact.

```
theorem unitInterval_IsCompact : IsCompact {m : HermitianMat d ℤ | 0 ≤ m ∧ m ≤ 1}
```

### theorem norm\_one

[\[source\]](#)

```
@[simp]
```

```
theorem norm_one : ||(1 : HermitianMat d ℤ)|| = √(Fintype.card d : ℝ)
```

### theorem norm\_eq\_trace\_sq

[\[source\]](#)

```
theorem norm_eq_trace_sq : ||A|| ^ 2 = (A.mat ^ 2).trace
```

### lemma inner\_eq\_doubly\_stochastic\_sum

[\[source\]](#)

The inner product  $\langle\langle A, B \rangle\rangle$  equals  $\sum_{ij} a_i b_j w_{ij}$  where  $a_i, b_j$  are eigenvalues and  $w_{ij} = \|C_{ij}\|^2$  for  $C = U_A^* U_B$  unitary.

```
lemma inner_eq_doubly_stochastic_sum {d : Type*} [Fintype d] [DecidableEq d]
  (A B : HermitianMat d ℂ) :
  let C
```

## 2.58 QuantumInfo.ForMathlib.HermitianMat.Jordan

[QuantumInfo/ForMathlib/HermitianMat/Jordan.lean](#) on GitHub.

### def symmMul

[\[source\]](#)

```
def symmMul : HermitianMat d ℤ
```

### theorem symmMul\_comm

[\[source\]](#)

```
theorem symmMul_comm : A.symmMul B = B.symmMul A
```

### theorem symmMul\_zero

[\[source\]](#)

```
@[simp]
```

```
theorem symmMul_zero : A.symmMul 0 = 0
```

### theorem zero\_symmMul

[\[source\]](#)

```
@[simp]
```

```
theorem zero_symmMul : symmMul 0 A = 0
```

**theorem symmMul\_toMat**[\[source\]](#)

```
theorem symmMul_toMat : (A.symmMul B).mat =
  (2 :  $\mathbb{k}$ )-1 • (A.mat * B.mat + B.mat * A.mat)
```

**theorem symmMul\_of\_commute**[\[source\]](#)

```
@[simp]
theorem symmMul_of_commute (hAB : Commute A.mat B.mat) :
  (A.symmMul B).mat = A.mat * B.mat
```

**theorem symmMul\_self**[\[source\]](#)

```
theorem symmMul_self : (symmMul A A).mat = A.mat * A.mat
```

**theorem symmMul\_one**[\[source\]](#)

```
@[simp]
theorem symmMul_one : A.symmMul 1 = A
```

**theorem one\_symmMul**[\[source\]](#)

```
@[simp]
theorem one_symmMul : symmMul 1 A = A
```

**theorem symmMul\_neg\_one**[\[source\]](#)

```
@[simp]
theorem symmMul_neg_one : A.symmMul (-1) = -A
```

**theorem neg\_one\_symmMul**[\[source\]](#)

```
@[simp]
theorem neg_one_symmMul : symmMul (-1) A = -A
```

**instance CommMagma (HermitianMat d  $\mathbb{k}$ )**[\[source\]](#)

```
scoped instance : CommMagma (HermitianMat d  $\mathbb{k}$ ) where
```

**theorem mul\_eq\_symmMul**[\[source\]](#)

```
theorem mul_eq_symmMul : A * B = A.symmMul B
```

**instance IsCommJordan (HermitianMat d  $\mathbb{k}$ )**[\[source\]](#)

```
scoped instance : IsCommJordan (HermitianMat d  $\mathbb{k}$ ) where
```

**instance MulZeroClass (HermitianMat d  $\mathbb{k}$ )**[\[source\]](#)

```
scoped instance : MulZeroClass (HermitianMat d  $\mathbb{k}$ ) where
```

**instance MulZeroOneClass (HermitianMat d  $\mathbb{k}$ )** [\[source\]](#)

**scoped instance** : MulZeroOneClass (HermitianMat d  $\mathbb{k}$ ) **where**

**instance NonUnitalNonAssocRing (HermitianMat d  $\mathbb{k}$ )** [\[source\]](#)

**scoped instance** : NonUnitalNonAssocRing (HermitianMat d  $\mathbb{k}$ ) **where**

**instance NonAssocRing (HermitianMat d  $\mathbb{k}$ )** [\[source\]](#)

**scoped instance** : NonAssocRing (HermitianMat d  $\mathbb{k}$ ) **where**

**instance IsScalarTower  $\mathbb{R}$  (HermitianMat d  $\mathbb{k}$ ) (HermitianMat d  $\mathbb{k}$ )** [\[source\]](#)

**scoped instance** : IsScalarTower  $\mathbb{R}$  (HermitianMat d  $\mathbb{k}$ ) (HermitianMat d  $\mathbb{k}$ ) **where**

## 2.59 QuantumInfo.ForMathlib.HermitianMat.LogExp

[QuantumInfo/ForMathlib/HermitianMat/LogExp.lean](#) on GitHub.

**theorem log\_smul\_of\_ne\_zero** [\[source\]](#)

**theorem** Matrix.IsHermitian.log\_smul\_of\_ne\_zero {A : Matrix d d  $\mathbb{k}$ } (hA : A.IsHermitian) (hx : x  $\neq 0$ ) :  
 cfc Real.log (x • A) = (Real.log x) • cfc (if • = 0 then (0 :  $\mathbb{R}$ ) else 1) A + cfc Real.log A

**def exp** [\[source\]](#)

*Matrix exponential of a Hermitian matrix, as given by the continuous functional calculus with Real.exp*

**def** exp (A : HermitianMat d  $\mathbb{k}$ ) : HermitianMat d  $\mathbb{k}$

**theorem exp\_left'** [\[source\]](#)

*Primed because Commute.exp\_left refers to NormedSpace.exp instead of HermitianMat.exp.*

@[aesop **unsafe** apply 50% (rule\_sets := [Commutes])]  
**theorem** \_root\_.Commute.exp\_left' (hAB : Commute A.mat B.mat) :  
 Commute (A.exp).mat B.mat

**theorem exp\_right'** [\[source\]](#)

*Primed because Commute.exp\_right refers to NormedSpace.exp instead of HermitianMat.exp.*

@[aesop **unsafe** apply 50% (rule\_sets := [Commutes])]  
**theorem** \_root\_.Commute.exp\_right' (hAB : Commute A.mat B.mat) :  
 Commute A.mat (B.exp).mat



**theorem reindex\_exp**[\[source\]](#)

```
@[simp]
theorem reindex_exp (e : d ≈ d₂) : (A.reindex e).exp = A.exp.reindex e
```

**instance nonSingular\_exp**[\[source\]](#)

```
instance nonSingular_exp : NonSingular A.exp
```

**theorem exp\_nonneg**[\[source\]](#)

*The matrix exponential of a Hermitian matrix is nonnegative.*

```
theorem exp_nonneg (A : HermitianMat d ℤ) : 0 ≤ A.exp
```

**theorem exp\_pos**[\[source\]](#)

*The matrix exponential of a Hermitian matrix is strictly positive (Loewner order). Requires Nonempty since over an empty index type every matrix equals zero and 0 < 0 is false.*

```
theorem exp_pos [i : Nonempty d] (A : HermitianMat d ℤ) : 0 < A.exp
```

**def evalHermitianMatExp**[\[source\]](#)

*Positivity extension for HermitianMat.exp: always strictly positive if Nonempty d. TODO: We could add a fallback to give nonnegative if Nonempty d is not available, possibly also print a warning. (Users might often not have Nonempty d in context, and they probably want to.)*

```
@[positivity HermitianMat.exp _]
def evalHermitianMatExp : PositivityExt where eval {_u _α} _zα _pα e
```

**def log**[\[source\]](#)

*Matrix logarithm (base e) of a Hermitian matrix, as given by the elementwise real logarithm of the diagonal in a diagonalized form, using Real.log*

*Note that this means that the nullspace of the image includes all of the nullspace of the original matrix. This contrasts to the standard definition, which is typically defined for positive \*definite\* matrices, and the nullspace of the image is exactly the (λ=1)-eigenspace of the original matrix. (We also get the (λ=-1)-eigenspace here!)*

*It coincides with a standard definition if A is positive definite.*

```
def log (A : HermitianMat d ℤ) : HermitianMat d ℤ
```

**theorem log\_left**[\[source\]](#)

```
@[aesop unsafe apply 50% (rule_sets := [Commutes])]
theorem _root_.Commute.log_left (hAB : Commute A.mat B.mat) :
  Commute (A.log).mat B.mat
```

**theorem log\_right**[\[source\]](#)

```
@[aesop unsafe apply 50% (rule_sets := [Commutes])]
theorem _root_.Commute.log_right (hAB : Commute A.mat B.mat) :
  Commute A.mat (B.log).mat
```

**theorem reindex\_log**[\[source\]](#)

```
@[simp]
theorem reindex_log (e : d ≈ d₂) : (A.reindex e).log = A.log.reindex e
```

**theorem log\_zero**[\[source\]](#)

```
@[simp]
theorem log_zero : (0 : HermitianMat d ℤ).log = 0
```

**theorem log\_one**[\[source\]](#)

```
@[simp]
theorem log_one : (1 : HermitianMat d ℤ).log = 0
```

**theorem log\_smul\_of\_pos**[\[source\]](#)

```
theorem log_smul_of_pos (A : HermitianMat d ℤ) (hx : x ≠ 0) :
  (x • A).log = Real.log x • A.supportProj + A.log
```

**theorem log\_smul**[\[source\]](#)

```
theorem log_smul {A : HermitianMat d ℤ} {x : ℝ} (hx : x ≠ 0) [NonSingular A] :
  (x • A).log = Real.log x • 1 + A.log
```

**theorem inv\_antitone**[\[source\]](#)

```
theorem inv_antitone (hA : A.mat.PosDef) (h : A ≤ B) : B-1 ≤ A-1
```

**lemma integral\_inv\_sub\_inv\_finite**[\[source\]](#)

```
lemma Real.integral_inv_sub_inv_finite (x T : ℝ) (hx : 0 < x) (hT : 0 < T) :
  ∫ t in (0)..T, (1 / (1 + t) - 1 / (x + t)) = Real.log x + Real.log ((1 + T) / (x + T))
```

**lemma tendsto\_log\_div\_add\_atTop**[\[source\]](#)

*The limit of  $\log((1+T)/(x+T))$  as  $T \rightarrow \infty$  is 0, for  $x > 0$ .*

```
lemma Real.tendsto_log_div_add_atTop (x : ℝ) :
  Filter.Tendsto (fun T => Real.log ((1 + T) / (x + T))) .atTop (nhds 0)
```

**theorem logApprox\_mono**[\[source\]](#)

*Monotonicity of the finite integral approximation of the logarithm.*

```
theorem logApprox_mono {x y : HermitianMat d  $\mathbb{K}$ } (hx : x.mat.PosDef) (hy : y.mat.PosDef)
  (hxy : x  $\leq$  y) (T :  $\mathbb{R}$ ) (hT : 0 < T) :
   $\int t \text{ in } (0)..T, ((1 + t)^{-1} \cdot (1 : \text{HermitianMat } d \mathbb{K}) - (x + t \cdot 1)^{-1}) \leq$ 
 $\int t \text{ in } (0)..T, ((1 + t)^{-1} \cdot (1 : \text{HermitianMat } d \mathbb{K}) - (y + t \cdot 1)^{-1})$ 
```

**def** logApprox

[\[source\]](#)

```
noncomputable def logApprox {n  $\mathbb{K}$  : Type*} [Fintype n] [DecidableEq n] [RCLike  $\mathbb{K}$ ]
  (x : HermitianMat n  $\mathbb{K}$ ) (T :  $\mathbb{R}$ ) : HermitianMat n  $\mathbb{K}$ 
```

**def** scalarLogApprox

[\[source\]](#)

```
noncomputable def scalarLogApprox (T :  $\mathbb{R}$ ) (u :  $\mathbb{R}$ ) :  $\mathbb{R}$ 
```

**theorem** scalarLogApprox\_eq

[\[source\]](#)

```
theorem scalarLogApprox_eq (x T :  $\mathbb{R}$ ) (hx : 0 < x) (hT : 0 < T) :
  scalarLogApprox T x = Real.log x + Real.log ((1 + T) / (x + T))
```

**lemma** integrand\_eq

[\[source\]](#)

*The integrand in the log approximation is the CFC of the scalar integrand.*

```
private lemma integrand_eq
  (x : HermitianMat d  $\mathbb{K}$ ) (hx : x.mat.PosDef) (t :  $\mathbb{R}$ ) (ht : 0  $\leq$  t) :
   $((1 + t)^{-1} \cdot (1 : \text{HermitianMat } d \mathbb{K}) - (x + t \cdot 1)^{-1}) = x.\text{cfc } (\text{fun } u \Rightarrow (1 + t)^{-1} - (u + t)^{-1})$ 
```

**theorem** logApprox\_eq\_cfc\_scalar

[\[source\]](#)

*The matrix log approximation is the CFC of the scalar log approximation.*

```
theorem logApprox_eq_cfc_scalar
  (x : HermitianMat d  $\mathbb{K}$ ) (hx : x.mat.PosDef) (T :  $\mathbb{R}$ ) (hT : 0 < T) :
  logApprox x T = x.cfc (scalarLogApprox T)
```

**theorem** logApprox\_eq\_log\_add\_error

[\[source\]](#)

*The log approximation is the log plus an error term.*

```
theorem logApprox_eq_log_add_error
  (x : HermitianMat d  $\mathbb{K}$ ) (hx : x.mat.PosDef) (T :  $\mathbb{R}$ ) (hT : 0 < T) :
  logApprox x T = x.log + x.cfc (fun u => Real.log ((1 + T) / (u + T)))
```

**lemma** tendsto\_cfc\_log\_div\_add\_atTop

[\[source\]](#)

*The error term in the log approximation tends to 0 as T goes to infinity.*

```
lemma tendsto_cfc_log_div_add_atTop (x : HermitianMat d  $\mathbb{K}$ ) :
  Tendsto (fun T => x.cfc (fun u => Real.log ((1 + T) / (u + T)))) atTop (nhds 0)
```

**lemma tendsto\_logApprox**[\[source\]](#)

*The log approximation converges to the matrix logarithm.*

```
lemma tendsto_logApprox {x : HermitianMat d ℚ} (hx : x.mat.PosDef) :
  Tendsto (fun T => logApprox x T) atTop (nhds x.log)
```

**theorem posDef\_of\_posDef\_le**[\[source\]](#)

```
theorem posDef_of_posDef_le (hA : A.mat.PosDef) (hAB : A ≤ B) : B.mat.PosDef
```

**theorem log\_mono**[\[source\]](#)

*The matrix logarithm is operator monotone.*

```
theorem log_mono (hA : A.mat.PosDef) (hAB : A ≤ B) : A.log ≤ B.log
```

**theorem le\_of\_exp\_commute**[\[source\]](#)

*Monotonicity of exp on commuting operators.*

```
theorem le_of_exp_commute (hAB₂ : A.exp ≤ B.exp) :
  A ≤ B
```

**lemma inv\_convex**[\[source\]](#)

*The inverse function is operator convex on positive definite matrices.*

```
lemma inv_convex {x y : HermitianMat d ℚ} (hx : x.mat.PosDef) (hy : y.mat.PosDef)
  {a b : ℝ} (ha : 0 ≤ a) (hb : 0 ≤ b) (hab : a + b = 1) :
  (a • x + b • y)⁻¹ ≤ a • x⁻¹ + b • y⁻¹
```

**lemma inv\_shift\_convex**[\[source\]](#)

*The shifted inverse function is operator convex on positive definite matrices.*

```
lemma inv_shift_convex {x y : HermitianMat d ℚ} (hx : x.mat.PosDef) (hy : y.mat.PosDef)
  {a b : ℝ} (ha : 0 ≤ a) (hb : 0 ≤ b) (hab : a + b = 1) (t : ℝ) (ht : 0 ≤ t) :
  (a • x + b • y + t • 1)⁻¹ ≤ a • (x + t • 1)⁻¹ + b • (y + t • 1)⁻¹
```

**lemma integrable\_inv\_shift**[\[source\]](#)

*Definition of the approximation of the matrix logarithm.*

```
lemma integrable_inv_shift {A : HermitianMat d ℚ} (hA : A.mat.PosDef) (b : ℝ) (hb : 0 ≤ b) :
  IntervalIntegrable (fun t => (A + t • 1)⁻¹) volume 0 b
```

**theorem logApprox\_concave**[\[source\]](#)

*The finite integral approximation of the matrix logarithm is operator concave.*

```
theorem logApprox_concave {n ℚ : Type*} [Fintype n] [DecidableEq n] [RCLike ℚ]
  {x y : HermitianMat n ℚ} (hx : x.mat.PosDef) (hy : y.mat.PosDef)
  {a b : ℝ} (ha : 0 ≤ a) (hb : 0 ≤ b) (hab : a + b = 1) (T : ℝ) (hT : 0 ≤ T) :
  a • x.logApprox T + b • y.logApprox T ≤ (a • x + b • y).logApprox T
```

**theorem log\_concave**[\[source\]](#)

*The matrix logarithm is operator concave.*

```
theorem log_concave {x y : HermitianMat d  $\mathbb{k}$ } (hx : x.mat.PosDef) (hy : y.mat.PosDef)
  {a b :  $\mathbb{R}$ } (ha : 0 ≤ a) (hb : 0 ≤ b) (hab : a + b = 1) :
  a • x.log + b • y.log ≤ (a • x + b • y).log
```

**lemma log\_kron\_diagonal**[\[source\]](#)

```
lemma log_kron_diagonal {m n  $\mathbb{k}$  : Type*} [Fintype m] [DecidableEq m] [Fintype n] [DecidableEq n]
  [RCLike  $\mathbb{k}$ ]
  {d1 : m →  $\mathbb{R}$ } {d2 : n →  $\mathbb{R}$ } (h1 : ∀ i, 0 < d1 i) (h2 : ∀ j, 0 < d2 j) :
  (diagonal  $\mathbb{k}$  d1 ⊗k diagonal  $\mathbb{k}$  d2).log =
  (diagonal  $\mathbb{k}$  d1).log ⊗k 1 + 1 ⊗k (diagonal  $\mathbb{k}$  d2).log
```

**lemma log\_conj\_unitary**[\[source\]](#)

*The logarithm of a Hermitian matrix conjugated by a unitary matrix is the conjugate of the logarithm.*

```
lemma log_conj_unitary (A : HermitianMat d  $\mathbb{k}$ ) (U : Matrix.unitaryGroup d  $\mathbb{k}$ ) :
  (A.conj U.val).log = A.log.conj U.val
```

**theorem inner\_log\_smul\_of**[\[source\]](#)

```
theorem inner_log_smul_of [NonSingular A] {x :  $\mathbb{R}$ } (hx : x ≠ 0) :
  ⟨⟨(x • A).log, B⟩⟩ = Real.log x * B.trace + ⟨⟨A.log, B⟩⟩
```

**lemma log\_kron\_diagonal\_with\_proj**[\[source\]](#)

```
lemma log_kron_diagonal_with_proj {f : d →  $\mathbb{R}$ } {g : d2 →  $\mathbb{R}$ } :
  (diagonal  $\mathbb{k}$  f ⊗k diagonal  $\mathbb{k}$  g).log =
  (diagonal  $\mathbb{k}$  f).log ⊗k (diagonal  $\mathbb{k}$  g).supportProj +
  (diagonal  $\mathbb{k}$  f).supportProj ⊗k (diagonal  $\mathbb{k}$  g).log
```

**lemma log\_kron\_with\_proj**[\[source\]](#)

*Generalization of HermitianMat.log\_kron for possibly singular matrices.*

```
lemma log_kron_with_proj : (A ⊗k B).log = A.log ⊗k B.supportProj + A.supportProj ⊗k B.log
```

**theorem log\_kron**[\[source\]](#)

*The matrix logarithm of the Kronecker product of two nonsingular Hermitian matrices is the sum of the Kronecker products of their logarithms with the identity matrix.*

```
theorem log_kron [NonSingular A] [NonSingular B] : (A ⊗k B).log = A.log ⊗k 1 + 1 ⊗k B.log
```

**2.60 QuantumInfo.ForMathlib.HermitianMat.NonSingular**

[QuantumInfo/ForMathlib/HermitianMat/NonSingular.lean](#) on GitHub.

**theorem isUnit\_smul**[\[source\]](#)

```
theorem isUnit_smul {c : R} (hA : IsUnit c) {M : Matrix d d S} (hM : IsUnit M) :
  IsUnit (c • M : Matrix d d S)
```

**theorem isUnit\_natCast**[\[source\]](#)

```
theorem isUnit_natCast {n : ℕ} (hn : n ≠ 0) [CharZero F] : IsUnit (n : Matrix d d F)
```

**theorem isUnit\_real\_smul**[\[source\]](#)

```
theorem isUnit_real_smul {r : ℝ} (hr : r ≠ 0) {M : Matrix d d ℚ} (hM : IsUnit M) :
  IsUnit (r • M : Matrix d d ℚ)
```

**theorem isUnit\_real\_cast**[\[source\]](#)

```
theorem isUnit_real_cast {r : ℝ} (hr : r ≠ 0) : IsUnit (r • 1 : Matrix d d ℚ)
```

**class NonSingular**[\[source\]](#)

```
class NonSingular (A : HermitianMat n R) : Prop where
  isUnit : IsUnit A.mat
```

**theorem isUnit\_mat\_of\_nonSingular**[\[source\]](#)

```
@[simp]
theorem isUnit_mat_of_nonSingular [NonSingular A] : IsUnit A.mat
```

**theorem nonsingular\_iff\_isUnit**[\[source\]](#)

```
theorem nonsingular_iff_isUnit : NonSingular A ↔ IsUnit A.mat
```

**instance instHasInv\_of\_invertible**[\[source\]](#)

```
instance instHasInv_of_invertible [i : Invertible A.mat] : NonSingular A
```

**instance instInvertible\_of\_hasInv**[\[source\]](#)

```
instance instInvertible_of_hasInv [h : NonSingular A] : Invertible A.mat
```

**instance NonSingular (1 : HermitianMat n R)**[\[source\]](#)

```
instance : NonSingular (1 : HermitianMat n R)
```

**theorem nonSingular\_of\_posDef**[\[source\]](#)

```
theorem nonSingular_of_posDef (hA : A.mat.PosDef) : NonSingular A
```

**theorem nonSingular\_iff\_posDef\_of\_PSD**[\[source\]](#)

```
theorem nonSingular_iff_posDef_of_PSD (hA : 0 ≤ A) :
  NonSingular A ↔ A.mat.PosDef
```

**theorem nonSingular\_smul**[\[source\]](#)

```
theorem nonSingular_smul [i : NonSingular A] {c : ℝ} (hc : IsUnit c) : NonSingular (c • A)
```

**theorem nonSingular\_iff\_zero\_notMem\_spectrum**[\[source\]](#)

```
theorem nonSingular_iff_zero_notMem_spectrum : NonSingular A ↔ 0 ∉ spectrum ℝ A.mat
```

**theorem nonSingular\_iff\_eigenvalue\_ne\_zero**[\[source\]](#)

```
theorem nonSingular_iff_eigenvalue_ne_zero : NonSingular A ↔ ∀ i, A.H.eigenvalues i ≠ 0
```

**theorem nonSingular\_iff\_det\_ne\_zero**[\[source\]](#)

```
theorem nonSingular_iff_det_ne_zero : NonSingular A ↔ A.mat.det ≠ 0
```

**theorem nonSingular\_iff\_ker\_bot**[\[source\]](#)

```
theorem nonSingular_iff_ker_bot : NonSingular A ↔ A.ker = ⊥
```

**theorem nonSingular\_iff\_support\_top**[\[source\]](#)

```
theorem nonSingular_iff_support_top : NonSingular A ↔ A.support = ⊤
```

**theorem nonSingular\_iff\_neg**[\[source\]](#)

```
@[simp]
theorem nonSingular_iff_neg : NonSingular (-A) ↔ NonSingular A
```

**theorem nonSingular\_iff\_inv**[\[source\]](#)

```
@[simp]
theorem nonSingular_iff_inv : NonSingular (A-1) ↔ NonSingular A
```

**theorem nonSingular\_iff\_kronecker**[\[source\]](#)

```
@[simp]
theorem nonSingular_iff_kronecker [Nonempty n] [Nonempty m] :
  NonSingular (A ⊗k B) ↔ NonSingular A ∧ NonSingular B
```

**theorem nonSingular\_iff\_conj**[\[source\]](#)

```
theorem nonSingular_iff_conj (hC : IsUnit C) : NonSingular (A.conj C) ↔ NonSingular A
```

**theorem nonSingular\_iff\_reindex**[\[source\]](#)

@[simp]

**theorem** nonSingular\_iff\_reindex (e : n ≈ m) : NonSingular (A.reindex e) ↔ NonSingular A**theorem nonSingular\_empty**[\[source\]](#)**theorem** nonSingular\_empty [IsEmpty n] : NonSingular A**theorem nonSingular\_det\_ne\_zero**[\[source\]](#)**theorem** nonSingular\_det\_ne\_zero : A.mat.det ≠ 0**theorem nonSingular\_ker\_bot**[\[source\]](#)

@[simp]

**theorem** nonSingular\_ker\_bot : A.ker = ⊥**theorem nonSingular\_support\_top**[\[source\]](#)

@[simp]

**theorem** nonSingular\_support\_top : A.support = τ**instance nonSingular\_neg**[\[source\]](#)**instance** nonSingular\_neg : NonSingular (-A)**instance nonSingular\_inv**[\[source\]](#)**instance** nonSingular\_inv : NonSingular (A<sup>-1</sup>)**instance nonSingular\_kron**[\[source\]](#)**instance** nonSingular\_kron [Nonempty n] [Nonempty m] : NonSingular (A ⊗<sub>k</sub> B)**theorem nonSingular\_conj**[\[source\]](#)**theorem** nonSingular\_conj (hC : IsUnit C) : NonSingular (A.conj C)**instance nonSingular\_conj\_isometry**[\[source\]](#)**instance** nonSingular\_conj\_isometry {B : HermitianMat n ℓ} [NonSingular B] :  
NonSingular (A.conj B.mat)**theorem nonSingular\_zero\_notMem\_spectrum**[\[source\]](#)**theorem** nonSingular\_zero\_notMem\_spectrum : 0 ∉ spectrum ℝ A.mat



**theorem nonSingular\_eigenvalue\_ne\_zero**[\[source\]](#)

```
theorem nonSingular_eigenvalue_ne_zero : ∀ i, A.H.eigenvalues i ≠ 0
```

**2.61 QuantumInfo.ForMathlib.HermitianMat.Order**

[QuantumInfo/ForMathlib/HermitianMat/Order.lean](#) on GitHub.

**instance PartialOrder (HermitianMat n  $\mathbb{k}$ )**[\[source\]](#)

*The MatrixOrder instance for Matrix (the Loewner order) we keep open for HermitianMat, always.*

```
instance : PartialOrder (HermitianMat n  $\mathbb{k}$ )
```

**instance IsOrderedAddMonoid (HermitianMat n  $\mathbb{k}$ )**[\[source\]](#)

```
instance : IsOrderedAddMonoid (HermitianMat n  $\mathbb{k}$ )
```

**theorem le\_iff**[\[source\]](#)

```
theorem le_iff : A ≤ B ↔ (B - A).mat.PosSemidef
```

**theorem zero\_le\_iff**[\[source\]](#)

```
theorem zero_le_iff : 0 ≤ A ↔ A.mat.PosSemidef
```

**theorem le\_iff\_mulVec\_le**[\[source\]](#)

```
theorem le_iff_mulVec_le : A ≤ B ↔
  ∀ x, star x ·v A.mat ·v x ≤ star x ·v B.mat ·v x
```

**instance ZeroLEOneClass (HermitianMat n  $\mathbb{k}$ )**[\[source\]](#)

```
instance [DecidableEq n] : ZeroLEOneClass (HermitianMat n  $\mathbb{k}$ ) where
```

**theorem lt\_iff\_posdef**[\[source\]](#)

```
theorem lt_iff_posdef : A < B ↔ (B - A).mat.PosSemidef ∧ A ≠ B
```

**instance IsStrictOrderedModule  $\mathbb{R}$  (HermitianMat n  $\mathbb{k}$ )**[\[source\]](#)

```
instance : IsStrictOrderedModule  $\mathbb{R}$  (HermitianMat n  $\mathbb{k}$ ) where
```

**theorem posSemidef\_iff\_spectrum\_Ici**[\[source\]](#)

```
theorem posSemidef_iff_spectrum_Ici [DecidableEq n] (A : HermitianMat n  $\mathbb{k}$ ) :
  0 ≤ A ↔ spectrum  $\mathbb{R}$  A.mat ⊆ Set.Ici 0
```

**theorem posSemidef\_iff\_spectrum\_nonneg**[\[source\]](#)

```
theorem posSemidef_iff_spectrum_nonneg [DecidableEq n] (A : HermitianMat n ℤ) :
  0 ≤ A ↔ ∀ x ∈ spectrum ℝ A.mat, 0 ≤ x
```

**theorem trace\_nonneg**[\[source\]](#)

```
theorem trace_nonneg (hA : 0 ≤ A) : 0 ≤ A.trace
```

**theorem trace\_pos**[\[source\]](#)

```
theorem trace_pos (hA : 0 < A) : 0 < A.trace
```

**def evalHermitianMatTrace**[\[source\]](#)

*Positivity extension for HermitianMat.trace: nonneg when the matrix is nonneg, positive when the matrix is positive.*

```
@[positivity HermitianMat.trace _]
def evalHermitianMatTrace : PositivityExt where eval {u _α} _zα _pα e
```

**instance PosSMulMono ℝ (HermitianMat n ℤ)**[\[source\]](#)

```
instance : PosSMulMono ℝ (HermitianMat n ℤ)
```

**instance SMulPosMono ℝ (HermitianMat n ℤ)**[\[source\]](#)

```
instance : SMulPosMono ℝ (HermitianMat n ℤ)
```

**instance PosSMulReflectLE ℝ (HermitianMat n ℤ)**[\[source\]](#)

```
instance : PosSMulReflectLE ℝ (HermitianMat n ℤ)
```

**theorem le\_trace\_smul\_one**[\[source\]](#)

```
theorem le_trace_smul_one [DecidableEq n] (hA : 0 ≤ A) : A ≤ A.trace • 1
```

**theorem kronecker\_nonneg**[\[source\]](#)

*The Kronecker product of two nonnegative Hermitian matrices is nonnegative.*

```
theorem kronecker_nonneg {A : HermitianMat m ℤ} (hA : 0 ≤ A) (hB : 0 ≤ B) : 0 ≤ A ⊗_k B
```

**theorem kronecker\_pos**[\[source\]](#)

*The Kronecker product of two positive Hermitian matrices is positive.*

```
theorem kronecker_pos {A : HermitianMat m ℤ} (hA : 0 < A) (hB : 0 < B) : 0 < A ⊗_k B
```

**theorem posSemidef\_to\_nonneg**[\[source\]](#)

```
theorem posSemidef_to_nonneg {A : Matrix n n  $\mathbb{K}$ } (hA : A.PosSemidef) :  $0 \leq A$ 
```

**theorem posDef\_to\_pos**[\[source\]](#)

```
theorem posDef_to_pos {A : Matrix n n  $\mathbb{K}$ } (hA : A.PosDef) [Nonempty n] :  $0 < A$ 
```

**def findMatrixPSDInExpr**[\[source\]](#)

*Given an expression  $e$  (a matrix) and a proof expression  $p$  whose type may be `Matrix.PosSemidef A`, `Matrix.PosDef A`, or `And P Q` (syntactically), attempt to find a proof of nonnegativity or positivity for  $e$ . Only syntactic matching on the head constant is used; `isDefEq` is used only to compare the matrix argument.*

```
private partial def findMatrixPSDInExpr (e : Expr) (p : Expr) (ty : Expr) :  
  MetaM (Option (Bool × Expr))
```

**def evalMatrixPSD**[\[source\]](#)

*Positivity extension for Matrix: looks for `A.PosSemidef` or `A.PosDef` in the local context (including syntactic `And` conjunctions) to prove  $0 \leq A$  or  $0 < A$ .*

```
@[positivity (_ : HermitianMat _ _)]  
def evalMatrixPSD : PositivityExt where eval {_u _α} _zα _pα e
```

**theorem mat\_posSemidef\_to\_nonneg**[\[source\]](#)

```
theorem mat_posSemidef_to_nonneg (hA : A.mat.PosSemidef) :  $0 \leq A$ 
```

**theorem mat\_posDef\_to\_pos**[\[source\]](#)

```
theorem mat_posDef_to_pos [Nonempty n] (hA : A.mat.PosDef) :  $0 < A$ 
```

**def findHermitianMatPSDInExpr**[\[source\]](#)

*Given an expression  $e$  (a `HermitianMat`) and a proof expression  $p$  whose type may be `Matrix.PosSemidef A.mat`, `Matrix.PosDef A.mat`, or `And P Q` (syntactically), attempt to find a proof of nonnegativity or positivity for  $e$ . Only syntactic matching on the head constant is used; `isDefEq` is used only to compare the `HermitianMat` argument.*

```
private partial def findHermitianMatPSDInExpr (e : Expr) (p : Expr) (ty : Expr) :  
  MetaM (Option (Bool × Expr))
```

**def evalHermitianMatPSD**[\[source\]](#)

*Positivity extension for HermitianMat: looks for `A.mat.PosSemidef` or `A.mat.PosDef` in the local context (including syntactic `And` conjunctions) to prove  $0 \leq A$  or  $0 < A$ .*

```
@[positivity (_ : HermitianMat _ _)]  
def evalHermitianMatPSD : PositivityExt where eval {_u _α} _zα _pα e
```

**def evalHermitianMatKronecker**[\[source\]](#)*Positivity extension for HermitianMat.kronecker: nonneg when both factors are.*

```
@[positivity HermitianMat.kronecker _ _]
def evalHermitianMatKronecker : PositivityExt where eval {_u _α} _zα _pα e
```

**theorem conj\_nonneg**[\[source\]](#)*Positivity extension for HermitianMat.conj: nonneg when the inner matrix is.*

```
theorem conj_nonneg (hA : 0 ≤ A) : 0 ≤ A.conj M
```

**theorem conj\_pos**[\[source\]](#)

```
theorem conj_pos [DecidableEq n] {A : HermitianMat n ℚ} {M : Matrix m n ℚ} (hA : 0 < A)
(h : LinearMap.ker M.toEuclideanLin ≤ A.ker) : 0 < A.conj M
```

**def evalHermitianMatConj**[\[source\]](#)*Positivity extension for HermitianMat.conj: nonneg when the inner matrix is.*

```
@[positivity HermitianMat.conj _ _]
def evalHermitianMatConj : PositivityExt where eval {_u _α} _zα _pα e
```

**theorem convex\_cone**[\[source\]](#)

```
theorem convex_cone (hA : 0 ≤ A) (hB : 0 ≤ B) {c1 c2 : ℝ} (hc1 : 0 ≤ c1) (hc2 : 0 ≤ c2) :
0 ≤ (c1 • A + c2 • B)
```

**theorem sq\_nonneg**[\[source\]](#)

```
theorem sq_nonneg [DecidableEq n] : 0 ≤ A ^ 2
```

**theorem ker\_antitone**[\[source\]](#)

```
theorem ker_antitone [DecidableEq n] (hA : 0 ≤ A) : A ≤ B → B.ker ≤ A.ker
```

**theorem conj\_mono**[\[source\]](#)

```
theorem conj_mono (h : A ≤ B) : A.conj M ≤ B.conj M
```

**lemma conj\_posDef**[\[source\]](#)

```
lemma conj_posDef [DecidableEq n] (hA : A.mat.PosDef) (hN : IsUnit N) :
(A.conj N).mat.PosDef
```

**lemma inv\_conj**[\[source\]](#)

```
lemma inv_conj [DecidableEq n] {M : Matrix n n ℚ} (hM : IsUnit M) :
(A.conj M)-1 = A-1.conj (M-1)H
```

**theorem le\_iff\_mulVec\_le\_mulVec**[\[source\]](#)

```
theorem le_iff_mulVec_le_mulVec (A B : HermitianMat n  $\mathbb{K}$ ) :
  A ≤ B ↔ ∀ v : n →  $\mathbb{K}$ , star v ·v A.mat *v v ≤ star v ·v B.mat *v v
```

**theorem inner\_mulVec\_nonneg**[\[source\]](#)

```
theorem inner_mulVec_nonneg (hA : 0 ≤ A) (v : n →  $\mathbb{K}$ ) :
  0 ≤ star v ·v A.mat *v v
```

**theorem mem\_ker\_of\_inner\_mulVec\_zero**[\[source\]](#)

```
theorem mem_ker_of_inner_mulVec_zero [DecidableEq n] (hA : 0 ≤ A) (v : EuclideanSpace  $\mathbb{K}$  n)
  (h : star v ·v A.mat *v v = 0) : v ∈ A.ker
```

**theorem ker\_add**[\[source\]](#)

```
theorem ker_add [DecidableEq n] (hA : 0 ≤ A) (hB : 0 ≤ B) :
  (A + B).ker = A.ker ⊔ B.ker
```

**theorem ker\_sum**[\[source\]](#)

```
theorem ker_sum [DecidableEq n] (f : ι → HermitianMat n  $\mathbb{K}$ ) (hf : ∀ i, 0 ≤ f i) :
  (∑ i, f i).ker = ⋂ i, (f i).ker
```

**theorem ker\_conj**[\[source\]](#)

```
theorem ker_conj [DecidableEq n] (hA : 0 ≤ A) (B : Matrix n n  $\mathbb{K}$ ) :
  (A.conj B).ker = Submodule.comap (Matrix.toEuclideanLin B.conjTranspose) A.ker
```

**theorem ker\_le\_of\_le\_smul**[\[source\]](#)

```
theorem ker_le_of_le_smul {α :  $\mathbb{R}$ } [DecidableEq n] (hα : α ≠ 0) (hA : 0 ≤ A) (hAB : A ≤ α • B) :
  B.ker ≤ A.ker
```

**theorem HermitianMat.eigenvalues\_nonneg**[\[source\]](#)

*If a HermitianMat is PSD, then its eigenvalues are nonneg.*

```
theorem eigenvalues_nonneg [DecidableEq n] (hA : 0 ≤ A) (i : n) :
  0 ≤ A.H.eigenvalues i
```

**theorem mat\_nonneg**[\[source\]](#)

*If a HermitianMat is PSD, its underlying matrix is nonneg in the Loewner order.*

```
theorem mat_nonneg (hA : 0 ≤ A) : 0 ≤ A.mat
```

**theorem mat\_pos**[\[source\]](#)

*If a HermitianMat is positive, its underlying matrix is positive in the Loewner order.*

```
theorem mat_pos (hA : 0 < A) : 0 < A.mat
```

**theorem nonneg\_conjTranspose\_mul\_self**

[\[source\]](#)

$M^H * M$  is nonneg in the Loewner order, for any matrix  $M$ .

```
theorem _root_.Matrix.nonneg_conjTranspose_mul_self {m : Type*} [Fintype m]
  (M : Matrix m n ℚ) : 0 ≤ M.conjTranspose * M
```

**theorem nonneg\_self\_mul\_conjTranspose**

[\[source\]](#)

$M * M^H$  is nonneg in the Loewner order, for any matrix  $M$ .

```
theorem _root_.Matrix.nonneg_self_mul_conjTranspose {m : Type*} [Fintype m]
  (M : Matrix n m ℚ) : 0 ≤ M * M.conjTranspose
```

**theorem subtype\_mk\_nonneg**

[\[source\]](#)

```
theorem subtype_mk_nonneg {M : Matrix m m ℚ} (h : 0 ≤ M) :
  0 ≤ ((M, (Matrix.LE.le.posSemidef h).isHermitian) : HermitianMat m ℚ)
```

**theorem subtype\_mk\_pos**

[\[source\]](#)

```
theorem subtype_mk_pos {M : Matrix m m ℚ} (h : 0 < M) :
  0 < ((M, (Matrix.LE.le.posSemidef h.le).isHermitian) : HermitianMat m ℚ)
```

**theorem Matrix.eigenvalues\_nonneg**

[\[source\]](#)

```
private theorem _root_.Matrix.eigenvalues_nonneg [DecidableEq n] {M : Matrix n n ℚ} (h : 0 ≤ M)
  (i : n) :
  0 ≤ (Matrix.LE.le.posSemidef h).isHermitian.eigenvalues i
```

**def evalHermitianMatMat**

[\[source\]](#)

Positivity extension for  $A.mat$  where  $A : HermitianMat$ : nonneg when  $0 \leq A$ .

```
@[positivity HermitianMat.mat _]
def evalHermitianMatMat : PositivityExt where eval {u α} _za _pa e
```

**def evalHermitianMatVal**

[\[source\]](#)

Positivity extension for  $A.mat$  where  $A : HermitianMat$ : nonneg when  $0 \leq A$ .

```
@[positivity Subtype.val (_ : HermitianMat _ _)]
def evalHermitianMatVal : PositivityExt where eval {u α} _za _pa e
```

**def evalMatrixSelfMulConjTranspose**

[\[source\]](#)

Positivity extension for  $M * M^H$  as a Matrix: always nonneg.

```
@[positivity HMul.hMul _ (Matrix.conjTranspose _)]
def evalMatrixSelfMulConjTranspose : PositivityExt where eval {u α} _za _pa e
```

**def evalMatrixConjTransposeMulSelf**[\[source\]](#)

*Positivity extension for  $M^H * M$  as a Matrix: always nonneg.*

```
@[positivity HMul.hMul (Matrix.conjTranspose _) _]
def evalMatrixConjTransposeMulSelf : PositivityExt where eval {_u _α} _zα _pα e
```

**def evalHermitianMatMk**[\[source\]](#)

*Positivity extension for  $\langle M, (pf : M.IsHermitian) \rangle$  as a HermitianMat: equivalent to  $\theta \leq M$  in MatrixOrder.*

```
@[positivity (Subtype.mk _ _ : HermitianMat _ _)]
def evalHermitianMatMk : PositivityExt where eval {_u _α} _zα _pα e
```

**def evalMatrixEigenvalues**[\[source\]](#)

*Positivity extension for eigenvalues of a Matrix:  $\theta \leq (_ : M.IsHermitian).eigenvalues$  i. Will try to prove  $\theta \leq M$  in the MatrixOrder. If the proof is A.H, i.e.  $M$  comes from a HermitianMat, this will give  $\theta \leq A.mat$  which becomes  $\theta \leq A$  later.*

```
@[positivity Matrix.IsHermitian.eigenvalues _ _]
def evalMatrixEigenvalues : PositivityExt where eval {_u _α} _zα _pα e
```

**2.62 QuantumInfo.ForMathlib.HermitianMat.Proj**

[QuantumInfo/ForMathlib/HermitianMat/Proj.lean](#) on GitHub.

**def projector**[\[source\]](#)

*Given a Submodule (EuclideanSpace ...) to HermitianMat, this gives the projector onto that subspace, i.e. a matrix that squares to itself, preserves vectors in the submodule, and zeroes out anything in the orthogonal complement of that submodule.*

```
noncomputable def projector (S : Submodule ℚ (EuclideanSpace ℚ n)) : HermitianMat n ℚ
```

**theorem projector\_add\_orthogonal**[\[source\]](#)

```
theorem projector_add_orthogonal : projector S + projector S⊥ = 1
```

**theorem trace\_projector**[\[source\]](#)

```
@[simp]
theorem trace_projector : (projector S).trace = (Module.finrank ℚ S : ℝ)
```

**def supportProj**[\[source\]](#)

*The HermitianMat.projector for the HermitianMat.support submodule.*

```
noncomputable def supportProj (A : HermitianMat n ℚ) : HermitianMat n ℚ
```

**def kerProj** [\[source\]](#)

*The HermitianMat.projector for the HermitianMat.ker submodule.*

```
noncomputable def kerProj (A : HermitianMat n ℤ) : HermitianMat n ℤ
```

**theorem kerProj\_add\_supportProj** [\[source\]](#)

@[simp]

```
theorem kerProj_add_supportProj : A.kerProj + A.supportProj = 1
```

**theorem kerProj\_of\_nonSingular** [\[source\]](#)

@[simp]

```
theorem kerProj_of_nonSingular [NonSingular A] : A.kerProj = 0
```

**theorem supportProj\_of\_nonSingular** [\[source\]](#)

@[simp]

```
theorem supportProj_of_nonSingular [NonSingular A] : A.supportProj = 1
```

**theorem projector\_eq\_sum\_rankOne** [\[source\]](#)

*The projector onto a submodule  $S$  is the sum of the outer products of the vectors in an orthonormal basis of  $S$ .*

```
theorem projector_eq_sum_rankOne (b : OrthonormalBasis ι ℤ S) :  
  (projector S).mat = ∑ i, Matrix.vecMulVec (S.subtype (b i)) (star (S.subtype (b i)))
```

**lemma projector\_support\_eq\_sum** [\[source\]](#)

*The projector onto the support of  $A$  is the sum of the projections onto the eigenvectors with non-zero eigenvalues.*

```
lemma projector_support_eq_sum : A.supportProj.mat =  
  ∑ i, (if A.H.eigenvalues i = 0 then 0 else 1) •  
    Matrix.vecMulVec (A.H.eigenvectorBasis i) (star (A.H.eigenvectorBasis i))
```

**theorem supportProj\_eq\_cfc** [\[source\]](#)

```
theorem supportProj_eq_cfc : A.supportProj = A.cfc (if · = 0 then 0 else 1)
```

**def projLE** [\[source\]](#)

*Projector onto the non-negative eigenspace of  $B - A$ . Accessible by the notation  $\{A \leq_p B\}$ , which is scoped to HermitianMat. This is the unique maximum operator  $P$  such that  $P^2 = P$  and  $P * A * P \leq P * B * P$  in the Loewner order.*

```
def projLE (A B : HermitianMat n ℤ) : HermitianMat n ℤ
```



**def projLT**[\[source\]](#)

*Projector onto the positive eigenspace of  $B - A$ . Accessible by the notation  $\{A <_p B\}$ , which is scoped to *HermitianMat*. Compare with *proj\_le*.*

```
noncomputable def projLT (A B : HermitianMat n ℤ) : HermitianMat n ℤ
```

**theorem projLE\_def**[\[source\]](#)

```
theorem projLE_def : {A ≤p B} = (B - A).cfc (fun x ↦ if 0 ≤ x then 1 else 0)
```

**theorem projLT\_def**[\[source\]](#)

```
theorem projLT_def : {A <p B} = (B - A).cfc (fun x ↦ if 0 < x then 1 else 0)
```

**theorem projLE\_sq**[\[source\]](#)

```
theorem projLE_sq : {A ≤p B}^2 = {A ≤p B}
```

**theorem projLT\_sq**[\[source\]](#)

```
theorem projLT_sq : {A <p B}^2 = {A <p B}
```

**theorem projLE\_zero\_cfc**[\[source\]](#)

```
theorem projLE_zero_cfc : {0 ≤p A} = A.cfc (fun x ↦ if 0 ≤ x then 1 else 0)
```

**theorem projLT\_zero\_cfc**[\[source\]](#)

```
theorem projLT_zero_cfc : {0 <p A} = A.cfc (fun x ↦ if 0 < x then 1 else 0)
```

**theorem projLE\_zero\_cfc'**[\[source\]](#)

```
theorem projLE_zero_cfc' : {A ≤p 0} = A.cfc (fun x ↦ if x ≤ 0 then 1 else 0)
```

**theorem projLT\_zero\_cfc'**[\[source\]](#)

```
theorem projLT_zero_cfc' : {A <p 0} = A.cfc (fun x ↦ if x < 0 then 1 else 0)
```

**theorem projLE\_nonneg**[\[source\]](#)

```
theorem projLE_nonneg : 0 ≤ {A ≤p B}
```

**theorem projLT\_nonneg**[\[source\]](#)

```
theorem projLT_nonneg : 0 ≤ {A <p B}
```

**theorem projLE\_le\_one**[\[source\]](#)

```
theorem projLE_le_one : {A ≤p B} ≤ 1
```

**theorem projLE\_mul\_nonneg**[\[source\]](#)

```
theorem projLE_mul_nonneg : 0 ≤ {A ≤p B}.mat * (B - A).mat
```

**theorem projLE\_mul\_le**[\[source\]](#)

```
theorem projLE_mul_le : {A ≤p B}.mat * A.mat ≤ {A ≤p B}.mat * B.mat
```

**theorem proj\_le\_add\_lt**[\[source\]](#)

```
@[simp]
```

```
theorem proj_le_add_lt : {A <p B} + {B ≤p A} = 1
```

**theorem conj\_lt\_add\_conj\_le**[\[source\]](#)

```
theorem conj_lt_add_conj_le : A.conj {A <p 0} + A.conj {0 ≤p A} = A
```

**theorem supportProj\_eq\_proj\_lt\_add\_proj\_lt**[\[source\]](#)

```
theorem supportProj_eq_proj_lt_add_proj_lt (A : HermitianMat n ℤ) :  
  A.supportProj = {A <p 0} + {0 <p A}
```

**instance PosPart (HermitianMat n ℤ)**[\[source\]](#)

*The positive part of a Hermitian matrix: the projection onto its positive eigenvalues.*

```
instance : PosPart (HermitianMat n ℤ) where
```

**instance NegPart (HermitianMat n ℤ)**[\[source\]](#)

*The negative part of a Hermitian matrix: the projection onto its negative eigenvalues.*

```
instance : NegPart (HermitianMat n ℤ) where
```

**theorem posPart\_eq\_cfc\_max**[\[source\]](#)

```
theorem posPart_eq_cfc_max : A+ = A.cfc (fun x ↦ x ⊔ 0)
```

**theorem negPart\_eq\_cfc\_min**[\[source\]](#)

```
theorem negPart_eq_cfc_min : A- = A.cfc (fun x ↦ -x ⊔ 0)
```

**theorem posPart\_eq\_cfc\_ite**[\[source\]](#)

```
theorem posPart_eq_cfc_ite : A+ = A.cfc (fun x ↦ if 0 ≤ x then x else 0)
```

**theorem negPart\_eq\_cfc\_ite**[\[source\]](#)

```
theorem negPart_eq_cfc_ite : A- = A.cfc (fun x ↦ if x ≤ 0 then -x else 0)
```

**theorem posPart\_eq\_posPart\_toMat**[\[source\]](#)

*There is an existing (very slow) PosPart instance on Matrix  $n \times n \mathbb{K}$ , this shows that this is equal.*

```
theorem posPart_eq_posPart_toMat : A+ = A.mat+
```

**theorem negPart\_eq\_negPart\_toMat**[\[source\]](#)

*There is an existing (very slow) PosPart instance on Matrix  $n \times n \mathbb{K}$ , this shows that this is equal.*

```
theorem negPart_eq_negPart_toMat : A- = A.mat-
```

**theorem posPart\_eq\_cfc\_lt**[\[source\]](#)

*The positive part can be equivalently described as the nonnegative part.*

```
theorem posPart_eq_cfc_lt : A+ = A.cfc (fun x ↦ if 0 < x then x else 0)
```

**theorem negPart\_eq\_cfc\_lt**[\[source\]](#)

*The negative part can be equivalently described as the nonpositive part.*

```
theorem negPart_eq_cfc_lt : A- = A.cfc (fun x ↦ if x < 0 then -x else 0)
```

**theorem posPart\_add\_negPart**[\[source\]](#)

```
theorem posPart_add_negPart : A+ - A- = A
```

**theorem posPart\_eq\_self**[\[source\]](#)

```
theorem posPart_eq_self {A : HermitianMat n  $\mathbb{K}$ } (hA : 0 ≤ A) :  
  A+ = A
```

**theorem posPart\_nonneg**[\[source\]](#)

```
theorem posPart_nonneg : 0 ≤ A+
```

**theorem negPart\_nonneg**[\[source\]](#)

```
theorem negPart_nonneg : 0 ≤ A-
```

**theorem posPart\_le**[\[source\]](#)

```
theorem posPart_le : A ≤ A+
```

**theorem posPart\_mul\_negPart**[\[source\]](#)

```
theorem posPart_mul_negPart : A+.mat * A-.mat = 0
```

**theorem projLE\_inner\_nonneg**[\[source\]](#)

**theorem** projLE\_inner\_nonneg :  $0 \leq \langle\langle A \leq_p B, (B - A) \rangle\rangle$

**theorem projLE\_inner\_le**[\[source\]](#)

**theorem** projLE\_inner\_le :  $\langle\langle A \leq_p B, A \rangle\rangle \leq \langle\langle A \leq_p B, B \rangle\rangle$

**theorem inner\_projLE\_nonneg**[\[source\]](#)

**theorem** inner\_projLE\_nonneg :  $0 \leq \langle\langle A \leq_p B, (B - A) \rangle\rangle$

**theorem inner\_projLE\_le**[\[source\]](#)

**theorem** inner\_projLE\_le :  $\langle\langle A \leq_p B, A \rangle\rangle \leq \langle\langle A \leq_p B, B \rangle\rangle$

**theorem posPart\_Continuous**[\[source\]](#)

@[fun\_prop]

**theorem** posPart\_Continuous : Continuous  $(\cdot^+ : \text{HermitianMat } n \mathbb{C} \rightarrow \_)$

**theorem negPart\_Continuous**[\[source\]](#)

@[fun\_prop]

**theorem** negPart\_Continuous : Continuous  $(\cdot^- : \text{HermitianMat } n \mathbb{C} \rightarrow \_)$

**theorem one\_sub\_projLT**[\[source\]](#)

**theorem** one\_sub\_projLT :  $1 - \{B \leq_p A\} = \{A <_p B\}$

**theorem projLT\_mul\_nonneg**[\[source\]](#)

**theorem** projLT\_mul\_nonneg :  $0 \leq \{A <_p B\}.\text{mat} * (B - A).\text{mat}$

**theorem proj\_lt\_mul\_lt**[\[source\]](#)

**theorem** proj\_lt\_mul\_lt :  $\{A <_p B\}.\text{mat} * A.\text{mat} \leq \{A <_p B\}.\text{mat} * B.\text{mat}$

**theorem inner\_negPart\_nonpos**[\[source\]](#)

**theorem** inner\_negPart\_nonpos :  $\langle\langle A, A^- \rangle\rangle \leq 0$

**theorem posPart\_inner\_negPart\_zero**[\[source\]](#)

@[simp]

**theorem** posPart\_inner\_negPart\_zero :  $\langle\langle A^+, A^- \rangle\rangle = 0$

**theorem inner\_negPart\_zero\_iff**[\[source\]](#)

**theorem** inner\_negPart\_zero\_iff :  $\langle\langle A, A^- \rangle\rangle = 0 \leftrightarrow 0 \leq A$

**theorem inner\_negPart\_neg\_iff**[\[source\]](#)

```
theorem inner_negPart_neg_iff :  $\langle\langle A, A^- \rangle\rangle < 0 \leftrightarrow \neg 0 \leq A$ 
```

**theorem nonneg\_iff\_inner\_nonneg**[\[source\]](#)

*The self-duality of the PSD cone: a matrix is PSD iff its inner product with all nonnegative matrices is non-negative.*

```
theorem nonneg_iff_inner_nonneg (A : HermitianMat n  $\mathbb{K}$ ) :  
   $0 \leq A \leftrightarrow \forall B, 0 \leq B \rightarrow 0 \leq \langle\langle A, B \rangle\rangle$ 
```

**2.63 QuantumInfo.ForMathlib.HermitianMat.Reindex**

[QuantumInfo/ForMathlib/HermitianMat/Reindex.lean](#) on GitHub.

**def reindex**[\[source\]](#)

```
def reindex (e : d  $\approx$  d2) : HermitianMat d2  $\mathbb{K}$ 
```

**theorem mat\_reindex**[\[source\]](#)

```
@[simp]  
theorem mat_reindex : (A.reindex e).mat = A.mat.reindex e e
```

**theorem reindex\_refl**[\[source\]](#)

```
@[simp]  
theorem reindex_refl (A : HermitianMat d  $\mathbb{K}$ ) :  
  A.reindex (.refl _) = A
```

**theorem reindex\_reindex**[\[source\]](#)

```
@[simp]  
theorem reindex_reindex (A : HermitianMat d  $\mathbb{K}$ ) (e : d  $\approx$  d2) (f : d2  $\approx$  d3) :  
  (A.reindex e).reindex f = A.reindex (e.trans f)
```

**theorem reindex\_zero**[\[source\]](#)

```
@[simp]  
theorem reindex_zero : (0 : HermitianMat d  $\mathbb{K}$ ).reindex e = 0
```

**theorem reindex\_one**[\[source\]](#)

```
@[simp]  
theorem reindex_one [DecidableEq d] [DecidableEq d2] :  
  (1 : HermitianMat d  $\mathbb{K}$ ).reindex e = 1
```

**theorem reindex\_add**[\[source\]](#)

```
@[simp]  
theorem reindex_add : A.reindex e + B.reindex e = (A + B).reindex e
```

**theorem reindex\_sub**[\[source\]](#)

```
@[simp]
theorem reindex_sub : A.reindex e - B.reindex e = (A - B).reindex e
```

**theorem reindex\_neg**[\[source\]](#)

```
@[simp]
theorem reindex_neg : (-A).reindex e = -(A.reindex e)
```

**theorem reindex\_smul**[\[source\]](#)

```
@[simp]
theorem reindex_smul (c : ℝ) : (c • A).reindex e = c • (A.reindex e)
```

**theorem reindex\_conj**[\[source\]](#)

```
@[simp]
theorem reindex_conj [Fintype d₂] [Fintype d] (B : Matrix d₃ d₂ ℤ) :
  (A.reindex e).conj B = A.conj (B.submatrix id e)
```

**theorem conj\_submatrix**[\[source\]](#)

```
theorem conj_submatrix (B : Matrix d₂ d₄ ℤ) (e : d₃ ≈ d₂) (f : d → d₄) :
  A.conj (B.submatrix e f) = (A.conj (B.submatrix id f)).reindex e.symm
```

**theorem reindex\_eq\_conj**[\[source\]](#)

```
theorem reindex_eq_conj [DecidableEq d] (e : d ≈ d₂) : A.reindex e = A.conj (Matrix.reindex e (.
  refl d) 1)
```

**theorem ker\_reindex**[\[source\]](#)

```
theorem ker_reindex :
  (A.reindex e).ker = A.ker.comap (LinearEquiv.euclidean_of_relabel ℤ e).toLinearMap
```

**theorem ker\_reindex\_le\_iff**[\[source\]](#)

```
@[simp]
theorem ker_reindex_le_iff :
  (A.reindex e).ker ≤ (B.reindex e).ker ↔ A.ker ≤ B.ker
```

**2.64 QuantumInfo.ForMathlib.HermitianMat.Rpow**

[QuantumInfo/ForMathlib/HermitianMat/Rpow.lean](#) on GitHub.

**def rpow**[\[source\]](#)

*Matrix power of a positive semidefinite matrix, as given by the elementwise real power of the diagonal in a diagonalized form.*

*Note that this has the usual `Real.rpow` caveats, such as 0 to the power -1 giving 0.*

```
def rpow (A : HermitianMat d ℤ) (r : ℝ) : HermitianMat d ℤ
```

**instance instRPow**

[\[source\]](#)

```
instance instRPow : Pow (HermitianMat d ℤ) ℝ
```

**theorem rpow\_conj\_unitary**

[\[source\]](#)

```
theorem rpow_conj_unitary (A : HermitianMat d ℤ) (U : Matrix.unitaryGroup d ℤ) (r : ℝ) :
  (HermitianMat.conj U.val A) ^ r = HermitianMat.conj U.val (A ^ r)
```

**theorem pow\_eq\_rpow**

[\[source\]](#)

```
theorem pow_eq_rpow : A ^ r = A.rpow r
```

**theorem rpow\_eq\_cfc**

[\[source\]](#)

```
theorem rpow_eq_cfc : A ^ r = A.cfc (· ^ r)
```

**theorem diagonal\_pow**

[\[source\]](#)

```
theorem diagonal_pow (f : d → ℝ) :
  (diagonal ℤ f) ^ r = diagonal ℤ (fun i ↦ (f i) ^ r)
```

**theorem rpow\_const\_continuous**

[\[source\]](#)

@[fun\_prop]

```
theorem rpow_const_continuous {r : ℝ} (hr : 0 ≤ r) : Continuous (fun A : HermitianMat d ℂ ↦ A ^ r)
```

**theorem const\_rpow\_continuous**

[\[source\]](#)

@[fun\_prop]

```
theorem const_rpow_continuous [NonSingular A] : Continuous (fun r : ℝ ↦ A ^ r)
```

**theorem continuousOn\_rpow\_pos**

[\[source\]](#)

*For a fixed Hermitian matrix  $A$ , the function  $x \mapsto A^x$  is continuous for  $x > 0$ .*

@[fun\_prop]

```
theorem continuousOn_rpow_pos (A : HermitianMat d ℂ) : ContinuousOn (fun x : ℝ ↦ A ^ x) (Set.Ioi 0)
```

**theorem continuousOn\_rpow\_neg**

[\[source\]](#)

*For a fixed Hermitian matrix  $A$ , the function  $x \mapsto A^x$  is continuous for  $x < 0$ .*

@[fun\_prop]

```
theorem continuousOn_rpow_neg (A : HermitianMat d ℂ) : ContinuousOn (fun x : ℝ ↦ A ^ x) (Set.Iio 0)
```

**theorem rpow\_one**[\[source\]](#)

@[simp]

**theorem** rpow\_one :  $A ^ (1 : \mathbb{R}) = A$ **lemma sqrt\_eq\_cfc\_rpow\_half**[\[source\]](#)*Functional calculus of  $\text{Real.sqrt}$  is equal to functional calculus of  $x^{(1/2)}$ .***lemma** sqrt\_eq\_cfc\_rpow\_half (A : HermitianMat d  $\mathbb{k}$ ) :A.sqrt = A.cfc (**fun** x  $\mapsto$   $x ^ (1/2 : \mathbb{R})$ )**theorem one\_rpow**[\[source\]](#)

@[simp]

**theorem** one\_rpow :  $(1 : \text{HermitianMat } d \mathbb{k}) ^ r = 1$ **lemma rpow\_zero**[\[source\]](#)

@[simp]

**lemma** rpow\_zero (A : HermitianMat d  $\mathbb{k}$ ) :  $A ^ (0 : \mathbb{R}) = 1$ **lemma rpow\_diagonal**[\[source\]](#)**lemma** rpow\_diagonal (a : d  $\rightarrow \mathbb{R}$ ) (r :  $\mathbb{R}$ ) : $(\text{diagonal } \mathbb{C} \ a) ^ r = \text{diagonal } \mathbb{C} \ (\text{fun } i \Rightarrow a \ i ^ r)$ **theorem reindex\_rpow**[\[source\]](#)*Keeps in line with our simp-normal form for moving reindex outwards.*

@[simp]

**theorem** reindex\_rpow (e : d  $\approx$  d<sub>2</sub>) :

A.reindex e ^ r = (A ^ r).reindex e

**theorem mat\_rpow\_add**[\[source\]](#)**theorem** mat\_rpow\_add (hA :  $0 \leq A$ ) {p q :  $\mathbb{R}$ } (hpq : p + q  $\neq$  0) : $(A ^ (p + q)).\text{mat} = (A ^ p).\text{mat} * (A ^ q).\text{mat}$ **theorem rpow\_mul**[\[source\]](#)**theorem** rpow\_mul (hA :  $0 \leq A$ ) {p q :  $\mathbb{R}$ } :  $A ^ (p * q) = (A ^ p) ^ q$ **theorem conj\_rpow**[\[source\]](#)**theorem** conj\_rpow (hA :  $0 \leq A$ ) (hq : q  $\neq$  0) (hqr : r + 2 \* q  $\neq$  0) : $(A ^ r).\text{conj } (A ^ q) = A ^ (r + 2 * q)$



**theorem pow\_half\_mul**[\[source\]](#)

```
theorem pow_half_mul (hA : 0 ≤ A) :
  (A ^ (1/2 : ℝ)).mat * (A ^ (1/2 : ℝ)).mat = A
```

**theorem rpow\_pos**[\[source\]](#)

```
theorem rpow_pos {A : HermitianMat d ℤ} (hA : 0 < A) {p : ℝ} : 0 < A ^ p
```

**theorem rpow\_nonneg**[\[source\]](#)

```
theorem rpow_nonneg (hA : 0 ≤ A) {p : ℝ} : 0 ≤ A ^ p
```

**theorem inv\_eq\_rpow\_neg\_one**[\[source\]](#)

```
theorem inv_eq_rpow_neg_one (hA : A.mat.PosDef) : A-1 = A ^ (-1 : ℝ)
```

**theorem sandwich\_self**[\[source\]](#)

```
theorem sandwich_self (hB : B.mat.PosDef) :
  (B.conj (B ^ (-1/2 : ℝ)).mat) = 1
```

**lemma rpow\_inv\_eq\_neg\_rpow**[\[source\]](#)

```
lemma rpow_inv_eq_neg_rpow (hA : A.mat.PosDef) (p : ℝ) : (A ^ p)-1 = A ^ (-p)
```

**lemma sandwich\_le\_one**[\[source\]](#)

```
lemma sandwich_le_one (hB : B.mat.PosDef) (h : A ≤ B) :
  (A.conj (B ^ (-1/2 : ℝ)).mat) ≤ 1
```

**lemma rpow\_neg\_mul\_rpow\_self**[\[source\]](#)

```
lemma rpow_neg_mul_rpow_self (hA : A.mat.PosDef) (p : ℝ) :
  (A ^ (-p)).mat * (A ^ p).mat = 1
```

**lemma isUnit\_rpow\_toMat**[\[source\]](#)

```
lemma isUnit_rpow_toMat (hA : A.mat.PosDef) (p : ℝ) : IsUnit (A ^ p).mat
```

**lemma sandwich\_inv**[\[source\]](#)

```
lemma sandwich_inv (hB : B.mat.PosDef) :
  (A.conj (B ^ (-1/2 : ℝ)).mat)-1 = A-1.conj (B ^ (1/2 : ℝ)).mat
```

**theorem ker\_rpow\_eq\_of\_nonneg**[\[source\]](#)

```
theorem ker_rpow_eq_of_nonneg {A : HermitianMat d ℂ} (hA : 0 ≤ A) (hp : r ≠ 0) :
  (A ^ r).ker = A.ker
```

**theorem ker\_rpow\_le\_of\_nonneg**[\[source\]](#)

```
theorem ker_rpow_le_of_nonneg {A : HermitianMat d ℂ} (hA : 0 ≤ A) :
  (A ^ r).ker ≤ A.ker
```

**lemma rpow\_kron\_diagonal**[\[source\]](#)

```
private lemma rpow_kron_diagonal
  (a : d → ℝ) (b : d₂ → ℝ) (r : ℝ) (ha : ∀ i, 0 ≤ a i) (hb : ∀ j, 0 ≤ b j) :
  ((diagonal ℂ a) ⊗ₖ (diagonal ℂ b)) ^ r =
  ((diagonal ℂ a) ^ r) ⊗ₖ ((diagonal ℂ b) ^ r)
```

**lemma conj\_kron**[\[source\]](#)

```
lemma conj_kron
  (A : Matrix d d ℤ) (B : Matrix d₂ d₂ ℤ) (C : HermitianMat d ℤ) (D : HermitianMat d₂ ℤ) :
  conj (A ⊗ₖ B) (C ⊗ₖ D) = conj A C ⊗ₖ conj B D
```

**lemma rpow\_kron**[\[source\]](#)

```
lemma rpow_kron
  {A : HermitianMat d ℂ} {B : HermitianMat d₂ ℂ} (r : ℝ) (hA : 0 ≤ A) (hB : 0 ≤ B) :
  (A ⊗ₖ B) ^ r = (A ^ r) ⊗ₖ (B ^ r)
```

**lemma continuous0n\_rpow\_uniform**[\[source\]](#)

```
lemma continuous0n_rpow_uniform {K : Set ℝ} (hK : IsCompact K) :
  Continuous0n (fun r : ℝ ↦ Uniform0nFun.ofFun {K} (fun t : ℝ ↦ t ^ r)) (Set.Ioi 0)
```

**theorem continuous0n\_rpow\_joint\_nonneg\_pos**[\[source\]](#)

*Joint continuity of matrix rpow for Hermitian matrices with positive exponent*

```
@[fun_prop]
theorem continuous0n_rpow_joint_nonneg_pos
  {X : Type*} [TopologicalSpace X]
  {A : X → HermitianMat d ℂ} {p : X → ℝ} {S : Set X}
  (hA : Continuous0n A S) (hp : Continuous0n p S)
  (hp_pos : ∀ x ∈ S, 0 < p x) :
  Continuous0n (fun x ↦ A x ^ p x) S
```

**theorem cfc\_sq\_rpow\_eq\_cfc\_rpow**[\[source\]](#)

```
theorem cfc_sq_rpow_eq_cfc_rpow
  (A : HermitianMat d ℂ) (hA : 0 ≤ A) (p : ℝ) (hp : 0 < p) :
  (A ^ 2).cfc (fun x => x ^ (p/2)) = A.cfc (fun x => x ^ p)
```

**lemma trace\_rpow\_eq\_sum**[\[source\]](#)

$Tr[A^{\widehat{p}}] = \sum_i \lambda_i \widehat{p}$  for a Hermitian matrix  $A$ .

```
lemma trace_rpow_eq_sum (A : HermitianMat d ℂ) (p : ℝ) :
  (A ^ p).trace = ∑ i, (A.H.eigenvalues i) ^ p
```

**def rpowApprox**[\[source\]](#)

*Finite integral approximation for the rpow monotonicity proof. Same integrand as logApprox but with weight  $t^q$ .*

```
noncomputable def rpowApprox (A : HermitianMat d ℂ) (q T : ℝ) : HermitianMat d ℂ
```

**theorem rpowApprox\_mono**[\[source\]](#)

```
theorem rpowApprox_mono {A B : HermitianMat d ℂ} (hA : A.mat.PosDef) (hB : B.mat.PosDef)
(hAB : A ≤ B) (hq : 0 ≤ q) (T : ℝ) (hT : 0 < T) :
rpowApprox A q T ≤ rpowApprox B q T
```

**def scalarRpowApprox**[\[source\]](#)

*The scalar function underlying rpowApprox via the CFC.*

```
noncomputable def scalarRpowApprox (q T x : ℝ) : ℝ
```

**theorem rpowApprox\_eq\_cfc\_scalar**[\[source\]](#)

```
theorem rpowApprox_eq_cfc_scalar (A : HermitianMat d ℂ) (hA : A.mat.PosDef) (q T : ℝ)
(hq : 0 ≤ q) (hT : 0 < T) :
rpowApprox A q T = A.cfc (scalarRpowApprox q T)
```

**def rpowConst**[\[source\]](#)

*The positive constant arising from the resolvent integral. Equal to  $\int u$  in Set.Ioi  $\theta$ ,  $u^{(q-1)/(1+u)} = \pi / \sin(\pi q)$ , but we only need its positivity.*

```
noncomputable def rpowConst (q : ℝ) : ℝ
```

**lemma rpowConst\_integrable0n**[\[source\]](#)

*The integrand  $u^{(q-1)/(1+u)}$  is integrable on  $(\theta, \infty)$  for  $\theta < q < 1$ .*

```
lemma rpowConst_integrable0n (hq : 0 < q) (hq1 : q < 1) :
Integrable0n (fun u : ℝ => u ^ (q - 1) / (1 + u)) (Set.Ioi θ)
```

**lemma rpowConst\_pos**[\[source\]](#)

```
lemma rpowConst_pos (hq : 0 < q) (hq1 : q < 1) : 0 < rpowConst q
```

**lemma scalarRpowApprox\_tendsto**[\[source\]](#)

*The scalar rpow approximation converges pointwise.  $\text{scalarRpowApprox } q T x \rightarrow \text{rpowConst } q * (x^q - 1)$  as  $T \rightarrow \infty$ .*

```
lemma scalarRpowApprox_tendsto {x : ℝ} (hx : 0 < x) (hq : 0 < q) (hq1 : q < 1) :
Filter.Tendsto (fun T => scalarRpowApprox q T x) atTop (nhds (rpowConst q * (x ^ q - 1)))
```

**lemma tendsto\_rpowApprox**[\[source\]](#)

The matrix rpow approximation converges:  $\text{rpowApprox } A \ q \ T \rightarrow \text{rpowConst } q \cdot (A^q - 1)$ .

```
lemma tendsto_rpowApprox (hA : A.mat.PosDef) (hq : 0 < q) (hq1 : q < 1) :
  Tendsto (rpowApprox A q) atTop (nhds (rpowConst q • (A ^ q - 1)))
```

**theorem rpow\_le\_rpow\_of\_posDef**[\[source\]](#)

```
theorem rpow_le_rpow_of_posDef (hA : A.mat.PosDef) (hAB : A ≤ B)
  (hq : 0 < q) (hq1 : q ≤ 1) : A ^ q ≤ B ^ q
```

**theorem rpow\_le\_rpow\_of\_le**[\[source\]](#)

The *Löwner—Heinz theorem*: for matrices  $A$  and  $B$ , if  $0 \leq A \leq B$  and  $0 < q \leq 1$ , then  $A^q \leq B^q$ . That is, real roots are operator monotone.

```
theorem rpow_le_rpow_of_le (hA : 0 ≤ A) (hAB : A ≤ B)
  (hq : 0 < q) (hq1 : q ≤ 1) : A ^ q ≤ B ^ q
```

**lemma lieb\_thirring\_le\_one**[\[source\]](#)

An inequality of Lieb-Thirring type. For  $0 < r \leq 1$ :  $\text{Tr}[B^r A^r B^r] \leq \text{Tr}[(B A B)^r]$ .

```
lemma lieb_thirring_le_one
  {A B : HermitianMat d ℂ} (hA : 0 ≤ A) (hB : 0 ≤ B)
  {r : ℝ} (hq0 : 0 < r) (hq1 : q ≤ 1) :
  ((A ^ r).conj (B ^ r).mat).trace ≤ ((A.conj B.mat) ^ r).trace
```

**lemma trace\_rpow\_add\_le**[\[source\]](#)

```
lemma trace_rpow_add_le
  {A B : HermitianMat d ℂ} (hA : 0 ≤ A) (hB : 0 ≤ B)
  (p : ℝ) (hp : 0 < p) (hp1 : p ≤ 1) :
  ((A + B) ^ p).trace ≤ (A ^ p).trace + (B ^ p).trace
```

**2.65 QuantumInfo.ForMathlib.HermitianMat.Schatten**

[QuantumInfo/ForMathlib/HermitianMat/Schatten.lean](#) on GitHub.

**def schattenNorm**[\[source\]](#)

The Schatten  $p$ -norm of a matrix  $A$  is  $(\text{Tr}[(A^* A)^{p/2}])^{1/p}$ .

```
noncomputable def schattenNorm (A : Matrix d d ℂ) (p : ℝ) : ℝ
```

**theorem schattenNorm\_hermitian\_pow**[\[source\]](#)

```
theorem schattenNorm_hermitian_pow {A : HermitianMat d ℂ} (hA : 0 ≤ A) {p : ℝ} (hp : 0 < p) :
  schattenNorm A.mat p = (A ^ p).trace ^ (1/p)
```

**lemma SchattenNorm\_nonneg**[\[source\]](#)

```
lemma SchattenNorm_nonneg (A : Matrix d d ℂ) (p : ℝ) :
  0 ≤ SchattenNorm A p
```

**lemma SchattenNorm\_pow\_eq**[\[source\]](#)

```
lemma SchattenNorm_pow_eq
  (A : HermitianMat d ℂ) (hA : 0 ≤ A) (p k : ℝ) (hp : 0 < p) (hk : 0 < k) :
  SchattenNorm (A ^ k).mat p = (SchattenNorm A.mat (k * p)) ^ k
```

**lemma trace\_eq\_SchattenNorm\_rpow**[\[source\]](#)

```
lemma trace_eq_SchattenNorm_rpow
  (A : HermitianMat d ℂ) (hA : 0 ≤ A) (r : ℝ) (hr : 0 < r) :
  (A ^ r).trace = (SchattenNorm A.mat r) ^ r
```

**lemma SchattenNorm\_trace\_as\_eigenvalue\_sum**[\[source\]](#)

```
lemma SchattenNorm_trace_as_eigenvalue_sum (A : Matrix d d ℂ) (p : ℝ) :
  RCLike.re ((Matrix.isHermitian_mul_conjTranspose_self A.conjTranspose).cfc (· ^ (p/2))).
  trace =
  ∑ i : d, ((Matrix.isHermitian_mul_conjTranspose_self A.conjTranspose).eigenvalues i) ^ (p
  /2)
```

**lemma SchattenNorm\_rpow\_eq\_sum\_singularValues**[\[source\]](#)

```
lemma SchattenNorm_rpow_eq_sum_singularValues (A : Matrix d d ℂ) {p : ℝ} (hp : 0 < p) :
  SchattenNorm A p ^ p = ∑ i : d, singularValues A i ^ p
```

**lemma SchattenNorm\_eq\_sum\_singularValues\_rpow**[\[source\]](#)

```
lemma SchattenNorm_eq_sum_singularValues_rpow (A : Matrix d d ℂ) {p : ℝ} (hp : 0 < p) :
  SchattenNorm A p = (∑ i : d, singularValues A i ^ p) ^ (1/p)
```

**lemma SchattenNorm\_rpow\_eq\_sum\_sorted**[\[source\]](#)

$\|A\|_p^p$  equals the same sum over sorted singular values.

```
lemma SchattenNorm_rpow_eq_sum_sorted (A : Matrix d d ℂ) {p : ℝ} (hp : 0 < p) :
  SchattenNorm A p ^ p =
  ∑ i : Fin (Fintype.card d), singularValuesSorted A i ^ p
```

**lemma trace\_young**[\[source\]](#)

Scalar trace Young inequality for PSD matrices:  $\langle\langle A, B \rangle\rangle \leq \text{Tr}[A^p]/p + \text{Tr}[B^q]/q$  for PSD  $A, B$  and conjugate  $p, q > 1$ .

```
lemma HermitianMat.trace_young
  (A B : HermitianMat d ℂ) (hA : 0 ≤ A) (hB : 0 ≤ B)
  (p q : ℝ) (hp : 1 < p) (hpq : 1/p + 1/q = 1) :
  ⟨⟨A, B⟩⟩_ℝ ≤ (A ^ p).trace / p + (B ^ q).trace / q
```

**lemma conjTranspose\_half\_mul\_eq\_conj**[\[source\]](#)

*For PSD  $A$  and Hermitian  $B$ , the product  $C = A^{\{1/2\}} * B$  satisfies  $C^* C = (A.conj B.mat).mat = B * A * B$ .*

**lemma conjTranspose\_half\_mul\_eq\_conj**

```
{A B : HermitianMat d ℂ} (hA : 0 ≤ A) :
((A ^ (1/2 : ℝ)).mat * B.mat).conjTranspose * ((A ^ (1/2 : ℝ)).mat * B.mat)
= (A.conj B.mat).mat
```

**lemma schattenNorm\_half\_mul\_rpow\_eq\_trace\_conj**[\[source\]](#)**lemma schattenNorm\_half\_mul\_rpow\_eq\_trace\_conj**

```
{A B : HermitianMat d ℂ} (hA : 0 ≤ A)
{α : ℝ} (hα : 0 < α) :
(schattenNorm ((A ^ (1/2 : ℝ)).mat * B.mat) (2 * α)) ^ (2 * α) =
((A.conj B.mat) ^ α).trace
```

**lemma schattenNorm\_mul\_le**[\[source\]](#)**lemma schattenNorm\_mul\_le** (A B : Matrix d d ℂ) {r p q : ℝ}

```
(hr : 0 < r) (hp : 0 < p) (hq : 0 < q) (hpqr : 1 / r = 1 / p + 1 / q) :
schattenNorm (A * B) r ≤ schattenNorm A p * schattenNorm B q
```

**lemma trace\_rpow\_conj\_le**[\[source\]](#)**lemma HermitianMat.trace\_rpow\_conj\_le**

```
{A B : HermitianMat d ℂ} (hA : 0 ≤ A) (hB : 0 ≤ B)
{α p q : ℝ} (hα : 0 < α) (hp : 0 < p) (hq : 0 < q)
(hpq : 1 / (2 * α) = 1 / p + 1 / q) :
((A.conj B.mat) ^ α).trace ≤
(((A ^ (p / 2)).trace) ^ (1 / p) * ((B ^ q).trace) ^ (1 / q)) ^ (2 * α)
```

**2.66 QuantumInfo.ForMathlib.HermitianMat.Sqrt**

[QuantumInfo/ForMathlib/HermitianMat/Sqrt.lean](#) on GitHub.

**def sqrt**[\[source\]](#)

*The square root of a Hermitian matrix. Negative eigenvalues are mapped to zero.*

```
noncomputable def sqrt (A : HermitianMat d ℤ) : HermitianMat d ℤ
```

**theorem sqrt\_sq\_eq\_proj**[\[source\]](#)**theorem sqrt\_sq\_eq\_proj** (A : HermitianMat d ℤ) :

```
A.sqrt.mat * A.sqrt.mat = A+
```

**theorem sqrt\_sq**[\[source\]](#)**theorem sqrt\_sq** (hA : 0 ≤ A) :

```
A.sqrt.mat * A.sqrt.mat = A
```

**theorem commute\_sqrt\_left**[\[source\]](#)

```
@[aesop unsafe apply 50% (rule_sets := [Commutes])]
theorem commute_sqrt_left (hAB : Commute A.mat B.mat) :
  Commute A.sqrt.mat B.mat
```

**theorem commute\_sqrt\_right**[\[source\]](#)

```
@[aesop unsafe apply 50% (rule_sets := [Commutes])]
theorem commute_sqrt_right (hAB : Commute A.mat B.mat) :
  Commute A.mat B.sqrt.mat
```

**lemma sqrt\_inv\_mul\_self\_mul\_sqrt\_inv\_eq\_one**[\[source\]](#)

*For a positive definite matrix  $A$ ,  $A^{-1/2} * A * A^{-1/2} = I$ .*

```
lemma sqrt_inv_mul_self_mul_sqrt_inv_eq_one {A : HermitianMat d ℤ} (hA : A.mat.PosDef) :
  A⁻¹.sqrt.mat * A.mat * A⁻¹.sqrt.mat = 1
```

**theorem sqrt\_nonneg**[\[source\]](#)

```
theorem sqrt_nonneg (A : HermitianMat d ℤ) : 0 ≤ A.sqrt
```

**theorem sqrt\_pos**[\[source\]](#)

```
theorem sqrt_pos (h : 0 < A) : 0 < A.sqrt
```

**theorem sqrt\_posDef**[\[source\]](#)

```
theorem sqrt_posDef {A : HermitianMat d ℤ} (hA : A.mat.PosDef) :
  A.sqrt.mat.PosDef
```

**def evalHermitianMatSqrt**[\[source\]](#)

*Positivity extension for HermitianMat.sqrt*

```
@[positivity HermitianMat.sqrt _]
def evalHermitianMatSqrt : PositivityExt where eval {_u _α} _zα _pα e
```

**2.67 QuantumInfo.ForMathlib.HermitianMat.Trace**

[QuantumInfo/ForMathlib/HermitianMat/Trace.lean](#) on GitHub.

**def trace**[\[source\]](#)

*The trace of the matrix. This requires a `IsMaximalSelfAdjoint R α` instance, and then maps from `HermitianMat n α` to `R`. This means that the trace of (say) a `HermitianMat n ℤ` gives values in `ℤ`, but that the trace of a `HermitianMat n ℂ` gives values in `ℝ`. The fact that traces are "automatically" real reduces coercions down the line.*

```
def trace (A : HermitianMat n α) : R
```

**theorem trace\_eq\_trace**[\[source\]](#)*HermitianMat.trace reduces to Matrix.trace in the algebra.*

```
theorem trace_eq_trace (A : HermitianMat n α) : algebraMap R α A.trace = Matrix.trace A.mat
```

**theorem trace\_smul**[\[source\]](#)

```
@[simp]
```

```
theorem trace_smul (A : HermitianMat n α) (r : R) : (r • A).trace = r * A.trace
```

**theorem trace\_zero**[\[source\]](#)

```
@[simp]
```

```
theorem trace_zero : (0 : HermitianMat n α).trace = 0
```

**theorem trace\_add**[\[source\]](#)

```
@[simp]
```

```
theorem trace_add (A B : HermitianMat n α) : (A + B).trace = A.trace + B.trace
```

**theorem trace\_neg**[\[source\]](#)

```
@[simp]
```

```
theorem trace_neg (A : HermitianMat n α) : (-A).trace = -A.trace
```

**theorem trace\_sub**[\[source\]](#)

```
@[simp]
```

```
theorem trace_sub (A B : HermitianMat n α) : (A - B).trace = A.trace - B.trace
```

**theorem isSelfAdjoint\_trace**[\[source\]](#)

```
theorem _root_.Matrix.IsHermitian.isSelfAdjoint_trace {A : Matrix n n α} (hA : A.IsHermitian) :  
  IsSelfAdjoint A.trace
```

**theorem trace\_kronecker**[\[source\]](#)

```
@[simp]
```

```
theorem trace_kronecker [FaithfulSMul R α] : (A ⊗k B).trace = A.trace * B.trace
```

**theorem trace\_eq\_trace\_trivial**[\[source\]](#)*HermitianMat.trace reduces to Matrix.trace when the elements are a TrivialStar.*

```
@[simp]
```

```
theorem trace_eq_trace_trivial (A : HermitianMat n ℝ) : A.trace = A.mat.trace
```

**theorem trace\_eq\_re\_trace**[\[source\]](#)

```
theorem trace_eq_re_trace (A : HermitianMat n ℚ) : A.trace = RCLike.re A.mat.trace
```



**theorem trace\_one**[\[source\]](#)

```
@[simp]
theorem trace_one [DecidableEq n] : (1 : HermitianMat n  $\mathbb{K}$ ).trace = Fintype.card n
```

**theorem trace\_eq\_trace\_rc**[\[source\]](#)

*HermitianMat.trace reduces to Matrix.trace when the elements are RCLike.*

```
@[simp]
theorem trace_eq_trace_rc (A : HermitianMat n  $\mathbb{K}$ ) : A.trace = A.mat.trace
```

**theorem trace\_diagonal**[\[source\]](#)

```
theorem trace_diagonal {T : Type*} [Fintype T] [DecidableEq T] (f : T →  $\mathbb{R}$ ) :
  (diagonal  $\mathbb{K}$  f).trace =  $\sum i, f i$ 
```

**theorem sum\_eigenvalues\_eq\_trace**[\[source\]](#)

```
theorem sum_eigenvalues_eq_trace [DecidableEq n] (A : HermitianMat n  $\mathbb{K}$ ) :
   $\sum i, A.H.eigenvalues i = A.trace$ 
```

**theorem trace\_eq\_zero\_iff**[\[source\]](#)

```
theorem trace_eq_zero_iff (A : HermitianMat n  $\mathbb{K}$ ) : A.trace = 0 ↔ A.mat.trace = 0
```

**theorem trace\_eq\_one\_iff**[\[source\]](#)

```
theorem trace_eq_one_iff (A : HermitianMat n  $\mathbb{K}$ ) : A.trace = 1 ↔ A.mat.trace = 1
```

**theorem trace\_reindex**[\[source\]](#)

```
@[simp]
theorem trace_reindex (A : HermitianMat n  $\mathbb{C}$ ) (e : n ≈ m) :
  (A.reindex e).trace = A.trace
```

**def traceLeft**[\[source\]](#)

```
def traceLeft (A : HermitianMat (m × n)  $\alpha$ ) : HermitianMat n  $\alpha$ 
```

**def traceRight**[\[source\]](#)

```
def traceRight (A : HermitianMat (m × n)  $\alpha$ ) : HermitianMat m  $\alpha$ 
```

**theorem traceLeft\_mat**[\[source\]](#)

```
@[simp]
theorem traceLeft_mat : A.traceLeft.mat = A.mat.traceLeft
```

**theorem traceLeft\_add**[\[source\]](#)

```
@[simp]
theorem traceLeft_add : (A + B).traceLeft = A.traceLeft + B.traceLeft
```

**theorem traceLeft\_neg**[\[source\]](#)

```
@[simp]
theorem traceLeft_neg : (-A).traceLeft = -A.traceLeft
```

**theorem traceLeft\_sub**[\[source\]](#)

```
@[simp]
theorem traceLeft_sub : (A - B).traceLeft = A.traceLeft - B.traceLeft
```

**theorem traceRight\_mat**[\[source\]](#)

```
@[simp]
theorem traceRight_mat :
  (traceRight A).mat = A.mat.traceRight
```

**theorem traceRight\_add**[\[source\]](#)

```
@[simp]
theorem traceRight_add : (A + B).traceRight = A.traceRight + B.traceRight
```

**theorem traceRight\_neg**[\[source\]](#)

```
@[simp]
theorem traceRight_neg : (-A).traceRight = -A.traceRight
```

**theorem traceRight\_sub**[\[source\]](#)

```
@[simp]
theorem traceRight_sub : (A - B).traceRight = A.traceRight - B.traceRight
```

**theorem traceLeft\_smul**[\[source\]](#)

```
@[simp]
theorem traceLeft_smul (r : ℝ) : (r • A).traceLeft = r • A.traceLeft
```

**theorem traceRight\_smul**[\[source\]](#)

```
@[simp]
theorem traceRight_smul (r : ℝ) : (r • A).traceRight = r • A.traceRight
```

**theorem traceLeft\_trace**[\[source\]](#)

```
@[simp]
theorem traceLeft_trace : A.traceLeft.trace = A.trace
```

**theorem traceRight\_trace**[\[source\]](#)

```
@[simp]
theorem traceRight_trace : A.traceRight.trace = A.trace
```

**theorem traceLeft\_kron**[\[source\]](#)

```
@[simp]
theorem traceLeft_kron [Fintype m] : (A  $\otimes_k$  B).traceLeft = A.trace • B
```

**theorem traceRight\_kron**[\[source\]](#)

```
@[simp]
theorem traceRight_kron [Fintype n] : (A  $\otimes_k$  B).traceRight = B.trace • A
```

**2.68 QuantumInfo.ForMathlib.HermitianMat.Unitary**

[QuantumInfo/ForMathlib/HermitianMat/Unitary.lean](#) on GitHub.

**theorem neg\_unitary\_val**[\[source\]](#)

```
@[simp]
theorem neg_unitary_val (u : U[ $\alpha$ ]) : (-u).val = -u
```

**theorem star\_kron**[\[source\]](#)

```
@[simp]
theorem star_kron (a : Matrix  $\alpha$   $\alpha$   $\mathbb{C}$ ) (b : Matrix  $\beta$   $\beta$   $\mathbb{C}$ ) : star (a  $\otimes_k$  b) = (star a)  $\otimes_k$  (star b)
```

**theorem kron\_unitary**[\[source\]](#)

```
theorem kron_unitary (a : U[ $\alpha$ ]) (b : U[ $\beta$ ]) : a.val  $\otimes_k$  b.val  $\in$  U[ $\alpha \times \beta$ ]
```

**def unitary\_kron**[\[source\]](#)

```
def unitary_kron (a : U[ $\alpha$ ]) (b : U[ $\beta$ ]) : U[ $\alpha \times \beta$ ]
```

**theorem unitary\_kron\_apply**[\[source\]](#)

```
@[simp]
theorem unitary_kron_apply (a : U[ $\alpha$ ]) (b : U[ $\beta$ ]) (i1 i2 :  $\alpha$ ) (j1 j2 :  $\beta$ ) :
  (a  $\otimes_u$  b) (i1, j1) (i2, j2) = (a i1 i2) * (b j1 j2)
```

**theorem unitary\_kron\_one\_one**[\[source\]](#)

```
@[simp]
theorem unitary_kron_one_one : (1 : U[ $\alpha$ ])  $\otimes_u$  (1 : U[ $\beta$ ]) = (1 : U[ $\alpha \times \beta$ ])
```

**lemma unitary\_row\_sum\_norm\_sq**[\[source\]](#)

*For a unitary matrix  $C$ , the row sums of  $\|C\|_2^2$  equal 1.*

```
lemma unitary_row_sum_norm_sq (C : Matrix d d ℂ) (hC : C * C.conjTranspose = 1) (i : d) :
  ∑ j, ‖C i j‖ ^ 2 = 1
```

**lemma unitary\_col\_sum\_norm\_sq**[\[source\]](#)

*For a unitary matrix  $C$ , the column sums of  $\|C\|_2^2$  equal 1.*

```
lemma unitary_col_sum_norm_sq (C : Matrix d d ℂ) (hC : C.conjTranspose * C = 1) (j : d) :
  ∑ i, ‖C i j‖ ^ 2 = 1
```

**2.69 QuantumInfo.ForMathlib.IsMaximalSelfAdjoint**

[QuantumInfo/ForMathlib/IsMaximalSelfAdjoint.lean](#) on GitHub.

**class IsMaximalSelfAdjoint**[\[source\]](#)

```
class IsMaximalSelfAdjoint (R : outParam Type*) (α : Type*) [Star α] [Star R] [CommSemiring R]
  [Semiring α] [TrivialStar R] [Algebra R α] where
  selfadjMap : α →+ R
  selfadj_smul : ∀ (r : R) (a : α), selfadjMap (r • a) = r * (selfadjMap a)
  selfadj_algebra : ∀ {a : α}, IsSelfAdjoint a → algebraMap _ _ (selfadjMap a) = a
```

**instance instTrivialStar\_IsMaximalSelfAdjoint**[\[source\]](#)

*Every TrivialStar CommSemiring is its own maximal self adjoints.*

```
instance instTrivialStar_IsMaximalSelfAdjoint {R} [Star R] [TrivialStar R] [CommSemiring R] :
  IsMaximalSelfAdjoint R where
```

**instance instRCLike\_IsMaximalSelfAdjoint**[\[source\]](#)

$\mathbb{R}$  is the maximal self adjoint elements over RCLike

```
instance instRCLike_IsMaximalSelfAdjoint {α} [RCLike α] : IsMaximalSelfAdjoint ℝ α where
```

**theorem trivial\_selfadjMap**[\[source\]](#)

```
@[simp]
theorem trivial_selfadjMap {R} [Star R] [TrivialStar R] [CommSemiring R] : (selfadjMap : R →+ R)
  = .id R
```

**theorem RCLike\_selfadjMap**[\[source\]](#)

```
@[simp]
theorem RCLike_selfadjMap {α} [RCLike α] : (selfadjMap : α →+ ℝ) = RCLike.re
```

**2.70 QuantumInfo.ForMathlib.Isometry**

[QuantumInfo/ForMathlib/Isometry.lean](#) on GitHub.

**def Isometry**[\[source\]](#)

*An isometry is a matrix  $A$  such that  $AA^H = 1$ . Compare with a unitary, which requires  $AA^H = A^HA = 1$ . It is common to claim that, in a finite-dimensional vector space, a two-sided isometry ( $A.\text{Isometry} \wedge A^H.\text{Isometry}$ ) must be square and therefore unitary; this does not work out so well here, since a `Matrix m n R` can be a two-sided isometry, but cannot be a unitary since the rows and columns are indexed by different labels.*

```
def Matrix.Isometry (A : Matrix d d₂ R) : Prop
```

**theorem submatrix\_one\_isometry**[\[source\]](#)

```
theorem Matrix.submatrix_one_isometry {e : d₂ → d} {f : d₃ → d} (he : e.Bijective) (hf : f.Injective) :
  (submatrix (α := R) 1 e f).Isometry
```

**theorem submatrix\_one\_id\_left\_isometry**[\[source\]](#)

```
theorem Matrix.submatrix_one_id_left_isometry {e : d₂ → d} (he : e.Bijective) :
  (submatrix (1 : Matrix d d R) e id).Isometry
```

**theorem submatrix\_one\_id\_right\_isometry**[\[source\]](#)

```
theorem Matrix.submatrix_one_id_right_isometry {e : d₂ → d} (he : e.Injective) :
  (submatrix (1 : Matrix d d R) id e).Isometry
```

**theorem mem\_unitaryGroup\_iff\_isometry**[\[source\]](#)

```
theorem Matrix.mem_unitaryGroup_iff_isometry (A : Matrix d d R) :
  A ∈ unitaryGroup d R ↔ A.Isometry ∧ A^H.Isometry
```

**theorem permMatrix\_mem\_unitaryGroup**[\[source\]](#)

```
theorem Equiv.Perm.permMatrix_mem_unitaryGroup (e : Perm d) :
  e.permMatrix R ∈ Matrix.unitaryGroup d R
```

**theorem reindex\_one\_isometry**[\[source\]](#)

```
theorem Matrix.reindex_one_isometry (e : d ≈ d₂) (f : d ≈ d₃) :
  (reindex (α := R) e f 1).Isometry
```

**theorem reindex\_one\_mem\_unitaryGroup**[\[source\]](#)

```
theorem Matrix.reindex_one_mem_unitaryGroup (e : d ≈ d₂) :
  reindex (α := R) e e 1 ∈ unitaryGroup d₂ R
```

**theorem reindex\_eq\_conj**[\[source\]](#)

```
theorem Matrix.reindex_eq_conj (A : Matrix d d R) (e : d ≈ d₂) : reindex e e A =
  (reindex (α := R) e (.refl d) 1) * A * (reindex (α := R) (.refl d) e 1)
```

**theorem reindex\_eq\_conj\_unitaryGroup'**[\[source\]](#)

```

theorem Matrix.reindex_eq_conj_unitaryGroup' (A : Matrix d d R) (e : Equiv.Perm d) : reindex e
  e A =
  (⟦_, e⁻¹.permMatrix_mem_unitaryGroup⟧ : unitaryGroup d R) * A * (⟦_, e.
    permMatrix_mem_unitaryGroup⟧ : unitaryGroup d R)

```

**theorem eigenvalue\_ext**[\[source\]](#)

```

theorem Matrix.IsHermitian.eigenvalue_ext (hA : A.IsHermitian)
  (h : ∀ (v : d → ℤ) (lam : ℤ), A *v v = lam • v → B *v v = lam • v) :
  A = B

```

**theorem cfc\_eq\_any\_isometry**[\[source\]](#)

*Generalizes Matrix.IsHermitian.cfc.eq\_1, which gives a definition for the matrix CFC in terms of Matrix.IsHermitian.eigenvalues and Matrix.IsHermitian.eigenvectorUnitary, to show that the CFC works similarly for \_\_any\_\_ diagonalization by a two-sided isometry.*

```

theorem Matrix.IsHermitian.cfc_eq_any_isometry {n m ℤ : Type*} [RCLike ℤ]
  [Fintype n] [DecidableEq n] [Fintype m] [DecidableEq m]
  {A : Matrix n n ℤ} (hA : A.IsHermitian) {U : Matrix n m ℤ}
  (hU₁ : U * UH = 1) (hU₂ : UH * U = 1) {D : m → ℝ}
  (hUD : A = (U * diagonal (RCLike.ofReal • D) : Matrix _ _ _) * UH) (f : ℝ → ℝ) :
  hA.cfc f = (U * diagonal (RCLike.ofReal • f • D) : Matrix _ _ _) * UH

```

**theorem cfc\_eq\_any\_unitary**[\[source\]](#)

*Generalizes Matrix.IsHermitian.cfc.eq\_1, which gives a definition for the matrix CFC in terms of Matrix.IsHermitian.eigenvalues and Matrix.IsHermitian.eigenvectorUnitary, to show that the CFC works similarly for \_\_any\_\_ diagonalization.*

```

theorem Matrix.IsHermitian.cfc_eq_any_unitary {n ℤ : Type*} [RCLike ℤ] [Fintype n] [DecidableEq n]
  {A : Matrix n n ℤ} (hA : A.IsHermitian) {U : unitaryGroup n ℤ} {D : n → ℝ}
  (hUD : A = U.val * diagonal (RCLike.ofReal • D) * star U.val) (f : ℝ → ℝ) :
  hA.cfc f = U.val * diagonal (RCLike.ofReal • f • D) * star U.val

```

**theorem cfc\_conj\_isometry'**[\[source\]](#)

```

private theorem Matrix.cfc_conj_isometry' (hA : A.IsHermitian) (f : ℝ → ℝ) {u : Matrix d₂ d ℤ}
  (hu₁ : u.Isometry) (hu₂ : uH.Isometry) :
  cfc f (u * A * uH) = u * (cfc f A) * uH

```

**theorem cfc\_conj\_isometry**[\[source\]](#)

```

theorem Matrix.cfc_conj_isometry (f : ℝ → ℝ) {u : Matrix d₂ d ℤ}
  (hu₁ : u.Isometry) (hu₂ : uH.Isometry) :
  cfc f (u * A * uH) = u * (cfc f A) * uH

```

**theorem cfc\_conj\_unitary**[\[source\]](#)

```
theorem Matrix.cfc_conj_unitary (f : ℝ → ℝ) (u : unitaryGroup d ℚ) :
  cfc f (u * A * u-1) = u * (cfc f A) * u-1
```

**theorem cfc\_conj\_unitary'**

[\[source\]](#)

```
theorem Matrix.cfc_conj_unitary' (f : ℝ → ℝ) (u : unitaryGroup d ℚ) :
  cfc f (uH * A * u.val) = uH * (cfc f A) * u.val
```

**theorem cfc\_reindex**

[\[source\]](#)

```
theorem Matrix.cfc_reindex (f : ℝ → ℝ) (e : d ≈ d2) :
  cfc f (reindex e e A) = reindex e e (cfc f A)
```

**theorem commute\_euclideanLin**

[\[source\]](#)

```
theorem Matrix.commute_euclideanLin (hAB : Commute A B) :
  Commute A.toEuclideanLin B.toEuclideanLin
```

**theorem orthogonalFamily\_eigenspace\_inf\_eigenspace'**

[\[source\]](#)

*Similar to LinearMap.IsSymmetric.orthogonalFamily\_eigenspace\_inf\_eigenspace, but here the direct sum is indexed by only the pairs of eigenvalues, as opposed to all pairs of ℚ values, giving a finite decomposition.*

```
theorem LinearMap.IsSymmetric.orthogonalFamily_eigenspace_inf_eigenspace' {ℚ E : Type*} [RCLike ℚ]
  [NormedAddCommGroup E] [InnerProductSpace ℚ E] {A B : E →ℓ [ℚ] E}
  (hA : A.IsSymmetric) (hB : B.IsSymmetric) :
  OrthogonalFamily ℚ (fun (μ12 : Eigenvalues A × Eigenvalues B) ↦
    1(eigenspace A μ12.1 ⊔ eigenspace B μ12.2)) fun μ12 ↦
    (eigenspace A μ12.1 ⊔ eigenspace B μ12.2).subtypeℓ 1
```

**theorem iSup\_mono\_bot**

[\[source\]](#)

*Variant of iSup\_mono' that allows for an easier handling of bottom elements.*

```
theorem iSup_mono_bot {α : Type*} {ι ι' : Sort*} [CompleteLattice α]
  {f : ι → α} {g : ι' → α} (h : ∀ (i : ι), f i = ⊥ ∨ ∃ i', f i ≤ g i') :
  iSup f ≤ iSup g
```

**def isSymmetric\_directSumDecomposition**

[\[source\]](#)

```
noncomputable def Commute.isSymmetric_directSumDecomposition {ℚ E : Type*} [RCLike ℚ]
  [NormedAddCommGroup E] [InnerProductSpace ℚ E] {A B : E →ℓ [ℚ] E} [FiniteDimensional ℚ E]
  (hA : A.IsSymmetric) (hB : B.IsSymmetric) (hAB : Commute A B) :
  DirectSum.Decomposition fun (μ12 : Eigenvalues A × Eigenvalues B) ↦
    (eigenspace A μ12.1 ⊔ eigenspace B μ12.2)
```

**theorem directSum\_isInternal\_of\_commute'**

[\[source\]](#)

*Similar to `LinearMap.IsSymmetric.directSum_isInternal_of_commute`, but here the direct sum is indexed by only the pairs of eigenvalues, as opposed to all pairs of  $\mathbb{k}$  values, giving a finite decomposition.*

```
theorem LinearMap.IsSymmetric.directSum_isInternal_of_commute' { $\mathbb{k}$  E : Type*} [RCLike  $\mathbb{k}$ ]
[NormedAddCommGroup E] [InnerProductSpace  $\mathbb{k}$  E] {A B : E  $\rightarrow_l$  [ $\mathbb{k}$ ] E} [FiniteDimensional  $\mathbb{k}$  E]
(hA : A.IsSymmetric) (hB : B.IsSymmetric) (hAB : Commute A B) :
  DirectSum.IsInternal fun ( $\mu_{12}$  : Eigenvalues A  $\times$  Eigenvalues B)  $\mapsto$ 
    eigenspace A  $\mu_{12}.1 \sqcap$  eigenspace B  $\mu_{12}.2$ 
```

### def LinearMap.sharedEigenbasis

[\[source\]](#)

```
noncomputable def LinearMap.sharedEigenbasis {A B : EuclideanSpace  $\mathbb{k}$  d  $\rightarrow_l$  [ $\mathbb{k}$ ] EuclideanSpace  $\mathbb{k}$  d}
}
(hA : A.IsSymmetric) (hB : B.IsSymmetric) (hAB : Commute A B) :
  OrthonormalBasis d  $\mathbb{k}$  (EuclideanSpace  $\mathbb{k}$  d)
```

### def sharedEigenvaluesA

[\[source\]](#)

```
noncomputable def LinearMap.sharedEigenvaluesA {A B : EuclideanSpace  $\mathbb{k}$  d  $\rightarrow_l$  [ $\mathbb{k}$ ] EuclideanSpace
 $\mathbb{k}$  d}
(hA : A.IsSymmetric) (hB : B.IsSymmetric) (hAB : Commute A B) : d  $\rightarrow$   $\mathbb{R}$ 
```

### def sharedEigenvaluesB

[\[source\]](#)

```
noncomputable def LinearMap.sharedEigenvaluesB {A B : EuclideanSpace  $\mathbb{k}$  d  $\rightarrow_l$  [ $\mathbb{k}$ ] EuclideanSpace
 $\mathbb{k}$  d}
(hA : A.IsSymmetric) (hB : B.IsSymmetric) (hAB : Commute A B) : d  $\rightarrow$   $\mathbb{R}$ 
```

### theorem mem\_eigenspace\_inf\_of\_sharedEigenbasis

[\[source\]](#)

```
theorem LinearMap.mem_eigenspace_inf_of_sharedEigenbasis {A B : EuclideanSpace  $\mathbb{k}$  d  $\rightarrow_l$  [ $\mathbb{k}$ ]
EuclideanSpace  $\mathbb{k}$  d}
(hA : A.IsSymmetric) (hB : B.IsSymmetric) (hAB : Commute A B) (i : d) :
 $\exists$  ( $\mu$  : Module.End.Eigenvalues A) ( $\nu$  : Module.End.Eigenvalues B),
  LinearMap.sharedEigenbasis hA hB hAB i  $\in$  Module.End.eigenspace A  $\mu \sqcap$  Module.End.
  eigenspace B  $\nu$ 
```

### theorem apply\_A\_sharedEigenbasis

[\[source\]](#)

```
theorem LinearMap.apply_A_sharedEigenbasis {A B : EuclideanSpace  $\mathbb{k}$  d  $\rightarrow_l$  [ $\mathbb{k}$ ] EuclideanSpace  $\mathbb{k}$  d}
(hA : A.IsSymmetric) (hB : B.IsSymmetric) (hAB : Commute A B) (i : d) :
  A (sharedEigenbasis hA hB hAB i) = (sharedEigenvaluesA hA hB hAB i :  $\mathbb{k}$ ) • (sharedEigenbasis
  hA hB hAB i)
```

### theorem apply\_B\_sharedEigenbasis

[\[source\]](#)

```
theorem LinearMap.apply_B_sharedEigenbasis {A B : EuclideanSpace  $\mathbb{k}$  d  $\rightarrow_l$  [ $\mathbb{k}$ ] EuclideanSpace  $\mathbb{k}$  d}
(hA : A.IsSymmetric) (hB : B.IsSymmetric) (hAB : Commute A B) (i : d) :
  B (sharedEigenbasis hA hB hAB i) = (sharedEigenvaluesB hA hB hAB i :  $\mathbb{k}$ ) • (sharedEigenbasis
  hA hB hAB i)
```



**def Matrix.sharedEigenbasis**[\[source\]](#)

```
noncomputable def Matrix.sharedEigenbasis
  (hA : A.IsHermitian) (hB : B.IsHermitian) (hAB : Commute A B) :
    OrthonormalBasis d  $\mathbb{K}$  (EuclideanSpace  $\mathbb{K}$  d)
```

**def sharedEigenvectorUnitary**[\[source\]](#)

```
noncomputable def Matrix.sharedEigenvectorUnitary (hA : A.IsHermitian) (hB : B.IsHermitian)
  (hAB : Commute A B) : Matrix.unitaryGroup d  $\mathbb{K}$ 
```

**theorem sharedEigenvectorUnitary\_mulVec**[\[source\]](#)

*Analogous to Matrix.IsHermitian.eigenvectorUnitary\_mulVec for the shared basis.*

```
theorem sharedEigenvectorUnitary_mulVec (j : d) : (sharedEigenvectorUnitary hA hB hAB) *v
  Pi.single j 1 = WithLp.ofLp (sharedEigenbasis hA hB hAB j)
```

**def sharedEigenvalueA**[\[source\]](#)

```
noncomputable def sharedEigenvalueA (j : d) :  $\mathbb{R}$ 
```

**def sharedEigenvalueB**[\[source\]](#)

```
noncomputable def sharedEigenvalueB (j : d) :  $\mathbb{R}$ 
```

**theorem mulVec\_sharedEigenbasisA**[\[source\]](#)

*Analogous to Matrix.IsHermitian.mulVec\_eigenvectorBasis for the shared basis.*

```
theorem mulVec_sharedEigenbasisA (j : d) :
  A *v (sharedEigenbasis hA hB hAB j) =
  (sharedEigenvalueA hA hB hAB) j • WithLp.ofLp (sharedEigenbasis hA hB hAB j)
```

**theorem mulVec\_sharedEigenbasisB**[\[source\]](#)

```
theorem mulVec_sharedEigenbasisB (j : d) :
  B *v (sharedEigenbasis hA hB hAB j) =
  (sharedEigenvalueB hA hB hAB) j • WithLp.ofLp (sharedEigenbasis hA hB hAB j)
```

**theorem star\_shared\_mul\_A\_mul\_IsDiag**[\[source\]](#)

```
theorem star_shared_mul_A_mul_IsDiag : IsDiag
  ((star (sharedEigenvectorUnitary hA hB hAB : Matrix d d  $\mathbb{K}$ )) * A *
    (sharedEigenvectorUnitary hA hB hAB : Matrix d d  $\mathbb{K}$ ))
```

**theorem star\_shared\_mul\_B\_mul\_IsDiag**[\[source\]](#)

*Analogous to Matrix.IsHermitian.star\_mul\_self\_mul\_eq\_diagonal for the shared basis.*

```
theorem star_shared_mul_B_mul_IsDiag : IsDiag
  ((star (sharedEigenvectorUnitary hA hB hAB : Matrix d d  $\mathbb{K}$ )) * B *
```

```
(sharedEigenvectorUnitary hA hB hAB : Matrix d d ℤ))
```

**theorem exists\_unitary**[\[source\]](#)

```
theorem Commute.exists_unitary (hA : A.IsHermitian) (hB : B.IsHermitian) (hAB : Commute A B) :
  ∃ U : Matrix.unitaryGroup d ℤ, (U.val * A * UH).IsDiag ∧ (U.val * B * UH).IsDiag
```

**instance instInvertibleUnitaryGroup**[\[source\]](#)

```
instance instInvertibleUnitaryGroup (U : Matrix.unitaryGroup d ℤ) : Invertible U
```

**instance Invertible U.val**[\[source\]](#)

```
instance (U : Matrix.unitaryGroup d ℤ) : Invertible U.val
```

**theorem IsDiag.exists\_cfc**[\[source\]](#)

*If a matrix is diagonalized by a unitary matrix, then it can be written as a CFC of a (particular, canonical) diagonal matrix.*

```
theorem Matrix.IsDiag.exists_cfc {U : Matrix.unitaryGroup d ℤ} {M : Matrix d d ℤ}
  (hU : (U.val * M * UH).IsDiag) (hM : M.IsHermitian) (e : d ≈ Fin (Fintype.card d)) :
  ∃ f : ℝ → ℝ,
  M = cfc f (UH * (Matrix.diagonal fun x => ↑(e x)) * U.val)
```

**theorem Commute.exists\_cfc**[\[source\]](#)

*If two Hermitian matrices commute, there exists a common matrix that they are both a CFC of.*

```
theorem Commute.exists_cfc (hA : A.IsHermitian) (hB : B.IsHermitian) (hAB : Commute A B) :
  ∃ C : Matrix d d ℤ, (∃ f : ℝ → ℝ, A = cfc f C) ∧ (∃ g : ℝ → ℝ, B = cfc g C)
```

**2.71 QuantumInfo.ForMathlib.Lieb**

[QuantumInfo/ForMathlib/Lieb.lean](#) on GitHub.

**theorem LiebConcavity**[\[source\]](#)

```
theorem LiebConcavity (K : Matrix n m ℂ) (hq : 0 ≤ q) (hr : 0 ≤ r) (hqr : q + r ≤ 1) :
  let F : (HermitianMat m ℂ × HermitianMat n ℂ) → ℝ
```

**2.72 QuantumInfo.ForMathlib.LimSupInf**

[QuantumInfo/ForMathlib/LimSupInf.lean](#) on GitHub.

**lemma exists\_strictMono\_seq\_le**[\[source\]](#)

```
lemma exists_strictMono_seq_le (y : ℝ≥0) (f : ℝ≥0 → ℕ → ℝ≥0∞) (hf : ∀ x > 0, Filter.atTop.
  liminf (f x) ≤ y) :
  ∃ n : ℕ → ℕ, StrictMono n ∧ ∀ k : ℕ, f ((k : ℝ≥0) + 1)-1 (n k) ≤ (y : ℝ≥0∞) + ((k : ℝ≥0) +
  1)-1
```

**lemma exists\_seq\_bound**[\[source\]](#)

```
lemma exists_seq_bound (y : ℝ≥0) (f : ℝ≥0 → ℕ → ℝ≥0∞) (hf : ∀ x > 0, Filter.atTop.limsup (f x) ≤ y) :
  ∃ M : ℕ → ℕ, StrictMono M ∧ ∀ k, ∀ n ≥ M k, f ((k + 1 : ℝ≥0)-1) n ≤ y + (k + 1 : ℝ≥0∞)-1
```

**lemma exists\_liminf\_zero\_of\_forall\_liminf\_le**[\[source\]](#)

```
lemma exists_liminf_zero_of_forall_liminf_le (y : ℝ≥0) (f : ℝ≥0 → ℕ → ℝ≥0∞)
  (hf : ∀ x > 0, Filter.atTop.liminf (f x) ≤ y) :
  ∃ g : ℕ → ℝ≥0, (∀ x, g x > 0) ∧ Filter.atTop.Tendsto g (ℵ 0) ∧
    Filter.atTop.liminf (fun n ↦ f (g n) n) ≤ y
```

**lemma exists\_liminf\_zero\_of\_forall\_liminf\_le\_with\_UB**[\[source\]](#)

```
lemma exists_liminf_zero_of_forall_liminf_le_with_UB (y : ℝ≥0) (f : ℝ≥0 → ℕ → ℝ≥0∞)
  {z : ℝ≥0} (hz : 0 < z)
  (hf : ∀ x, x > 0 → Filter.atTop.liminf (f x) ≤ y) :
  ∃ g : ℕ → ℝ≥0, (∀ x, g x > 0) ∧ (∀ x, g x < z) ∧ (Filter.atTop.Tendsto g (ℵ 0)) ∧
    (Filter.atTop.liminf (fun n ↦ f (g n) n) ≤ y)
```

**lemma exists\_limsup\_zero\_of\_forall\_limsup\_le**[\[source\]](#)

```
lemma exists_limsup_zero_of_forall_limsup_le (y : ℝ≥0) (f : ℝ≥0 → ℕ → ℝ≥0∞)
  (hf : ∀ x, x > 0 → Filter.atTop.limsup (f x) ≤ y) :
  ∃ g : ℕ → ℝ≥0, (∀ x, g x > 0) ∧ (Filter.atTop.Tendsto g (ℵ 0)) ∧
    (Filter.atTop.limsup (fun n ↦ f (g n) n) ≤ y)
```

**lemma tendsto\_of\_block\_sequence**[\[source\]](#)

```
lemma tendsto_of_block_sequence {α : Type*} [TopologicalSpace α] {x : ℕ → α} {T : ℕ → ℕ}
  (hT : StrictMono T) {L : α} (hx : Filter.atTop.Tendsto x (ℵ L)) (g : ℕ → α) (hg : ∀ k, ∀ n
    ∈ Set.Ico (T k) (T (k + 1)), g n = x k) :
  Filter.atTop.Tendsto g (ℵ L)
```

**lemma exists\_increasing\_sequence\_with\_property**[\[source\]](#)

```
lemma exists_increasing_sequence_with_property (M : ℕ → ℕ) (P : ℕ → ℕ → Prop) (hP : ∀ k L, ∃ n
  ≥ L, P k n) :
  ∃ T : ℕ → ℕ, StrictMono T ∧ (∀ k, T k ≥ M k) ∧ (∀ k, ∃ n ∈ Set.Ico (T k) (T (k + 1)), P k n)
```

**lemma liminf\_le\_of\_block\_sequence\_witnesses**[\[source\]](#)

```
lemma liminf_le_of_block_sequence_witnesses {α : Type*} (y : ℝ≥0) (f : α → ℕ → ℝ≥0∞)
  (T : ℕ → ℕ) (hT : StrictMono T) (x : ℕ → α) (g : ℕ → α)
  (hg : ∀ k, ∀ n ∈ Set.Ico (T k) (T (k + 1)), g n = x k)
  (hwit : ∀ k, ∃ n ∈ Set.Ico (T k) (T (k + 1)), f (x k) n ≤ (y : ℝ≥0∞) + (k + 1 : ℝ≥0)-1) :
  Filter.atTop.liminf (fun n ↦ f (g n) n) ≤ y
```

**lemma limsup\_le\_of\_block\_sequence\_bound**[\[source\]](#)

```

lemma limsup_le_of_block_sequence_bound {α : Type*} (y : ℝ≥0) (f : α → ℕ → ℝ≥0∞)
  (T : ℕ → ℕ) (hT : StrictMono T) (x : ℕ → α) (g : ℕ → α)
  (hg : ∀ k, ∀ n ∈ Set.Ico (T k) (T (k + 1)), g n = x k)
  (hbound : ∀ k, ∀ n ∈ Set.Ico (T k) (T (k + 1)), f (x k) n ≤ (y : ℝ≥0∞) + (k + 1 : ℝ≥0)-1) :
  Filter.atTop.limsup (fun n ↦ f (g n) n) ≤ y

```

**lemma exists\_liminf\_zero\_of\_forall\_liminf\_limsup\_le\_with\_UB**

[\[source\]](#)

```

lemma exists_liminf_zero_of_forall_liminf_limsup_le_with_UB (y₁ y₂ : ℝ≥0) (f₁ f₂ : ℝ≥0 → ℕ →
  ℝ≥0∞)
  {z : ℝ≥0} (hz : 0 < z)
  (hf₁ : ∀ x > 0, Filter.atTop.liminf (f₁ x) ≤ y₁)
  (hf₂ : ∀ x > 0, Filter.atTop.limsup (f₂ x) ≤ y₂) :
  ∃ g : ℕ → ℝ≥0, (∀ x, g x > 0) ∧ (∀ x, g x < z) ∧
  Filter.atTop.Tendsto g (ℕ 0) ∧
  Filter.atTop.liminf (fun n ↦ f₁ (g n) n) ≤ y₁ ∧
  Filter.atTop.limsup (fun n ↦ f₂ (g n) n) ≤ y₂

```

**theorem extracted\_limsup\_inequality**

[\[source\]](#)

```

theorem extracted_limsup_inequality (z : ℝ≥0∞) (hz : z ≠ ⊤) (y x : ℕ → ℝ≥0∞) (h_lem5 : ∀ (n :
  ℕ), x n ≤ y n + z)
  : Filter.atTop.limsup (fun n ↦ x n / n) ≤ Filter.atTop.limsup (fun n ↦ y n / n)

```

**theorem tendsto\_sub\_atTop\_iff\_nat**

[\[source\]](#)

*Like Filter.tendsto\_add\_atTop\_iff\_nat, but with nat subtraction.*

```

theorem _root_.Filter.tendsto_sub_atTop_iff_nat {α : Type*} {f : ℕ → α} {l : Filter α} (k : ℕ)
  :
  Tendsto (fun (n : ℕ) ↦ f (n - k)) atTop l ↔ Tendsto f atTop l

```

**theorem tendsto\_sub\_const\_nhds\_zero\_iff**

[\[source\]](#)

*Sort of dual to ENNReal.tendsto\_const\_sub\_nhds\_zero\_iff. Takes a substantially different form though, since we don't actually have equality of the limits, or even the fact that the other one converges, which is why we need to use limsup.*

```

theorem _root_.ENNReal.tendsto_sub_const_nhds_zero_iff {α : Type*} {l : Filter α} {f : α → ℝ≥0
  ∞} {a : ℝ≥0∞}
  : Tendsto (f · - a) l (ℕ 0) ↔ limsup f l ≤ a

```

## 2.73 QuantumInfo.ForMathlib.LinearEquiv

[QuantumInfo/ForMathlib/LinearEquiv.lean](#) on GitHub.

**def of\_relabel**

[\[source\]](#)

```

@[simps]
def of_relabel (e : d ≈ d₂) : (d₂ → R) ≈[R] (d → R)

```

**def euclidean\_of\_relabel**[\[source\]](#)

```
@[simps!]
def euclidean_of_relabel (e : d ≈ d₂) : EuclideanSpace ℤ d₂ ≈ℓ[ℤ] EuclideanSpace ℤ d
```

**theorem of\_relabel\_refl**[\[source\]](#)

```
@[simp]
theorem of_relabel_refl : of_relabel R (.refl d) = LinearEquiv.refl R (d → R)
```

**theorem euclidean\_of\_relabel\_refl**[\[source\]](#)

```
@[simp]
theorem euclidean_of_relabel_refl : euclidean_of_relabel ℤ (.refl d) =
  LinearEquiv.refl ℤ (EuclideanSpace ℤ d)
```

**theorem reindex\_toLin'**[\[source\]](#)

```
theorem reindex_toLin' (e : d₁ ≈ d₃) (f : d₂ ≈ d) (M : Matrix d₁ d₂ R) :
  (M.reindex e f).toLin' = (LinearEquiv.of_relabel R e.symm) ∘ℓ
  M.toLin' ∘ℓ (LinearEquiv.of_relabel R f)
```

**theorem reindex\_toEuclideanLin**[\[source\]](#)

```
theorem reindex_toEuclideanLin (e : d₁ ≈ d₃) (f : d₂ ≈ d) (M : Matrix d₁ d₂ ℤ) :
  (M.reindex e f).toEuclideanLin = (LinearEquiv.euclidean_of_relabel ℤ e.symm) ∘ℓ
  M.toEuclideanLin ∘ℓ (LinearEquiv.euclidean_of_relabel ℤ f)
```

**theorem reindex\_right\_toLin'**[\[source\]](#)

```
theorem reindex_right_toLin' (e : d ≈ d₂) (M : Matrix d₃ d R) :
  (M.reindex (.refl d₃) e).toLin' = M.toLin' ∘ℓ (LinearEquiv.of_relabel R e)
```

**theorem reindex\_right\_toEuclideanLin**[\[source\]](#)

```
theorem reindex_right_toEuclideanLin (e : d ≈ d₂) (M : Matrix d₃ d ℤ) :
  (M.reindex (.refl d₃) e).toEuclideanLin =
  M.toEuclideanLin ∘ℓ (LinearEquiv.euclidean_of_relabel ℤ e)
```

**theorem reindex\_left\_toLin'**[\[source\]](#)

```
theorem reindex_left_toLin' (e : d₁ ≈ d₃) (M : Matrix d₁ d₂ R) :
  (M.reindex e (.refl d₂)).toLin' = (LinearEquiv.of_relabel R e.symm) ∘ M.toLin'
```

**theorem reindex\_left\_toEuclideanLin**[\[source\]](#)

```
theorem reindex_left_toEuclideanLin (e : d₁ ≈ d₃) (M : Matrix d₁ d₂ ℤ) :
  (M.reindex e (.refl d₂)).toEuclideanLin =
  (LinearEquiv.euclidean_of_relabel ℤ e.symm) ∘ M.toEuclideanLin
```

## 2.74 QuantumInfo.ForMathlib.Majorization

[QuantumInfo/ForMathlib/Majorization.lean](#) on GitHub.

**def singularValues** [\[source\]](#)

*The singular values of a square complex matrix  $A$ , defined as the square roots of the eigenvalues of  $A^\dagger A$ . These are indexed by  $d$  without a particular ordering.*

*Note: We use  $A.conjTranspose$  as the argument to `isHermitian_mul_conjTranspose_self` so that the underlying Hermitian matrix is  $A^\dagger * A$  (matching the convention in `schattenNorm`).*

```
def singularValues (A : Matrix d d ℂ) : d → ℝ
```

**lemma singularValues\_nonneg** [\[source\]](#)

*Singular values are nonneg.*

```
lemma singularValues_nonneg (A : Matrix d d ℂ) (i : d) :  
  0 ≤ singularValues A i
```

**def singularValuesSorted** [\[source\]](#)

*The sorted singular values of a square complex matrix, in decreasing order, indexed by  $\text{Fin } (\text{Fintype.card } d)$ .*

*We define them by sorting the multiset of singular values.*

```
noncomputable def singularValuesSorted (A : Matrix d d ℂ) :  
  Fin (Fintype.card d) → ℝ
```

**lemma singularValuesSorted\_nonneg** [\[source\]](#)

*Sorted singular values are nonneg.*

```
lemma singularValuesSorted_nonneg (A : Matrix d d ℂ) (i : Fin (Fintype.card d)) :  
  0 ≤ singularValuesSorted A i
```

**lemma sum\_singularValues\_rpow\_eq\_sum\_sorted** [\[source\]](#)

*The sum  $\sum \text{singularValues } A \ i \ ^p$  equals the sum over sorted singular values.*

```
lemma sum_singularValues_rpow_eq_sum_sorted (A : Matrix d d ℂ) (p : ℝ) :  
  ∑ i : d, singularValues A i ^ p =  
  ∑ i : Fin (Fintype.card d), singularValuesSorted A i ^ p
```

**lemma singularValuesSorted\_antitone** [\[source\]](#)

*Sorted singular values are antitone (decreasing).*

```
lemma singularValuesSorted_antitone (A : Matrix d d ℂ) :  
  Antitone (singularValuesSorted A)
```

**lemma antitone\_mul\_of\_antitone\_nonneg**[\[source\]](#)

*The product of nonneg antitone sequences is antitone.*

```
lemma antitone_mul_of_antitone_nonneg {n : ℕ}
  {f g : Fin n → ℝ} (hf : Antitone f) (hg : Antitone g)
  (hf_nn : ∀ i, 0 ≤ f i) (hg_nn : ∀ i, 0 ≤ g i) :
  Antitone (fun i => f i * g i)
```

**instance fintypeLinearOrderClassical**[\[source\]](#)

*A LinearOrder on any Fintype, obtained classically via well-ordering.*

```
scoped instance fintypeLinearOrderClassical (α : Type*) [Fintype α] [DecidableEq α] :
  LinearOrder α
```

**def compoundMatrix**[\[source\]](#)

*The  $k$ -th compound (exterior-power) matrix of  $M$ .*

```
noncomputable def compoundMatrix (M : Matrix d d ℂ) (k : ℕ) :
  Matrix {S : Finset d // S.card = k} {S : Finset d // S.card = k} ℂ
```

**lemma cauchyBinet**[\[source\]](#)

*\*\*Cauchy–Binet formula\*\* for rectangular matrices: if  $A$  is  $m \times n$  and  $B$  is  $n \times m$ , then  $\det(A * B) = \sum_S \det(A[:, S]) * \det(B[S, :])$  where the sum is over  $m$ -element subsets  $S$  of the column/row index.*

```
lemma cauchyBinet {m : ℕ} {n : Type*} [Fintype n] [DecidableEq n] [LinearOrder n]
  {R : Type*} [CommRing R]
  (A : Matrix (Fin m) n R) (B : Matrix n (Fin m) R) :
  (A * B).det = ∑ S : {S : Finset n // S.card = m},
    (A.submatrix id (S.1.orderEmbOffFin S.2)).det *
    (B.submatrix (S.1.orderEmbOffFin S.2) id).det
```

**lemma compoundMatrix\_mul**[\[source\]](#)

*The compoundMatrix of a product is the product of the compoundMatrixs.*

```
lemma compoundMatrix_mul (M N : Matrix d d ℂ) (k : ℕ) :
  compoundMatrix (M * N) k = compoundMatrix M k * compoundMatrix N k
```

**lemma compoundMatrix\_conjTranspose**[\[source\]](#)

*compoundMatrix commutes with conjTranspose.*

```
lemma compoundMatrix_conjTranspose (M : Matrix d d ℂ) (k : ℕ) :
  compoundMatrix M.conjTranspose k = (compoundMatrix M k).conjTranspose
```

**lemma compoundMatrix\_diagonal**[\[source\]](#)

*The compound matrix of a diagonal matrix is diagonal, with entries being products of eigenvalues over  $k$ -subsets.*

```

lemma compoundMatrix_diagonal (f : d → ℂ) (k : ℕ) :
  compoundMatrix (Matrix.diagonal f) k =
  Matrix.diagonal (fun S : {S : Finset d // S.card = k} =>
    [] i : Fin k, f (S.1.orderEmb0fFin S.2 i))

```

### lemma singularValues\_compoundMatrix\_eq

[\[source\]](#)

*The eigenvalues of the compound matrix of a Hermitian matrix are the products of eigenvalues over  $k$ -subsets. More precisely, the singular values of `compoundMatrix M`  $k$  are the square roots of products of eigenvalues of  $M^\dagger M$  over  $k$ -subsets.*

```

lemma singularValues_compoundMatrix_eq (M : Matrix d d ℂ) (k : ℕ) :
  ∀ (S : {S : Finset d // S.card = k}),
  ∃ (j : {S : Finset d // S.card = k}),
  singularValues (compoundMatrix M k) j =
  [] i : Fin k, singularValues M (S.1.orderEmb0fFin S.2 i)

```

### lemma prod\_le\_prod\_sorted

[\[source\]](#)

*The product of nonneg values over a  $k$ -subset is at most the product of the  $k$  largest values.*

```

lemma prod_le_prod_sorted {n : ℕ} {f : Fin n → ℝ}
  (hf : Antitone f) (hf_nn : ∀ i, 0 ≤ f i)
  (k : ℕ) (hk : k ≤ n)
  (g : Fin k → Fin n) (hg : Function.Injective g) :
  [] i : Fin k, f (g i) ≤ [] i : Fin k, f ⟨i.val, by omega⟩

```

### lemma singularValuesSorted\_zero\_eq\_sup

[\[source\]](#)

*The 0th sorted singular value is the maximum of the singular values.*

```

lemma singularValuesSorted_zero_eq_sup {e : Type*} [Fintype e] [DecidableEq e]
  (A : Matrix e e ℂ) (h : 0 < Fintype.card e) :
  singularValuesSorted A ⟨0, h⟩ = Finset.sup' Finset.univ
    (Finset.univ_nonempty_iff.mpr (Fintype.card_pos_iff.mp h))
    (singularValues A)

```

### lemma singularValues\_mem\_sorted

[\[source\]](#)

*Each singular value appears in the sorted list.*

```

lemma singularValues_mem_sorted {e : Type*} [Fintype e] [DecidableEq e]
  (A : Matrix e e ℂ) (i : e) :
  ∃ j : Fin (Fintype.card e), singularValues A i = singularValuesSorted A j

```

### lemma singularValuesSorted\_mem\_values

[\[source\]](#)

*Each sorted singular value appears among the original singular values.*

```

lemma singularValuesSorted_mem_values {e : Type*} [Fintype e] [DecidableEq e]
  (A : Matrix e e ℂ) (j : Fin (Fintype.card e)) :
  ∃ i : e, singularValuesSorted A j = singularValues A i

```



**lemma singularValues\_compoundMatrix\_perm**[\[source\]](#)

*Stronger version of `singularValues_compoundMatrix_eq` that exposes the permutation.*

```
lemma singularValues_compoundMatrix_perm (M : Matrix d d ℂ) (k : ℕ) :
  ∃ σ : {S : Finset d // S.card = k} ≈ {S : Finset d // S.card = k},
  ∀ S, singularValues (compoundMatrix M k) (σ S) =
  [] i : Fin k, singularValues M (S.1.orderEmb0fFin S.2 i)
```

**lemma singularValues\_compoundMatrix\_rev**[\[source\]](#)

*Converse of `singularValues_compoundMatrix_eq`: every singular value of the compound matrix is a product of singular values of  $M$  over some  $k$ -subset.*

```
lemma singularValues_compoundMatrix_rev (M : Matrix d d ℂ) (k : ℕ)
  (j : {S : Finset d // S.card = k}) :
  ∃ (S : {S : Finset d // S.card = k}),
  singularValues (compoundMatrix M k) j =
  [] i : Fin k, singularValues M (S.1.orderEmb0fFin S.2 i)
```

**lemma exists\_sorting\_equiv**[\[source\]](#)

*There exists a bijection  $\sigma : \text{Fin}(\text{card } d) \approx d$  such that  $\text{singularValues } M (\sigma i) = \text{singularValuesSorted } M i$  for all  $i$ .*

```
lemma exists_sorting_equiv (M : Matrix d d ℂ) :
  ∃ σ : Fin (Fintype.card d) ≈ d,
  ∀ i, singularValues M (σ i) = singularValuesSorted M i
```

**lemma prod\_singularValues\_subset\_le\_sorted\_prod**[\[source\]](#)

*For any  $k$ -subset  $S$  of  $d$ , the product of singular values over  $S$  is  $\leq$  the product of the top  $k$  sorted singular values.*

```
lemma prod_singularValues_subset_le_sorted_prod (M : Matrix d d ℂ) (k : ℕ)
  (hk : k ≤ Fintype.card d) (S : {S : Finset d // S.card = k}) :
  [] i : Fin k, singularValues M (S.1.orderEmb0fFin S.2 i) ≤
  [] i : Fin k, singularValuesSorted M {i.val, by omega}
```

**lemma exists\_subset\_prod\_eq\_sorted\_prod**[\[source\]](#)

```
lemma exists_subset_prod_eq_sorted_prod (M : Matrix d d ℂ) (k : ℕ)
  (hk : k ≤ Fintype.card d) :
  ∃ S : {S : Finset d // S.card = k},
  [] i : Fin k, singularValues M (S.1.orderEmb0fFin S.2 i) =
  [] i : Fin k, singularValuesSorted M {i.val, by omega}
```

**lemma prod\_singularValuesSorted\_eq\_compoundSV**[\[source\]](#)

```
lemma prod_singularValuesSorted_eq_compoundSV (M : Matrix d d ℂ) (k : ℕ)
  (hk : k ≤ Fintype.card d) :
  [] i : Fin k, singularValuesSorted M {i.val, by omega} =
  singularValuesSorted (compoundMatrix M k) {0, by
    have : Fintype.card {S : Finset d // S.card = k} = (Fintype.card d).choose k := by
      simp [Fintype.card_subtype]}
```

```
rw [this]; exact Nat.choose_pos hk)
```

### lemma inner\_le\_sup\_eigenvalue\_mul\_inner

[\[source\]](#)

The *\*\*Rayleigh quotient bound\*\**: For a Hermitian matrix  $H$  with eigenvalues  $\lambda$ , we have  $\forall v \neq 0, \frac{v^* H v}{v^* v} \leq \max \lambda$ .

```
lemma IsHermitian.inner_le_sup_eigenvalue_mul_inner
  {e : Type*} [Fintype e] [DecidableEq e]
  (H : Matrix e e ℂ) (hH : H.IsHermitian)
  (he : 0 < Fintype.card e)
  (v : e → ℂ) :
  Complex.re (star v ·v H.mulVec v) ≤
  Finset.sup' Finset.univ
    (Finset.univ_nonempty_iff.mpr (Fintype.card_pos_iff.mp he))
  hH.eigenvalues * Complex.re (star v ·v v)
```

### lemma eigenvalue\_le\_singularValuesSorted\_sq

[\[source\]](#)

```
lemma eigenvalue_le_singularValuesSorted_sq {e : Type*} [Fintype e] [DecidableEq e]
  (A : Matrix e e ℂ) (h : 0 < Fintype.card e) (i : e) :
  (isHermitian_mul_conjTranspose_self A.conjTranspose).eigenvalues i ≤
  (singularValuesSorted A {0, h}) ^ 2
```

### lemma quadratic\_form\_le\_singularValuesSorted\_sq

[\[source\]](#)

The quadratic form of  $A^* A$  is bounded by  $(\max \text{singular value})^2 \cdot \|v\|^2$ .

```
lemma quadratic_form_le_singularValuesSorted_sq {e : Type*} [Fintype e] [DecidableEq e]
  (A : Matrix e e ℂ) (h : 0 < Fintype.card e) (v : e → ℂ) :
  Complex.re (star v ·v (A.conjTranspose * A).mulVec v) ≤
  (singularValuesSorted A {0, h}) ^ 2 * Complex.re (star v ·v v)
```

### lemma singularValuesSorted\_mul\_le

[\[source\]](#)

The largest singular value of a matrix product is at most the product of the largest singular values:  $\sigma_1(M * N) \leq \sigma_1(M) * \sigma_1(N)$ . This is operator-norm submultiplicativity.

```
lemma singularValuesSorted_mul_le {e : Type*} [Fintype e] [DecidableEq e]
  (M N : Matrix e e ℂ) (h : 0 < Fintype.card e) :
  singularValuesSorted (M * N) {0, h} ≤
  singularValuesSorted M {0, h} * singularValuesSorted N {0, h}
```

### lemma horn\_weak\_log\_majorization

[\[source\]](#)

Horn's inequality (weak log-majorization of singular values): For all  $k$ ,  $\prod_{i=1}^k \sigma_i(AB) \leq \prod_{i=1}^k \sigma_i(A) \cdot \prod_{i=1}^k \sigma_i(B)$ . This follows from submultiplicativity of the operator norm applied to exterior powers of the matrices.

```
lemma horn_weak_log_majorization (A B : Matrix d d ℂ) (k : ℕ)
  (hk : k ≤ Fintype.card d) :
  ∏ i : Fin k, singularValuesSorted (A * B) (i.val, by omega) ≤
  ∏ i : Fin k, (singularValuesSorted A (i.val, by omega) *
    singularValuesSorted B (i.val, by omega))
```

**lemma rpow\_antitone\_of\_nonneg\_antitone**[\[source\]](#)

*Raising nonneg antitone sequences to a positive power preserves antitonicity.*

```
lemma rpow_antitone_of_nonneg_antitone {n : ℕ}
  {f : Fin n → ℝ} (hf : Antitone f) (hf_nn : ∀ i, 0 ≤ f i)
  {r : ℝ} (hr : 0 < r) :
  Antitone (fun i => f i ^ r)
```

**lemma rpow\_preserves\_weak\_log\_maj**[\[source\]](#)

*Weak log-majorization is preserved under positive powers.*

```
lemma rpow_preserves_weak_log_maj {n : ℕ}
  {x y : Fin n → ℝ}
  (hx_nn : ∀ i, 0 ≤ x i) (hy_nn : ∀ i, 0 ≤ y i)
  (h_log_maj : ∀ (k : ℕ) (_ : k ≤ n),
    ⌈ i : Fin k, x (i.val, by omega) ≤
    ⌈ i : Fin k, y (i.val, by omega))
  {r : ℝ} (hr : 0 < r) :
  ∀ (k : ℕ) (_ : k ≤ n),
    ⌈ i : Fin k, (fun j => x j ^ r) (i.val, by omega) ≤
    ⌈ i : Fin k, (fun j => y j ^ r) (i.val, by omega)
```

**lemma sum\_mul\_log\_nonneg\_of\_weak\_log\_maj**[\[source\]](#)

```
lemma sum_mul_log_nonneg_of_weak_log_maj {n : ℕ}
  {x y : Fin n → ℝ}
  (hx_pos : ∀ i, 0 < x i) (hy_pos : ∀ i, 0 < y i)
  (hx_anti : Antitone x)
  (h_log_maj : ∀ (k : ℕ) (_ : k ≤ n),
    ⌈ i : Fin k, x (i.val, by omega) ≤
    ⌈ i : Fin k, y (i.val, by omega)) :
  0 ≤ ∑ i, x i * Real.log (y i / x i)
```

**lemma sub\_ge\_mul\_log\_div**[\[source\]](#)

```
lemma sub_ge_mul_log_div {a b : ℝ} (ha : 0 < a) (hb : 0 < b) :
  b - a ≥ a * Real.log (b / a)
```

**lemma weak\_log\_maj\_sum\_le**[\[source\]](#)

```
lemma weak_log_maj_sum_le {n : ℕ}
  {x y : Fin n → ℝ}
  (hx_nn : ∀ i, 0 ≤ x i) (hy_nn : ∀ i, 0 ≤ y i)
  (hx_anti : Antitone x) (hy_anti : Antitone y)
  (h_log_maj : ∀ (k : ℕ) (_ : k ≤ n),
    ⌈ i : Fin k, x (i.val, by omega) ≤
    ⌈ i : Fin k, y (i.val, by omega)) :
  ∑ i, x i ≤ ∑ i, y i
```

**lemma weak\_log\_maj\_sum\_rpow\_le**[\[source\]](#)

*Weak log-majorization of nonneg antitone sequences implies the sum of powers inequality.*

```

lemma weak_log_maj_sum_rpow_le {n : ℕ}
  {x y : Fin n → ℝ}
  (hx_nn : ∀ i, 0 ≤ x i) (hy_nn : ∀ i, 0 ≤ y i)
  (hx_anti : Antitone x) (hy_anti : Antitone y)
  (h_log_maj : ∀ (k : ℕ) (_ : k ≤ n),
    ∏ i : Fin k, x (i.val, by omega) ≤
    ∏ i : Fin k, y (i.val, by omega))
  {r : ℝ} (hr : 0 < r) :
  ∑ i, x i ^ r ≤ ∑ i, y i ^ r

```

### lemma sum\_rpow\_singularValues\_mul\_le

[\[source\]](#)

```

lemma sum_rpow_singularValues_mul_le (A B : Matrix d d ℂ) {r : ℝ} (hr : 0 < r) :
  ∑ i : Fin (Fintype.card d), singularValuesSorted (A * B) i ^ r ≤
  ∑ i : Fin (Fintype.card d),
    (singularValuesSorted A i ^ r * singularValuesSorted B i ^ r)

```

### lemma holder\_step\_for\_singularValues

[\[source\]](#)

The finite-sum Hölder inequality applied to sequences of  $r$ -th powers of sorted singular values.

With conjugate exponents  $p' = p/r > 1$  and  $q' = q/r > 1$  (which satisfy  $1/p' + 1/q' = 1$  when  $1/r = 1/p + 1/q$ ), this gives:  $\sum \sigma_{\downarrow i}(A)^r \cdot \sigma_{\downarrow i}(B)^r \leq (\sum \sigma_{\downarrow i}(A)^p)^{r/p} \cdot (\sum \sigma_{\downarrow i}(B)^q)^{r/q}$

Note: The sums on the RHS don't depend on the ordering, so we can replace sorted singular values with unsorted ones.

```

lemma holder_step_for_singularValues (A B : Matrix d d ℂ)
  {r p q : ℝ} (hr : 0 < r) (hp : 0 < p) (hq : 0 < q)
  (hpqr : 1 / r = 1 / p + 1 / q) :
  (∑ i : Fin (Fintype.card d),
    (singularValuesSorted A i ^ r * singularValuesSorted B i ^ r)) ≤
  (∑ i : Fin (Fintype.card d), singularValuesSorted A i ^ p) ^ (r / p) *
  (∑ i : Fin (Fintype.card d), singularValuesSorted B i ^ q) ^ (r / q)

```

## 2.75 QuantumInfo.ForMathlib.Matrix

[QuantumInfo/ForMathlib/Matrix.lean](#) on GitHub.

### theorem zero\_rank\_eq\_zero

[\[source\]](#)

```

theorem zero_rank_eq_zero {A : Matrix n n ℤ} [Fintype n] (hA : A.rank = 0) : A = 0

```

### theorem smul\_selfAdjoint

[\[source\]](#)

```

theorem smul_selfAdjoint {c : ℤ} (hc : _root_.IsSelfAdjoint c) : (c • A).IsHermitian

```

### theorem smul\_im\_zero

[\[source\]](#)

```

theorem smul_im_zero {c : ℤ} (h : RCLike.im c = 0) : (c • A).IsHermitian

```

**theorem smul\_real**[\[source\]](#)

```
theorem smul_real (c : ℝ) : (c • A).IsHermitian
```

**def HermitianSubspace**[\[source\]](#)

```
def HermitianSubspace (n ℚ : Type*) [Fintype n] [RCLike ℚ] : Subspace ℝ (Matrix n n ℚ) where
```

**theorem re\_trace\_eq\_trace**[\[source\]](#)

```
@[simp]
theorem re_trace_eq_trace : RCLike.re (A.trace) = A.trace
```

**theorem sum\_eigenvalues\_eq\_trace**[\[source\]](#)

*The sum of the eigenvalues of a Hermitian matrix is equal to its trace.*

```
theorem sum_eigenvalues_eq_trace : ∑ i, hA.eigenvalues i = A.trace
```

**theorem eigenvalues\_zero\_eq\_zero**[\[source\]](#)

*If all eigenvalues are equal to zero, then the matrix is zero.*

```
theorem eigenvalues_zero_eq_zero (h : ∀ i, hA.eigenvalues i = 0) : A = 0
```

**theorem kroneckerMap\_conjTranspose**[\[source\]](#)

```
theorem kroneckerMap_conjTranspose : (A ⊗ℚ B)H = (AH ⊗ℚ BH)
```

**theorem kroneckerMap\_IsHermitian**[\[source\]](#)

```
theorem kroneckerMap_IsHermitian : (A ⊗ℚ B).IsHermitian
```

**theorem trace\_zero**[\[source\]](#)

```
theorem trace_zero : A.trace = 0 → A = 0
```

**theorem trace\_zero\_iff**[\[source\]](#)

```
@[simp]
theorem trace_zero_iff : A.trace = 0 ↔ A = 0
```

**theorem normSq\_eq\_conj\_mul\_self**[\[source\]](#)

```
theorem _root_.RCLike.normSq_eq_conj_mul_self {z : ℚ} : RCLike.normSq z = conj z * z
```

**theorem sum\_eq\_ite**[\[source\]](#)

```
theorem Finsupp.sum_eq_ite
  {α : Type u_1} {M : Type u_8} {N : Type u_10} [Zero M] [AddCommMonoid N] [Fintype α]
  [DecidableEq M] (f : α →0 M) (g : α → M → N) :
  f.sum g = ∑ i, if f i ≠ 0 then g i (f i) else 0
```

**theorem outer\_self\_conj**[\[source\]](#)

```
theorem outer_self_conj (v : n → ℝ) : PosSemidef (vecMulVec v (conj v))
```

**theorem convex\_cone**[\[source\]](#)

```
theorem convex_cone {c₁ c₂ : ℝ} (hc₁ : 0 ≤ c₁) (hc₂ : 0 ≤ c₂) : (c₁ • A + c₂ • B).PosSemidef
```

**theorem stdBasisMatrix\_iff\_eq**[\[source\]](#)

*A standard basis matrix (with a positive entry) is positive semidefinite iff the entry is on the diagonal.*

```
theorem stdBasisMatrix_iff_eq (i j : m) {c : ℝ} (hc : 0 < c) : (single i j c).PosSemidef ↔ i = j
```

**theorem PosSemidef\_kronecker**[\[source\]](#)

```
theorem PosSemidef_kronecker : (A ⊗ₖ B).PosSemidef
```

**theorem zero\_dotProduct\_zero\_iff**[\[source\]](#)

```
theorem zero_dotProduct_zero_iff : (∀ x : m → ℝ, 0 = star x ·ᵥ A.mulVec x) ↔ A = 0
```

**theorem pos\_smul**[\[source\]](#)

```
theorem pos_smul {c : ℝ} (hA : (c • A).PosSemidef) (hc : 0 < c) : A.PosSemidef
```

**theorem zero\_posSemidef\_neg\_posSemidef\_iff**[\[source\]](#)

```
theorem zero_posSemidef_neg_posSemidef_iff : A.PosSemidef ∧ (-A).PosSemidef ↔ A = 0
```

**theorem toLin\_ker\_eq\_bot**[\[source\]](#)

```
theorem toLin_ker_eq_bot (hA : A.PosDef) : LinearMap.ker A.toLin' = ⊥
```

**theorem of\_toLin\_ker\_eq\_bot**[\[source\]](#)

```
theorem of_toLin_ker_eq_bot (hA : LinearMap.ker A.toLin' = ⊥) (hA₂ : A.PosSemidef) : A.PosDef
```

**theorem ker\_range\_antitone**[\[source\]](#)

```
theorem ker_range_antitone {d : Type*} [Fintype d] [DecidableEq d] {A B : Matrix d d ℝ}
  (hA : A.IsHermitian) (hB : B.IsHermitian) :
  LinearMap.ker A.toEuclideanLin ≤ LinearMap.ker B.toEuclideanLin ↔
  LinearMap.range B.toEuclideanLin ≤ LinearMap.range A.toEuclideanLin
```

**theorem le\_of\_nonneg\_imp**[\[source\]](#)

```
theorem le_of_nonneg_imp {R : Type*} [AddCommGroup R] [PartialOrder R] [IsOrderedAddMonoid R]
  (f : Matrix n n  $\mathbb{K}$   $\rightarrow$  R) (h :  $\forall A, A.\text{PosSemidef} \rightarrow 0 \leq f A$ ) :
  (A  $\leq$  B  $\rightarrow$  f A  $\leq$  f B)
```

**theorem le\_of\_nonneg\_imp'**[\[source\]](#)

```
theorem le_of_nonneg_imp' {R : Type*} [AddCommGroup R] [PartialOrder R] [IsOrderedAddMonoid R]
  {x y : R} (f : R  $\rightarrow$  Matrix n n  $\mathbb{K}$ ) (h :  $\forall x, 0 \leq x \rightarrow (f x).\text{PosSemidef}$ ) :
  (x  $\leq$  y  $\rightarrow$  f x  $\leq$  f y)
```

**theorem mul\_mul\_conjTranspose\_mono**[\[source\]](#)

```
theorem mul_mul_conjTranspose_mono (C : Matrix m n  $\mathbb{K}$ ) :
  A  $\leq$  B  $\rightarrow$  C * A * C.conjTranspose  $\leq$  C * B * C.conjTranspose
```

**theorem conjTranspose\_mul\_mul\_mono**[\[source\]](#)

```
theorem conjTranspose_mul_mul_mono (C : Matrix n m  $\mathbb{K}$ ) :
  A  $\leq$  B  $\rightarrow$  C.conjTranspose * A * C  $\leq$  C.conjTranspose * B * C
```

**theorem nonneg\_iff\_eigenvalue\_nonneg**[\[source\]](#)

```
theorem nonneg_iff_eigenvalue_nonneg [DecidableEq n] : 0  $\leq$  A  $\leftrightarrow \forall x, 0 \leq hA.\text{eigenvalues } x$ 
```

**theorem diag\_monotone**[\[source\]](#)

```
theorem diag_monotone : Monotone (diag : Matrix n n  $\mathbb{K}$   $\rightarrow$  (n  $\rightarrow$   $\mathbb{K}$ ))
```

**theorem diag\_mono**[\[source\]](#)

```
theorem diag_mono : A  $\leq$  B  $\rightarrow \forall i, A.\text{diag } i \leq B.\text{diag } i$ 
```

**theorem trace\_monotone**[\[source\]](#)

```
theorem trace_monotone : Monotone (@trace n  $\mathbb{K}$  _ _)
```

**theorem trace\_mono**[\[source\]](#)

```
theorem trace_mono : A  $\leq$  B  $\rightarrow$  A.trace  $\leq$  B.trace
```

**theorem diagonal\_monotone**[\[source\]](#)

```
theorem diagonal_monotone : Monotone (diagonal : (n  $\rightarrow$   $\mathbb{K}$ )  $\rightarrow$  _)
```

**theorem diagonal\_mono**[\[source\]](#)

```
theorem diagonal_mono {d1 d2 : n  $\rightarrow$   $\mathbb{K}$ } : d1  $\leq$  d2  $\rightarrow$  diagonal d1  $\leq$  diagonal d2
```

**theorem diagonal\_le\_iff**[\[source\]](#)

```
theorem diagonal_le_iff {d1 d2 : n → ℤ} : d1 ≤ d2 ↔ diagonal d1 ≤ diagonal d2
```

**theorem le\_smul\_one\_of\_eigenvalues\_iff**[\[source\]](#)

```
theorem le_smul_one_of_eigenvalues_iff (hA : A.IsHermitian) (c : ℝ) :  
  (∀ i, hA.eigenvalues i ≤ c) ↔ A ≤ c • (1 : Matrix n n ℤ)
```

**theorem smul\_one\_le\_of\_eigenvalues\_iff**[\[source\]](#)

```
theorem smul_one_le_of_eigenvalues_iff (hA : A.IsHermitian) (c : ℝ) :  
  (∀ i, c ≤ hA.eigenvalues i) ↔ c • (1 : Matrix n n ℤ) ≤ A
```

**def Matrix.traceLeft**[\[source\]](#)

```
def traceLeft (m : Matrix (d × d1) (d × d2) R) : Matrix d1 d2 R
```

**def Matrix.traceRight**[\[source\]](#)

```
def traceRight (m : Matrix (d1 × d) (d2 × d) R) : Matrix d1 d2 R
```

**theorem traceLeft\_trace**[\[source\]](#)

```
@[simp]  
theorem traceLeft_trace (A : Matrix (d1 × d2) (d1 × d2) R) : A.traceLeft.trace = A.trace
```

**theorem traceRight\_trace**[\[source\]](#)

```
@[simp]  
theorem traceRight_trace (A : Matrix (d1 × d2) (d1 × d2) R) : A.traceRight.trace = A.trace
```

**theorem IsHermitian.traceLeft**[\[source\]](#)

```
theorem IsHermitian.traceLeft {A : Matrix (d × d1) (d × d1) R} (hA : A.IsHermitian) : A.  
  traceLeft.IsHermitian
```

**theorem IsHermitian.traceRight**[\[source\]](#)

```
theorem IsHermitian.traceRight {A : Matrix (d1 × d) (d1 × d) R} (hA : A.IsHermitian) : A.  
  traceRight.IsHermitian
```

**theorem trace\_mul\_kron\_one\_right**[\[source\]](#)

$$\text{Tr}(M (A \otimes I)) = \text{Tr}(\text{Tr}_B(M) A)$$

```
theorem trace_mul_kron_one_right {R : Type*} [Ring R]  
  (M : Matrix (dA × dB) (dA × dB) R) (A : Matrix dA dA R) :  
  (M * (A ⊗k (1 : Matrix dB dB R))).trace = (M.traceRight * A).trace
```



**theorem trace\_mul\_one\_kron\_right**[\[source\]](#)

$$\text{Tr}(M (I \otimes B)) = \text{Tr}(\text{Tr}_A(M) B)$$

```
theorem trace_mul_one_kron_right {R : Type*} [Ring R]
  (M : Matrix (dA × dB) (dA × dB) R) (B : Matrix dB dB R) :
  (M * ((1 : Matrix dA dA R) ⊗ B)).trace = (M.traceLeft * B).trace
```

**theorem PosSemidef.traceLeft**[\[source\]](#)

```
theorem PosSemidef.traceLeft [DecidableEq d₁] (hA : A.PosSemidef) : A.traceLeft.PosSemidef
```

**theorem PosSemidef.traceRight**[\[source\]](#)

```
theorem PosSemidef.traceRight [DecidableEq d₂] (hA : A.PosSemidef) : A.traceRight.PosSemidef
```

**theorem kron**[\[source\]](#)

```
theorem PosDef.kron {d₁ d₂ ℤ : Type*} [Fintype d₁] [DecidableEq d₁] [Fintype d₂] [DecidableEq d₂] [RCLike ℤ]
  {A : Matrix d₁ d₁ ℤ} {B : Matrix d₂ d₂ ℤ} (hA : A.PosDef) (hB : B.PosDef) : (A ⊗ B).PosDef
```

**theorem submatrix**[\[source\]](#)

```
theorem PosDef.submatrix {d₁ d₂ ℤ : Type*} [Fintype d₁] [DecidableEq d₁] [Fintype d₂] [DecidableEq d₂] [RCLike ℤ]
  {M : Matrix d₁ d₁ ℤ} (hM : M.PosDef) {e : d₂ → d₁} (he : Function.Injective e) : (M.submatrix e e).PosDef
```

**theorem reindex**[\[source\]](#)

```
theorem PosDef.reindex {d₁ d₂ ℤ : Type*} [Fintype d₁] [DecidableEq d₁] [Fintype d₂] [DecidableEq d₂] [RCLike ℤ]
  {M : Matrix d₁ d₁ ℤ} (hM : M.PosDef) (e : d₁ ≈ d₂) : (M.reindex e e).PosDef
```

**theorem reindex\_iff**[\[source\]](#)

```
@[simp]
theorem PosDef.reindex_iff {d₁ d₂ ℤ : Type*} [Fintype d₁] [DecidableEq d₁] [Fintype d₂] [DecidableEq d₂] [RCLike ℤ]
  {M : Matrix d₁ d₁ ℤ} (e : d₁ ≈ d₂) : (M.reindex e e).PosDef ↔ M.PosDef
```

**theorem rsmul**[\[source\]](#)

```
theorem PosSemidef.rsmul {n : Type*} [Fintype n] {M : Matrix n n ℂ} (hM : M.PosSemidef) {c : ℝ}
  (hc : 0 ≤ c) :
  (c • M).PosSemidef
```

**theorem Convex**[\[source\]](#)

```
theorem PosDef.Convex {n ℤ : Type*} [Fintype n] [RCLike ℤ] : Convex ℝ (Matrix.PosDef (n := n) (R := ℤ))
```

**theorem PosDef\_iff\_eigenvalues'**[\[source\]](#)

```
theorem PosDef_iff_eigenvalues' (M : Matrix d d  $\mathbb{k}$ ) :
  M.PosDef  $\leftrightarrow$   $\exists$  (h : M.IsHermitian),  $\forall$  i,  $0 < h.eigenvalues\ i$ 
```

**theorem cfc\_eigenvalues**[\[source\]](#)

```
theorem IsHermitian.cfc_eigenvalues {M : Matrix d d  $\mathbb{k}$ } (hM : M.IsHermitian) (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) :
   $\exists$  (e : d  $\approx$  d), Matrix.IsHermitian.eigenvalues (cfc_predicate f M) = f  $\circ$  hM.eigenvalues  $\circ$  e
```

**lemma eigenvalues\_eq\_of\_unitary\_similarity\_diagonal**[\[source\]](#)

*If a Hermitian matrix  $A$  is unitarily similar to a diagonal matrix with real entries  $f$ , then the eigenvalues of  $A$  are a permutation of  $f$ .*

```
lemma IsHermitian.eigenvalues_eq_of_unitary_similarity_diagonal {d  $\mathbb{k}$  : Type*}
  [Fintype d] [DecidableEq d] [RCLike  $\mathbb{k}$ ]
  {A : Matrix d d  $\mathbb{k}$ } (hA : A.IsHermitian)
  {U : Matrix d d  $\mathbb{k}$ } (hU : U  $\in$  Matrix.unitaryGroup d  $\mathbb{k}$ )
  {f : d  $\rightarrow \mathbb{R}$ }
  (h : A = U * Matrix.diagonal (fun i => (RCLike.ofReal (f i) :  $\mathbb{k}$ )) * Matrix.conjTranspose U)
  :
   $\exists$   $\sigma$  : d  $\approx$  d, hA.eigenvalues  $\circ$   $\sigma$  = f
```

**theorem toEuclideanLin\_one**[\[source\]](#)

```
@[simp]
theorem toEuclideanLin_one : Matrix.toEuclideanLin (1 : Matrix n n  $\alpha$ ) = .id
```

**theorem cfc\_diagonal**[\[source\]](#)

```
@[simp]
theorem cfc_diagonal (g : d  $\rightarrow \mathbb{R}$ ) (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) :
  cfc f (Matrix.diagonal (fun x  $\mapsto$  (g x :  $\mathbb{k}$ ))) = diagonal (RCLike.ofReal  $\circ$  f  $\circ$  g)
```

**theorem pos\_of\_mem\_spectrum**[\[source\]](#)

```
theorem PosSemidef.pos_of_mem_spectrum {A : Matrix d d  $\mathbb{k}$ } (hA : A.PosSemidef) (r :  $\mathbb{R}$ ) :
  r  $\in$  spectrum  $\mathbb{R}$  A  $\rightarrow 0 \leq r$ 
```

**theorem pow\_add**[\[source\]](#)

```
theorem PosSemidef.pow_add {A : Matrix d d  $\mathbb{k}$ } (hA : A.PosSemidef) {x y :  $\mathbb{R}$ } (hxy : x + y  $\neq$  0) :
  cfc ( $\cdot$  ^ (x + y) :  $\mathbb{R} \rightarrow \mathbb{R}$ ) A = cfc (fun r  $\mapsto$  r ^ x * r ^ y :  $\mathbb{R} \rightarrow \mathbb{R}$ ) A
```

**theorem pow\_mul**[\[source\]](#)

```
theorem PosSemidef.pow_mul {A : Matrix d d  $\mathbb{k}$ } {x y :  $\mathbb{R}$ } (hA : A.PosSemidef) :
  cfc ( $\cdot$  ^ (x * y) :  $\mathbb{R} \rightarrow \mathbb{R}$ ) A = cfc (fun r  $\mapsto$  (r ^ x) ^ y :  $\mathbb{R} \rightarrow \mathbb{R}$ ) A
```

**theorem trace\_submatrix**[\[source\]](#)

```
@[simp]
theorem trace_submatrix
  (A : Matrix d1 d1 α) (e : d2 ≈ d1) :
  (A.submatrix e e).trace = A.trace
```

**lemma spectrum\_prod\_complex**[\[source\]](#)

```
private lemma spectrum_prod_complex {d d2 : Type*}
  [Fintype d] [DecidableEq d] [Fintype d2] [DecidableEq d2]
  {A : Matrix d d ℚ} {B : Matrix d2 d2 ℚ}
  (hA : A.IsHermitian) (hB : B.IsHermitian) :
  ∀ x : ℚ, x ∈ spectrum ℚ (A ⊗ℚ B) → ∃ a ∈ spectrum ℚ A, ∃ b ∈ spectrum ℚ B, x = a * b
```

**lemma spectrum\_prod\_le**[\[source\]](#)

```
private lemma spectrum_prod_le {d d2 : Type*}
  [Fintype d] [DecidableEq d] [Fintype d2] [DecidableEq d2]
  {A : Matrix d d ℚ} {B : Matrix d2 d2 ℚ}
  (hA : A.IsHermitian) (hB : B.IsHermitian) :
  spectrum ℝ (A ⊗ℚ B) ≤ spectrum ℝ A * spectrum ℝ B
```

**theorem spectrum\_prod**[\[source\]](#)

```
theorem spectrum_prod {d d2 : Type*}
  [Fintype d] [DecidableEq d] [Fintype d2] [DecidableEq d2]
  {A : Matrix d d ℚ} {B : Matrix d2 d2 ℚ}
  (hA : A.IsHermitian) (hB : B.IsHermitian) :
  spectrum ℝ (A ⊗ℚ B) = spectrum ℝ A * spectrum ℝ B
```

**theorem zero\_lt**[\[source\]](#)

```
theorem PosDef.zero_lt {n : Type*} [Nonempty n] [Fintype n] {A : Matrix n n ℂ} (hA : A.PosDef)
  : 0 < A
```

**lemma spectrum\_eq\_image\_eigenvalues**[\[source\]](#)

```
lemma IsHermitian.spectrum_eq_image_eigenvalues [Fintype n] {A : Matrix n n ℂ} (hA : A.
  IsHermitian) :
  spectrum ℝ A = Finset.univ.image hA.eigenvalues
```

**lemma card\_spectrum\_eq\_image**[\[source\]](#)

```
lemma IsHermitian.card_spectrum_eq_image [Fintype n] {A : Matrix n n ℂ} (hA : A.IsHermitian)
  [Fintype (spectrum ℝ A)] :
  Fintype.card (spectrum ℝ A) = (Finset.univ.image hA.eigenvalues).card
```

**lemma sub\_iInf\_eigenvalues**[\[source\]](#)

```
lemma sub_iInf_eigenvalues (hA : A.IsHermitian) :
  (A - iInf hA.eigenvalues • 1).PosSemidef
```

**lemma iInf\_eigenvalues\_le\_dotProduct\_mulVec**[\[source\]](#)

```
lemma iInf_eigenvalues_le_dotProduct_mulVec (hA : A.IsHermitian) (v : d → ℂ) :
  iInf hA.eigenvalues * (star v ·v v) ≤ star v ·v A *v v
```

**lemma iInf\_eigenvalues\_le\_of\_posSemidef**[\[source\]](#)

```
lemma iInf_eigenvalues_le_of_posSemidef
  (hAB : (B - A).PosSemidef) (hA : A.IsHermitian) (hB : B.IsHermitian) :
  iInf hA.eigenvalues ≤ iInf hB.eigenvalues
```

**lemma iInf\_eigenvalues\_le**[\[source\]](#)

```
lemma iInf_eigenvalues_le (hAB : A ≤ B) (hA : A.IsHermitian) (hB : B.IsHermitian) :
  iInf hA.eigenvalues ≤ iInf hB.eigenvalues
```

**lemma iInf\_eigenvalues\_smul\_one\_le**[\[source\]](#)

```
lemma iInf_eigenvalues_smul_one_le (hA : A.IsHermitian) : iInf hA.eigenvalues • 1 ≤ A
```

**lemma \_shortcut\_posSMulMono**[\[source\]](#)

```
lemma _shortcut_posSMulMono : PosSMulMono ℝ (Matrix d d ℂ)
```

**lemma \_shortcut\_posSmulReflectLE**[\[source\]](#)

```
lemma _shortcut_posSmulReflectLE : PosSMulReflectLE ℝ (Matrix d d ℂ)
```

**theorem spectrum\_subset\_Ici\_of\_sub**[\[source\]](#)

```
theorem IsHermitian.spectrum_subset_Ici_of_sub {d ℤ : Type*} [Fintype d] [DecidableEq d] [
  RCLike ℤ]
  {A x : Matrix d d ℤ} (hA : A.IsHermitian) (hl : (x - A).PosSemidef) :
  spectrum ℝ x ⊆ Set.Ici (⌊ i, hA.eigenvalues i)
```

**theorem spectrum\_subset\_Iic\_of\_sub**[\[source\]](#)

```
theorem IsHermitian.spectrum_subset_Iic_of_sub {d ℤ : Type*} [Fintype d] [DecidableEq d] [
  RCLike ℤ]
  {A x : Matrix d d ℤ} (hA : A.IsHermitian) (hl : (A - x).PosSemidef) :
  spectrum ℝ x ⊆ Set.Iic (⌊ i, hA.eigenvalues i)
```

**theorem spectrum\_subset\_of\_mem\_Icc**[\[source\]](#)

```
theorem IsHermitian.spectrum_subset_of_mem_Icc {d ℤ : Type*} [Fintype d] [DecidableEq d] [
  RCLike ℤ]
  {A B x : Matrix d d ℤ} (hA : A.IsHermitian) (hB : B.IsHermitian)
  (hl : (x - A).PosSemidef) (hr : (B - x).PosSemidef) :
  spectrum ℝ x ⊆ Set.Icc (⌊ i, hA.eigenvalues i) (⌊ i, hB.eigenvalues i)
```

**theorem traceRight\_eq\_traceLeft\_reindex**[\[source\]](#)

*The right partial trace of a matrix is equal to the left partial trace of the matrix reindexed by swapping the tensor factors.*

```
theorem traceRight_eq_traceLeft_reindex {n m R : Type*} [Fintype m] [AddCommMonoid R]
  (M : Matrix (n × m) (n × m) R) :
  M.traceRight = (M.reindex (.prodComm ..) (.prodComm ..)).traceLeft
```

**theorem trace\_pos**[\[source\]](#)

```
theorem PosSemidef.trace_pos {n ℤ : Type*} [Fintype n] [RCLike ℤ]
  {A : Matrix n n ℤ} (hA : A.PosSemidef) (h : A ≠ 0) : 0 < A.trace
```

**theorem traceLeft\_add**[\[source\]](#)

```
@[simp]
theorem traceLeft_add : (A + B).traceLeft = A.traceLeft + B.traceLeft
```

**theorem traceLeft\_neg**[\[source\]](#)

```
@[simp]
theorem traceLeft_neg : (-A).traceLeft = -A.traceLeft
```

**theorem traceLeft\_sub**[\[source\]](#)

```
@[simp]
theorem traceLeft_sub : (A - B).traceLeft = A.traceLeft - B.traceLeft
```

**theorem traceRight\_add**[\[source\]](#)

```
@[simp]
theorem traceRight_add : (A + B).traceRight = A.traceRight + B.traceRight
```

**theorem traceRight\_neg**[\[source\]](#)

```
@[simp]
theorem traceRight_neg : (-A).traceRight = -A.traceRight
```

**theorem traceRight\_sub**[\[source\]](#)

```
@[simp]
theorem traceRight_sub : (A - B).traceRight = A.traceRight - B.traceRight
```

**theorem traceLeft\_smul**[\[source\]](#)

```
@[simp]
theorem traceLeft_smul {A : Matrix (m × n) (m × n) α} (r : R) :
  (r • A).traceLeft = r • A.traceLeft
```

**theorem traceRight\_smul**[\[source\]](#)

```
@[simp]
theorem traceRight_smul {A : Matrix (n × m) (n × m) α} (r : R) :
  (r • A).traceRight = r • A.traceRight
```

**theorem unitaryGroup\_row\_norm**[\[source\]](#)

```
theorem unitaryGroup_row_norm [Fintype n] (U : Matrix.unitaryGroup n ℂ) (i : n) :
  ∑ j, ‖U j i‖2 = 1
```

**def Matrix.piProd**[\[source\]](#)

```
def piProd [CommMonoid R] (A : ∀ i, Matrix (d i) (d i) R) : Matrix (∀ i, d i) (∀ i, d i) R
```

**theorem IsHermitian.piProd**[\[source\]](#)

```
theorem IsHermitian.piProd [CommSemiring R] [StarRing R] (hA : ∀ i, (A i).IsHermitian) :
  (piProd A).IsHermitian
```

**theorem trace\_piProd**[\[source\]](#)

```
theorem trace_piProd [CommSemiring R] :
  (piProd A).trace = ∏ i, (A i).trace
```

**theorem PosSemidef.piProd**[\[source\]](#)

```
theorem PosSemidef.piProd [RCLike R] (hA : ∀ i, (A i).PosSemidef) :
  (piProd A).PosSemidef
```

**theorem submatrix\_eq\_mul\_mul**[\[source\]](#)

```
theorem submatrix_eq_mul_mul {d d₂ d₃ R : Type*} [DecidableEq d] [Fintype d] [Semiring R]
  (A : Matrix d d R) (e : d₂ → d) (f : d₃ → d) :
  A.submatrix e f = (submatrix (α := R) 1 e id : Matrix d₂ d R) * A * (submatrix (α := R) 1
  id f)
```

**lemma kronecker\_conj\_eq**[\[source\]](#)

*The conjugate of a Kronecker product by a Kronecker product is the Kronecker product of the conjugates (for matrices).*

```
lemma kronecker_conj_eq {m n p q α : Type*} [CommSemiring α] [StarRing α] [Fintype m] [Fintype
  n]
  (A : Matrix m m α) (B : Matrix n n α) (C : Matrix p m α) (D : Matrix q n α) :
  (C ⊗k D) * (A ⊗k B) * (C ⊗k D)H = (C * A * CH) ⊗k (D * B * DH)
```

**2.76 QuantumInfo.ForMathlib.MatrixNorm.TraceNorm**

[QuantumInfo/ForMathlib/MatrixNorm/TraceNorm.lean](#) on GitHub.

**def traceNorm**[\[source\]](#)

*The trace norm of a matrix:  $\text{Tr}[\sqrt{(A^\dagger A)}]$ .*

```
def traceNorm (A : Matrix m n R) : ℝ
```

**theorem traceNorm\_zero**[\[source\]](#)

```
@[simp]
```

```
theorem traceNorm_zero : traceNorm (0 : Matrix m n R) = 0
```

**theorem traceNorm\_eq\_neg\_self**[\[source\]](#)

*The trace norm of the negative is equal to the trace norm.*

```
@[simp]
```

```
theorem traceNorm_eq_neg_self (A : Matrix m n R) : traceNorm (-A) = traceNorm A
```

**lemma cfc\_sqrt\_isometry\_conj**[\[source\]](#)

```
lemma cfc_sqrt_isometry_conj {A : Matrix n n R} (hA : 0 ≤ A)
  {u : Matrix m n R} (hu₁ : u.Isometry) :
  CFC.sqrt (u * A * uH) = u * CFC.sqrt A * uH
```

**theorem traceNorm\_isometry\_left**[\[source\]](#)

```
theorem traceNorm_isometry_left [Fintype k] {A : Matrix n m R} {u : Matrix k n R}
  (hu₁ : u.Isometry) : traceNorm (u * A) = traceNorm A
```

**theorem traceNorm\_isometry\_right**[\[source\]](#)

```
theorem traceNorm_isometry_right [Fintype k] {A : Matrix n m R} {u : Matrix k m R}
  (hu₁ : u.Isometry) : traceNorm (A * uH) = traceNorm A
```

**theorem traceNorm\_isometry\_conj**[\[source\]](#)

*The trace norm is invariant under isometries u and v, Property 9.1.4 in Wilde*

```
theorem traceNorm_isometry_conj {A : Matrix n n R} {u : Matrix m n R}
  (hu : u.Isometry) {v : Matrix m n R} (hv : v.Isometry) :
  traceNorm (u * A * vH) = traceNorm A
```

**theorem traceNorm\_unitary\_conj**[\[source\]](#)

```
@[simp]
```

```
theorem traceNorm_unitary_conj {A : Matrix n n R} {U : Matrix.unitaryGroup n R} :
  traceNorm (U.val * A * U.valH) = traceNorm A
```

**theorem traceNorm\_Hermitian\_eq\_sum\_abs\_eigenvalues**[\[source\]](#)

```
theorem traceNorm_Hermitian_eq_sum_abs_eigenvalues {A : Matrix n n R} (hA : A.IsHermitian) :
  A.traceNorm = ∑ i, abs (hA.eigenvalues i)
```

**theorem traceNorm\_nonneg**[\[source\]](#)

*The trace norm is nonnegative. Property 9.1.1 in Wilde*

```
theorem traceNorm_nonneg (A : Matrix m n R) : 0 ≤ A.traceNorm
```

**theorem traceNorm\_zero\_iff**[\[source\]](#)

*The trace norm is zero iff. the matrix is zero.*

```
theorem traceNorm_zero_iff (A : Matrix m n R) : A.traceNorm = 0 ↔ A = 0
```

**theorem traceNorm\_smul**[\[source\]](#)

*Trace norm is linear under scalar multiplication. Property 9.1.2 in Wilde*

```
theorem traceNorm_smul (A : Matrix m n R) (c : R) : (c • A).traceNorm = ||c|| * A.traceNorm
```

**theorem traceNorm\_eq\_max\_tr\_U**[\[source\]](#)

*For square matrices, the trace norm is  $\max \text{Tr}[U * A]$  over unitaries  $U$ .*

```
theorem traceNorm_eq_max_tr_U (A : Matrix n n R) : IsGreatest {x | ∃ (U : unitaryGroup n R), (U.  
.1 * A).trace = x} A.traceNorm
```

**theorem traceNorm\_triangleIneq**[\[source\]](#)

*the trace norm satisfies the triangle inequality (for square matrices). TODO: Prove in general.*

```
theorem traceNorm_triangleIneq (A B : Matrix n n R) : (A + B).traceNorm ≤ A.traceNorm + B.  
traceNorm
```

**theorem traceNorm\_triangleIneq'**[\[source\]](#)

```
theorem traceNorm_triangleIneq' (A B : Matrix n n R) : (A - B).traceNorm ≤ A.traceNorm + B.  
traceNorm
```

**theorem traceNorm\_PSD\_eq\_trace**[\[source\]](#)

```
theorem PosSemidef.traceNorm_PSD_eq_trace {A : Matrix m m R} (hA : A.PosSemidef) : A.traceNorm  
= A.trace
```

**theorem traceNorm\_convex**[\[source\]](#)

*The trace norm is convex. Property 9.1.5 in Wilde*

```
theorem traceNorm_convex (M N : Matrix n n R) (l : ℝ) (hl : 0 ≤ l ∧ l ≤ 1) :  
((l : R) • M + ((1 - l) : R) • N).traceNorm ≤ l * M.traceNorm + (1 - l) * N.traceNorm
```

**2.77 QuantumInfo.ForMathlib.Minimax**

[QuantumInfo/ForMathlib/Minimax.lean](#) on GitHub.



**theorem exists\_isMinOn\_lowerSemicontinuousOn**[\[source\]](#)

```

theorem _root_.IsCompact.exists_isMinOn_lowerSemicontinuousOn {α β : Type*}
  [LinearOrder α] [TopologicalSpace α] [TopologicalSpace β] [ClosedIicTopology α]
  {s : Set β} (hs : IsCompact s) (ne_s : s.Nonempty) {f : β → α} (hf : LowerSemicontinuousOn f
    s) :
  ∃ x ∈ s, IsMinOn f s x

```

**theorem quasilinearOn**[\[source\]](#)

```

@[fun_prop]
theorem LinearMap.quasilinearOn {E β ℤ : Type*} [Semiring ℤ] [PartialOrder ℤ]
  [AddCommMonoid E] [Module ℤ E]
  [PartialOrder β] [AddCommMonoid β] [IsOrderedAddMonoid β] [Module ℤ β] [PosSMulMono ℤ β]
  (f : E →ℤ β) {s : Set E} (hs : Convex ℤ s) :
  QuasilinearOn ℤ s f

```

**theorem quasiconvexOn**[\[source\]](#)

```

@[fun_prop]
theorem LinearMap.quasiconvexOn {E β ℤ : Type*} [Semiring ℤ] [PartialOrder ℤ]
  [AddCommMonoid E] [Module ℤ E]
  [PartialOrder β] [AddCommMonoid β] [IsOrderedAddMonoid β] [Module ℤ β] [PosSMulMono ℤ β]
  (f : E →ℤ β) {s : Set E} (hs : Convex ℤ s) :
  QuasiconvexOn ℤ s f

```

**theorem quasiconcaveOn**[\[source\]](#)

```

@[fun_prop]
theorem LinearMap.quasiconcaveOn {E β ℤ : Type*} [Semiring ℤ] [PartialOrder ℤ]
  [AddCommMonoid E] [Module ℤ E]
  [PartialOrder β] [AddCommMonoid β] [IsOrderedAddMonoid β] [Module ℤ β] [PosSMulMono ℤ β]
  (f : E →ℤ β) {s : Set E} (hs : Convex ℤ s) :
  QuasiconcaveOn ℤ s f

```

**theorem**[\[source\]](#)

```

theorem continuous_stupid.{u_2, u_1} {M : Type u_1} [inst : NormedAddCommGroup M] [inst_1 :
  Module ℝ M]
  [inst_3 : ContinuousSMul ℝ M] {N : Type u_2} [inst_4 : NormedAddCommGroup N]
  [inst_5 : Module ℝ N]
  [FiniteDimensional ℝ M]
  (f : N →L[ℝ] M →L[ℝ] ℝ) :
  Continuous fun (x : N × M) ↦ (f x.1) x.2

```

**theorem minimax**[\[source\]](#)

*The minimax theorem, at the level of generality we need. For convex, compact, nonempty sets  $S$  and  $T$  in a real topological vector space  $M$ , and a bilinear function  $f$  on  $M$ , we can exchange the order of minimizing and maximizing.*

```

theorem minimax
  {M : Type*} [NormedAddCommGroup M] [Module ℝ M] [ContinuousAdd M] [ContinuousSMul ℝ M] [
    FiniteDimensional ℝ M]
  {N : Type*} [NormedAddCommGroup N] [Module ℝ N] [ContinuousAdd N] [ContinuousSMul ℝ N]

```

```
(f : N → L[ℝ] M → L[ℝ] ℝ)
(S : Set M) (T : Set N) (hS1 : IsCompact S) (hT1 : IsCompact T)
(hS2 : Convex ℝ S) (hT2 : Convex ℝ T) (hS3 : S.Nonempty) (hT3 : T.Nonempty)
:  $\bigcap x : T, \bigcup y : S, f x y = \bigcup y : S, \bigcap x : T, f x y$ 
```

**theorem BilinForm.minimax**

[\[source\]](#)

*\*\*Von-Neumann's Minimax Theorem\*\*, specialized to bilinear forms.*

```
theorem LinearMap.BilinForm.minimax
{M : Type*} [NormedAddCommGroup M] [Module ℝ M] [ContinuousAdd M] [ContinuousSMul ℝ M] [
  FiniteDimensional ℝ M]
(f : LinearMap.BilinForm ℝ M)
(S : Set M) (T : Set M) (hS1 : IsCompact S) (hT1 : IsCompact T)
(hS2 : Convex ℝ S) (hT2 : Convex ℝ T) (hS3 : S.Nonempty) (hT3 : T.Nonempty)
:  $\bigcap x : T, \bigcup y : S, f x y = \bigcup y : S, \bigcap x : T, f x y$ 
```

**theorem minimax'**

[\[source\]](#)

*Convenience form of `LinearMap.BilinForm.minimax` with the order *inf/sup* arguments supplied to *f* flipped.*

```
theorem LinearMap.BilinForm.minimax'
{M : Type*} [NormedAddCommGroup M] [Module ℝ M] [ContinuousAdd M] [ContinuousSMul ℝ M] [
  FiniteDimensional ℝ M]
(f : LinearMap.BilinForm ℝ M)
(S : Set M) (T : Set M) (hS1 : IsCompact S) (hT1 : IsCompact T)
(hS2 : Convex ℝ S) (hT2 : Convex ℝ T) (hS3 : S.Nonempty) (hT3 : T.Nonempty)
:  $\bigcup x : S, \bigcap y : T, f x y = \bigcap y : T, \bigcup x : S, f x y$ 
```

## 2.78 QuantumInfo.ForMathlib.Misc

[QuantumInfo/ForMathlib/Misc.lean](#) on GitHub.

**theorem ite\_eq\_top**

[\[source\]](#)

```
@[aesop (rule_sets := [finiteness]) unsafe apply]
theorem ite_eq_top {α : Type*} [Top α] (h : Prop) [Decidable h] {x y : α} (hx : x ≠ ⊤) (hy : y
  ≠ ⊤) :
  (if h then x else y) ≠ ⊤
```

**theorem subtype\_val\_iSup**

[\[source\]](#)

```
theorem subtype_val_iSup (h : ∀ i, f i ∈ Set.Icc a b) :
  (⋂ i, ((f i, h i) : ↑(Set.Icc a b))).val = ⋂ i, f i
```

**theorem subtype\_val\_iSup'**

[\[source\]](#)

```
theorem subtype_val_iSup' (h : ∀ i, f i ∈ Set.Icc a b) :
  ⋂ i, ((f i, h i) : ↑(Set.Icc a b)) =
  (⋂ i, f i, ((h i.some).1.trans (le_ciSup {b, by intro; grind} _), ciSup_le (h · |>.2)))
```

**theorem subtype\_val\_iInf**[\[source\]](#)

```
theorem subtype_val_iInf (h : ∀ i, f i ∈ Set.Icc a b) :
  (⋂ i, ((f i, h i) : ↑(Set.Icc a b))).val = ⋂ i, f i
```

**theorem subtype\_val\_iInf'**[\[source\]](#)

```
theorem subtype_val_iInf' (h : ∀ i, f i ∈ Set.Icc a b) :
  ⋂ i, ((f i, h i) : ↑(Set.Icc a b)) =
  { ⋂ i, f i, (le_ciInf (h ·|>.1), (ciInf_le (a, by intro; grind) _).trans (h i.some).2) }
```

**lemma bdd\_le\_mul\_tendsto\_zero**[\[source\]](#)

*Analogous to bdd\_le\_mul\_tendsto\_zero, for ENNReal (which otherwise lacks a continuous multiplication function). The product of a sequence that tends to zero with any bounded sequence also tends to zero.*

```
protected lemma ENNReal.bdd_le_mul_tendsto_zero
  {α : Type*} {l : Filter α} {f g : α → ℝ≥0∞} {b : ℝ≥0∞} (hb : b ≠ ⊤)
  (hf : l.Tendsto f (ℕ 0)) (hg : ∀f (x : α) in l, g x ≤ b) :
  l.Tendsto (fun x ↦ f x * g x) (ℕ 0)
```

**theorem csInf\_mul\_nonneg**[\[source\]](#)

```
theorem csInf_mul_nonneg {s t : Set ℝ}
  (hs₀ : s.Nonempty) (hs₁ : ∀ x ∈ s, 0 ≤ x) (ht₀ : t.Nonempty) (ht₁ : ∀ x ∈ t, 0 ≤ x) :
  sInf (s * t) = sInf s * sInf t
```

**lemma map\_univ\_eq\_iff**[\[source\]](#)

*If two functions from finite types have the same multiset of values, there exists a bijection between the domains that commutes with the functions.*

```
lemma Multiset.map_univ_eq_iff {α β : Type*} [Fintype α] (f g : α → β) :
  Multiset.map f Finset.univ.val = Multiset.map g Finset.univ.val ↔ ∃ (e : α ≈ α), f = g ∘ e
```

**lemma exists\_equiv\_of\_multiset\_map\_eq**[\[source\]](#)

*If two functions from finite types have the same multiset of values, there exists a bijection between the domains that commutes with the functions.*

```
lemma exists_equiv_of_multiset_map_eq {α β γ : Type*} [Fintype α] [Fintype β] [DecidableEq γ]
  (f : α → γ) (g : β → γ) (h : Multiset.map f Finset.univ.val = Multiset.map g Finset.univ.val) :
  ∃ e : α ≈ β, f = g ∘ e
```

**2.79 QuantumInfo.ForMathlib.SionMinimax**

[QuantumInfo/ForMathlib/SionMinimax.lean](#) on GitHub.

**theorem image2\_flip**[\[source\]](#)

```
@[simp]
theorem Set.image2_flip {α β γ : Type*} {f : α → β → γ} (s : Set α) (t : Set β) :
```

```
image2 (flip f) t s = image2 f s t
```

### theorem ciSup\_ciInf\_le\_ciInf\_ciSup

[\[source\]](#)

The *\*\*max-min theorem\*\**. A version of *iSup\_iInf\_le\_iInf\_iSup* for conditionally complete lattices.

```
theorem ciSup_ciInf_le_ciInf_ciSup {ι' : Type*} [Nonempty ι]
  (f : ι → ι' → α) (Ha : ∀ j, BddAbove (Set.range (f · j))) (Hb : ∀ i, BddBelow (Set.range (f i
  ))) :
  ⋒ i, ⋓ j, f i j ≤ ⋓ j, ⋒ i, f i j
```

### theorem range\_max

[\[source\]](#)

```
theorem BddAbove.range_max (hf : BddAbove (Set.range f)) (hg : BddAbove (Set.range g)) :
  BddAbove (Set.range (max f g))
```

### theorem range\_min

[\[source\]](#)

```
theorem BddBelow.range_min (hf : BddBelow (Set.range f)) (hg : BddBelow (Set.range g)) :
  BddBelow (Set.range (min f g))
```

### theorem ciInf\_eq\_min\_cInf\_inter\_diff

[\[source\]](#)

```
theorem ciInf_eq_min_cInf_inter_diff (S T : Set ι)
  [Nonempty (S ∩ T : Set ι)] [Nonempty (S \ T : Set ι)] (hf : BddBelow (f '' S)) :
  ⋒ i : S, f i = (⋒ i : (S ∩ T : Set ι), f i) ⋓ ⋒ i : (S \ T : Set ι), f i
```

### theorem lt\_ciInf\_iff

[\[source\]](#)

```
theorem lt_ciInf_iff (hf : BddBelow (Set.range f)) :
  a < iInf f ↔ ∃ b, a < b ∧ ∀ (i : ι), b ≤ f i
```

### theorem ciSup\_sup\_eq

[\[source\]](#)

```
theorem ciSup_sup_eq (hf : BddAbove (Set.range f)) (hg : BddAbove (Set.range g)) : ⋒ x, f x ⋒
  g x = (⋒ x, f x) ⋒ ⋒ x, g x
```

### theorem ciInf\_inf\_eq

[\[source\]](#)

```
theorem ciInf_inf_eq (hf : BddBelow (Set.range f)) (hg : BddBelow (Set.range g)) : ⋓ x, f x
  ⋓ g x = (⋓ x, f x) ⋓ ⋓ x, g x
```

### theorem sup\_ciSup

[\[source\]](#)

```
theorem sup_ciSup (hf : BddAbove (Set.range f)) : a ⋒ ⋒ x, f x = ⋒ x, a ⋒ f x
```

### theorem inf\_ciInf

[\[source\]](#)

```
theorem inf_ciInf (hf : BddBelow (Set.range f)) : a ⋓ ⋓ x, f x = ⋓ x, a ⋓ f x
```

**theorem ciInf\_sup\_ciInf\_le**[\[source\]](#)

```
theorem ciInf_sup_ciInf_le (hf : BddBelow (Set.range f)) (hg : BddBelow (Set.range g)) :
  (⋂ i, f i) ⊔ ⋂ i, g i ≤ ⋂ i, f i ⊔ g i
```

**theorem le\_ciSup\_inf\_ciSup**[\[source\]](#)

```
theorem le_ciSup_inf_ciSup (hf : BddAbove (Set.range f)) (hg : BddAbove (Set.range g)) :
  ⋃ (i : ι), f i ⊓ g i ≤ (⋃ (i : ι), f i) ⊓ ⋃ (i : ι), g i
```

**theorem subset**[\[source\]](#)

```
theorem Quasiconvex0n.subset
  {ℓ E β : Type*} [Semiring ℓ] [PartialOrder ℓ] [AddCommMonoid E] [LE β] [SMul ℓ E]
  {s : Set E} {f : E → β} (h : Quasiconvex0n ℓ s f) {t : Set E} (hts : t ⊆ s) (ht : Convex ℓ t)
  :
  Quasiconvex0n ℓ t f
```

**theorem mem\_segment\_le\_max**[\[source\]](#)

```
theorem Quasiconvex0n.mem_segment_le_max
  {ℓ E β : Type*} [Semiring ℓ] [PartialOrder ℓ] [AddCommMonoid E] [SemilatticeSup β] [SMul ℓ E]
  {s : Set E} {f : E → β} (h : Quasiconvex0n ℓ s f)
  {x y z : E} (hx : x ∈ s) (hy : y ∈ s) (hz : z ∈ segment ℓ x y):
  f z ≤ f x ⊔ f y
```

**theorem min\_le\_mem\_segment**[\[source\]](#)

```
theorem Quasiconcave0n.min_le_mem_segment
  {ℓ E β : Type*} [Semiring ℓ] [PartialOrder ℓ] [AddCommMonoid E] [SemilatticeInf β] [SMul ℓ E]
  {s : Set E} {f : E → β} (h : Quasiconcave0n ℓ s f)
  {x y z : E} (hx : x ∈ s) (hy : y ∈ s) (hz : z ∈ segment ℓ x y):
  f x ⊓ f y ≤ f z
```

**theorem bddBelow**[\[source\]](#)

```
theorem LowerSemicontinuous0n.bddBelow {α : Type*} [TopologicalSpace α] {S : Set α} {g : α → ℝ}
  (hg : LowerSemicontinuous0n g S) (hS : IsCompact S) : BddBelow (g '' S)
```

**theorem max**[\[source\]](#)

```
theorem LowerSemicontinuous0n.max {α : Type*} [TopologicalSpace α] {S : Set α} {f g : α → ℝ}
  (hf : LowerSemicontinuous0n f S) (hg : LowerSemicontinuous0n g S) :
  LowerSemicontinuous0n (fun x ↦ max (f x) (g x)) S
```

**theorem lowerSemicontinuous0n\_iff\_isClosed\_preimage**[\[source\]](#)

```
theorem lowerSemicontinuous0n_iff_isClosed_preimage {f : α → γ} [IsClosed s] :
  LowerSemicontinuous0n f s ↔ ∀ y, IsClosed (s ∩ f-1 (Set.Iic y))
```

**theorem isConnected**[\[source\]](#)

```
theorem segment.isConnected {E : Type u_1} [AddCommGroup E] [Module ℝ E] [TopologicalSpace E] [
  ContinuousAdd E] [ContinuousSMul ℝ E] (a b : E) :
  IsConnected (segment ℝ a b)
```

**theorem range\_inf\_of\_image2**[\[source\]](#)

```
theorem BddAbove.range_inf_of_image2 {M N α : Type*} {f : M → N → α} [
  ConditionallyCompleteLinearOrder α]
  {S : Set M} {T : Set N} (h_bddA : BddAbove (Set.image2 f S T)) (h_bddB : BddBelow (Set.image2
    f S T)) :
  BddAbove (Set.range fun y : T ↦  $\bigcap$  x : S, f x y)
```

**theorem range\_sup\_of\_image2**[\[source\]](#)

```
theorem BddBelow.range_sup_of_image2 {M N α : Type*} {f : M → N → α} [
  ConditionallyCompleteLinearOrder α]
  {S : Set M} {T : Set N} (h_bddA : BddAbove (Set.image2 f S T)) (h_bddB : BddBelow (Set.image2
    f S T)) :
  BddBelow (Set.range fun y : T ↦  $\bigcup$  x : S, f x y)
```

**theorem ciInf\_le\_ciInf\_of\_subset**[\[source\]](#)

```
theorem ciInf_le_ciInf_of_subset {α β : Type*} [ConditionallyCompleteLattice α]
  {f : β → α} {s t : Set β} (hs : s.Nonempty) (hf : BddBelow (f '' t)) (hst : s ⊆ t) :
   $\bigcap$  x : t, f x ≤  $\bigcap$  x : s, f x
```

**theorem dite\_top**[\[source\]](#)

```
theorem LowerSemicontinuous0n.dite_top {α β : Type*} [TopologicalSpace α] [Preorder β] [
  OrderTop β]
  {s : Set α} (p : α → Prop) [DecidablePred p] {f : (a : α) → p a → β}
  (hf : LowerSemicontinuous0n (fun x : Subtype p ↦ f x.val x.prop)) {x | x.val ∈ s}
  (h_relatively_closed : ∃ U : Set α, IsClosed U ∧ s ∩ U = s ∩ setOf p) :
  LowerSemicontinuous0n (fun x ↦ dite (p x) (f x) (fun _ ↦ T)) s
```

**theorem LowerSemicontinuous0n.comp\_continuous0n**[\[source\]](#)

```
theorem LowerSemicontinuous0n.comp_continuous0n {α β γ : Type*}
  [TopologicalSpace α] [TopologicalSpace β] [Preorder γ] {f : α → β} {s : Set α} {g : β → γ} {t
    : Set β}
  (hg : LowerSemicontinuous0n g t) (hf : Continuous0n f s) (h : Set.MapsTo f s t) :
  LowerSemicontinuous0n (g ∘ f) s
```

**theorem UpperSemicontinuous0n.comp\_continuous0n**[\[source\]](#)

```
theorem UpperSemicontinuous0n.comp_continuous0n {α β γ : Type*}
  [TopologicalSpace α] [TopologicalSpace β] [Preorder γ] {f : α → β} {s : Set α} {g : β → γ} {t
    : Set β}
  (hg : UpperSemicontinuous0n g t) (hf : Continuous0n f s) (h : Set.MapsTo f s t) :
  UpperSemicontinuous0n (g ∘ f) s
```

**theorem ite\_top**[\[source\]](#)

```
theorem LowerSemicontinuous0n.ite_top {α β : Type*} [TopologicalSpace α] [Preorder β] [OrderTop β]
  {s : Set α} (p : α → Prop) [DecidablePred p] {f : (a : α) → β} (hf : LowerSemicontinuous0n f
    (s n setOf p))
  (h_relatively_closed : ∃ U : Set α, IsClosed U ∧ s n U = s n setOf p) :
  LowerSemicontinuous0n (fun x ↦ ite (p x) (f x) T) s
```

**theorem comp\_lowerSemicontinuous0n\_strong\_assumptions**[\[source\]](#)

```
theorem LeftOrdContinuous.comp_lowerSemicontinuous0n_strong_assumptions {α γ δ : Type*}
  [TopologicalSpace α] [LinearOrder γ] [LinearOrder δ] [TopologicalSpace δ] [OrderTopology δ]
  [TopologicalSpace γ] [OrderTopology γ] [DenselyOrdered γ] [DenselyOrdered δ]
  {s : Set α} {g : γ → δ} {f : α → γ} (hg : LeftOrdContinuous g) (hf : LowerSemicontinuous0n f
    s) (hg2 : Monotone g) :
  LowerSemicontinuous0n (g ∘ f) s
```

**theorem frequently\_lt\_of\_tendsto**[\[source\]](#)

```
theorem UpperSemicontinuous0n.frequently_lt_of_tendsto {α β γ : Type*} [TopologicalSpace β] [
  Preorder γ]
  {f : β → γ} {T : Set β} (hf : UpperSemicontinuous0n f T) {c : γ} {zs : α → β} {z : β}
  {l : Filter α} [l.NeBot] (hzs : l.Tendsto zs (nhds z)) (hx₂ : f z < c) (hzI : ∀ a, zs a ∈ T)
  (hzT : z ∈ T) :
  ∀f a in l, f (zs a) < c
```

**theorem ciInf\_insert**[\[source\]](#)

```
theorem Finset.ciInf_insert {α β : Type*} [DecidableEq α] [ConditionallyCompleteLattice β]
  (t : Finset α) (ht : t.Nonempty) (x : α) (f : α → β) :
  ⌊ (a : (insert x t : _)), f a = f x ⌋ ⌊ (a : t), f a
```

**theorem ciSup\_insert**[\[source\]](#)

```
theorem Finset.ciSup_insert {α β : Type*} [DecidableEq α] [ConditionallyCompleteLattice β]
  (t : Finset α) (ht : t.Nonempty) (x : α) (f : α → β) :
  ⌈ (a : (insert x t : _)), f a = f x ⌈ ⌈ (a : t), f a
```

**theorem sion\_exists\_min\_lowerSemi**[\[source\]](#)

```
private theorem sion_exists_min_lowerSemi (a : ℝ) (hc : ∀ y₀ : T, ⌊ (x : S), f (↑x) y₀ ≤ a) (
  z : N) (hzT : z ∈ T) :
  ∃ x ∈ S, f x z ≤ a
```

**lemma sion\_exists\_min\_2**[\[source\]](#)

```
private lemma sion_exists_min_2 (y₁ y₂ : N) (hy₁ : y₁ ∈ T) (hy₂ : y₂ ∈ T)
  (a : ℝ) (ha : a < ⌊ x : S, (max (f x y₁) (f x y₂)))
  : ∃ y₀ : T, a < ⌊ x : S, f x y₀
```

**lemma sion\_exists\_min\_fin**[\[source\]](#)

```
private lemma sion_exists_min_fin
  (h_bddA : BddAbove (Set.image2 f S T)) (h_bddB : BddBelow (Set.image2 f S T))
  (ys : Finset N) (hys_n : ys.Nonempty) (hys : (ys : Set N) ⊆ T)
  (a : ℝ) (ha : a < ⋂ x : S, ⋂ yi : ys, f x yi)
  : ∃ y₀ : T, a < ⋂ x : S, f x y₀
```

**theorem sion\_minimax**[\[source\]](#)

*\*\*Sion's Minimax theorem\*\*. Because of `ciSup` and `ciInf` junk values when `f` isn't bounded, we need to assume that it's bounded above and below.*

```
theorem sion_minimax
  (h_bddA : BddAbove (Set.image2 f S T))
  (h_bddB : BddBelow (Set.image2 f S T))
  : ⋂ x : S, ⋂ y : T, f x y = ⋂ y : T, ⋂ x : S, f x y
```

**2.80 QuantumInfo.ForMathlib.Superadditive**

[QuantumInfo/ForMathlib/Superadditive.lean](#) on GitHub.

**def Superadditive**[\[source\]](#)

```
def Superadditive (u : ℕ → ℝ) : Prop
```

**theorem to\_Subadditive**[\[source\]](#)

```
theorem to_Subadditive : Subadditive (-u ·)
```

**def lim**[\[source\]](#)

```
noncomputable def lim (_h : Superadditive u)
```

**theorem tendsto\_lim**[\[source\]](#)

*Fekete's lemma for superadditive sequences: a superadditive sequence which is bounded above converges.*

```
theorem tendsto_lim (hbdd : BddAbove (Set.range fun n => u n / n)) :
  Filter.Tendsto (fun n => u n / n) .atTop (nhds (h.lim))
```

**theorem to\_Superadditive**[\[source\]](#)

```
theorem to_Superadditive : Superadditive (-u ·)
```

**2.81 QuantumInfo.ForMathlib.ULift**

[QuantumInfo/ForMathlib/ULift.lean](#) on GitHub.

**instance ulift**[\[source\]](#)

```
instance AddZero.ulift {α : Type u} [AddZero α] : AddZero (ULift.{v, u} α) where
```



**instance instStar**[\[source\]](#)

```
instance ULift.instStar {ℓ : Type u} [Star ℓ] : Star (ULift.{v,u} ℓ) where
```

**theorem star\_eq**[\[source\]](#)

```
@[simp]
theorem ULift.star_eq {ℓ : Type u} [Star ℓ] (x : ULift.{v,u} ℓ) : star x = .up (star x.down)
```

**instance instInvolutiveStar**[\[source\]](#)

```
instance ULift.instInvolutiveStar {ℓ : Type u} [InvolutiveStar ℓ] : InvolutiveStar (ULift.{v,u} ℓ) where
```

**instance instStarMul**[\[source\]](#)

```
instance ULift.instStarMul {ℓ : Type u} [Mul ℓ] [StarMul ℓ] : StarMul (ULift.{v,u} ℓ) where
```

**instance StarRing (ULift.{v,u} ℓ)**[\[source\]](#)

```
instance {ℓ : Type u} [NonUnitalNonAssocSemiring ℓ] [StarRing ℓ] : StarRing (ULift.{v,u} ℓ)
where
```

**theorem starRingEnd\_down**[\[source\]](#)

```
@[simp]
theorem ULift.starRingEnd_down {ℓ : Type u} (x : ULift.{v,u} ℓ) [CommSemiring ℓ] [StarRing ℓ] :
  ((starRingEnd (ULift.{v, u} ℓ)) x).down = star x.down
```

**instance NormedField (ULift.{v,u} ℓ)**[\[source\]](#)

```
instance {ℓ : Type u} [NormedField ℓ] : NormedField (ULift.{v,u} ℓ) where
```

**instance DenselyNormedField (ULift.{v,u} ℓ)**[\[source\]](#)

```
instance {ℓ : Type u} [DenselyNormedField ℓ] : DenselyNormedField (ULift.{v,u} ℓ) where
```

**theorem ulift\_apply**[\[source\]](#)

```
@[simp]
theorem AddEquiv.ulift_apply {α : Type u} [Add α] (x : ULift.{v, u} α) :
  AddEquiv.ulift.{u, v} x = x.down
```

**instance RCLike (ULift.{v,u} ℓ)**[\[source\]](#)

```
noncomputable instance {ℓ : Type u} [RCLike ℓ] : RCLike (ULift.{v,u} ℓ) where
```

**2.82 QuantumInfo.ForMathlib.Unitary**

[QuantumInfo/ForMathlib/Unitary.lean](#) on GitHub.

**theorem unitary\_star\_apply\_eq**[\[source\]](#)

```
@[simp]
theorem unitary_star_apply_eq (U : unitary (E →l[ $\mathbb{K}$ ] E)) (v : E) :
  (star U.val) (U.val v) = v
```

**theorem unitary\_apply\_star\_eq**[\[source\]](#)

```
@[simp]
theorem unitary_apply_star_eq (U : unitary (E →l[ $\mathbb{K}$ ] E)) (v : E) :
  U.val ((star U.val) v) = v
```

**def conj\_unitary\_eigenspace\_equiv**[\[source\]](#)

*Conjugating a linear map by a unitary operator gives a map whose  $\mu$ -eigenspace is isomorphic (same dimension) as those of the original linear map.*

```
noncomputable def conj_unitary_eigenspace_equiv (T : E →l[ $\mathbb{K}$ ] E) (U : unitary (E →l[ $\mathbb{K}$ ] E)) ( $\mu$  :  $\mathbb{K}$ ) :
  eigenspace T  $\mu$  ≈l[ $\mathbb{K}$ ] eigenspace (U.val * T * star (U.val))  $\mu$  where
```

**theorem conj\_unitary\_IsSymmetric**[\[source\]](#)

*A symmetric operator conjugated by a unitary is symmetric.*

```
theorem conj_unitary_IsSymmetric (U : unitary (E →l[ $\mathbb{K}$ ] E)) (hT : T.IsSymmetric) :
  (U.val * T * star U.val).IsSymmetric
```

**def conj\_unitary\_eigenvalue\_equiv**[\[source\]](#)

*There is an equivalence between the eigenvalues of a finite dimensional symmetric operator, and the eigenvalues of that operator conjugated by a unitary.*

```
def conj_unitary_eigenvalue_equiv (U : unitary (E →l[ $\mathbb{K}$ ] E)) (hT : T.IsSymmetric) :
  {  $\sigma$  : Equiv.Perm (Fin n) // (hT.conj_unitary_IsSymmetric U).eigenvalues hn = hT.eigenvalues hn ∘  $\sigma$  }
```

**2.83 QuantumInfo.Regularized**

[QuantumInfo/Regularized.lean](#) on GitHub.

**def InfRegularized**[\[source\]](#)

*An InfRegularized value is the  $\limsup$  of value at each natural number, but requires a proof of lower- and upper-bounds to be defined.*

```
noncomputable def InfRegularized (fn :  $\mathbb{N}$  → T) {lb ub : T}
  ( $\_$  :  $\forall$  n, lb ≤ fn n) ( $\_$  :  $\forall$  n, fn n ≤ ub) : T
```

**def SupRegularized**[\[source\]](#)

*A SupRegularized value is the  $\limsup$  of value at each natural number, but requires a proof of lower- and upper-bounds to be defined.*

```

noncomputable def SupRegularized (fn :  $\mathbb{N} \rightarrow T$ ) {lb ub : T}
  (_ :  $\forall n, lb \leq fn\ n$ ) (_ :  $\forall n, fn\ n \leq ub$ ) : T

```

### **theorem InfRegularized.lb**

[\[source\]](#)

*The InfRegularized value is also lower bounded.*

```

theorem lb : _lb  $\leq$  InfRegularized fn hl hu

```

### **theorem InfRegularized.ub**

[\[source\]](#)

*The InfRegularized value is also upper bounded.*

```

theorem ub : InfRegularized fn hl hu  $\leq$  _ub

```

### **theorem anti\_inf**

[\[source\]](#)

*For Antitone functions, the InfRegularized is the supremum of values.*

```

theorem anti_inf (h : Antitone fn) :
  InfRegularized fn hl hu = sInf (Set.range fn)

```

### **theorem anti\_ub**

[\[source\]](#)

*For Antitone functions, the InfRegularized is lower bounded by any particular value.*

```

theorem anti_ub (h : Antitone fn) :  $\forall n, \text{InfRegularized fn hl hu} \leq fn\ n$ 

```

### **theorem SupRegularized.lb**

[\[source\]](#)

*The SupRegularized value is also lower bounded.*

```

theorem lb : _lb  $\leq$  SupRegularized fn hl hu

```

### **theorem SupRegularized.ub**

[\[source\]](#)

*The SupRegularized value is also upper bounded.*

```

theorem ub : SupRegularized fn hl hu  $\leq$  _ub

```

### **theorem mono\_sup**

[\[source\]](#)

*For Monotone functions, the SupRegularized is the supremum of values.*

```

theorem mono_sup (h : Monotone fn) :
  SupRegularized fn hl hu = sSup { fn n | n :  $\mathbb{N}$  }

```

### **theorem mono\_lb**

[\[source\]](#)

*For Monotone functions, the SupRegularized is lower bounded by any particular value.*

```

theorem mono_lb (h : Monotone fn) :  $\forall n, fn\ n \leq \text{SupRegularized fn hl hu}$ 

```

**theorem to\_SupRegularized**[\[source\]](#)

```
theorem InfRegularized.to_SupRegularized : InfRegularized fn hl hu = -SupRegularized (-fn ·)
  (lb := -_ub) (ub := -_lb) (neg_le_neg_iff.mpr <| hu ·) (neg_le_neg_iff.mpr <| hl ·)
```

**theorem to\_InfRegularized**[\[source\]](#)

```
theorem SupRegularized.to_InfRegularized : SupRegularized fn hl hu = -InfRegularized (-fn ·)
  (lb := -_ub) (ub := -_lb) (neg_le_neg_iff.mpr <| hu ·) (neg_le_neg_iff.mpr <| hl ·)
```

**theorem anti\_tendsto**[\[source\]](#)

*For Antitone functions, the value `Filter.Tendsto` the `InfRegularized` value.*

```
theorem InfRegularized.anti_tendsto (h : Antitone fn) :
  Filter.Tendsto fn .atTop (nhds (InfRegularized fn hl hu))
```

**theorem of\_Subadditive**[\[source\]](#)

```
theorem InfRegularized.of_Subadditive (hf : Subadditive (fun n ↦ fn n * n))
  :
  hf.lim = InfRegularized fn hl hu
```

**2.84 QuantumInfo.StatMech.Hamiltonian**

[QuantumInfo/StatMech/Hamiltonian.lean](#) on GitHub.

**structure MicroHamiltonian**[\[source\]](#)

*The Hamiltonian for a microcanonical ensemble, parameterized by any extrinsic parameters of data  $D$ . Since it's an arbitrary type, 'T' would be nice, but we use 'D' for data to avoid confusion with temperature. We define Hamiltonians as an energy function on a real manifold, whose dimension  $n$  possibly depends on the data  $D$ . Energy is a `WithTop  $\mathbb{R}$` ,  $\top$  is used to mean "excluded as a possibility in phase space". This formalization does exclude the possibility of discrete degrees of freedom; it is not completely general. A better formalization could have an arbitrary measurable space.*

```
structure MicroHamiltonian (D : Type) where
  dim : D → Type
  [dim_fin : ∀ d, Fintype (dim d)]
  H : {d : D} → (dim d → ℝ) → WithTop ℝ
```

**instance microHamiltonianFintype**[\[source\]](#)

*We add the `dim_fin` to the instance cache so that things like the measure can be synthesized*

```
instance microHamiltonianFintype (H : MicroHamiltonian D) (d : D) : Fintype (H.dim d)
```

**abbrev NVEHamiltonian**[\[source\]](#)

*The standard microcanonical ensemble Hamiltonian, where the data is the particle number  $N$  and the volume  $V$ .*

**abbrev** NVEHamiltonian

**def** N [source]

*Helper to get the number in an N-V Hamiltonian*

**def** NVEHamiltonian.N :  $(\mathbb{N} \times \mathbb{R}) \rightarrow \mathbb{N}$

**def** V [source]

*Helper to get the volume in an N-V Hamiltonian*

**def** NVEHamiltonian.V :  $(\mathbb{N} \times \mathbb{R}) \rightarrow \mathbb{R}$

## 2.85 QuantumInfo.StatMech.IdealGas

[QuantumInfo/StatMech/IdealGas.lean](#) on GitHub.

**def** IdealGas [source]

*The Hamiltonian for an ideal gas: particles live in a cube of volume  $V^{1/3}$ , and each contributes an energy  $p^2/2$ . The per-particle mass is normalized to 1.*

**def** IdealGas : NVEHamiltonian **where**

**theorem** PartitionZ\_eq [source]

*The partition function  $Z$  for an ideal gas.*

**theorem** PartitionZ\_eq (hV :  $0 < V$ ) (hβ :  $0 < \beta$ ) :  
IdealGas.PartitionZ (n,V) β =  $V^n * (2 * \text{Real.pi} / \beta)^{(3 * n / 2 : \mathbb{R})}$

**theorem** HelmholtzA\_eq [source]

*The Helmholtz Free Energy  $A$  for an ideal gas.*

**theorem** HelmholtzA\_eq (hV :  $0 < V$ ) (hT :  $0 < T$ ) : IdealGas.HelmholtzA (n,V) T =  
 $-n * T * (\text{Real.log } V + (3/2) * \text{Real.log } (2 * \text{Real.pi} * T))$

**theorem** ZIntegrable [source]

**theorem** ZIntegrable (hV :  $0 < V$ ) (hβ :  $0 < \beta$ ) : IdealGas.ZIntegrable (n,V) β

**theorem** IdealGasLaw [source]

*The ideal gas law:  $PV = nRT$ . In our unitsless system,  $R = 1$ .*

**theorem** IdealGasLaw (hV :  $0 < V$ ) (hT :  $0 < T$ ) :  
**let** P

## 2.86 QuantumInfo.StatMech.ThermoQuantities

[QuantumInfo/StatMech/ThermoQuantities.lean](#) on GitHub.

**def PartitionZ**[\[source\]](#)

*The partition function corresponding to a given MicroHamiltonian. This is a function taking a thermodynamic  $\beta$ , not a temperature. It also depends on the data  $D$  defining the system extrinsics.*

*\* Ideally this would be an NNReal, but  $\int$  (NNReal) doesn't work right now, so it would just be a separate proof anyway*

```
def PartitionZ ( $\beta$  :  $\mathbb{R}$ ) :  $\mathbb{R}$ 
```

**def PartitionZT**[\[source\]](#)

*The partition function as a function of temperature  $T$  instead of  $\beta$ .*

```
def PartitionZT ( $T$  :  $\mathbb{R}$ ) :  $\mathbb{R}$ 
```

**def InternalU**[\[source\]](#)

*The Internal Energy,  $U$  or  $E$ , defined as  $-\partial(\ln Z)/\partial\beta$ . Parameterized here with  $\beta$ .*

```
def InternalU ( $\beta$  :  $\mathbb{R}$ ) :  $\mathbb{R}$ 
```

**def HelmholtzA**[\[source\]](#)

*The Helmholtz Free Energy,  $-T * \ln Z$ . Also denoted  $F$ . Parameterized here with temperature  $T$ , not  $\beta$ .*

```
def HelmholtzA ( $T$  :  $\mathbb{R}$ ) :  $\mathbb{R}$ 
```

**def EntropyS**[\[source\]](#)

*The entropy, defined as the  $-\partial A/\partial T$ . Function of  $T$ .*

```
def EntropyS ( $T$  :  $\mathbb{R}$ ) :  $\mathbb{R}$ 
```

**def EntropyS $\beta$** [\[source\]](#)

*The entropy, defined as  $\ln Z + \beta * U$ . Function of  $\beta$ .*

```
def EntropyS $\beta$  ( $\beta$  :  $\mathbb{R}$ ) :  $\mathbb{R}$ 
```

**def ZIntegrable**[\[source\]](#)

*To be able to compute or define anything from a Hamiltonian, we need its partition function to be a computable integral. A Hamiltonian is ZIntegrable at  $\beta$  if PartitionZ is Lebesgue integrable and nonzero.*

```
def ZIntegrable ( $\beta$  :  $\mathbb{R}$ ) : Prop
```

**def Positive $\beta$ Integrable**[\[source\]](#)

*This Prop defines the most common case of ZIntegrable, that it is integrable at all finite temperatures (aka all positive  $\beta$ ).*

```
def PositiveβIntegrable : Prop
```

**theorem DifferentiableAt\_Z\_if\_ZIntegrable**

[\[source\]](#)

```
theorem DifferentiableAt_Z_if_ZIntegrable {β : ℝ} (h : H.ZIntegrable d β) : ContDiffAt ℝ ω (H.
  PartitionZ d) β
```

**theorem entropy\_A\_eq\_entropy\_Z**

[\[source\]](#)

*The two definitions of entropy, in terms of  $T$  or  $\beta$ , are equivalent.*

```
theorem entropy_A_eq_entropy_Z (T β : ℝ) (hβT : T * β = 1) (hi : H.ZIntegrable d β)
  : EntropyS H d T = EntropySβ H d β
```

**theorem β\_eq\_deriv\_S\_U**

[\[source\]](#)

*The "definition of temperature from entropy":  $1/T = (\partial S/\partial U)$ , when the derivative is at constant extrinsic  $d$  (typically  $N/V$ ). Here we use  $\beta$  instead of  $1/T$  on the left, and express the right actually as  $(\partial S/\partial \beta)/(\partial U/\partial \beta)$ , as all our things are ultimately parameterized by  $\beta$ .*

```
theorem β_eq_deriv_S_U {β : ℝ} (hi : H.ZIntegrable d β) : β = (deriv (H.EntropySβ d) β) / deriv
  (H.InternalU d) β
```

**def Pressure**

[\[source\]](#)

*Pressure, as a function of  $T$ . Defined as the conjugate variable to volume.*

```
def Pressure (T : ℝ) : ℝ
```